

AIML U1

Artificial Intelligence - Overview

What is Artificial Intelligence?

Artificial Intelligence is a way of making a computer, a computer-controlled robot, or a software think intelligently, in the similar manner the intelligent humans think.

AI is accomplished by studying how human brain thinks, and how humans learn, decide, and work while trying to solve a problem, and then using the outcomes of this study as a basis of developing intelligent software and systems.

The term artificial intelligence was first coined decades ago in the year 1956 by John Mctarty at the Dartmouth conference. He defined artificial intelligence as the science and engineering of making intelligent machines.

Many AI applications, are not perceived as AI because we often tend to think of artificial as robots doing our daily chores. But the truth is artificial intelligence has found its way into our daily lives. It has become so general that we don't realize we use it all the time. For instance Google is able to give you accurate search results. Facebook feed always gives you content based on your interest. This is because of the application of artificial intelligence.

Philosophy of AI

While exploiting the power of the computer systems, the curiosity of human, lead him to wonder, *"Can a machine think and behave like humans do?"*

Thus, the development of AI started with the intention of creating similar intelligence in machines that we find and regard high in humans.

Why Artificial Intelligence?

Before Learning about Artificial Intelligence, we should know that what is the importance of AI and why should we learn it. Following are some main reasons to learn about AI:

- With the help of AI, you can create such software or devices which can solve real-world problems very easily and with accuracy such as health issues, marketing, traffic issues, etc.
- With the help of AI, you can create your personal virtual Assistant, such as Cortana, Google Assistant, Siri, etc.
- With the help of AI, you can build such Robots which can work in an environment where survival of humans can be at risk. Eg: spirit and curiosity and Vikram.
- AI opens a path for other new technologies, new devices, and new Opportunities.

Goals of AI

- **To Create Expert Systems** – The systems which exhibit intelligent behavior, learn, demonstrate, explain, and advice its users.
- **To Implement Human Intelligence in Machines** – Creating systems that understand, think, learn, and behave like humans.

What Contributes to AI?

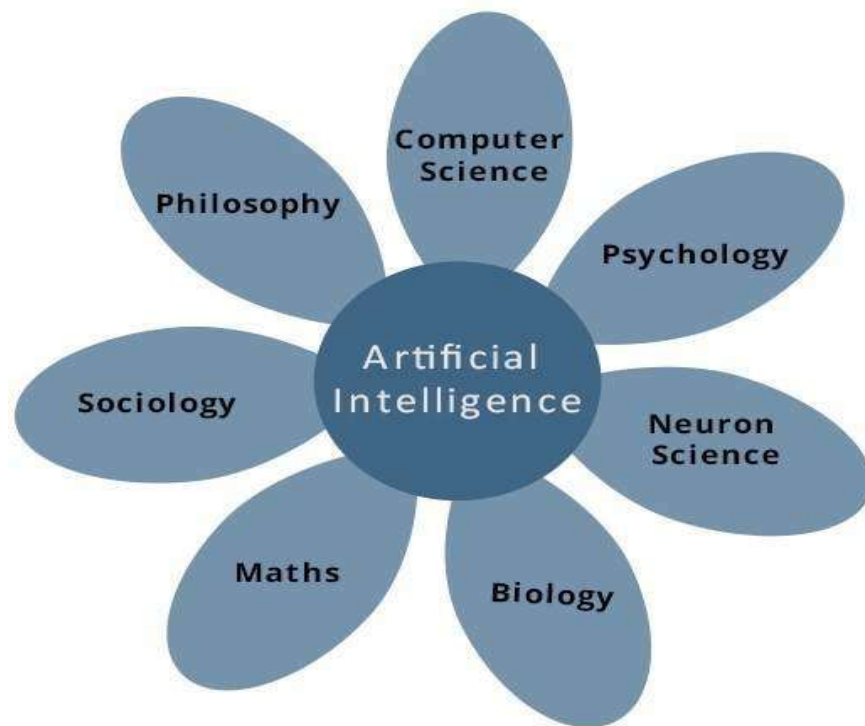
Artificial intelligence is a science and technology based on disciplines such as

- Computer Science,
- Biology,
- Psychology,
- Linguistics,
- Mathematics, and
- Engineering

The major thrust of AI: A major thrust of AI is in the development of computer functions associated with human intelligence, such as

- reasoning,
- learning, and
- problem solving.

Out of the following areas, one or multiple areas can contribute to build an intelligent system.



Advantages of Artificial Intelligence

Following are some main advantages of Artificial Intelligence:

- **High Accuracy with less errors:** AI machines or systems are prone to less errors and high accuracy as it takes decisions as per pre-experience or information.

- **High-Speed:** AI systems can be of very high-speed and fast-decision making, because of that AI systems can beat a chess champion in the Chess game.
- **High reliability:** AI machines are highly reliable and can perform the same action multiple times with high accuracy.
- **Useful for risky areas:** AI machines can be helpful in situations such as defusing a bomb, exploring the ocean floor, where to employ a human can be risky.
- **Digital Assistant:** AI can be very useful to provide digital assistant to the users such as AI technology is currently used by various E-commerce websites to show the products as per customer requirement.
- **Useful as a public utility:** AI can be very useful for public utilities such as a self-driving car which can make our journey safer and hassle-free, facial recognition for security purpose, Natural language processing to communicate with the human in human-language, etc.

Disadvantages of Artificial Intelligence

Every technology has some disadvantages, and the same goes for Artificial intelligence. Being so advantageous technology still, it has some disadvantages which we need to keep in our mind while creating an AI system. Following are the disadvantages of AI:

High Cost: The hardware and software requirement of AI is very costly as it requires lots of maintenance to meet current world requirements.

Can't think out of the box: Even though we are making smarter machines with AI, but still they cannot work out of the box, as the robot will only do that work for which they are trained, or programmed.

No feelings and emotions: AI machines can be an outstanding performer, but still it does not have the feeling so it cannot make any kind of emotional attachment with human, and may sometime be harmful for users if the proper care is not taken.

Increase dependency on machines: With the increment of technology, people are getting more dependent on devices and hence they are losing their mental capabilities.

No Original Creativity: As humans are so creative and can imagine some new ideas but still AI machines cannot beat this power of human intelligence and cannot be creative and imaginative.

Applications of AI

Artificial Intelligence has various applications in today's society. It is becoming essential for today's time because it can solve complex problems with an efficient way in multiple industries, such as Healthcare, entertainment, finance, education, etc. AI is making our daily life more comfortable and fast.

Following are some sectors which have the application of Artificial Intelligence:

- **Gaming** – AI plays crucial role in strategic games such as chess, poker, tic-tac-toe, etc., where machine can think of large number of possible positions based on heuristic knowledge.
- **Natural Language Processing** – It is possible to interact with the computer that understands natural language spoken by humans.
- **Expert Systems** – There are some applications which integrate machine, software, and special information to impart reasoning and advising. They provide explanation and advice to the users.
- **Vision Systems** – These systems understand, interpret, and comprehend visual input on the computer. For example,
 - A spying aeroplane takes photographs, which are used to figure out spatial information or map of the areas.
 - Doctors use clinical expert system to diagnose the patient.
 - Police use computer software that can recognize the face of criminal with the stored portrait made by forensic artist.
- **Speech Recognition** – Some intelligent systems are capable of hearing and comprehending the language in terms of sentences and their meanings while a human talks to it. It can handle different accents, slang words, noise in the background, change in human's noise due to cold, etc.
- **Handwriting Recognition** – The handwriting recognition software reads the text written on paper by a pen or on screen by a stylus. It can recognize the shapes of the letters and convert it into editable text.

- **Intelligent Robots** – Robots are able to perform the tasks given by a human. They have sensors to detect physical data from the real world such as light, heat, temperature, movement, sound, bump, and pressure. They have efficient processors, multiple sensors and huge memory, to exhibit intelligence. In addition, they are capable of learning from their mistakes and they can adapt to the new environment.
- **AI in Astronomy:** Artificial Intelligence can be very useful to solve complex universe problems. AI technology can be helpful for understanding the universe such as how it works, origin, etc. Spotting distant planets in solar systems which is 2500 light years away.
- **AI in Healthcare:** In the last, five to ten years, AI becoming more advantageous for the healthcare industry and going to have a significant impact on this industry.

Healthcare Industries are applying AI to make a better and faster diagnosis than humans. AI can help doctors with diagnoses and can inform when patients are worsening so that medical help can reach to the patient before hospitalization. eg: Apple watch

IBM is one of the pioneers that has developed AI software specifically for medicine. More than 230 healthcare organizations worldwide use IBM Watson technology. In 2016, IBM Watson AI technology was able to cross-reference 20 million oncology records and correctly diagnose a rare leukaemia condition in a patient.

Google's AI eye doctor, is another initiative taken by google where they are working with an Indian ikea chain to develop a AI system which can examine retinal scans and identify a condition called diabetic retinopathy which causes blindness.

- **The google predictive** search is one of the most famous AI applications. When you begin typing a search term and google makes recommendations for you to choose from. Predictive searches are based on data that Google collects about you, such as your location, your age and other personal

details, By using AI the search engine attempts to guess what you might be trying to find.

- **Virtual assistants:** Virtual assistants like Siri, Alexa, and Cortana are examples of artificial intelligence. A newly released google virtual assistant called google duplex can respond to calls and book appointments for you, it adds a human touch.
- **AI in Finance:** AI and finance industries are the best matches for each other. The finance industry is implementing automation, chatbot, adaptive intelligence, algorithm trading, and machine learning into financial processes.

In the finance sector JP Morgan's G's contract intelligent platform uses artificial intelligence, machine learning and image recognition software to analyze legal documents and extract important data points and clauses in a matter of seconds. Manually reviewing 12000 agreements takes over 36000 hours but AI was able to do this in a matter of seconds.

- **AI in Data Security:** The security of data is crucial for every company and cyber-attacks are growing very rapidly in the digital world. AI can be used to make your data more safe and secure. Some examples such as AEG bot, AI2 Platform, are used to determine software bug and cyber-attacks in a better way.
- **AI in Social Media:** Social Media sites such as Facebook, Twitter, and Snapchat contain billions of user profiles, which need to be stored and managed in a very efficient way. AI can organize and manage massive amounts of data. AI can analyze lots of data to identify the latest trends, hashtag, and requirement of different users.

In social media platforms like facebook, artificial intelligence is used for face verification where in machine learning and deep learning concepts are used to detect facial features and tag your friends.

Twitter's AI is being used to identify hate speech and terroristic languages in tweets. It makes use of machine learning, deep learning, and natural language processing to filter out offensive content. The company discovered

and banned three hundred thousand terrorist linked accounts, 95% of which were found by non human artificially intelligent machines.

- **AI in Travel & Transport:** AI is becoming highly demanding for travel industries. AI is capable of doing various travel related works such as from making travel arrangement to suggesting the hotels, flights, and best routes to the customers. Travel industries are using AI-powered chatbots which can make human-like interaction with customers for better and fast response.
- **AI in Automotive Industry:** Some Automotive industries are using AI to provide virtual assistant to their user for better performance. Such as Tesla has introduced TeslaBot, an intelligent virtual assistant.

Various Industries are currently working for developing self-driven cars which can make your journey more safe and secure.

Self Driving Cars: AI implements computer vision, image detection and deep learning to build cars, that can automatically detect objects and drive around without human intervention. Elon Musk talks about how AI is implemented in Tesla's self driving cars and auto-pilot features. Their Robo taxi version one that can ferry passengers without anyone behind the wheel.

- **AI in Robotics:** Artificial Intelligence has a remarkable role in Robotics. Usually, general robots are programmed such that they can perform some repetitive task, but with the help of AI, we can create intelligent robots which can perform tasks with their own experiences without pre-programmed.

Humanoid Robots are best examples for AI in robotics, recently the intelligent Humanoid robot named as Erica and Sophia has been developed which can talk and behave like humans.

- **AI in Entertainment:** We are currently using some AI based applications in our daily life with some entertainment services such as Netflix or Amazon. With the help of ML/AI algorithms, these services show the recommendations for programs or shows.
- **AI in Agriculture:** Agriculture is an area which requires various resources, labor, money, and time for best result. Now a day's agriculture is becoming digital, and AI is emerging in this field. Agriculture is applying AI as

agriculture robotics, soil and crop monitoring, predictive analysis. AI in agriculture can be very helpful for farmers.

- **AI in E-commerce:** AI is providing a competitive edge to the e-commerce industry, and it is becoming more demanding in the e-commerce business. AI is helping shoppers to discover associated products with recommended size, color, or even brand.
- **AI in education:** AI can automate grading so that the tutor can have more time to teach. AI chatbot can communicate with students as a teaching assistant.

AI in the future can be work as a personal virtual tutor for students, which will be accessible easily at any time and any place.

- **Literature:** composing sonnets and poems

Common Misconceptions:

People often tend to think that

- artificial intelligence
- machine learning and
- deep learning

are the same since they have common applications.

For example Siri is an application of AI, machine learning and deep learning. So how are these technologies related?

Artificial intelligence is the science of getting machines to mimic the behavior of humans.

Machine learning is a subset of AI that focuses on getting machines to make decisions by feeding them data.

Deep learning is a subset of machine learning that uses the concept of neural networks to solve complex problems.

Therefore artificial intelligence, machine learning and deep learning are interconnected fields.

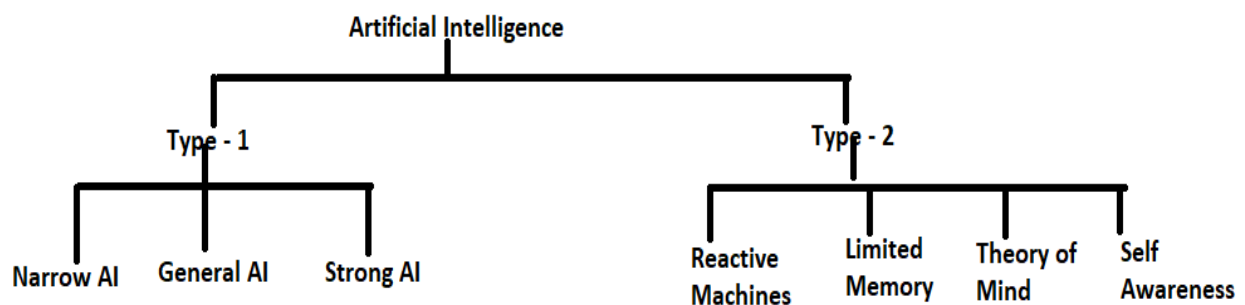
Machine learning and deep learning aids artificial intelligence by providing a set of algorithms and neural networks to solve data driven problems.

But AI is not restricted to only machine learning and deep learning. It covers a vast domain of fields including

- Natural Language Processing,
- Object Detection,
- Computer Vision,
- robotics,
- expert systems, and so on.

Types of Artificial Intelligence:

Artificial Intelligence can be divided in various types, there are mainly two types of main categorization which are based on capabilities and based on functionality of AI. Following is flow diagram which explain the types of AI.



AI type-1: Based on Capabilities

1. Weak AI or Narrow AI:

- Narrow AI is a type of AI which is able to perform a dedicated task with intelligence. The most common and currently available AI is Narrow AI in the world of Artificial Intelligence.
- Narrow AI cannot perform beyond its field or limitations, as it is only trained for one specific task. Hence it is also termed as weak AI. Narrow AI can fail in unpredictable ways if it goes beyond its limits.
- Apple Siri is a good example of Narrow AI, but it operates with a limited pre-defined range of functions.
- IBM's Watson supercomputer also comes under Narrow AI, as it uses an Expert system approach combined with Machine learning and natural language processing.
- Some Examples of Narrow AI are playing chess, purchasing suggestions on e-commerce site, self-driving cars, speech recognition, and image recognition.

2. General AI:

- General AI is a type of intelligence which could perform any intellectual task with efficiency like a human.
- The idea behind the general AI to make such a system which could be smarter and think like a human by its own.
- Currently, there is no such system exist which could come under general AI and can perform any task as perfect as a human.
- The worldwide researchers are now focused on developing machines with General AI.
- As systems with general AI are still under research, and it will take lots of efforts and time to develop such systems.

3. Super AI:

- Super AI is a level of Intelligence of Systems at which machines could surpass human intelligence, and can perform any task better than human with cognitive properties. It is an outcome of general AI.
- Some key characteristics of strong AI include capability include the ability to think, to reason, solve the puzzle, make judgments, plan, learn, and communicate by its own.
- Super AI is still a hypothetical concept of Artificial Intelligence. Development of such systems in real is still world changing task.

Narrow AI – Dedicated for one task

General AI – Performs like human

Super AI – Intelligent than human

Artificial Intelligence type-2: Based on functionality

1. Reactive Machines

- Purely reactive machines are the most basic types of Artificial Intelligence.
- Such AI systems do not store memories or past experiences for future actions.
- These machines only focus on current scenarios and react on it as per possible best action.
- IBM's Deep Blue system is an example of reactive machines.
- Google's AlphaGo is also an example of reactive machines.

2. Limited Memory

- Limited memory machines can store past experiences or some data for a short period of time.

- These machines can use stored data for a limited time period only.
- Self-driving cars are one of the best examples of Limited Memory systems. These cars can store recent speed of nearby cars, the distance of other cars, speed limit, and other information to navigate the road.

3. Theory of Mind

- Theory of Mind AI should understand the human emotions, people, beliefs, and be able to interact socially like humans.
- This type of AI machines are still not developed, but researchers are making lots of efforts and improvement for developing such AI machines.

4. Self-Awareness

- Self-awareness AI is the future of Artificial Intelligence. These machines will be super intelligent, and will have their own consciousness, sentiments, and self-awareness.
- These machines will be smarter than human mind.
- Self-Awareness AI does not exist in reality still and it is a hypothetical concept.

Since the emergence of AI in the 1950s, we have seen an exponential growth in its potential. AI cover domains such as machine learning, deep learning, neural networks, natural language processing, knowledge base, expert systems, and so on. It has also made its way into computer vision and image processing.

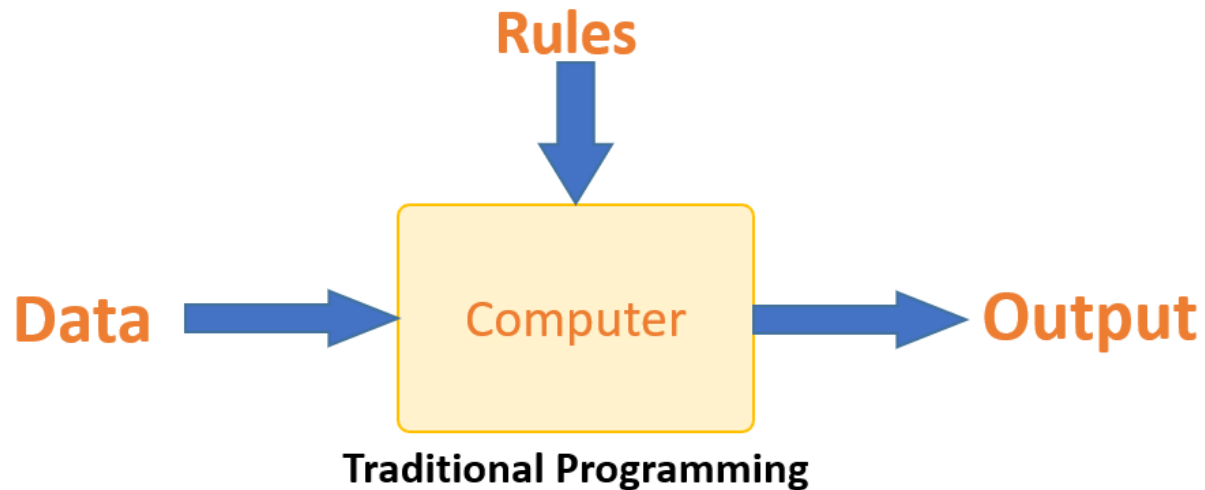
AIML U2

What is Machine Learning?

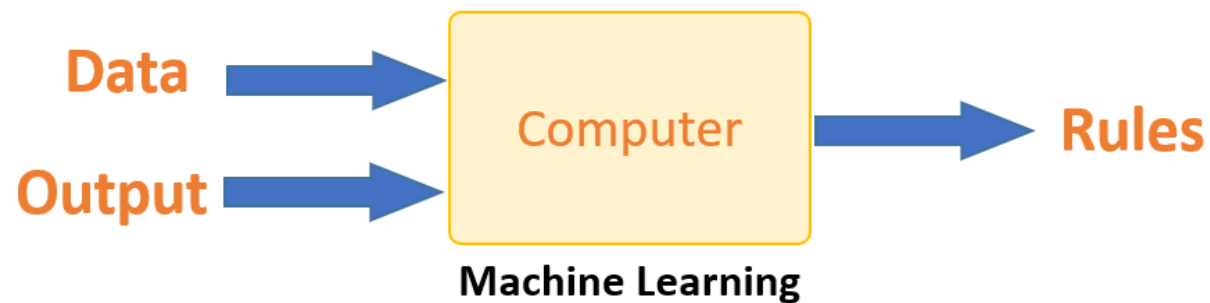
- Machine learning is a growing technology which enables computers to learn automatically from past data.
- Machine learning uses various algorithms for **building mathematical models and making predictions using historical data or information**.
- Currently, it is being used for various tasks such as **image recognition, speech recognition, email filtering, Facebook auto-tagging, recommender system**, and many more.
- Machine Learning is said as a subset of **artificial intelligence** that is mainly concerned with the development of algorithms which allow a computer to learn from the data and past experiences on their own.
- The term machine learning was first introduced by **Arthur Samuel** in **1959**. We can define it in a summarized way as:
 - Machine learning enables a machine to automatically learn from data, improve performance from experiences, and predict things without being explicitly programmed.
- With the help of sample historical data, which is known as **training data**, machine learning algorithms build a **mathematical model** that helps in making predictions or decisions without being explicitly programmed.
- Machine learning brings computer science and statistics together for creating predictive models.
- Machine learning constructs or uses the algorithms that learn from historical data.
- The more we will provide the information, the higher will be the performance.

Machine Learning vs. Traditional Programming

- Traditional programming differs significantly from machine learning.
- In traditional programming, a programmer code all the rules in consultation with an expert in the industry for which software is being developed.
- Each rule is based on a logical foundation; the machine will execute an output following the logical statement.
- When the system grows complex, more rules need to be written. It can quickly become unsustainable to maintain.



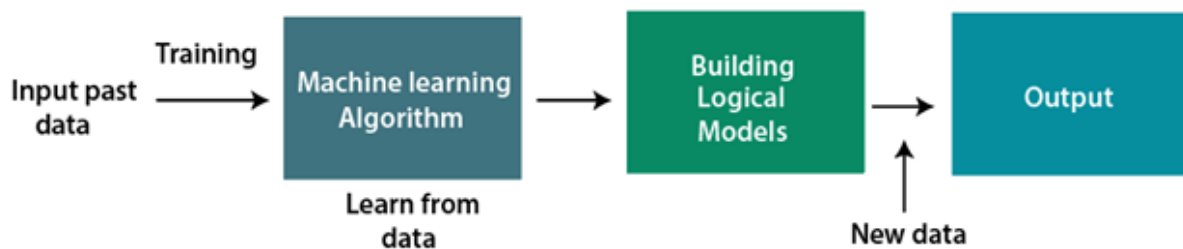
- Machine learning is supposed to overcome this issue.
- The machine learns how the input and output data are correlated and it writes a rule.
- The programmers do not need to write new rules each time there is new data.
- The algorithms adapt in response to new data and experiences to improve efficacy over time.



How does Machine Learning Work?

- Machine learning is the brain where all the learning takes place.
- The way the machine learns is similar to the human being.
- Humans learn from experience.
- The more we know, the more easily we can predict.
- When we face an unknown situation, the likelihood of success is lower than the known situation.
- Machines are trained the same way.
- To make an accurate prediction, the machine sees an example.
- When we give the machine a similar example, it can figure out the outcome.
- However, like a human, if it is fed a previously unseen example, the machine has difficulties to predict.

- That is, a Machine Learning system **learns from historical data, builds the prediction models, and whenever it receives new data, predicts the output for it.**
- The accuracy of predicted output depends upon the amount of data, as the huge amount of data helps to build a better model which predicts the output more accurately.
- Suppose we have a complex problem, where we need to perform some predictions, instead of writing a code for it, we just need to feed the data to generic algorithms, and with the help of these algorithms, machine builds the logic as per the data and predict the output.
- Machine learning has changed our way of thinking about the problem. The below block diagram explains the working of Machine Learning algorithm:



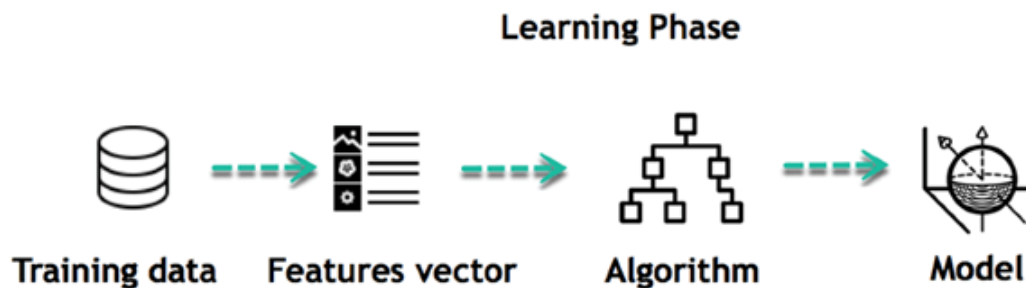
Learning, Inference and Data:

- The core objective of machine learning is the **learning** and **inference**.
- First of all, the machine learns through the discovery of patterns.
- This discovery is made by using the **data**.
- One crucial part of the data scientist is to choose carefully which data to provide to the machine.
- The list of attributes used to solve a problem is called a **feature vector**.
- You can think of a feature vector as a subset of data that is used to tackle a problem.

Model:

The machine uses some algorithms to simplify the reality and transform this discovery into a **model**.

Therefore, the learning stage is used to describe the data and summarize it into a model.



- For example, the machine is trying to understand the relationship between the wage of an individual and the likelihood to go to a fancy restaurant.

- The machine finds a positive relationship between wage and going to a high-end restaurant: This is the model

Inferring

- When the model is built, it is possible to test how powerful it is on never-seen-before data.
- The new data are transformed into a features vector, go through the model and give a prediction.
- There is no need to update the rules or train again the model.
- You can use the model previously trained to make inference on new data.

Inference from Model



Steps in Machine Learning:

- The life of Machine Learning programs is straightforward and can be summarized in the following points:
 1. Define a question
 2. Collect data
 3. Visualize data
 4. Train algorithm
 5. Test the Algorithm
 6. Collect feedback
 7. Refine the algorithm
 8. Loop 4-7 until the results are satisfying
 9. Use the model to make a prediction
- Once the algorithm gets good at drawing the right conclusions, it applies that knowledge to new sets of data.

Features of Machine Learning:

- Machine learning uses data to detect various patterns in a given dataset.
- It can learn from past data and improve automatically.

- It is a data-driven technology.
- Machine learning is much similar to data mining as it also deals with the huge amount of the data.

Need for Machine Learning

- The need for machine learning is increasing day by day. The reason behind the need for machine learning is that it is capable of doing tasks that are too complex for a person to implement directly.
- As a human, we have some limitations as we cannot access the huge amount of data manually, so for this, we need some computer systems and here comes the machine learning to make things easy for us.
- We can train machine learning algorithms by providing them the huge amount of data and let them explore the data, construct the models, and predict the required output automatically.
- The performance of the machine learning algorithm depends on the amount of data, and it can be determined by the cost function.
- With the help of machine learning, we can save both time and money.
- The importance of machine learning can be easily understood by its uses cases,
- Currently, machine learning is used in **self-driving cars, cyber fraud detection, face recognition, and friend suggestion by Facebook**, etc.
- Various top companies such as Netflix and Amazon have built machine learning models that are using a vast amount of data to analyze the user interest and recommend product accordingly.

Following are some key points which show the importance of Machine Learning:

- Rapid increment in the production of data
- Solving complex problems, which are difficult for a human
- Decision making in various sector including finance
- Finding hidden patterns and extracting useful information from data.

Challenges in Machine Learning

- We are living in a situation with numerous cutting edge applications developed using machine learning. There are certain **challenges** an ML practitioner might face while developing an application from **the initial stage** to bringing them to **production**.

1. Data Collection

- Data plays a key role in any use case. 60% of the work of a data scientist lies in collecting the data. For beginners to experiment with machine learning, they can easily find data from Kaggle, UCI ML Repository, etc.

- To implement real case scenarios, you need to collect the data through web-scraping or (through APIs like twitter) or for solving business problems you need to attain data from clients (here ML engineers need to coordinate with domain experts to collect the data). Another Example: Linkdin problem.
- Once the data is collected, we need to structure the data and store it in the database. This requires knowledge of Big data (or data engineer) which plays a major role here.

2. Less Amount of Training Data

- Once the data is collected you need to validate if the quantity is sufficient for the use case (if it is a time-series data, we need a minimum of 3–5 years of data).
- The two important things we do while doing a machine learning project are **selecting a learning algorithm** and **training the model using some of the acquired data**.
- So as humans, we naturally tend to make mistakes and as a result, things may go wrong. Here, the mistakes could be opting for the wrong model or selecting data which is bad.
- Imagine your machine learning model is a baby, and you plan on teaching the baby to distinguish between a cat and a dog. So we begin with pointing at a cat and saying ‘ it’s a **CAT**’ and do the same thing with a **DOG** (possibly repeating this procedure many times). Now the child will be able to distinguish between dog and cat, by identifying shapes, colors, or any other features. And just like that, the baby becomes an expert in distinguishing objects.
- Similarly, we train the model with a lot of data. A child may distinguish the animal with less number of samples, but a machine learning model requires thousands of examples for even simple problems. For complex problems like Image Classification and Speech Recognition, it may require data in a count of millions.
- Therefore, we need to train a model with sufficient **DATA**.

3. Non-representative Training Data

- The training data should be representative of the new cases to generalize well i.e., the data we use for training should cover all the cases that occurred and that is going to occur. By using a non-representative training set, the trained model is not likely to make accurate predictions.

- Systems which are developed to make predictions for generalized cases in business problem view are said to be good machine learning models. It will help the model to perform well even for the data which the data model has never seen.
- If the number of training samples is low, we have *sampling noise* which is unrepresentative data, again countless training tests bring **sampling bias** if the strategy utilized for training is defective.
- A popular case of examining sampling bias occurred during the US Presidential election in 1936 (Landon against Roosevelt), a very large poll was conducted by the *Literary Digest* by sending mail to around ten million people out of which 2.4 million answered, and predicted that Landon is going to get 57% of votes with high confidence. But, Roosevelt won with 62% of the votes.
- The problem here is in the sampling method, to get the addresses for conducting the poll, *Literary Digest* used magazine subscribers, club membership lists, and the likes, which are utilized by wealthier individuals who are bound to cast a ballot Republican, (hence Landon). Also, *non-response* bias comes into the picture as only 25% of people answered the poll.
- To make accurate predictions without any drifts, the training datasets must be representative.

4. Poor Quality of Data

- In reality, we don't directly start training the model.
- Analyzing data is the most important step. But the data we collected might not be ready for training, some samples are abnormal from others having outliers or missing values for instance. (300 kg man/woman or 9 feet height)
- In these cases, we can remove the outliers, or fill the missing features/values using median or mean (to fill height) or simply remove the attributes/instances with missing values, or train the model with and without these instances.
- We don't want our system to make false predictions. So the quality of data is very important to get accurate results. Data preprocessing needs to be done by filtering missing values, extract & rearrange what the model needs.

5. Irrelevant/Unwanted Features

- If the training data contains a large number of irrelevant features and enough relevant features, the machine learning system will not give the results as expected. One of the

important aspects required for the success of a machine learning project is the selection of good features to train the model also known as Feature Selection.

- For example if we need to predict the **number of hours a person needs to exercise** based on the input features that we collected — age, gender, weight, height, and location (i.e., where he/she lives).
 - Among these 5 features, **location** value might not impact our output function. This is an irrelevant feature, we know that we can have better results without this feature.
 - Also, we can combine two features to produce a more useful one i.e., *Feature Extraction*. In our example, we can produce a feature called **BMI** by eliminating weight and height. We can apply transformations on the dataset too.
 - Creating new features by gathering more data also helps.

6. Overfitting the Training Data

- For example you visited a restaurant in a new city. You looked at the menu to order something and found that the cost or bill is too high. You may say that *‘all the restaurants in the city are too costly and not affordable’*. Overgeneralizing is something that is done frequently,
- The frameworks can do the same and in AI, we call it overfitting.
- It means the model is performing well, making likely predictions on the training dataset, but it is not generalized well.
- Another example is suppose you are attempting to implement an Image Classification model to classify apple, peach, oranges, bananas with training samples of — 3000, 500, 500, 500 respectively. If we train the model with these samples the system is more likely to classify oranges as apples as the number of training samples for apples is too high. This can be referred to as Oversampling.
- At the point when the model is excessively unpredictable when compared to the noisiness of the training dataset, overfitting occurs. We can avoid it by:
 - Gathering more training data.
 - Selecting a model with fewer features, a higher degree polynomial model is not preferred compared to the linear model.
 - Fix data errors, remove the outliers, and reduce the number of instances in the training set.

7. Underfitting the Training data

- Underfitting which is opposite to overfitting generally occurs when the model is too simple to understand the base structure of the data.

- It generally happens when we have less information to construct an exact model.
Example: Human behavior.
- Main options to reduce underfitting are:
 - Feature Engineering — feeding better features to the learning algorithm.
 - Removing noise from the data.
 - Increasing parameters and selecting a powerful model.

8. Offline Learning & Deployment of the model

- Machine Learning engineering follows these steps while building an application
 - 1) Data collection
 - 2) Data cleaning
 - 3) Feature engineering
 - 4) Analyzing patterns
 - 5) Training the model and Optimization
 - 6) Deployment.
- A lot of machine learning practitioners can perform all steps but can lack the skills for deployment.
- Bringing their applications into production has become one of the biggest challenges due to lack of practice and dependencies issues, low understanding of underlying models with business, understanding of business problems, unstable models.
- Generally, many of the developers collect data from websites like Kaggle and start training the model. But in reality, we need to make a source for data collection, that varies dynamically. (like stock ticker data, or continuous data)
- Offline learning or Batch learning may not be used for this type of variable data. The system is trained and then it is launched into production, runs without learning anymore. Here the data might drift as it changes dynamically.
- It is always preferred to build a pipeline to collect, analyze, build/train, test & validate the dataset for any machine learning project and train the model in batches.

A system doesn't perform well if the training set is too small, or if the data is not generalized, noisy, and corrupted with irrelevant features.

Classification of Machine Learning

- At a broad level, machine learning can be classified into three types:
 1. **Supervised learning**
 2. **Unsupervised learning**

3. Reinforcement learning

1) Supervised Learning

- In supervised learning we provide data to the machine learning system that are already labelled for training.

Example

Width	Height	Label
0.9	1	paddy
0.7	1.2	Paddy
12	30	Arecanut
15	20	Arecanut
1	1.6	Paddy
20	40	Arecanut

- This labelled data is used for training the model and predicting the output.
- Supervised learning can be grouped further in two categories of algorithms:
 - **Classification**
 - **Regression**

2) Unsupervised Learning

- In unsupervised learning the data is not labelled.

Width	Height
0.9	1
0.7	1.2
12	30
15	20
1	1.6
20	40

- The unlabelled data is provided to the machine learning algorithm.
- The algorithm itself will discover patterns in the data and use it for classification or prediction.
- Hence there is no supervision.
- Unsupervised learning can be classified into two types.

- **Clustering (football players)**
- **Association (bread jam)**

3) Reinforcement Learning

- Reinforcement learning is, training the system based on feedback.
- The agent is rewarded for right action taken and penalised for wrong action taken.
- The agent learns automatically from these feedbacks and improves its performance.
- The goal of an agent is to get the most reward points, and hence, it improves its performance.
- The robotic dog, which automatically learns the movement of his arms, is an example of Reinforcement learning.

Key differences between Artificial Intelligence (AI) and Machine learning (ML):

Artificial Intelligence	Machine learning
Artificial intelligence is a technology which enables a machine to simulate human behavior.	Machine learning is a subset of AI which allows a machine to automatically learn from past data without programming explicitly.
The goal of AI is to make a smart computer system like humans to solve complex problems.	The goal of ML is to allow machines to learn from data so that they can give accurate output.
In AI, we make intelligent systems to perform any task like a human.	In ML, we teach machines with data to perform a particular task and give an accurate result.
Machine learning and deep learning are the two main subsets of AI.	Deep learning is a main subset of machine learning.
AI has a very wide range of scope.	Machine learning has a limited scope.
AI is working to create an intelligent system which can perform various complex tasks.	Machine learning is working to create machines that can perform only those specific tasks for which they are trained.
AI system is concerned about maximizing the chances of success.	Machine learning is mainly concerned about accuracy and patterns.

The main applications of AI are Siri, customer support using chatbots, Expert System, Online game playing, intelligent humanoid robot, etc.	The main applications of machine learning are Online recommender system, Google search algorithms, Facebook auto friend tagging suggestions, etc.
AI can be divided into three types, which are, Weak AI, General AI, and Strong AI.	Machine learning can also be divided into mainly three types that are Supervised learning, Unsupervised learning, and Reinforcement learning.
AI includes learning, reasoning, and self-correction.	ML includes learning and self-correction when introduced with new data.
AI completely deals with Structured, semi-structured, and unstructured data.	Machine learning deals with Structured and semi-structured data.

Datasets for Machine Learning

- In the field of machine learning or for a data scientist it is important to practice with different types of datasets.
- But finding suitable datasets for different types of machine learning projects is a difficult task.
- The sources of various datasets are provided below.

What is a dataset?

- **A dataset is a collection of data organized and stored in a well ordered and well formatted manner.**
- An example for a dataset is given below.

Country	Age	Salary	Purchased
India	38	48000	No
France	43	45000	Yes
Germany	30	54000	No
France	48	65000	No
Germany	40		Yes
India	35	58000	Yes

- A dataset can contain a collection of rows.
- Each row represents an observation or an instance.
- Each column in a row represents a feature or an attribute or a parameter.
- A dataset can contain data in a array or a csv file or a database table., excel file
- A dataset is most commonly stored in a "**Comma Separated File,**" or **CSV file.**
- To store a "tree-like data," we can use a JSON file.

Types of data in datasets

- **Numerical data:**Numerical data represents numbers such as integers, real numbers. For example house price, temperature, etc.,
- **Categorical data:**Categorical data represents categories or groups. For example 0/1, Yes/No, Graduate/Not graduate, etc.,
- **Ordinal data:**Ordinal data are similar to categorical data. But they are comparative information or that represent a range.

For example

economic status as “low income”, “middle income”, “high income”,

education level as “high school”, “Bachelors”, “Masters”, “PhD”

income as “less than 50K”, “50-100K”, “over 100K”

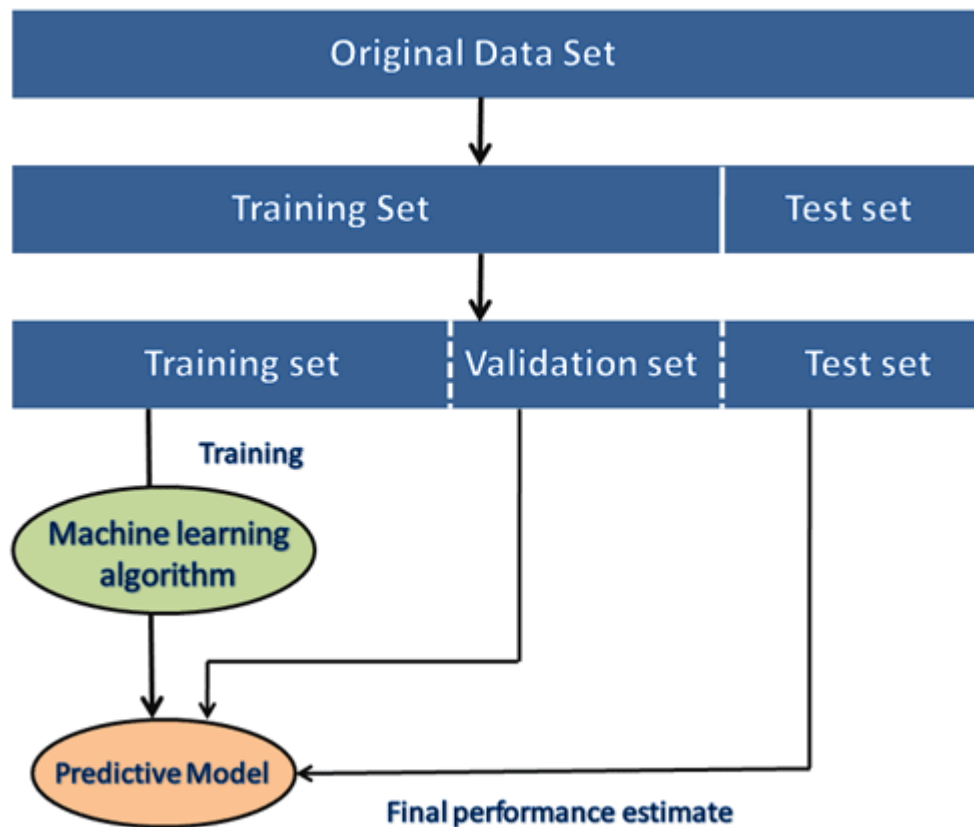
quality as “poor”, “good”, “very good”, “excellent”.

- A real world dataset will be huge in size and difficult to manage and requires powerful resources.
- You can use dummy datasets or datasets that are readily available for practice.

Need for Well Prepared Dataset

- Collecting and preparing a dataset is the most important step in a Machine Learning or AI project.
- If the data is not well prepared or not pre-processed, then the machine learning technology applied will not provide efficient or accurate results.

- During the development stage of a machine learning project, the developers completely rely on the dataset.
- In a machine learning project, the data is divided into two parts
 - **Training dataset:**
 - **Test Dataset**



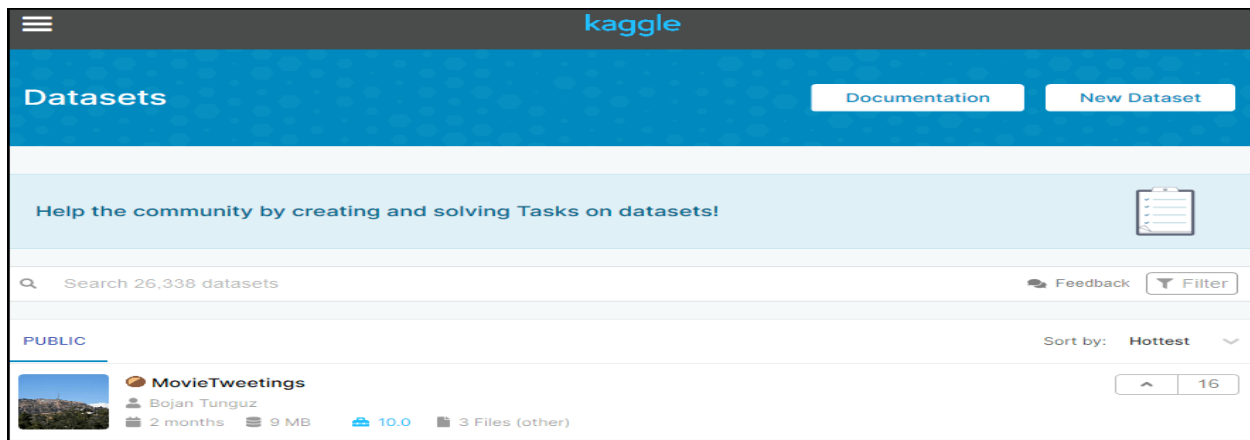
Ready made datasets available for Machine Learning:

Below is the list of datasets which are freely available for the public to work on it:

1. Kaggle Datasets

- Kaggle is a commonly and considered to be one of the best source for readymade datasets.
- It provides high quality datasets in different formats.
- A user can search, download and publish his/her own datasets.

- A developer can collaborate with other data scientists to work on machine learning projects.
- The link for the Kaggle dataset is <https://www.kaggle.com/datasets>.



2. UCI Machine Learning Repository

- UCI Machine learning repository is one of the great sources for ML datasets.
- It contains databases, domain theories and data generators that are widely used by developers of ML projects.
- Since the year 1987, it has been widely used by students, professors, researchers.
- It has datasets classified based on the types of tasks for which they are required like, **Regression, Classification, Clustering, etc.**
- It also contains some of the popular datasets such as the **Iris dataset, Car Evaluation dataset, Poker Hand dataset, etc.**
- The link for the UCI machine learning repository is <https://archive.ics.uci.edu/ml/index.php>.

UCI



Machine Learning Repository

Center for Machine Learning and Intelligent Systems

About

Citation Policy

Donate a Data Set

Contact

Search

☐ Repository

☐ Web

Google

View ALL Data Sets

Browse Through: 488 Data Sets

Table View

List View

Default Task

Classification (360)

Regression (107)

Clustering (90)

Other (55)

Attribute Type

Categorical (38)

Numerical (325)

Mixed (55)

Data Type

Multivariate (374)

Univariate (24)

Sequential (51)

Time-Series (99)

Text (55)

Domain Theory (23)

Other (21)

Area

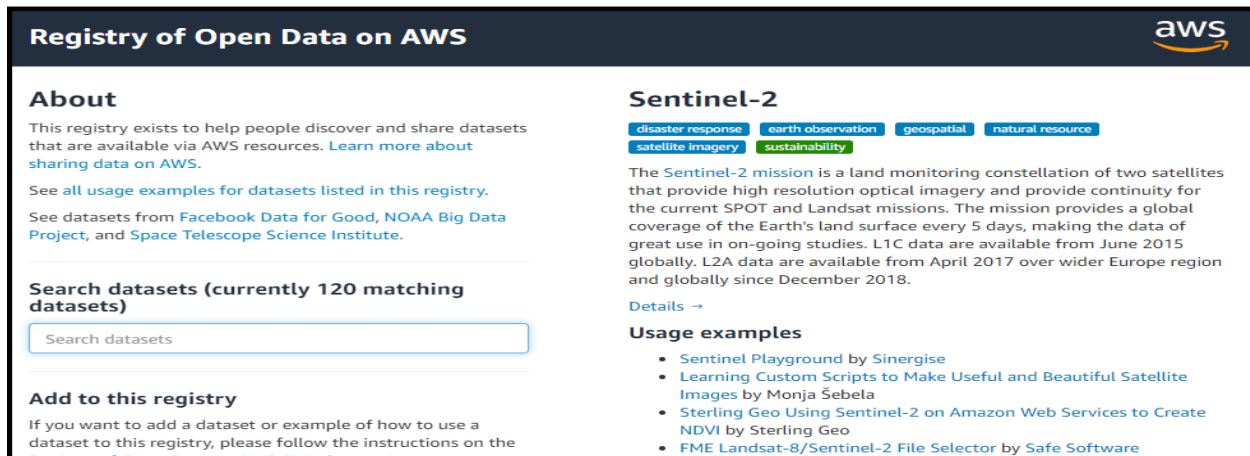
Life Sciences (110)

Physical Sciences (52)

Name	Data Types	Default Task	Attribute Types	# Instances	# Attributes	Year
 Abalone	Multivariate	Classification	Categorical, Integer, Real	4177	8	1995
 Adult	Multivariate	Classification	Categorical, Integer	48842	14	1996
 Annealing	Multivariate	Classification	Categorical, Integer, Real	798	38	
 Anonymous Microsoft Web Data		Recommender-Systems	Categorical	37711	294	1998
 Arrhythmia	Multivariate	Classification	Categorical, Integer, Real	452	279	1998

3. Datasets via AWS

- AWS allows us to search download, access and share publicly available datasets using its resources.
- The datasets are provided and maintained by government organizations, researches, businesses or individuals.
- We can search, download, access, and share the datasets that are publicly available via AWS resources. These datasets can be accessed through AWS resources but provided and maintained by different government organizations, researchers, businesses, or individuals.
- The datasets are available on the cloud. Anybody can use them.
- The link for the resource is <https://registry.opendata.aws/>.



4. Google's Dataset Search Engine

- **Google dataset search engine** is a search engine launched by **Google** on **September 5, 2018**.
- It helps users or researches get datasets online for free.
- The link for the Google dataset search engine is <https://toolbox.google.com/datasetsearch>.

The screenshot shows the Google Dataset Search interface. The search bar contains the word "classification". Below the search bar, there are filters for "Updated Date", "Download Format", "Usage Rights", and "Free". The results section shows "100+ results found". The first result is "Classification" from catalog.data.gov, data.nasa.gov, and +1 more, updated May 2, 2019. Below this, there are two results from Kaggle: "Mushroom Classification" (updated Dec 1, 2016) and "Question Classification". On the right side, there is a detailed view of the "Classification" dataset, including buttons to "Explore at catalog.data.gov", "Explore at Rally - Open Data Portal", and "Explore at data.wu.ac.at". It also mentions the dataset was updated May 2, 2019, provided by Dashlink, and includes a description of supervised learning tasks.

5. Microsoft Datasets

- The Microsoft has launched the "**Microsoft Research Open data**" repository.
- It provides datasets for free in areas like **natural language processing, computer vision, and domain-specific sciences**.
- Users can either download and use the datasets or directly use them on the cloud.
- The link to download or use the dataset from this resource is <https://msropendata.com/>.

The screenshot shows the Microsoft Research Open Data website. The header includes the Microsoft logo, the site name "Microsoft Research Open Data", and navigation links for "Categories", "About", "FAQs", "Feedback", and "Login". The main section features the title "Microsoft Research Open Data" with a "BETA" badge. Below the title is a search bar labeled "Search datasets". The text below the search bar describes the collection of free datasets from Microsoft Research, aimed at advancing state-of-the-art research in areas like natural language processing, computer vision, and domain-specific sciences. It also mentions that users can download or copy datasets directly to a cloud-based Data Science Virtual Machine for a seamless development experience.

Dataset Categories

6. Awesome Public Dataset Collection

 **Awesome Public Datasets**



NOTICE: This repo is automatically generated by [apd-core](#). Please **DO NOT** modify this file directly. We have provided [a new way](#) to contribute to Awesome Public Datasets. The original PR entrance directly on repo is closed forever.

- 🟢 I am well.
- 🟠 Please fix me.

This list of [a topic-centric public data sources](#) in high quality. They are collected and tidied from blogs, answers, and user responses. Most of the data sets listed below are free, however, some are not. Other amazingly awesome lists can be found in [sindresorhus's awesome](#) list.

Table of Contents

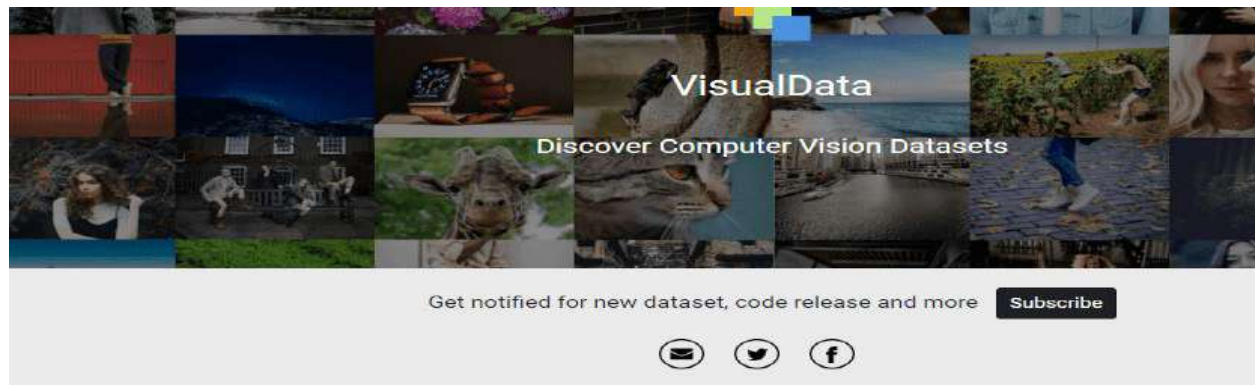
- [Agriculture](#)
- [Biology](#)
- [Climate+Weather](#)
- [ComplexNetworks](#)
- [ComputerNetworks](#)

- Awesome public dataset collection provides high-quality datasets that are arranged in a **well-organized manner according to various topics such as Agriculture, Biology, Climate, Complex networks, etc.**
- **Most of the datasets are available free, but some are for payment.**
- The link to download the dataset from Awesome public dataset collection is <https://github.com/awesomedata/awesome-public-datasets>.

7. Government Datasets

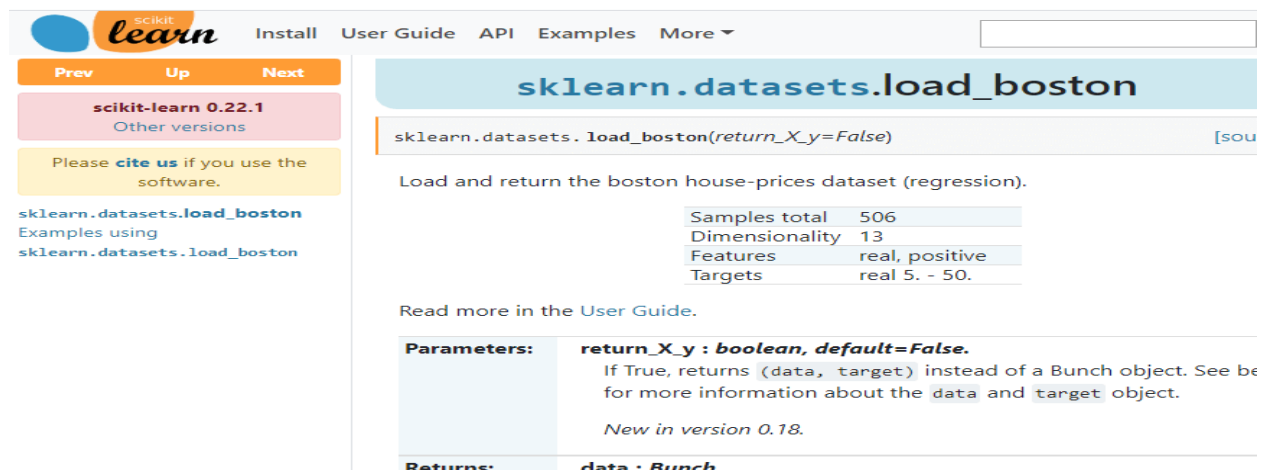
- There are different sources that provide government datasets belonging to various departments of various countries.
- Below are some links of government datasets:
 - [Indian Government dataset](#)
 - [US Government Dataset](#)
 - [Northern Ireland Public Sector Datasets](#)
 - [European Union Open Data Portal](#)

8. Computer Vision Datasets



- Visual data provides many datasets that are related to computer visions such as Image Classification, Video classification, Image Segmentation, etc.
- For Image processing projects and deep learning projects you can use the datasets available here.
- The link for downloading the dataset from this source is <https://www.visualdata.io/>.

9. Scikit-learn dataset

A screenshot of the Scikit-learn website showing the documentation for the `sklearn.datasets.load_boston` function. The page includes navigation links like "Prev", "Up", and "Next". The main content area shows the function signature `sklearn.datasets.load_boston(return_X_y=False)` and a description: "Load and return the boston house-prices dataset (regression)." Below this, a table lists the dataset's characteristics: Samples total (506), Dimensionality (13), Features (real, positive), and Targets (real 5. - 50.). The "Parameters" section explains the `return_X_y` parameter, and the "Returns" section indicates it returns a `Bunch` object.

- Scikit-learn is a very good source for machine learning datasets.
- It provides both dummy as well as real-world datasets.
- These datasets can be accessed using `sklearn.datasets` package and using the Scikit-learn API.
- The link to download datasets from this source is <https://scikit-learn.org/stable/datasets/index.html>.

Data Preprocessing in Machine learning

- Data maybe of three types:
 - Structured (database/ Excel)
 - Semi structured (XML/HTML)
 - Unstructured (Text)
- The raw data that is collected may not be suitable for applying machine learning models.
- Raw data may have missing values or noise or outliers or may not be in the format that is required for processing.
- So the raw data needs to be converted to a form that can be used by the machine learning algorithms efficiently.
- That is the data needs to be cleaned.
- To do this the missing values needs to be filled with either median values or may need to be deleted.
- Categorical data may need to be converted to numeric form.
- Unstructured or Text data may have to be converted into numerical data.
- Outliers may have to be deleted.
- Multiple attributes may have to be clubbed to form a single attribute.
- Therefore data preprocessing is the process of preparing the raw data to be suitable for machine learning task.
- Data preprocessing is the first step in a machine learning project.
- If the data is not prepared, it may lead to failure of the system or lower accuracy.
- Well prepared data will lead to higher accuracy and higher efficiency of the machine learning model.
- Data pre-processing involves the following steps.
 - **Getting the dataset**
 - **Importing libraries**
 - **Importing datasets**
 - **Finding Missing Data**
 - **Encoding Categorical Data**
 - **Splitting dataset into training and test set**
 - **Feature scaling (reduce distance between the data)**

1) Get the Dataset

- This is the first step in a machine learning process.
- You can use dummy dataset or dataset available in repositories like kaggle.
- Twitter datasets, speeches, image datasets, movie reviews, etc., are available. Eg: nltk library
- Electronic health records are difficult to obtain.
- Images of leaves, paddy plant, arecanut can be captured.
- Bacteria can be cultured and high resolution images taken.
- Web page contents can be scraped using customized search engines.
- Readymade scraping tools can be used.
- Readymade APIs also can be used to get the dataset. Eg: nltk library

2) Importing Libraries

- Python has inbuilt libraries for various jobs.
- In Python the 3 important libraries used for data preprocessing are:
 - **Numpy:** Numpy is a Python library to manipulate arrays and matrices, mathematical operations and scientific calculations.

import numpy as nm

- Matplotlib is a library which is useful for data visualization. It helps in plotting 2D charts. It will be imported as below:

import matplotlib.pyplot as mtp

- **Pandas:** Pandas is one of the most popular of the Python libraries. It is used for managing, manipulating and analyzing datasets. It is open source.

```
1 # importing Libraries
2 import numpy as nm
3 import matplotlib.pyplot as mtp
4 import pandas as pd
5
```

3) Importing the Datasets

To read and load the dataset which is in csv file, in Python we use the following statement from the Pandas library..

read_csv() function:

```
data_set= pd.read_csv('Dataset.csv')
```

Introduction

Machine Learning algorithms are completely dependent on data because it is the most crucial aspect that makes model training possible. On the other hand, if we won't be able to make sense out of that data, before feeding it to ML algorithms, a machine will be useless. In simple words, we always need to feed right data i.e. the data in correct scale, format and containing meaningful features, for the problem we want machine to solve.

This makes data preparation the most important step in ML process. Data preparation may be defined as the procedure that makes our dataset more appropriate for ML process.

Why Data Pre-processing?

After selecting the raw data for ML training, the most important task is data pre-processing. In broad sense, data preprocessing will convert the selected data into a form we can work with or can feed to ML algorithms. We always need to preprocess our data so that it can be as per the expectation of machine learning algorithm.

Data Pre-processing Techniques

We have the following data preprocessing techniques that can be applied on data set to produce data for ML algorithms –

Scaling

Most probably our dataset comprises of the attributes with varying scale, but we cannot provide such data to ML algorithm hence it requires rescaling. Data rescaling makes sure that attributes are at same scale. Generally, attributes are rescaled into the range of 0 and 1. ML algorithms like gradient descent and k-Nearest Neighbors requires scaled data. We can rescale the data with the help of MinMaxScaler class of scikit-learn Python library.

Example

In this example we will rescale the data of Pima Indians Diabetes dataset which we used earlier. First, the CSV data will be loaded (as done in the previous chapters) and then with the help of MinMaxScaler class, it will be rescaled in the range of 0 and 1.

The first few lines of the following script are same as we have written in previous chapters while loading CSV data.

```
from pandas import read_csv
from numpy import set_printoptions
from sklearn import preprocessing
path = r'C:\pima-indians-diabetes.csv'
names = ['preg', 'plas', 'pres', 'skin', 'test', 'mass', 'pedi',
'age', 'class']
dataframe = read_csv(path, names=names)
array = dataframe.values
```

Now, we can use MinMaxScaler class to rescale the data in the range of 0 and 1.

```
data_scaler = preprocessing.MinMaxScaler(feature_range=(0,1))
data_rescaled = data_scaler.fit_transform(array)
```

We can also summarize the data for output as per our choice. Here, we are setting the precision to 1 and showing the first 10 rows in the output.

```
set_printoptions(precision=1)
print ("\nScaled data:\n", data_rescaled[0:10])
```

Output

Scaled data:

```
[
  [0.4 0.7 0.6 0.4 0.  0.5 0.2 0.5 1. ]
  [0.1 0.4 0.5 0.3 0.  0.4 0.1 0.2 0. ]
  [0.5 0.9 0.5 0.  0.  0.3 0.3 0.2 1. ]
  [0.1 0.4 0.5 0.2 0.1 0.4 0.  0.  0. ]
  [0.  0.7 0.3 0.4 0.2 0.6 0.9 0.2 1. ]
  [0.3 0.6 0.6 0.  0.  0.4 0.1 0.2 0. ]
  [0.2 0.4 0.4 0.3 0.1 0.5 0.1 0.1 1. ]
  [0.6 0.6 0.  0.  0.  0.5 0.  0.1 0. ]
  [0.1 1.  0.6 0.5 0.6 0.5 0.  0.5 1. ]
  [0.5 0.6 0.8 0.  0.  0.  0.1 0.6 1. ]
]
```

From the above output, all the data got rescaled into the range of 0 and 1.

Normalization

Another useful data preprocessing technique is Normalization. This is used to rescale each row of data to have a length of 1. It is mainly useful in Sparse dataset where we have lots of zeros. We can rescale the data with the help of Normalizer class of scikit-learn Python library.

Types of Normalization

In machine learning, there are two types of normalization preprocessing techniques as follows –

L1 Normalization

It may be defined as the normalization technique that modifies the dataset values in a way that in each row the sum of the absolute values will always be up to 1. It is also called Least Absolute Deviations.

Example

In this example, we use L1 Normalize technique to normalize the data of Pima Indians Diabetes dataset which we used earlier. First, the CSV data will be loaded and then with the help of Normalizer class it will be normalized.

The first few lines of following script are same as we have written in previous chapters while loading CSV data.

```
from pandas import read_csv
from numpy import set_printoptions
from sklearn.preprocessing import Normalizer
path = r'C:\pima-indians-diabetes.csv'
names = ['preg', 'plas', 'pres', 'skin', 'test', 'mass', 'pedi',
'age', 'class']
dataframe = read_csv (path, names=names)
array = dataframe.values
```

Now, we can use Normalizer class with L1 to normalize the data.

```
Data_normalizer = Normalizer(norm='l1').fit(array)
Data_normalized = Data_normalizer.transform(array)
```

We can also summarize the data for output as per our choice. Here, we are setting the precision to 2 and showing the first 3 rows in the output.

```
set_printoptions(precision=2)
print ("\nNormalized data:\n", Data_normalized [0:3])
```

Output

Normalized data:

```
[
  [0.02 0.43 0.21 0.1  0.  0.1  0.  0.14 0. ]
  [0.   0.36 0.28 0.12 0.  0.11 0.  0.13 0. ]
```

```
[0.03 0.59 0.21 0.    0. 0.07 0. 0.1  0. ]
]
```

L2 Normalization

It may be defined as the normalization technique that modifies the dataset values in a way that in each row the sum of the squares will always be up to 1. It is also called least squares.

Example

In this example, we use L2 Normalization technique to normalize the data of Pima Indians Diabetes dataset which we used earlier. First, the CSV data will be loaded (as done in previous chapters) and then with the help of Normalizer class it will be normalized.

The first few lines of following script are same as we have written in previous chapters while loading CSV data.

```
from pandas import read_csv
from numpy import set_printoptions
from sklearn.preprocessing import Normalizer
path = r'C:\pima-indians-diabetes.csv'
names = ['preg', 'plas', 'pres', 'skin', 'test', 'mass', 'pedi',
'age', 'class']
dataframe = read_csv (path, names=names)
array = dataframe.values
```

Now, we can use Normalizer class with L1 to normalize the data.

```
Data_normalizer = Normalizer(norm='l2').fit(array)
Data_normalized = Data_normalizer.transform(array)
```

We can also summarize the data for output as per our choice. Here, we are setting the precision to 2 and showing the first 3 rows in the output.

```
set_printoptions(precision=2)
print ("\nNormalized data:\n", Data_normalized [0:3])
```

Output

```
Normalized data:
[
  [0.03 0.83 0.4  0.2  0. 0.19 0. 0.28 0.01]
  [0.01 0.72 0.56 0.24 0. 0.22 0. 0.26 0. ]
  [0.04 0.92 0.32 0.   0. 0.12 0. 0.16 0.01]
]
```

Binarization

As the name suggests, this is the technique with the help of which we can make our data binary. We can use a binary threshold for making our data binary. The values above that threshold value will be converted to 1 and below that threshold will be converted to 0. For example, if we choose threshold value = 0.5, then the dataset value above it will become 1 and below this will become 0. That is why we can call it **binarizing** the data or **thresholding** the data. This technique is useful when we have probabilities in our dataset and want to convert them into crisp values.

We can binarize the data with the help of Binarizer class of scikit-learn Python library.

Example

In this example, we will rescale the data of Pima Indians Diabetes dataset which we used earlier. First, the CSV data will be loaded and then with the help of Binarizer class it will be converted into binary values i.e. 0 and 1 depending upon the threshold value. We are taking 0.5 as threshold value.

The first few lines of following script are same as we have written in previous chapters while loading CSV data.

```
from pandas import read_csv
from sklearn.preprocessing import Binarizer
path = r'C:\pima-indians-diabetes.csv'
names = ['preg', 'plas', 'pres', 'skin', 'test', 'mass', 'pedi',
         'age', 'class']
dataframe = read_csv(path, names=names)
array = dataframe.values
```

Now, we can use Binarize class to convert the data into binary values.

```
binarizer = Binarizer(threshold=0.5).fit(array)
Data_binarized = binarizer.transform(array)
```

Here, we are showing the first 5 rows in the output.

```
print ("\nBinary data:\n", Data_binarized [0:5])
```

Output

Binary data:

```
[
  [1. 1. 1. 1. 0. 1. 1. 1. 1.]
  [1. 1. 1. 1. 0. 1. 0. 1. 0.]
  [1. 1. 1. 0. 0. 1. 1. 1. 1.]
  [1. 1. 1. 1. 1. 1. 0. 1. 0.]
  [0. 1. 1. 1. 1. 1. 1. 1. 1.]
]
```

Standardization

Another useful data preprocessing technique which is basically used to transform the data attributes with a Gaussian distribution. It differs the mean and SD (Standard Deviation) to a standard Gaussian distribution with a mean of 0 and a SD of 1. This technique is useful in ML algorithms like linear regression, logistic regression that assumes a Gaussian distribution in input dataset and produce better results with rescaled data. We can standardize the data (mean = 0 and SD =1) with the help of StandardScaler class of scikit-learn Python library.

Example

In this example, we will rescale the data of Pima Indians Diabetes dataset which we used earlier. First, the CSV data will be loaded and then with the help of StandardScaler class it will be converted into Gaussian Distribution with mean = 0 and SD = 1.

The first few lines of following script are same as we have written in previous chapters while loading CSV data.

```
from sklearn.preprocessing import StandardScaler
from pandas import read_csv
from numpy import set_printoptions
path = r'C:\pima-indians-diabetes.csv'
names = ['preg', 'plas', 'pres', 'skin', 'test', 'mass', 'pedi',
'age', 'class']
dataframe = read_csv(path, names=names)
array = dataframe.values
```

Now, we can use StandardScaler class to rescale the data.

```
data_scaler = StandardScaler().fit(array)
data_rescaled = data_scaler.transform(array)
```

We can also summarize the data for output as per our choice. Here, we are setting the precision to 2 and showing the first 5 rows in the output.

```
set_printoptions(precision=2)
print ("\nRescaled data:\n", data_rescaled [0:5])
```

Output

Rescaled data:

```
[
  [ 0.64  0.85  0.15  0.91 -0.69  0.2   0.47  1.43  1.37]
 [-0.84 -1.12 -0.16  0.53 -0.69 -0.68 -0.37 -0.19 -0.73]
 [ 1.23  1.94 -0.26 -1.29 -0.69 -1.1   0.6  -0.11  1.37]
 [-0.84 -1.   -0.16  0.15  0.12 -0.49 -0.92 -1.04 -0.73]
 [-1.14  0.5  -1.5   0.91  0.77  1.41  5.48 -0.02  1.37]
]
```

Data Labeling

We discussed the importance of good data for ML algorithms as well as some techniques to pre-process the data before sending it to ML algorithms. One more aspect in this regard is data labeling. It is also very important to send the data to ML algorithms having proper labeling. For example, in case of classification problems, lot of labels in the form of words, numbers etc. are there on the data.

What is Label Encoding?

Most of the sklearn functions expect that the data with number labels rather than word labels. Hence, we need to convert such labels into number labels. This process is called label encoding. We can perform label encoding of data with the help of LabelEncoder() function of scikit-learn Python library.

Example

In the following example, Python script will perform the label encoding.

First, import the required Python libraries as follows –

```
import numpy as np
from sklearn import preprocessing
```

Now, we need to provide the input labels as follows –

```
input_labels =
['red', 'black', 'red', 'green', 'black', 'yellow', 'white']
```

The next line of code will create the label encoder and train it.

```
encoder = preprocessing.LabelEncoder()
encoder.fit(input_labels)
```

The next lines of script will check the performance by encoding the random ordered list –

```
test_labels = ['green', 'red', 'black']
encoded_values = encoder.transform(test_labels)
print("\nLabels =", test_labels)
print("Encoded values =", list(encoded_values))
encoded_values = [3, 0, 4, 1]
decoded_list = encoder.inverse_transform(encoded_values)
```

We can get the list of encoded values with the help of following python script –

```
print("\nEncoded values =", encoded_values)
print("\nDecoded labels =", list(decoded_list))
```

Output

```
Labels = ['green', 'red', 'black']  
Encoded values = [1, 2, 0]  
Encoded values = [3, 0, 4, 1]  
Decoded labels = ['white', 'black', 'yellow', 'green']
```

Types of Data:

Text, Image, Audio, Video

Data preprocessing is a process to convert raw data into meaningful data using different techniques

Why Data Preprocessing:

Data in the real world is dirty

- Incomplete

- Noisy

- Inconsistent

- Duplicate

Data Quality Elements

- Accuracy

- Completeness

- Consistency

- Believability

- Interpretability

Machine Learning Algorithms follow the rule (like kids)

So it is important to train the algorithm with correct and accurate data.

Because we take decisions based on the ML decisions.

Garbage in Garbage out

Steps/Techniques in Data Preprocessing

Data cleaning

Data integration

Data Reduction

Data Transformation

Data discretization

What is Data Cleaning

Data cleaning means fill in missing values, smooth out noise while identifying outliers, and correct inconsistencies in the data.

Fill missing values – nan values

Ignore outliers

What is Data Integration

Data Integration is a technique to merge data from multiple sources into a coherent data store, such as data warehouse.

Integrate data from Mysql database, json file, csv file, xml file that too columnwise

What is Data Reduction

Data Reduction is a technique used to reduce the data size by aggregating, eliminating redundant features, or clustering for instance.

Eg: filtering cricketers

What is data transformation

Data transformation means data are transformed or consolidated into forms appropriate for ML model training, such as normalization, maybe applied where data are scaled to fall within a smaller range like 0.0 to 1.0 or 0.0 to -1.0.

Aggregation

Feature Type Conversion

Normalization

Attribute/feature construction

Eg: age may vary between 22 to 60 and salary may range from 20,000 to 2 lakh. So values are in different range which may confuse the algorithm.

What is Data Discretization

Data discretization technique transforms numeric data by mapping values to interval or concept labels.

It can be used to reduce the number of values for a given continuous attributes by dividing the range of the attributes into intervals.

Discretization techniques include:

Binning

Histogram analysis

Cluster analysis

Decision tree analysis

Correlation Analysis.

Eg: Age with values 1,2,3,4,5,6,7,8,9

Bins: 1-3, 4-6, 7-9

Simple Linear Regression

What is Regression?

- Regression analysis is a form of predictive modelling technique which investigates the relationship between a dependent and independent variable.
- A regression shows the changes in a dependent variable on the Y axis to the changes in the explanatory variable on the X axis.

Uses of Regression:

- **Determining the strength of predictors** : that is the strength of the effect of the independent variable on the dependent variable. **Eg: what is the strength of the relationship between sales and marketing spend or the strength of the relationship between kilometers travelled by a vehicle and its price.**
- **Forecasting an effect**: that is forecast effects or impact of changes. That is how much the dependent variable changes with unit change in the independent variable(s). **Eg: how much how much additional sale income will I get for each thousand dollars spent on marketing.**
- **Trend forecasting**: In this the regression analysis predicts trends and future values. The regression analysis can be used to get Point estimates. Like for **eg: what will be the price of Bitcoin in the next six months.**

Types of Regressions:

- Simple Linear Regression
- Multi Linear Regression
- Logistic Regression
- Ridge Regression
- Lasso Regression
- Polynomial Regression

Linear Regression:

- Linear Regression is the easiest algorithm among the machine learning algorithms.
- It is a statistical model that attempts to show the relationship between two variables with a linear equation.

Linear Regression Selection Criteria.

Classification and Regression Capabilities.

- Regression model predict a continuous variable. Such as the sales made on a day or predict the temperature of a city

- But for classification it can be used but not efficient, because for every new data point the model needs to be recreated from scratch.

Data Quality:

- Data Quality matters. In simple linear regression the outliers can significantly disrupt the outcome of the algorithm.

Computational Complexity:

- Linear Regression is not computationally expensive as compared to the decision tree or clustering algorithms. The order of complexity for n training example and X features usually fall in either big O of X square that is $O(X^2)$ or Big O of Xn , $O(Xn)$

Comprehensible and Transparent:

- Linear Regression are easily comprehensible and transparent in nature.
- They can be represented by a simple mathematical notation to anyone and can be understood very easily.

So above are some of the criteria based on which we will select Linear Regression algorithm.

Where Linear Regression can be used?

- **Evaluating Trends and Sales Estimates.**

Linear regression can be used in business to evaluate trends and make estimates or forecast. For eg: if a company sales has increased steadily every month for past few years then conducting a linear analysis on the sales data with monthly sales on the y axis and time on the x axis. This will give you a line that predicts the upward trends in the sale. After creating the trend line the company could use the slope of the line to forecast the sale in future months.

- **Analysing the impact of price changes with linear regression**

Linear regression can be used to analyse the effect of pricing on consumer behavior. If a company changes the price on certain product several times, it can record the quantity itself for each price level. And then perform a linear regression with sold quantity as the dependent variable and price as the independent variable. This would result in a line that depicts the extent to which the customer reduce their consumption of the product as the price increase. This would help us in future pricing decisions.

- **Assessment of risk in financial services and insurance domain.**

A health insurance company may conduct a linear regression by plotting the number of claims per customer against its age and they might discover that the old customers tend to make more health insurance claim. Such analysis might guide important business decisions.

Discussion:

Suppose you have 2 variables x and y which are independent and dependent variables plotted on a graph.

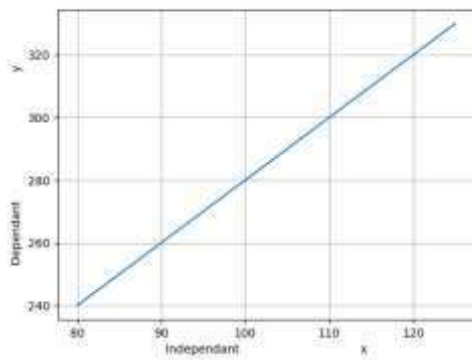
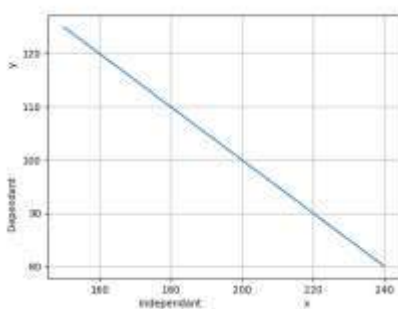


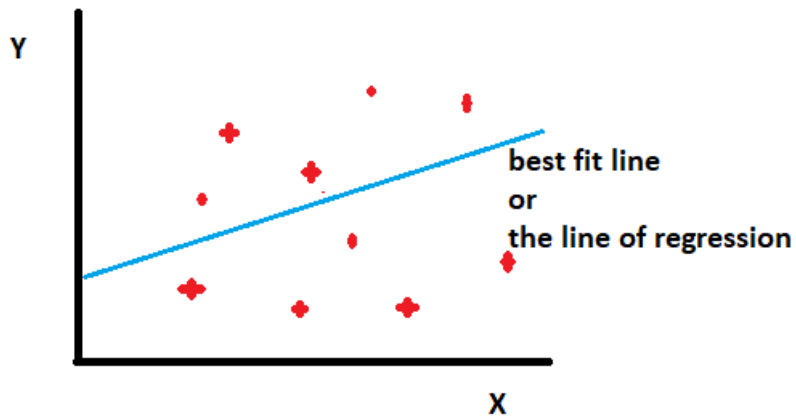
Image: Positive Linear Regression

- When the independent variable increases on the x axis, if the dependent variable also increases on the y axis, you would get a positive linear regression.
- The slope of the line will be positive.
- When the independent variable decreases, if the dependent variable decreases, then you would get a negative linear regression.
- The slope of the line will be negative.

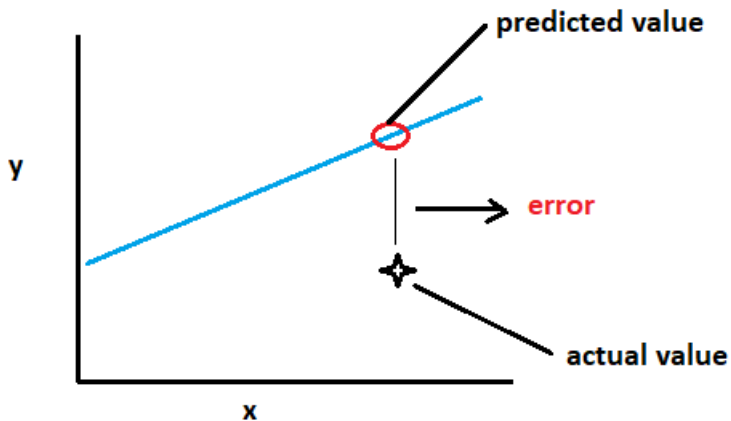


- This line of regression shows the relation between the independent and dependent variable.

- This line is represented by the equation $y=mx+c$
- If we have plotted some data points on the graph,



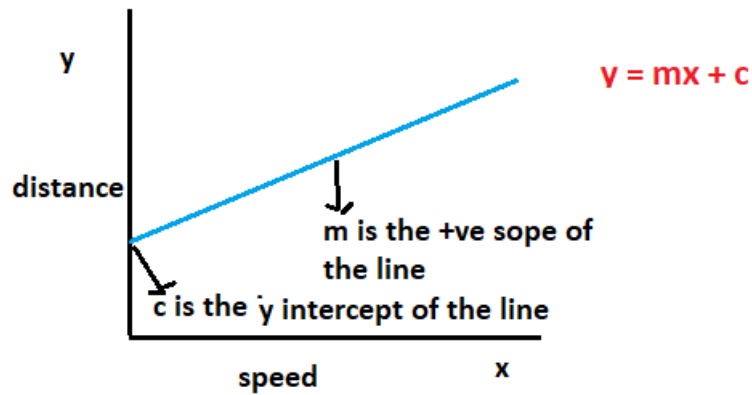
- Let us say the task is to plot the best fit line or the line of regression for these data points.
- Once our best fit line is drawn, now we have the task of prediction.
- Suppose we have an actual value and a predicted value.



- Our main goal is to reduce the error or distance between the actual value and the predicted value.
- The best fit line is the one which has the least error or least difference between the predicted value and the actual value.
- In other words we have to minimize the error.

Mathematical implementation:

- First let us see a graph where speed is in x axis and distance is in y axis for a fixed duration of time.
- If you plot a line or a graph, then we will get a positive relationship.



- If the equation of the line is

$$Y = mx + c$$

Where

Y is the distance travelled in a fixed duration of time

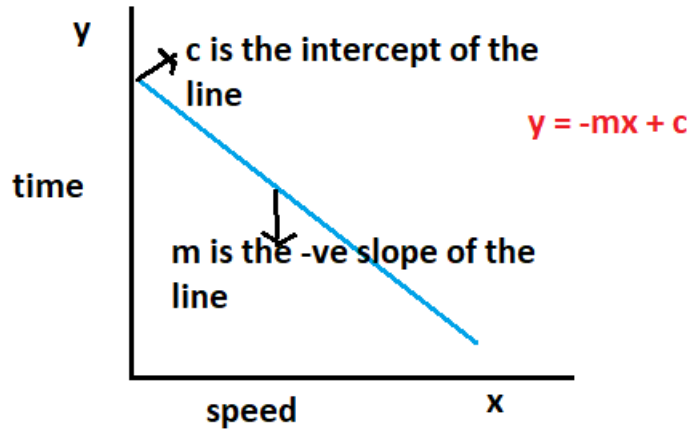
X is the speed of the vehicle

M is the positive slope of the line

C is the y intercept of the line.

- But suppose if we plot a graph between the speed of the vehicle on the x axis and time taken to travel a fixed distance, then in that case we will get a negative relationship.

Negative Relationship



- The slope of the line is negative.
- Here the equation of the line changes to

$$Y = -mx + c$$

Where

y is the time taken to travel a fixed distance.

X is the speed of the vehicle

M is the -ve slope of the line

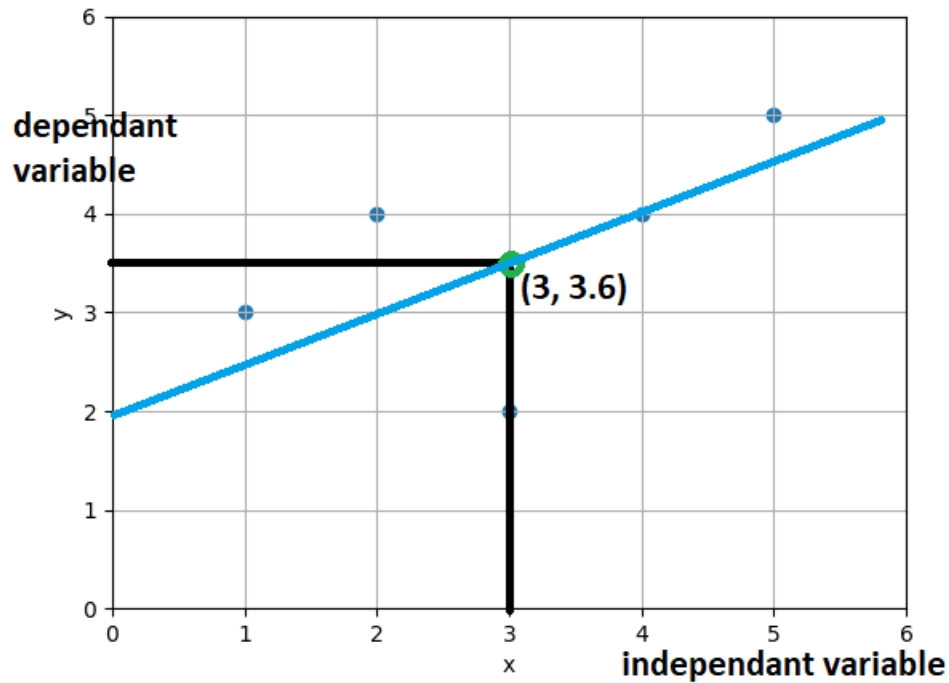
C is the y intercept of the line.

The mathematical implementation.

Let us consider the values of x and y as follows.

X	Y
1	3
2	4
3	2
4	4
5	5
Mean of x = 3	Mean of y = 3.6

Lets plot them on the X axis.



$$\bar{x} = \frac{1+2+3+4+5}{5} = 3$$

$$\bar{y} = \frac{3+4+2+4+5}{5} = 3.6$$

- Let us calculate the mean of x and y and plot it on the graph.
- That is let us plot the mean of x = 3 and mean of y=3.6 on the graph.
- Our goal is to find or predict the best fit line using the least square method.
- In order to find that we first need to find the equation of the line.
- Let us find the equation of the regression line.
- Suppose below given is the equation of the regression line.

$$Y = mx + c.$$

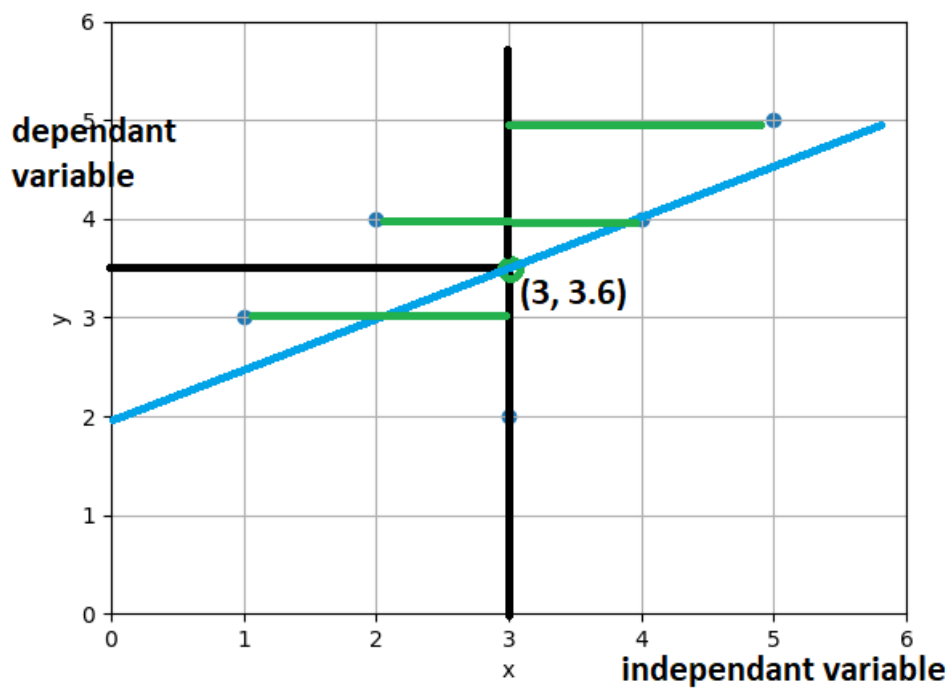
- We need to find the value of m and c.

$$m = \frac{\sum(x - \bar{x})(y - \bar{y})}{\sum(x - \bar{x})^2}$$

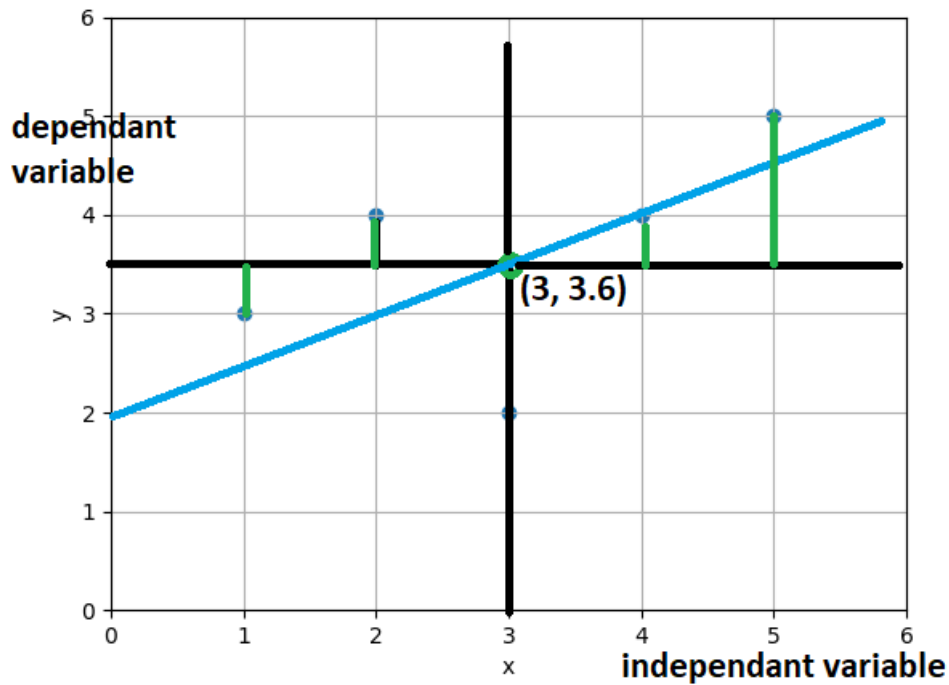
Let us calculate .

X	Y	$x - \bar{x}$	$y - \bar{y}$
1	3	-2	-0.6
2	4	-1	0.4
3	2	0	-1.6
4	4	1	0.4
5	5	2	1.4
Mean of x=3	Mean of y=3.6		

- So $x - \bar{x}$ is nothing but the distance of all the points through the line $y=3$.



- $y - \bar{y}$ implies the distance of all the points from the line $x=3.6$



- So let us calculate the value of $y - \bar{y}$, $(x - \bar{x})^2$, $(x - \bar{x})(y - \bar{y})$

X	Y	$x - \bar{x}$	$y - \bar{y}$	$(x - \bar{x})^2$	$(x - \bar{x})(y - \bar{y})$
1	3	-2	-0.6	4	1.2
2	4	-1	0.4	1	-0.4
3	2	0	-1.6	0	0
4	4	1	0.4	1	0.4
5	5	2	1.4	4	2.8
Mean of x=3	Mean of y=3.6			$\sum (x - \bar{x})^2 = 10$	$\sum (x - \bar{x})(y - \bar{y}) = 4$

- Let us get the summation of last 2 columns which is 10 and 4.
- Therefore the value of m will be

$$m = \frac{\sum (x - \bar{x})(y - \bar{y})}{\sum (x - \bar{x})^2} = \frac{4}{10} = 0.4$$

- Let us put this value of $m=0.4$ in our line $y=mx+c$
- Let us fill all the points into the equation and find the value of c.
- Mean of y is = 3.6
- $m=0.4$
- Mean of x = 3

- and we have the equation as

$$y=mx+c$$

that is

$$3.6 = 0.4 * 3 + c$$

$$3.6 - 1.2 = c$$

Therefore

$$c=2.4$$

So this is the regression line.

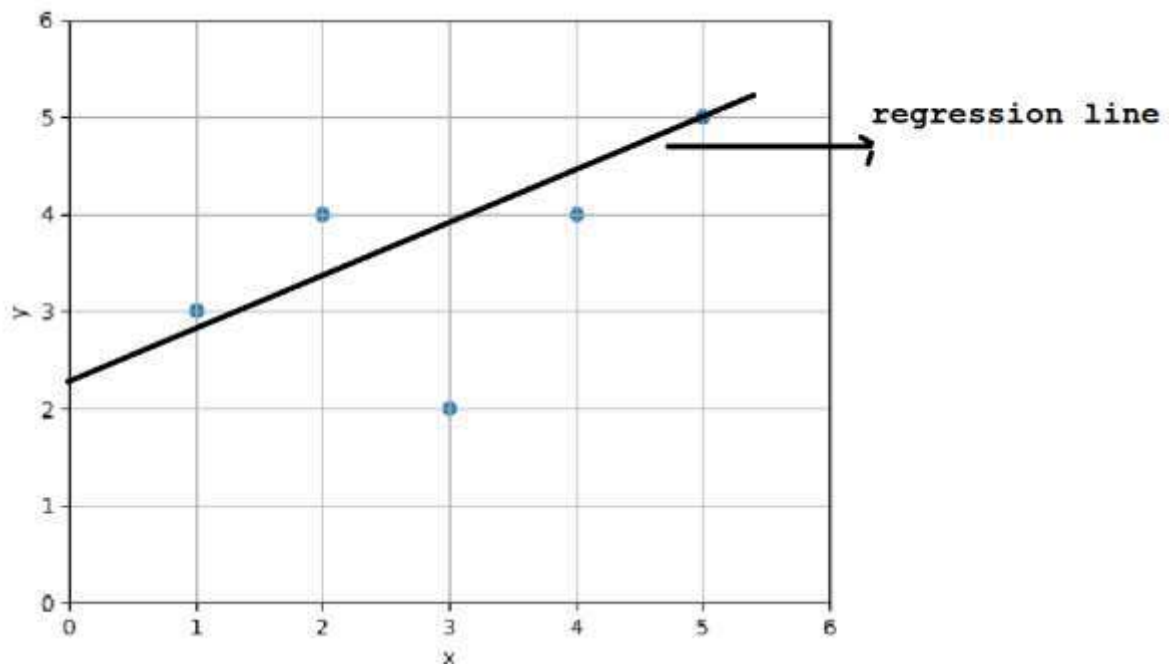
So this is how the points will be plotted

Note: $m = 0.4$

$$C = 2.4$$

$$Y=0.4x+2.4$$

Below are the actual points.



- Now for given $m=0.4$ and $c=2.4$, let us predict values for y for $x=[1,2,3,4,5]$

$$Y=0.4*1 + 2.4 = 2.8$$

$$Y=0.4*2 + 2.4 = 3.2$$

$$Y=0.4*3 + 2.4 =3.6$$

$$Y=0.4*4 + 2.4 =4.0$$

$$Y=0.4*5+2.4=4.4$$

- The above are all predicted y values for each x value.
- Let us plot the predicted y values for the values of x for which they were predicted.
- The line cuts the y axis at 2.4 as the line of regression.

Mean Square Error:

- The **mean squared error** (MSE) tells you how close a regression line is to a set of points. It does this by taking the distances from the points to the regression line (these distances are the “errors”) and squaring them. The squaring is necessary to remove any negative signs. It also gives more weight to larger differences. It’s called the mean squared error as you’re finding the average of a set of errors. The lower the MSE, the better the forecast.

$$\text{MSE formula} = (1/n) * \Sigma(\text{actual} - \text{predicted})^2$$

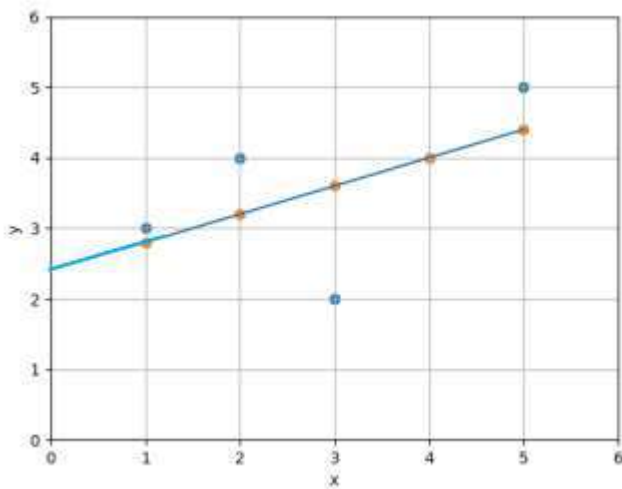
Where:

- n = number of items,
- Σ = summation notation,
- Actual = original or observed y-value,
- Forecast = y-value from regression, that is predicted value.

General steps to calculate the MSE from a set of X and Y values:

1. Find the regression line.
2. Insert your X values into the linear regression equation to find the new Y values (Y’).
3. Subtract the new Y value from the original to get the error.
4. Square the errors.
5. Add up the errors (the Σ in the formula is summation notation).
6. Find the mean.

x	y	yp	Error (y-yp)	Error Squared
1	3	2.8	0.2	0.04
2	4	3.2	0.8	0.64
3	2	3.6	-1.6	2.56
4	4	4	0	0
5	5	4.4	0.6	0.36
				Sum of errors=3.6
				Mean of Squared Error=0.72



- Now our task is to calculate the distance between the actual and predicted values.
- And our job is to reduce the distance, or in other words you have to reduce the error between the actual and the predicted values.
- The line with the least error will be the line of linear regression or regression line and it will also be the best fit line.
- So below is how different regression lines are created.

Iteration=1

M=0.04

iteration=2

m=0.07

iteration=3

m=0.11

B=0.0

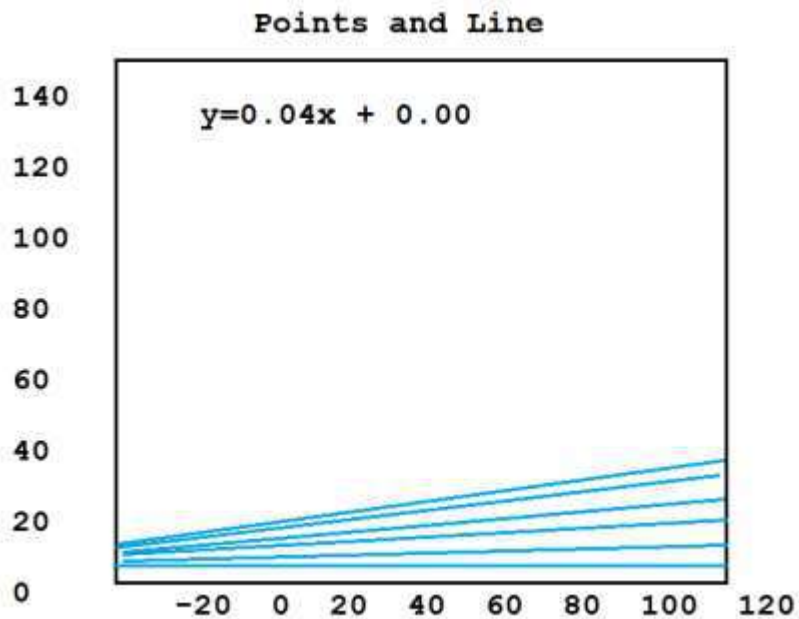
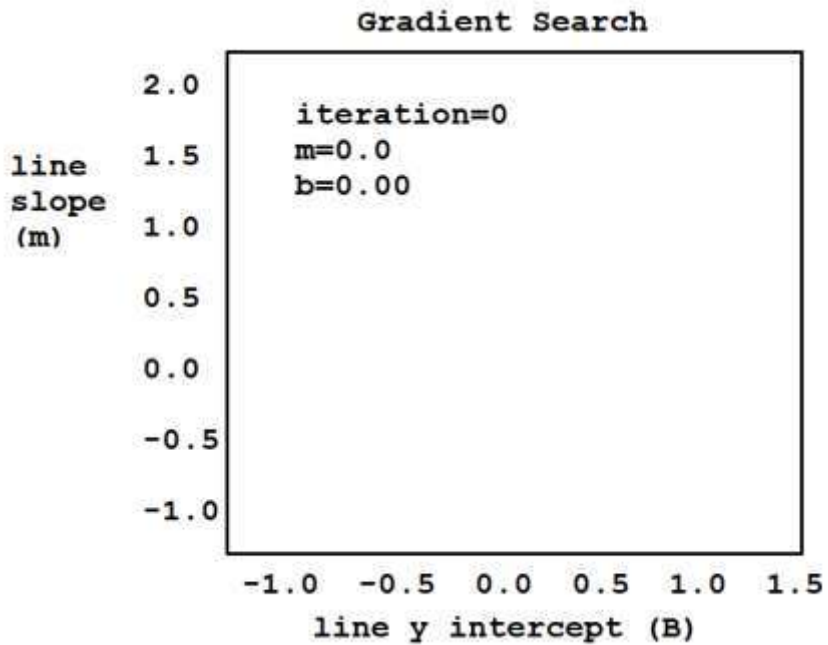
b=0.00

b=0.00

$Y=0.04x * 0.00$

$Y=0.07x * 0.00$

$Y=0.11x * 0.00$



- We perform n number of iterations for different values of m. For different values of m, it will calculate the equation of the line where $y=mx+c$
- As the value of m changes, the line is changing.

- The iteration will start from 1.
- And it will perform a number of iterations.
- So after every iteration, it will calculate the predicted value of m for which the distance between the actual and the predicted value is minimum.
- That line will be selected as the best fit line.
- Now that we have calculated the best fit line, now it is time to check how good our model is performing.
- In order to do that we have a method called the R-Squared method.

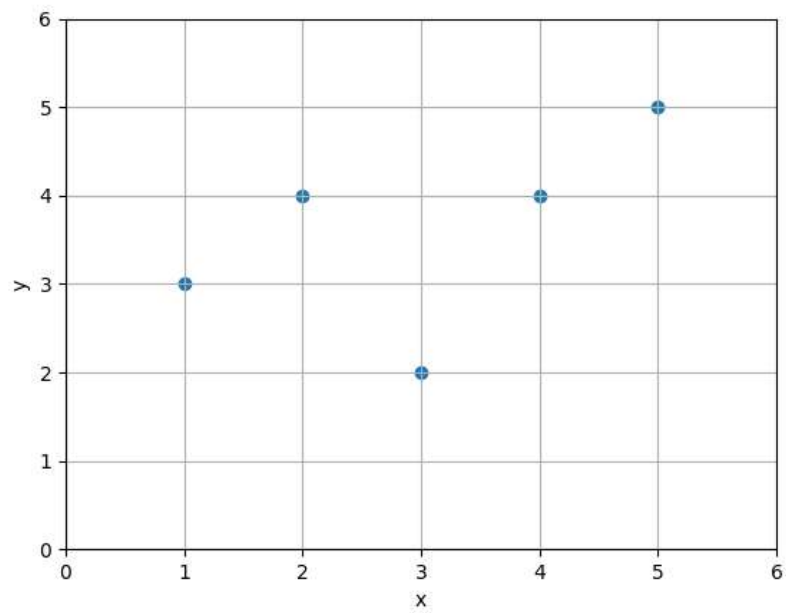
What is R-Squared Method?

- R-Squared value is a statistical measure of how close the data are to the fitted regression line.
- It is also known as the coefficient of determination or coefficient of multiple determination for multiple regression.
- 1(100%) is the best score and 0(0%) is worst score..
- In general it is considered that a high R-Squared value model is a good model. But you can also have a lower R-Squared value for a good model as well.

Calculation of R^2 :

- Let us see how a R-Squared value is calculated.
- Below are the actual values plotted on the graph.

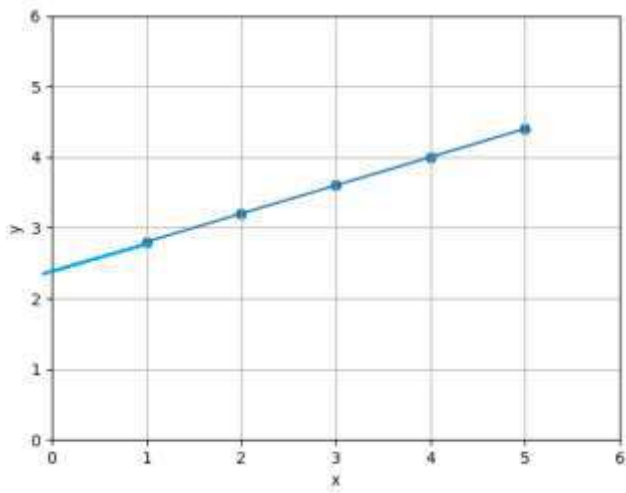
X	y
1	3
2	4
3	2
4	4
5	5



- We had calculated the predicted values.

X	y_p
1	2.8
2	3.2
3	3.6
4	4.0
5	4.4

- The regression line was as follows.



- We had calculated the predicted values of y for the equation

$$Y_p = 0.14x + 2.4$$

for every $x=[1,2,3,4,5]$

- From there we got the predicted values of y.

That is $y_p = [2.8, 3.2, 3.6, 4.0, 4.4]$

- Let us plot it on the above graph.
- So these are the points and line passing through these points are nothing but the regression line.
- Now we need to check and compare the distance of actual – mean versus the distance of predicted – mean.

That is

Distance actual - mean

Vs

Distance predicted – mean

This is nothing but $R^2 = \frac{\sum(y_p - \bar{y})^2}{\sum(y - \bar{y})^2}$

- Here we are calculating the distance of actual value to the mean to the distance of the predicted value to the mean.
- In the equation y is the actual value and y_p is the predicted value and $\bar{y}$ is the mean value of y , that is nothing but 3.6.
- Let us calculate $y - \bar{y}$

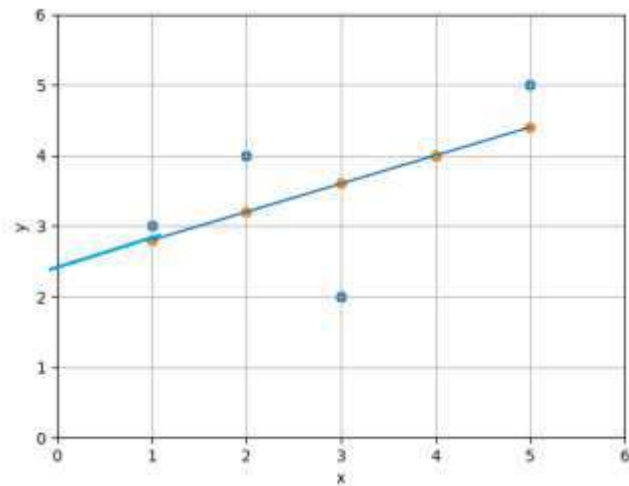
x	y	y - $\bar{y}$	(y - $\bar{y}$) ²	y _p	y _p - $\bar{y}$	(y _p - $\bar{y}$) ²
1	3	-0.6	0.36	2.8	-0.8	0.64
2	4	0.4	0.16	3.2	-0.4	0.16
3	2	-1.6	2.54	3.6	0	0
4	4	0.4	0.16	4.0	0.4	0.16
5	5	1.4	1.96	4.4	0.8	0.64
	Mean $\bar{y} = 3.6$		$\sum = 5.2$			$\sum = 1.6$

$$\sum (y - \bar{y})^2 = 5.2$$

$$\sum (y_p - \bar{y})^2 = 1.6$$

$$\text{So } R^2 = \frac{\sum(y_p - \bar{y})^2}{\sum(y - \bar{y})^2} = \frac{1.6}{5.2}$$

- So R^2 is approximately equal to 0.3
- This is not a good fit.
- It suggests that the data points are far away from the regression line.
- So the above graph is how it will look like when R square is 0.3.



- When you increase the value of R^2 to 0.7, you will see that the actual value would be closer to the regression line.
- When it reaches 0.9, it comes more closer and when the value is approximately equals to 1, then the actual values lies on the regression line itself.
- For example, if you get a very low value of R square as 0.02, in that case what we will see the actual values are very far away from the regression line or you can say that there are too many outliers in your data. You cannot forecast anything from the data.

MULTI LINEAR REGRESSION

Multi Linear Regression

Here there is

- One dependent variable
- Multiple independent variables

For Example:

Features



Area	Bedrooms	Age	Price
2600	3	20	550000
3000	4	15	565000
3200	4	18	610000
3600	3	30	595000
4000	5	8	760000

Here:

- Price is the dependent variable
- Area, Bedroom and Age are the independent variables
- Area, Bedroom and Age are also called attributes or features

The equation for Multi Linear Regression is

$$Y = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \beta_3 X_3 + \cdots \cdots \cdots + \beta_n X_n + \epsilon$$

Where

Y is the dependent variable

$X_1, X_2, X_3, \cdots \cdots \cdots X_n$ are the independent variables

β_0 is the Y intercept

$\beta_1, \beta_2, \beta_3, \cdots \cdots \cdots \beta_n$ are the coefficients of X or the slopes

ϵ is the error factor

Finding the Coefficients:

The equation is:

$$\hat{\beta} = (X^T X)^{-1} X^T Y$$

So, Given the Data:

	Y	X1	X2	X3
0	43	30	63	33
1	63	45	47	52
2	71	68	67	62
3	61	46	83	42
4	81	66	84	42

We need to find the best fit line using the dependent and independent variables.

So, We need to solve:

$$Y = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \beta_3 X_3$$

We need to find $\beta_0, \beta_1, \beta_2, \beta_3$

Y is known

X is known

Y	X0	X1	X2	X3
43	1	30	63	33
63	1	45	47	52
71	1	68	67	62
61	1	46	83	42
81	1	66	84	42
43	1	36	50	66

Now let us solve the equation:

$$\hat{\beta} = (X^T X)^{-1} X^T Y$$

$$X^T = \begin{array}{cccccc} 1 & 1 & 1 & 1 & 1 & 1 \\ 30 & 45 & 68 & 46 & 66 & 36 \\ 63 & 47 & 67 & 83 & 84 & 50 \\ 33 & 52 & 62 & 42 & 42 & 66 \end{array}$$

$$X^T X =$$

$$X^T = \begin{bmatrix} 1 & 1 & 1 & 1 & 1 & 1 \\ 30 & 45 & 68 & 46 & 66 & 36 \\ 63 & 47 & 67 & 83 & 84 & 50 \\ 33 & 52 & 62 & 42 & 42 & 66 \end{bmatrix} \quad X = \begin{bmatrix} 1 & 30 & 63 & 33 \\ 1 & 45 & 47 & 52 \\ 1 & 68 & 67 & 62 \\ 1 & 46 & 83 & 42 \\ 1 & 66 & 84 & 42 \\ 1 & 36 & 50 & 66 \end{bmatrix}$$

$$X^T X = \begin{bmatrix} 6 & 291 & 394 & 297 \\ 291 & 15317 & 19723 & 14626 \\ 394 & 19723 & 27112 & 18991 \\ 297 & 14626 & 18991 & 15521 \end{bmatrix}$$

$$X^T X =$$

$$\begin{array}{cccc} 6 & 291 & 394 & 297 \\ 291 & 15317 & 19723 & 14626 \\ 394 & 19723 & 27112 & 18991 \\ 297 & 14626 & 18991 & 15521 \end{array}$$

$$(X^T X)^{-1} =$$

$$\begin{array}{cccc} 16.95802 & 0.083191 & -0.17313 & -0.19106 \\ 0.083191 & 0.001894 & -0.00155 & -0.00148 \\ -0.17313 & -0.00155 & 0.002356 & 0.001891 \\ -0.19106 & -0.00148 & 0.001891 & 0.002802 \end{array}$$

$$X^T = \begin{bmatrix} 1 & 1 & 1 & 1 & 1 & 1 \\ 30 & 45 & 68 & 46 & 66 & 36 \\ 63 & 47 & 67 & 83 & 84 & 50 \\ 33 & 52 & 62 & 42 & 42 & 66 \end{bmatrix}$$

$$Y = \begin{bmatrix} 43 \\ 63 \\ 71 \\ 61 \\ 81 \\ 43 \end{bmatrix}$$

$$X^T Y = \begin{bmatrix} 362 \\ 18653 \\ 24444 \\ 17899 \end{bmatrix}$$

Now Solving the Equation

$$\hat{\beta} = (X^T X)^{-1} X^T Y$$

$$(X^T X)^{-1} = \begin{bmatrix} 16.95802 & 0.083191 & -0.17313 & -0.19106 \\ 0.083191 & 0.001894 & -0.00155 & -0.00148 \\ -0.17313 & -0.00155 & 0.002356 & 0.001891 \\ -0.19106 & -0.00148 & 0.001891 & 0.002802 \end{bmatrix}$$

$$X^T Y = \begin{bmatrix} 362 \\ 18653 \\ 24444 \\ 17899 \end{bmatrix}$$

Therefore:

$$\hat{\beta} = (X^T X)^{-1} X^T Y$$

$$\hat{\beta} = \begin{matrix} 38.8731 \\ 1.062949 \\ -0.15181 \\ -0.40655 \end{matrix}$$

That is: The intercept and the Coefficients are:

$$\beta_0 = 38.8731 \quad \beta_1 = 1.062949 \quad \beta_2 = -0.15181 \quad \beta_3 = -0.40655$$

Therefore for a new set of Features:

Y	X1	X2	X3
?	39	59	40

Applying:

$$Y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \beta_3 x_3$$

$$Y = 38.8731 + (1.062949 * 39) + (-0.15181 * 59) + (-0.40655 * 40)$$

$$Y = 55.10958$$

Mean Squared Error:

	Y	X1	X2	X3
0	43	30	63	33
1	63	45	47	52
2	71	68	67	62
3	61	46	83	42
4	81	66	84	42

Mean Squared Error:

$$Y_{p1} = 38.8731 + (1.062949 * 43) + (-0.15181 * 30) + (-0.40655 * 63)$$

$$Y_{p2} = 38.8731 + (1.062949 * 63) + (-0.15181 * 45) + (-0.40655 * 47)$$

$$Y_{p3} = 38.8731 + (1.062949 * 71) + (-0.15181 * 68) + (-0.40655 * 67)$$

$$Y_{p4} = 38.8731 + (1.062949 * 61) + (-0.15181 * 46) + (-0.40655 * 83)$$

$$Y_{p5} = 38.8731 + (1.062949 * 81) + (-0.15181 * 66) + (-0.40655 * 84)$$

Mean Squared Error:

Y	Yp	Error(Y-Yp)	Error Squared
43	47.78166	-4.78166	22.86429328
63	58.43034	4.569658	20.88177075
71	75.77656	-4.77656	22.81554469
61	58.09379	2.906214	8.446078129
81	79.20097	1.799032	3.236515623
43	42.71668	0.283321	0.080270799

Sum of Errors 78.32447328

Mean Squared Error 13.05407888

R Squared: The coefficient of Determination

$$R^2 = \frac{\sum (y_p - \bar{y})^2}{\sum (y - \bar{y})^2}$$

- R-Squared value is a statistical measure of how close the data are to the fitted regression line.
- It is also known as the coefficient of determination or coefficient of multiple determination for multiple regression.
- 1(100%) is the best score and 0(0%) is worst score..
- In general it is considered that a high R-Squared value model is a good model. But you can also have a lower R-Squared value for a good model as well.

Y	Y-Y_Mean	(Y-Y_Mean)^2	Yp	Yp-Y_Mean	(Yp-Y_Mean)^2
43	-17.3333	300.4444444	47.78166219	-12.5517	157.5444
63	2.666667	7.111111111	58.43034238	-1.90299	3.621375
71	10.66667	113.7777778	75.77656202	15.44323	238.4933
61	0.666667	0.444444444	58.09378629	-2.23955	5.015571
81	20.66667	427.1111111	79.20096814	18.86763	355.9876
43	-17.3333	300.4444444	42.71667898	-17.6167	310.3465
Mean		Sum			Sum
60.33333		1149.333333			1071.009

$$R_Squared = 1079.009 / 1149.333333$$

$$R_Squared = 0.931852256$$

Polynomial Regression

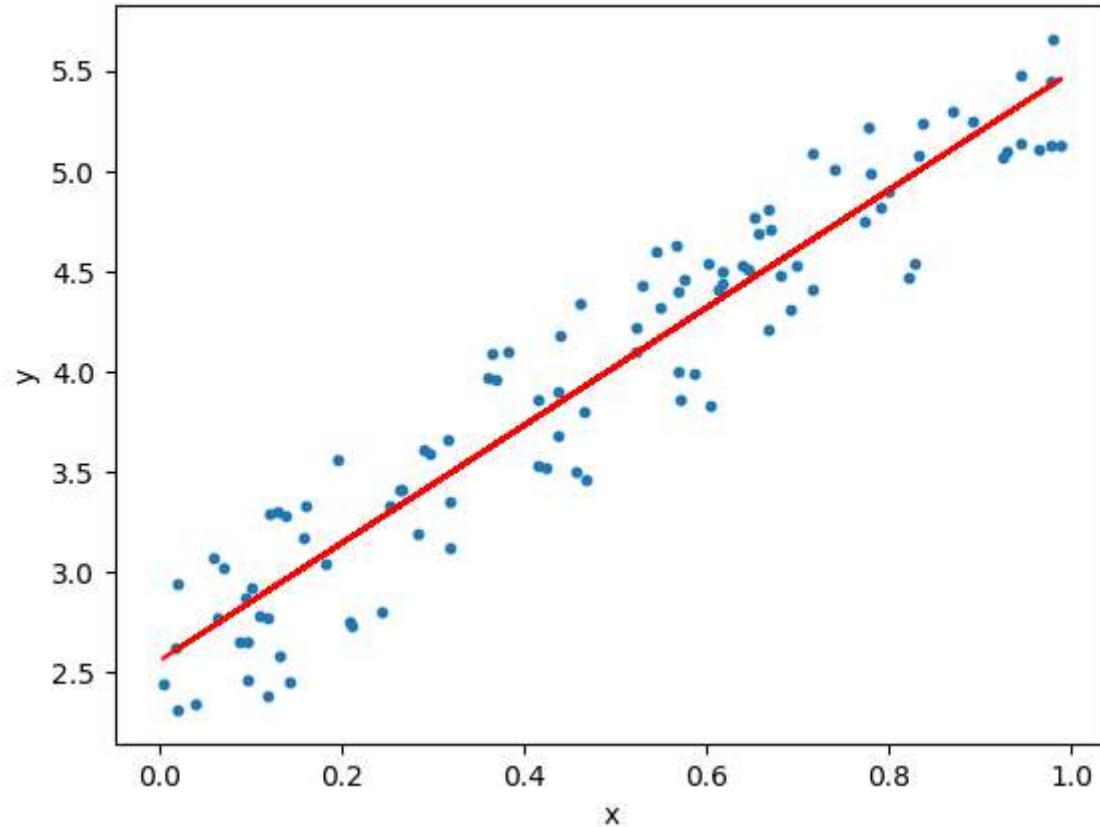
Simple Linear Regression

In Linear Regression, the relationship between the dependent and independent variable is linear

But here you have one dependent and one independent variable.

Equation

$$Y = \beta_0 + \beta_1 X + \epsilon$$



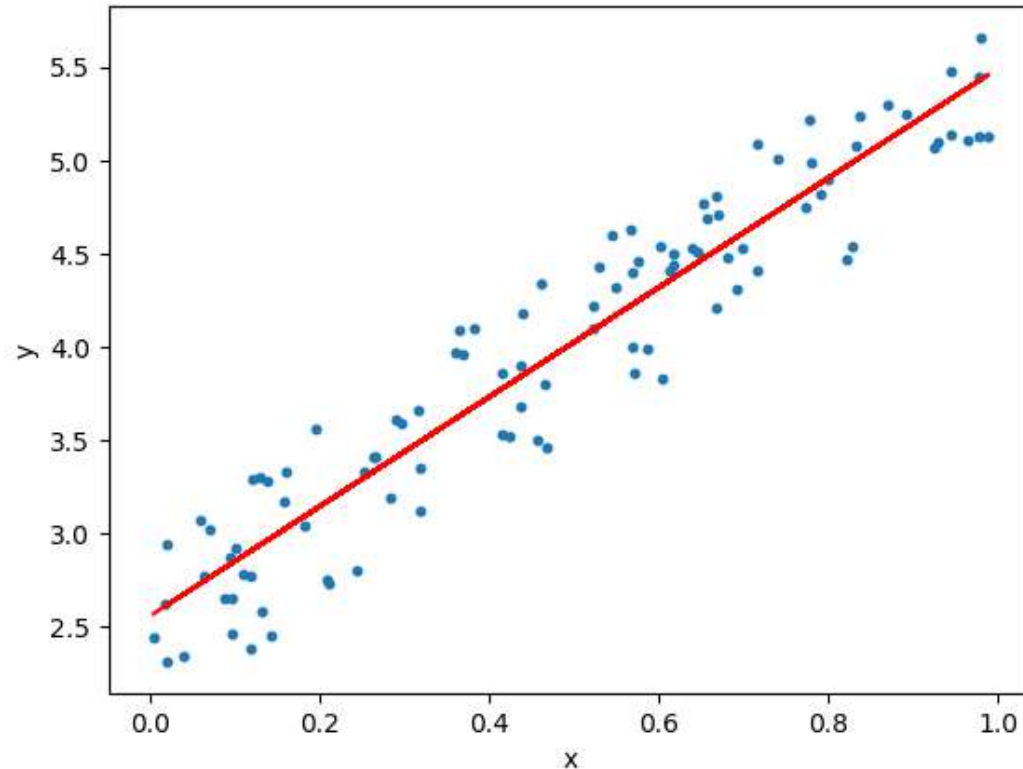
MultiLinear Regression

In Linear Regression, the relationship between the dependent and independent variable is linear

But here you have one dependent and multiple independent variable.

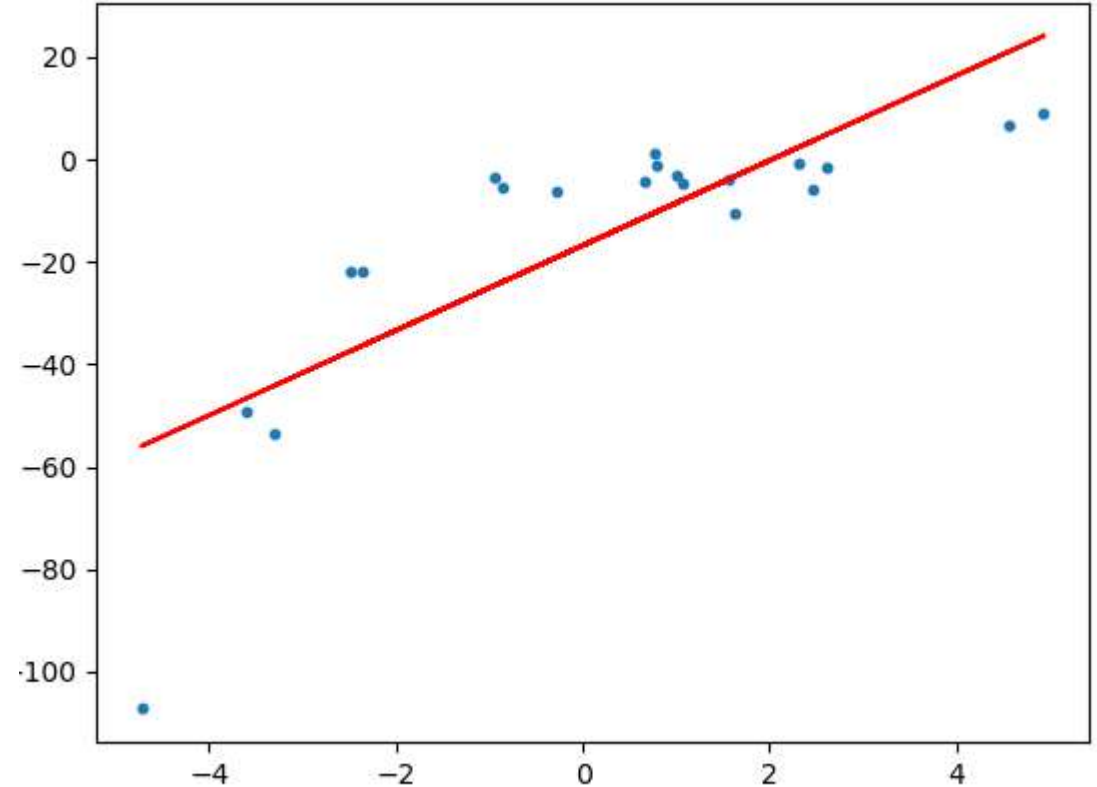
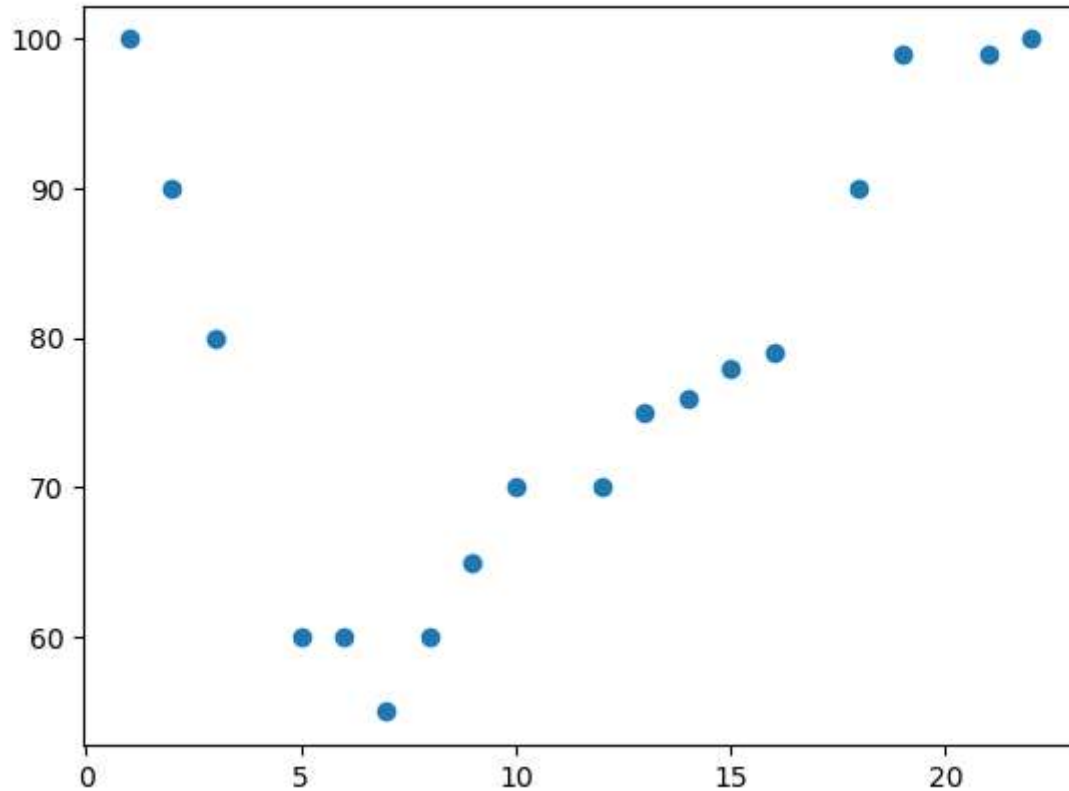
Equation

$$Y = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \beta_3 X_3 + \cdots \cdots \cdots + \beta_n X_n + \epsilon$$



Complex Data:

*Some times the data or the distribution of the data is more complex.
The data may not be Linear.
The Linear models we have seen will not be able to fit non linear data.*

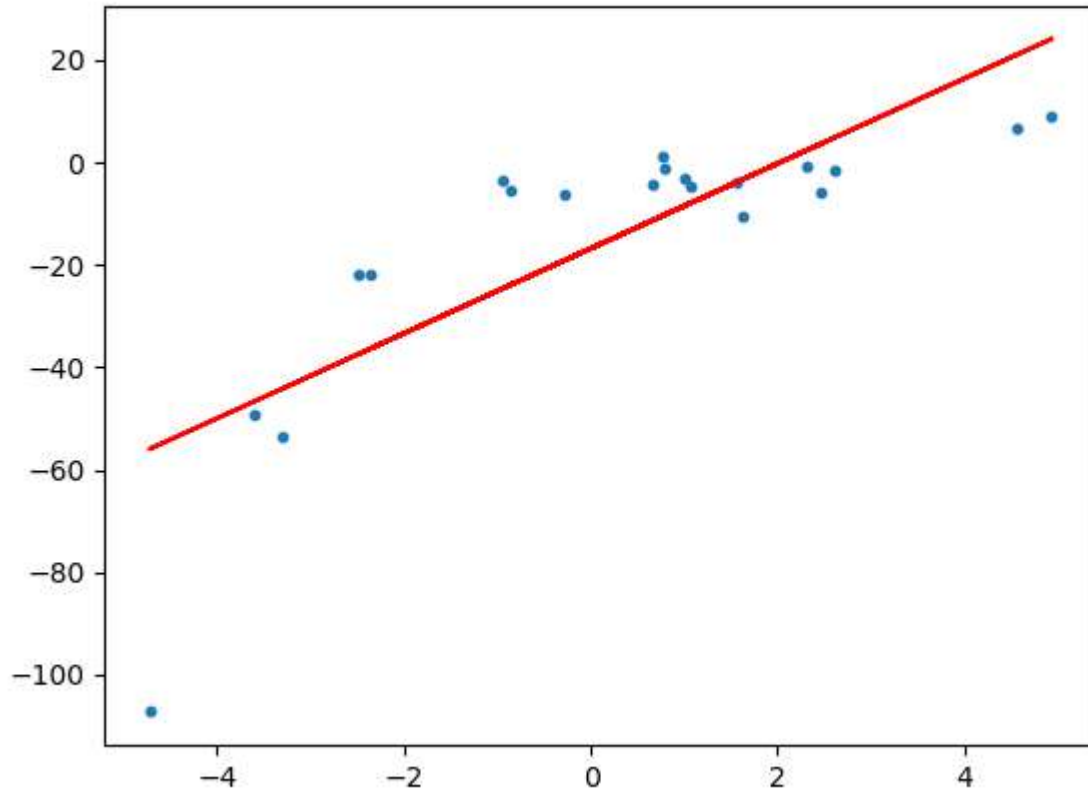


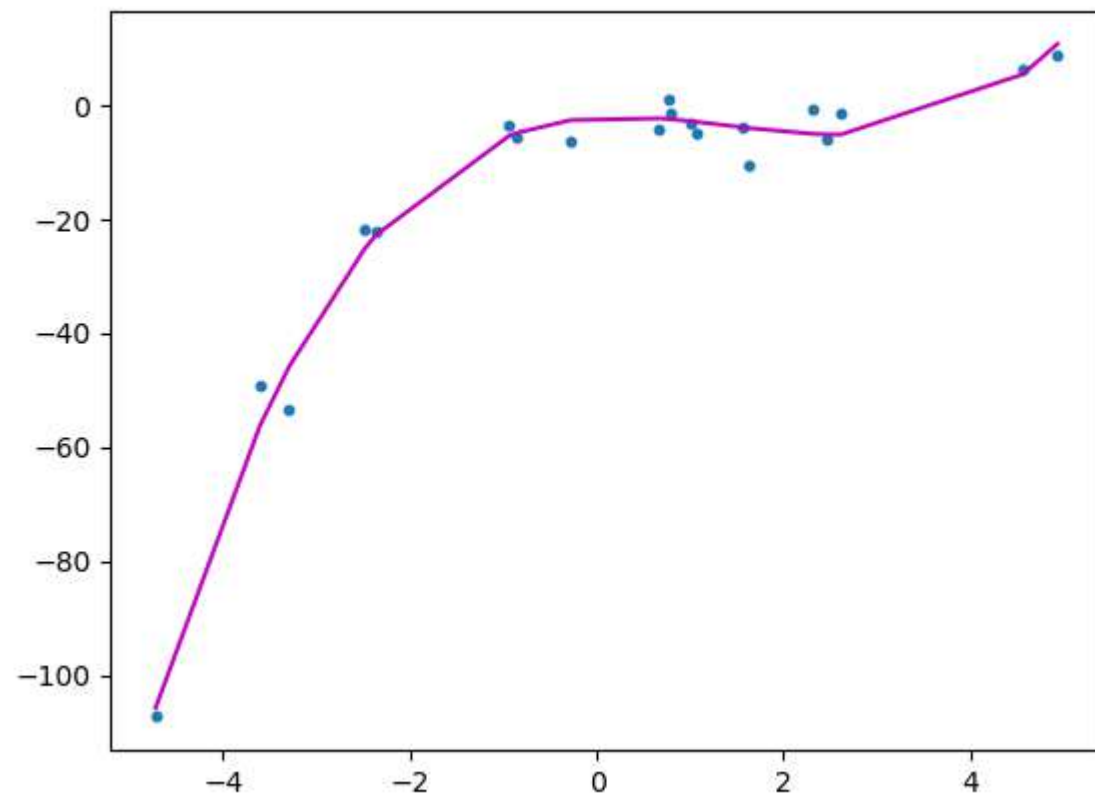
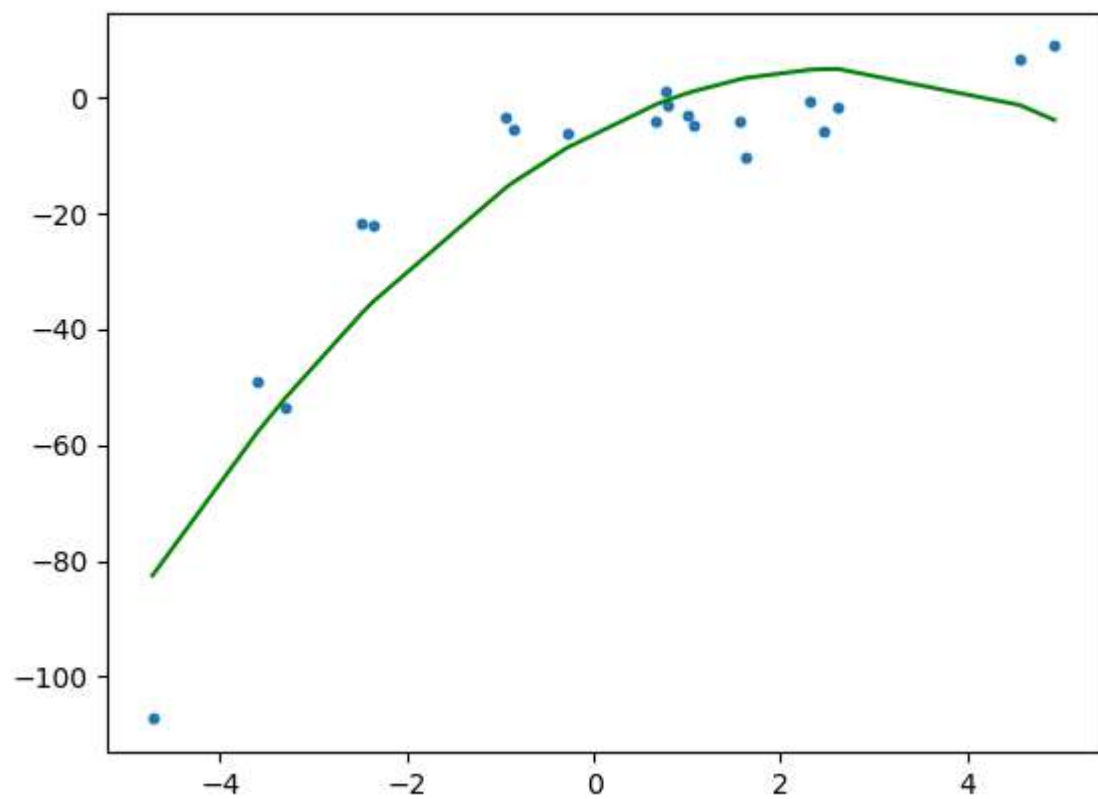
The above line under fits the data points.

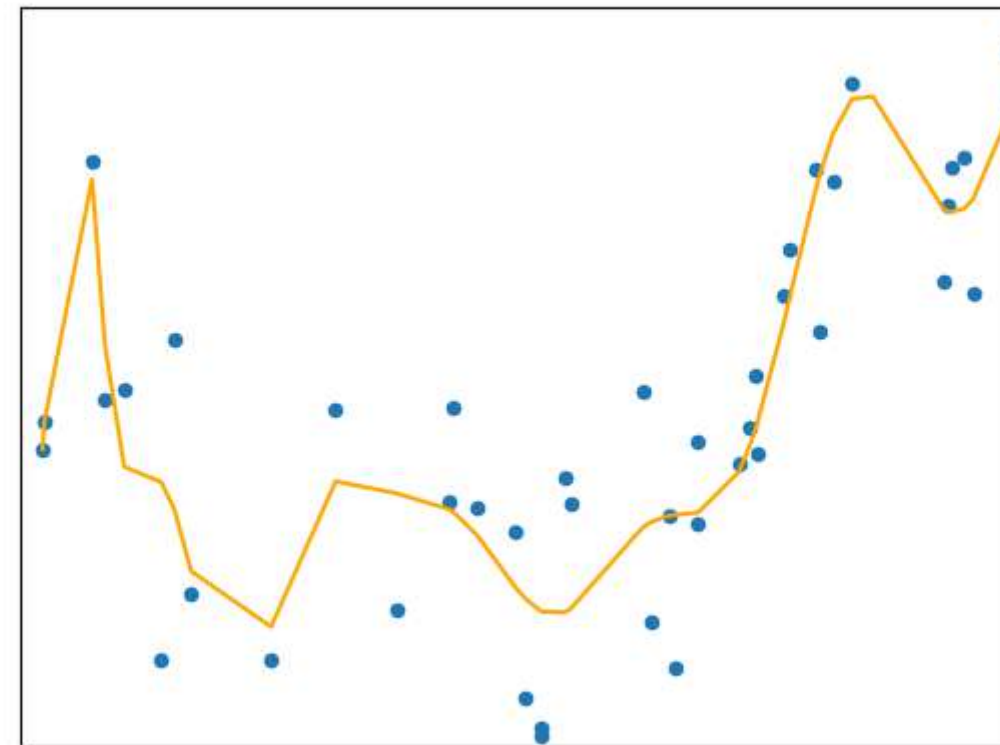
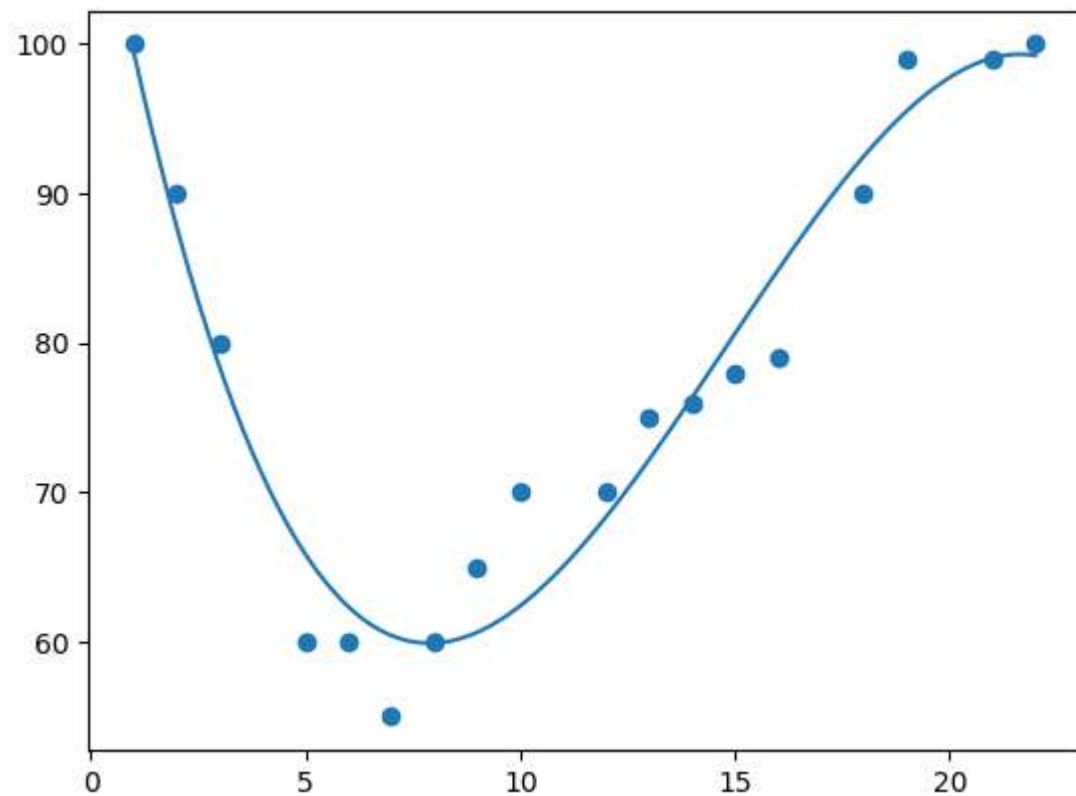
Polynomial Regression:

Polynomial regression, like linear regression, uses the relationship between the variables x and y to find the best way to draw a line through the data points.

So instead of a straight line we require a curve fitting line.







It is clear from the above plot that the quadratic curve is able to fit the data better than the linear line.

To overcome under-fitting, we need to increase the complexity of the model.

To generate a higher order equation we can add powers of the original features as new features.

Therefore the linear model,

$$Y = \beta_0 + \beta_1 X$$

Can be transformed to

$$Y = \beta_0 + \beta_1 X + \beta_2 X^2 + \beta_3 X^3 + \dots \dots \dots + \beta_n X^n + \epsilon$$

The order can be 2 or 3 or 4, so on till n.

But the most we will go for is the 2nd order.

This is still considered to be **linear model** as the coefficients/weights associated with the features are still linear.

x^2 is only a feature.

However the curve that we are fitting is **quadratic** in nature.

$$Y = \beta_0 + \beta_1 X + \beta_2 X^2 + \epsilon$$

The order can be 2 or 3 or 4, so on till n.

But the most we will go for is the 2nd order.

This is still considered to be **linear model** as the coefficients/weights associated with the features are still linear. x^2 is only a feature.

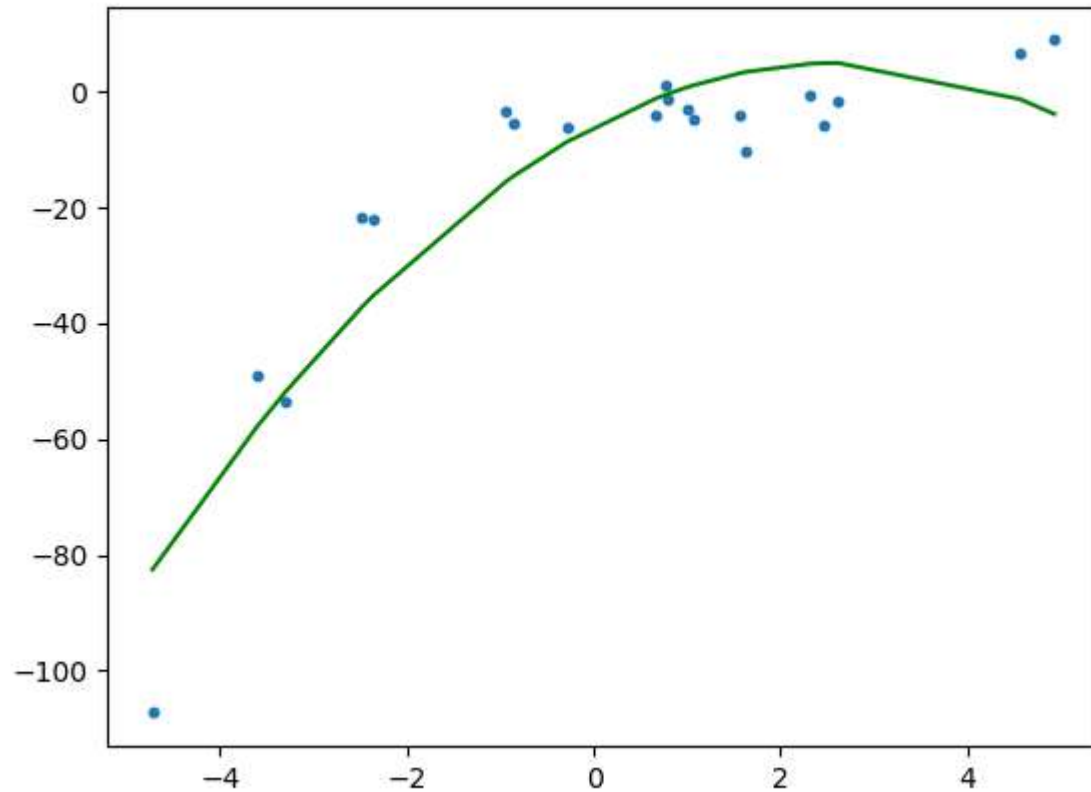
However the curve that we are fitting is **quadratic** in nature.

It can be cubic also.

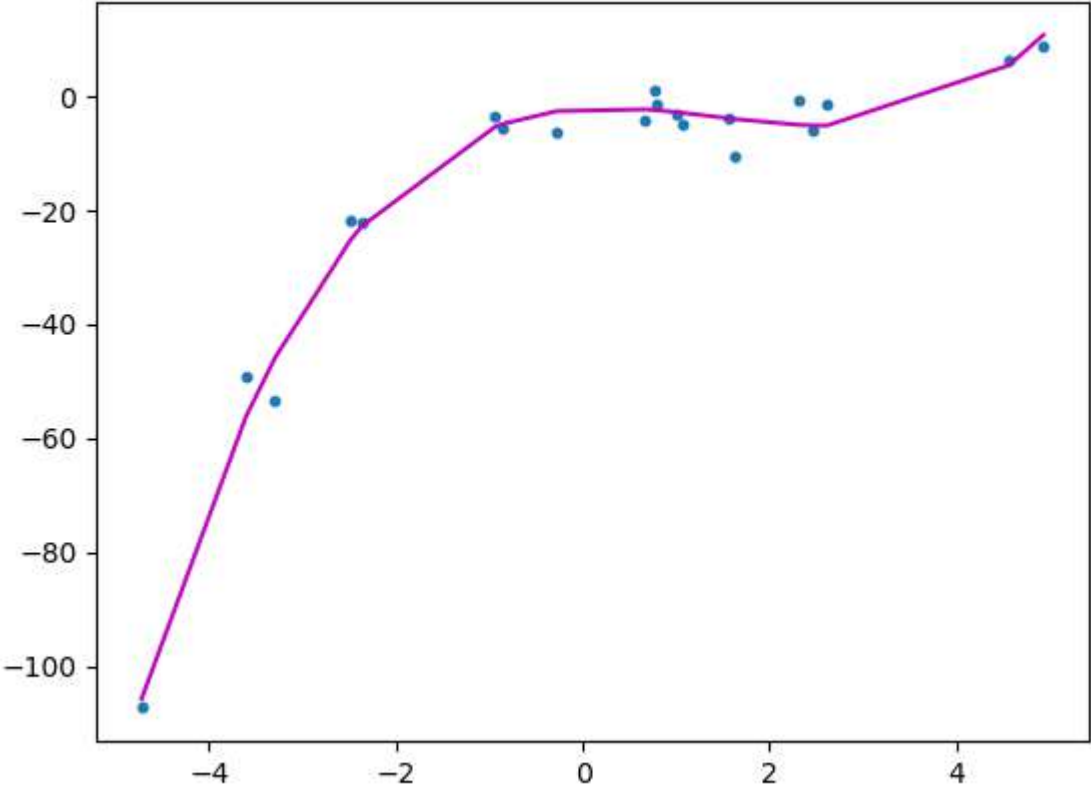
It can be 15 or 20 and so on.

2nd Order: Quadratic

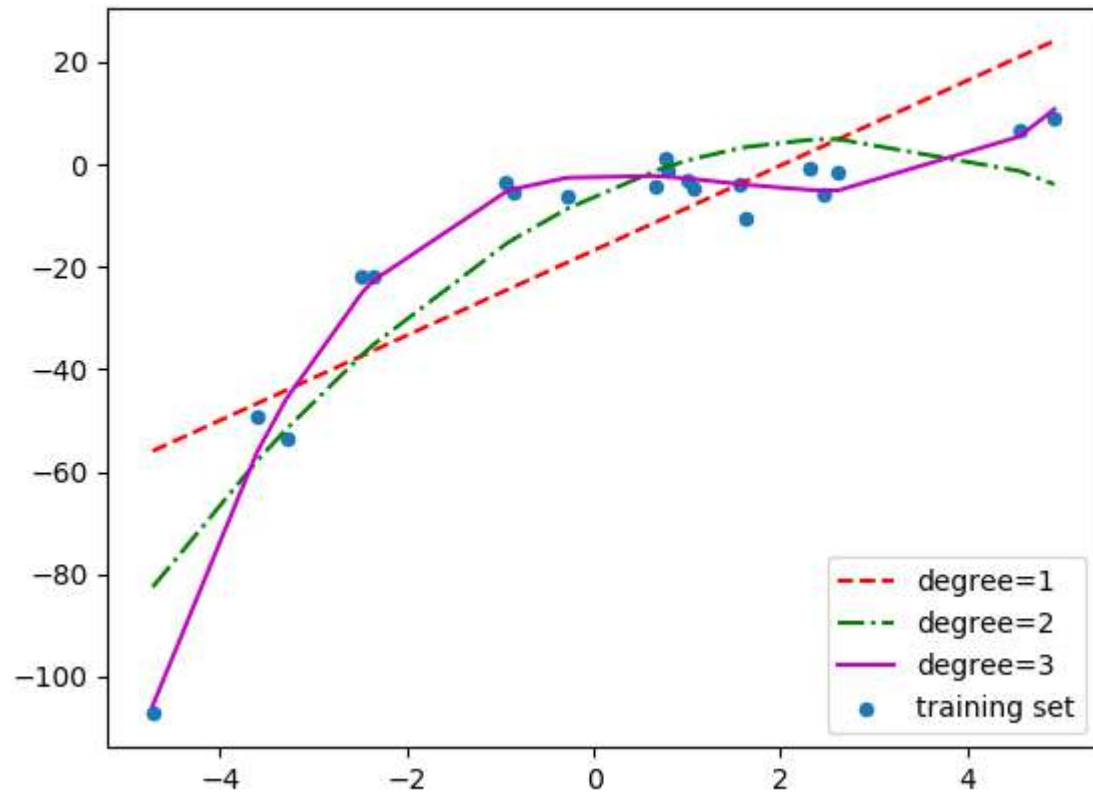
$$Y = \beta_0 + \beta_1 X + \beta_2 X^2 + \epsilon$$



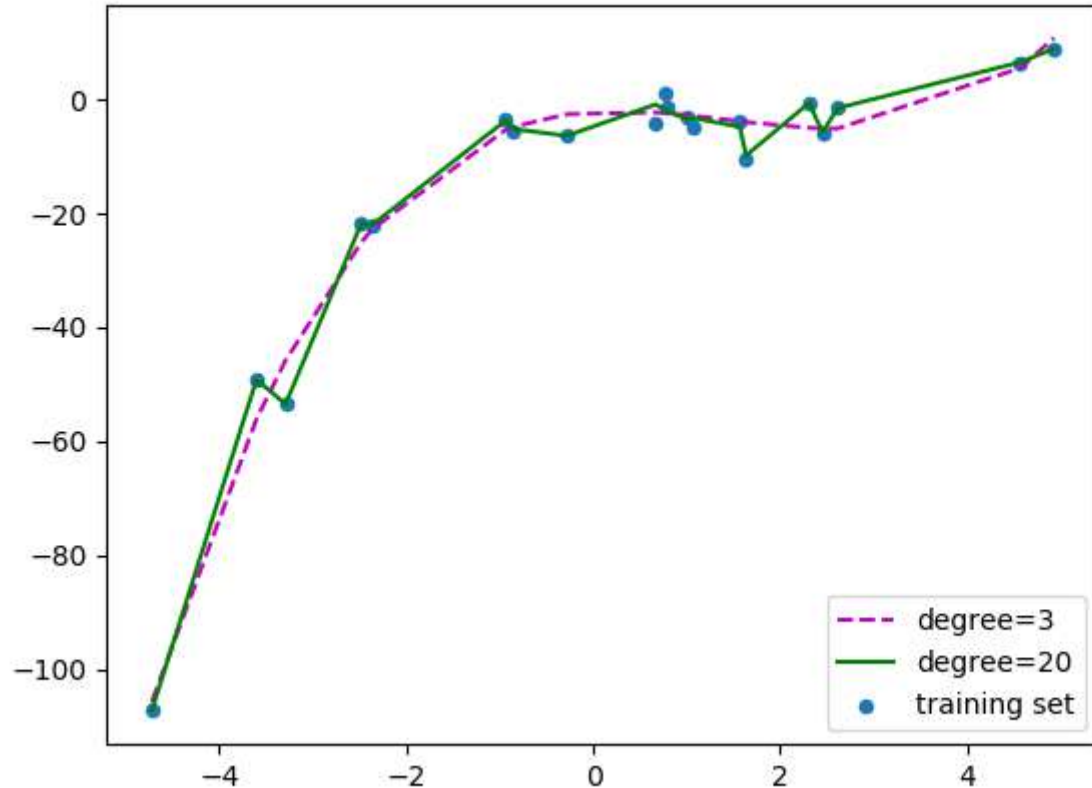
3rd Order: Cubic



Below is a comparison of fitting linear, quadratic and cubic curves on the dataset.



If we further increase the degree to 20, we can see that the curve passes through more data points. Below is a comparison of curves for degree 3 and 20.



For degree=20, the model is also capturing the noise in the data. This is an example of **over-fitting**. Even though this model passes through most of the data, it will fail to generalize on unseen data.

To prevent over-fitting, we can add more training samples so that the algorithm doesn't learn the noise in the system and can become more generalized.

It is always preferred to have the order of the polynomial to be less than or equal to 2.

Equation:

$$Y = \beta_0 + \beta_1 X + \beta_2 X^2 + \epsilon$$

$$y_i = \beta_0 + \beta_1 x_i + \beta_2 x_i^2 + \cdots + \beta_m x_i^m + \varepsilon_i \quad (i = 1, 2, \dots, n)$$

can be expressed in matrix form in terms of a design matrix $\mathbf{X}$, a response vector $\vec{y}$, a parameter vector $\vec{\beta}$, and a vector $\vec{\varepsilon}$ of random errors. The i -th row of $\mathbf{X}$ and $\vec{y}$ will contain the x and y value for the i -th data sample. Then the model can be written as a system of linear equations:

$$\begin{bmatrix} y_1 \\ y_2 \\ y_3 \\ \vdots \\ y_n \end{bmatrix} = \begin{bmatrix} 1 & x_1 & x_1^2 & \cdots & x_1^m \\ 1 & x_2 & x_2^2 & \cdots & x_2^m \\ 1 & x_3 & x_3^2 & \cdots & x_3^m \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & x_n & x_n^2 & \cdots & x_n^m \end{bmatrix} \begin{bmatrix} \beta_0 \\ \beta_1 \\ \beta_2 \\ \vdots \\ \beta_m \end{bmatrix} + \begin{bmatrix} \varepsilon_1 \\ \varepsilon_2 \\ \varepsilon_3 \\ \vdots \\ \varepsilon_n \end{bmatrix},$$

which when using pure matrix notation is written as

$$\vec{y} = \mathbf{X}\vec{\beta} + \vec{\varepsilon}.$$

Equation:

$$Y = \beta_0 + \beta_1 X + \beta_2 X^2 + \epsilon$$

How to find the X coefficients, that is

$\beta_0, \beta_1, \beta_2$ and so on

The equation is the same as Linear Regression

$$\hat{\beta} = (X^T X)^{-1} X^T Y$$

Once $\beta_0, \beta_1, \beta_2$ and so on is known, we can fit them in the below equation

$$Y = \beta_0 + \beta_1 X + \beta_2 X^2 + \epsilon$$

If more number features are there, then the equation will change to:

For example if there are 2 features X_1 and X_2 , then the equation will change to:

$$Y = \beta_0 + \beta_1 X_1 + \beta_2 X_1^2 + \beta_1 X_2 + \beta_2 X_2^2 + \epsilon$$

The major issue with Multivariate Polynomial Regression is the problem of **Multicollinearity**. When there are multiple regression variables, there are high chances that the variables are interdependent on each other.

The vector of estimated polynomial regression coefficients (using ordinary least squares estimation) is

$$\hat{\vec{\beta}} = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \vec{y},$$

LOGISTIC REGRESSION

Consider the Data:

Time	Clicked on Ad
68.95	No
80.23	No
69.45	No
74.15	No
50	Yes
55.5	Yes
80	No
70.5	No

In the above we have 2 variables

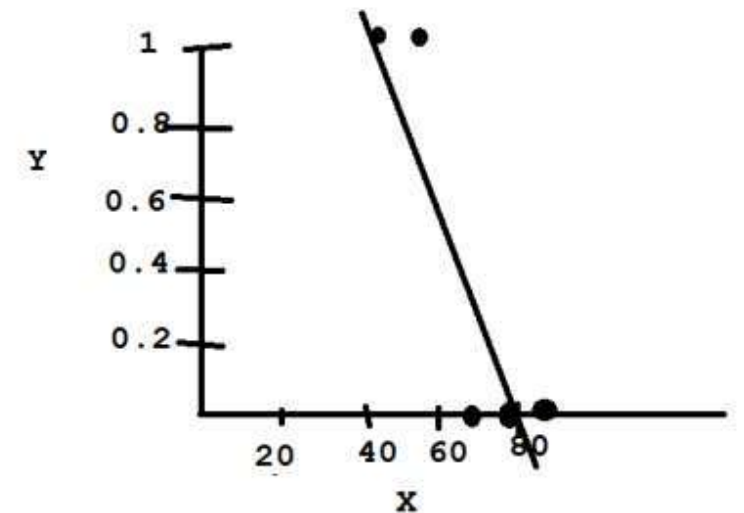
The independent variable X is the Time spent by user on a web site

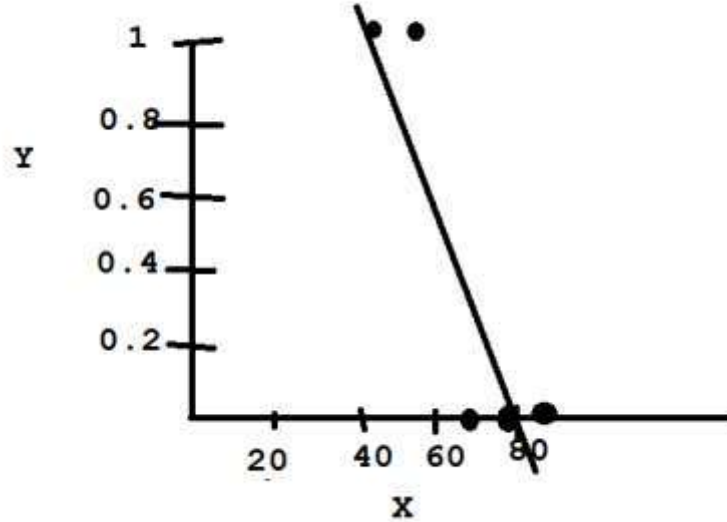
The dependent variable Y is whether the user clicked on any Ads

Now suppose we apply Linear Regression to the data, the points to be considered are:

There is no continuous numerical data
We have categorical data

Now when we plot the graph, we get





The above provides no use in predicting the dependent variables by using the independent variable.

Using the above, it is not possible for us to predict if a user will click on ads for a given time.

Therefore we cannot use Linear Regression for this scenario.

The main reason is because we have categorical data as dependent variable.

So when we have categorical data in the dependent variable, it is better not to use Linear regression

Or if you have yes/no or 0/1 or true/false etc, then linear regression will not be of use.

LOGISTIC REGRESSION:

In such a case go for Logistic Regression.

That is, basically Logistic Regression is used for Classification.

Sigmoid Function:

$$Y = \frac{1}{1 + e^{-x}}$$

The Sigmoid function is the most important function used in Logistic Regression.

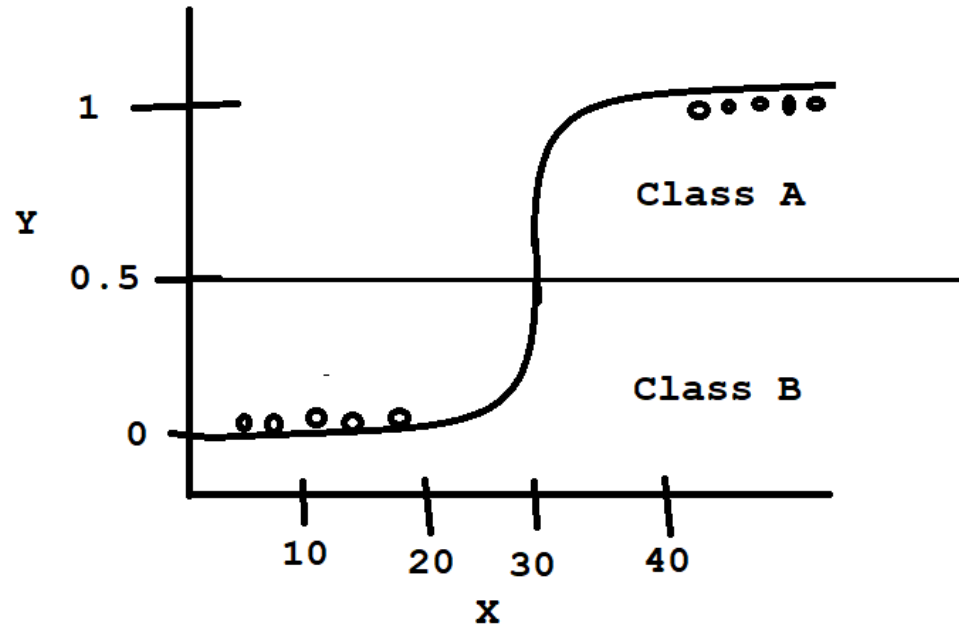
Here x is the independent variable that you want to transform.

E is the Euler's constant which is 2.718

Y is the output or the dependent variable

The Sigmoid function tries to convert the independent variable into an expression of probability that ranges between 0 and 1 with respect to the dependent variable. It will not go below 0 and above 1.

So you can depict the logistic regression as shown below.



As shown above, logistic regression works between 0 and 1.

We can consider 0.5 as cut off point.

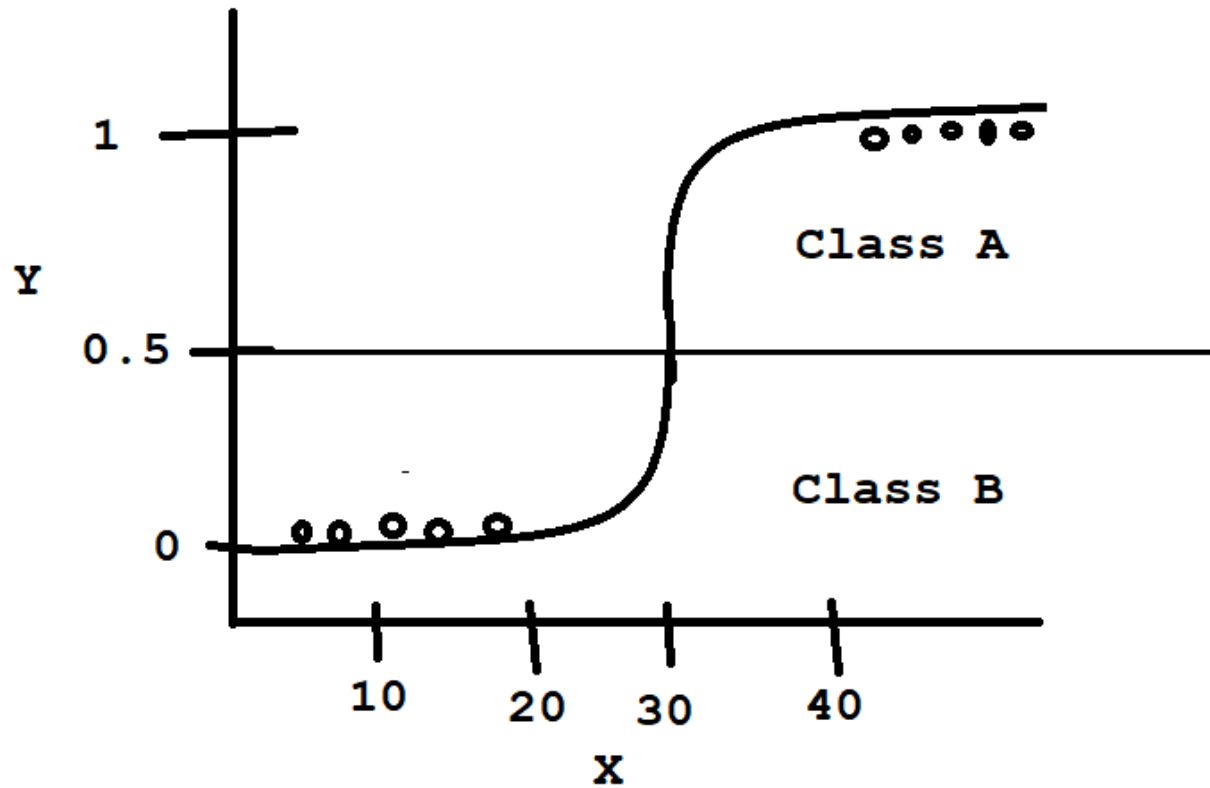
So there are some points below 0.5 and some points above 0.5.

Therefore the logistic regression acts as a strong binary classifier.

It maps the real values of the independent variable in the interval of 0 and 1

The points above 0.5 will be considered as Class A and the points below 0.5 will be considered as Class B

The points that are plotted are the actual output



As shown above, logistic regression works between 0 and 1.

We can consider 0.5 as cut off point.

So there are some points below 0.5 and some points above 0.5.

Therefore the logistic regression acts as a strong binary classifier.

It maps the real values of the independent variable in the interval of 0 and 1

The points above 0.5 will be considered as Class A and the points below 0.5 will be considered as Class B

The points that are plotted are the actual output

So the outputs are classified as belonging to Class A and Class B.

The Y axis is always between 0 and 1.

All the data points will be between 0 and 1.

0 means no possibility of occurrence

1 means highest possibility of occurrence.

So the data points nearer to 1 belong to one class and their possibility is the highest.

Where as the data points nearer to 0 belong to another class and their probability of occurrence is next to impossible.

If the data points are located at 0.5, then they are unclassifiable. But this is rare.

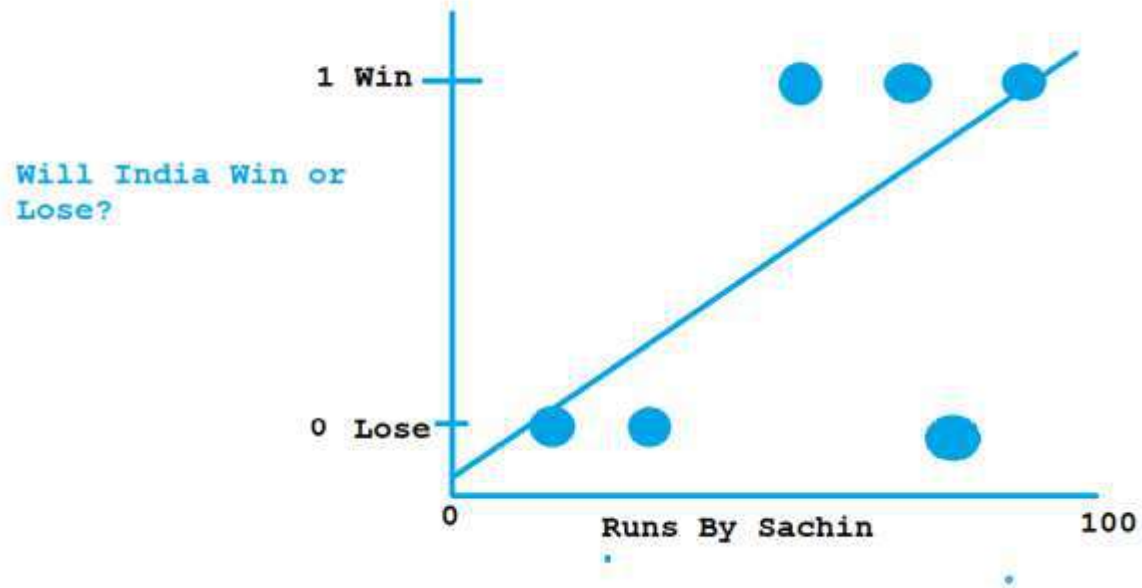
Applications:

Fraud detection

Disease Diagnosis

Emergency Detection

Spam , no spam

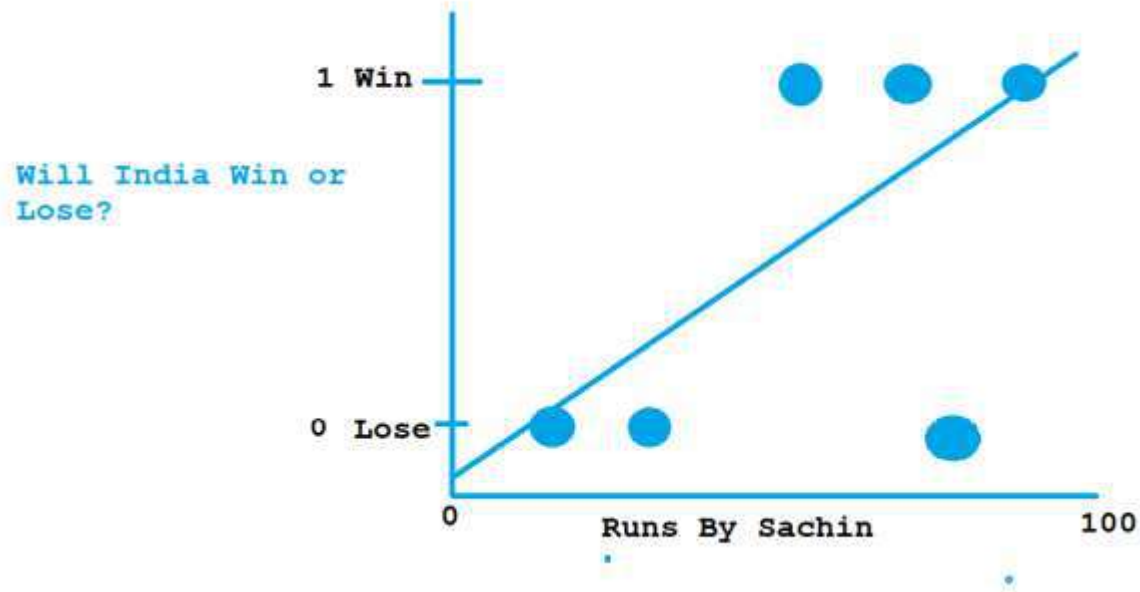


Another example is Scores by Sachin and whether India has won or lost.

If score by Sachin is plotted against X axis and win/lose against Y axis

The below points may represent scores 10, 40 and 80 and the matches were lost

In the above case, the scores might be 60, 80 and 90 and the matches were won.



So if we want to predict if Sachin scores 57, then will the match be won?

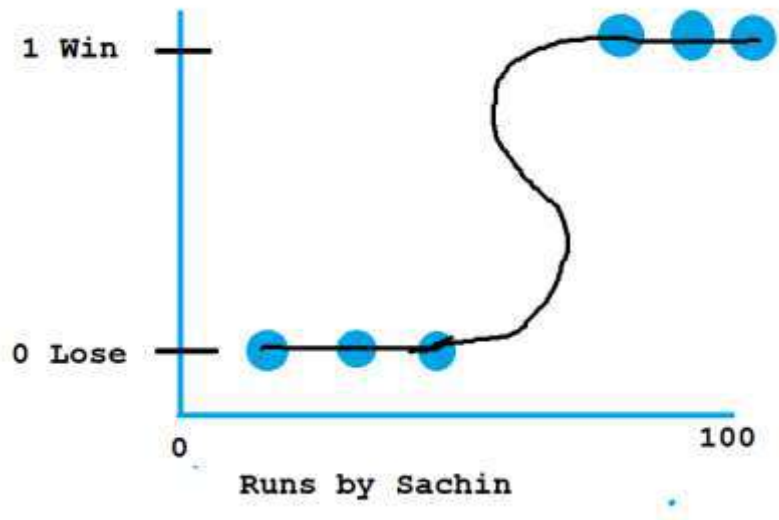
For this drawing a regression line will not work or it will not show the relationship between the score and the result of the match.

Because a Linear Regression Line will not show the Y value as 0(lose) or 1(Win).

Because it is a straight line and the line may go above 1 or below 0.

So suppose the score is 140, then the Y value may be 3 or 4.

But we want the Y value to be either 0 or 1 or maybe in between.

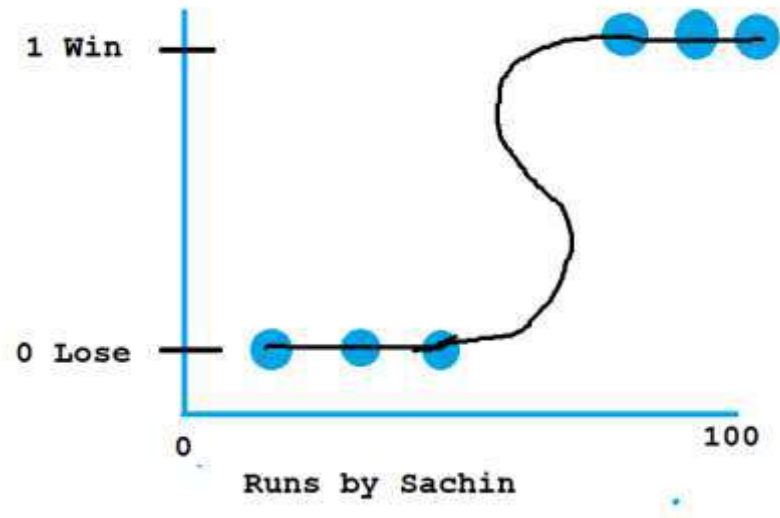


Here we are not predicting a value, but we are classifying, whether India will win or lose the match. In such cases where we are doing a binary classification, we go for Logistic Regression.

We do not draw a regression line, instead we draw the Regression curve. This curve we call it the Sigmoid Curve or the S curve.

There can be any number of data points, but the result will be 1 or 0.

So here we have applied logistic regression and created a Sigmoid curve. The Sigmoid curve will tell you the probability of the result Y based on X value. That is it will give the winning probability of the match based on the runs scored by Sachin.



Here we see that when the runs scored is 20, 30, 40, the probability is that the match will be lost.

When the runs scored is 70, 80, 90, the probability is that the match will be won.

So as the runs scored increases, the probability of winning the match is more.

The probability of winning will not go above 1 or below 0.

The dependent variable Y will always be categorical.
So Logistic Regression is used for binary classification.

Another Example is Smoking(Yes/No)-----→Cancer(Yes/No)

The Formula for the Sigmoid Curve is as follows:

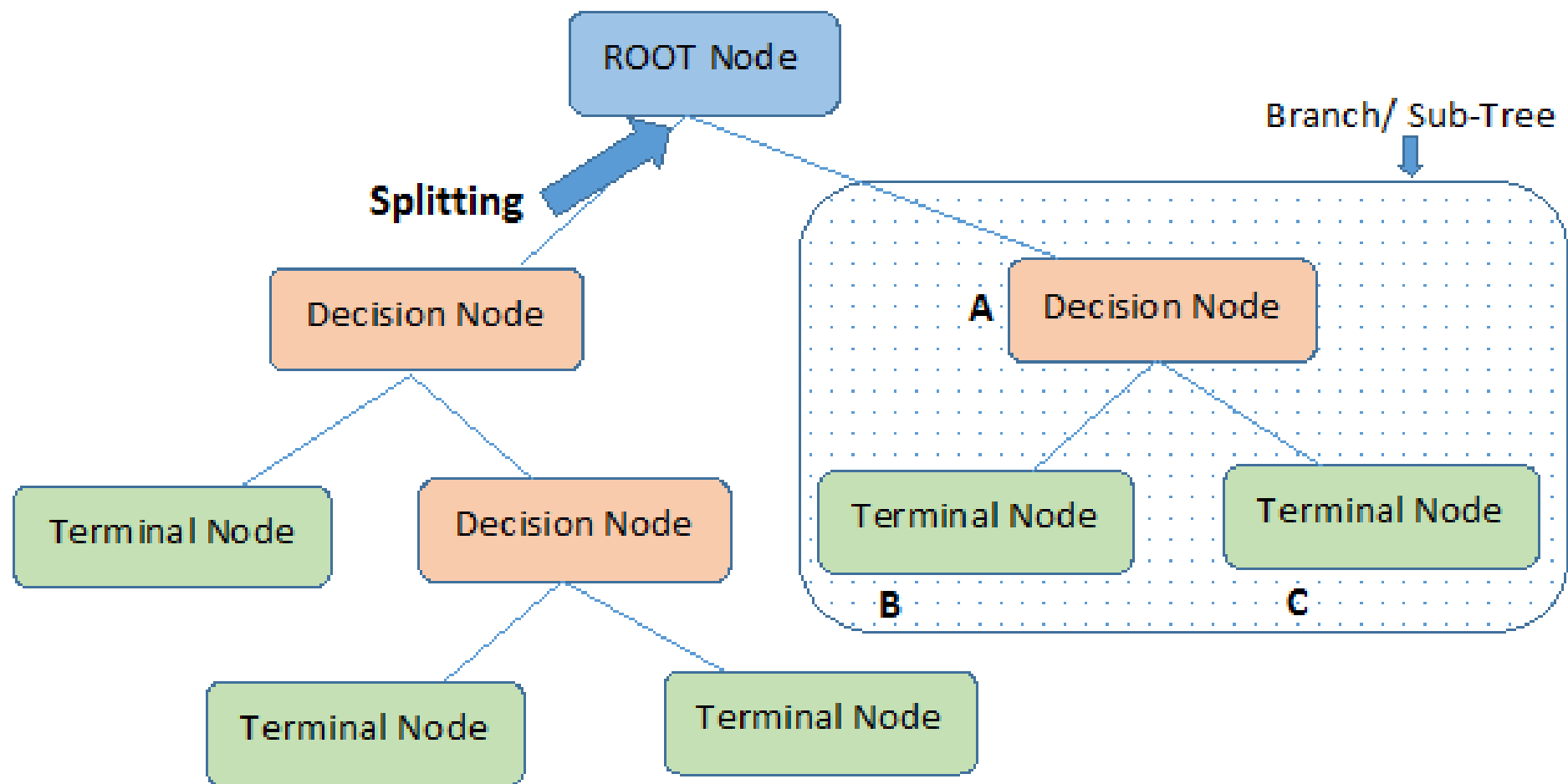
$$Y = \frac{1}{1 + e^{-x}}$$

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

Here x is $\beta^T X$

Where $\beta^T X$ is $\beta_0 + \beta_1 X_1 + \beta_2 X_2 + \dots + \beta_n X_n$

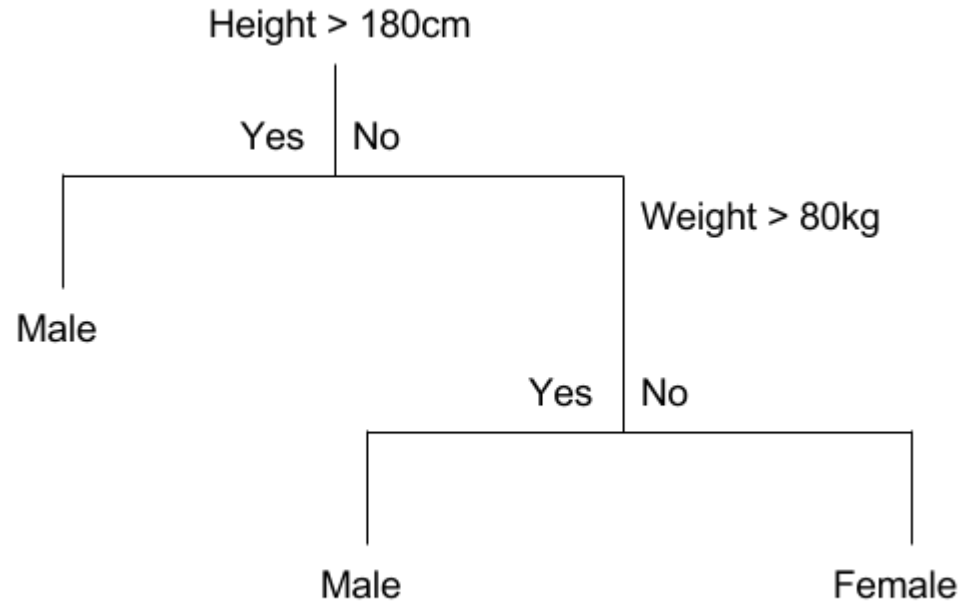
Decision Tree



The decision tree is one of the most important machine learning algorithms. It is used for both classification and regression problems. Here, we will go through the classification part.

What is a decision tree?

A decision tree is a classification and prediction tool having a tree-like structure, where each internal node denotes a test on an attribute, each branch represents an outcome of the test, and each leaf node (terminal node) holds a class label.



Above we have a small decision tree.

An important advantage of the decision tree is that it is highly interpretable.

Here If Height > 180cm or if height < 180cm and weight > 80kg person is male. Otherwise female.

important terms related to decision trees.

Entropy

In machine learning, entropy is a measure of the randomness in the information being processed. The higher the entropy, the harder it is to draw any conclusions from that information.

$$H(X) = - \sum_{i=1}^n P(x_i) \log_b P(x_i)$$

Information Gain

Information gain can be defined as the amount of information gained about a random variable or signal from observing another random variable.

It can be considered as the difference between the entropy of parent node and weighted average entropy of child nodes.

Information gain is the amount of knowledge acquired during a certain decision, whereas entropy is a measure of uncertainty or unpredictability.

Information gain is the reduction in entropy by transforming the dataset.

IG is a statistical property that measures how well a given attribute separates the training examples according to their target classification. IG v

$$IG(S, A) = H(S) - H(S, A)$$

Alternatively,

$$IG(S, A) = H(S) - \sum_{i=0}^n P(x) * H(x)$$

Gini Impurity

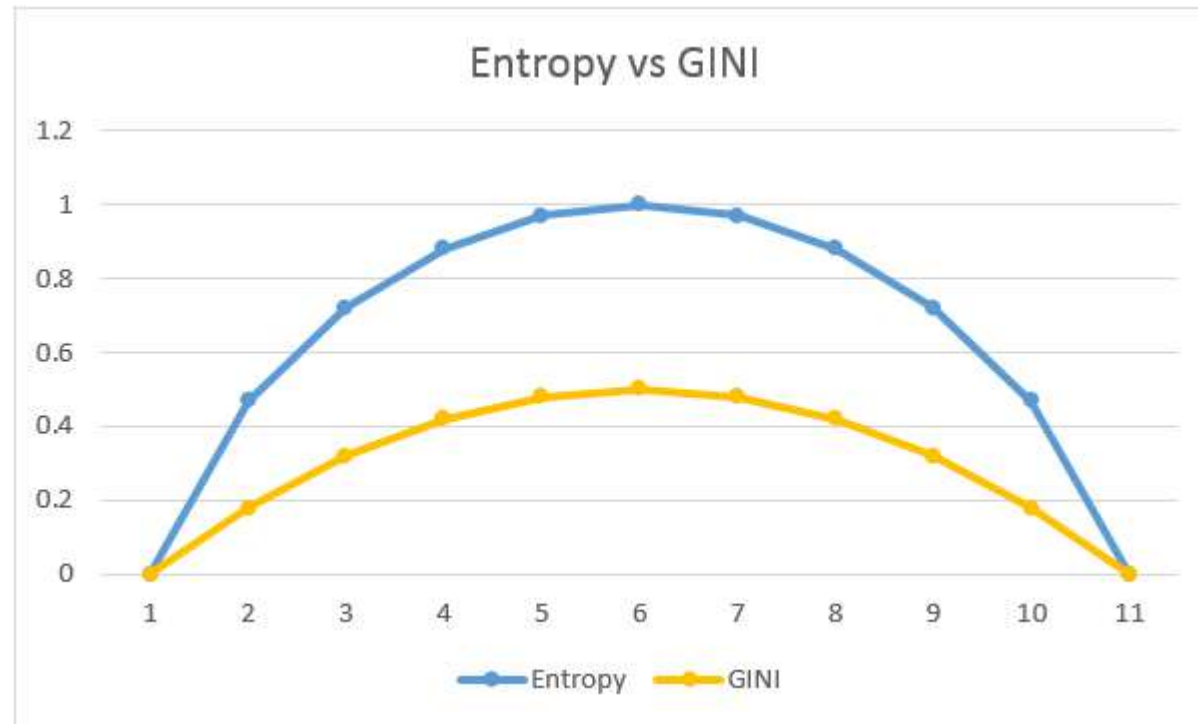
Gini impurity is a measure of how often a randomly chosen element from the set would be incorrectly labeled if it was randomly labeled according to the distribution of labels in the subset.

Gini index is the measure of impurity or entropy in the values of a dataset.

Gini index is the probability of misclassification when an instance is randomly classified.

$$Gini(E) = 1 - \sum_{j=1}^c p_j^2$$

Gini impurity is **lower bounded by 0**, with 0 occurring if the data set contains only one class.



There are many algorithms there to build a decision tree. They are

- 1.CART** (Classification and Regression Trees) — This makes use of Gini impurity as the metric.
- 2.ID3** (Iterative Dichotomiser 3) — This uses entropy and information gain as metric.

Classification using the ID3 algorithm

Consider whether a dataset based on which we will determine whether to play football or not.

Outlook	Temperature	Humidity	Wind	Played football(yes/no)
Sunny	Hot	High	Weak	No
Sunny	Hot	High	Strong	No
Overcast	Hot	High	Weak	Yes
Rain	Mild	High	Weak	Yes
Rain	Cool	Normal	Weak	Yes
Rain	Cool	Normal	Strong	No
Overcast	Cool	Normal	Strong	Yes
Sunny	Mild	High	Weak	No
Sunny	Cool	Normal	Weak	Yes
Rain	Mild	Normal	Weak	Yes
Sunny	Mild	Normal	Strong	Yes
Overcast	Mild	High	Strong	Yes
Overcast	Hot	Normal	Weak	Yes
Rain	Mild	High	Strong	No

In the dataset there are four independent variables to determine the dependent variable. The independent variables are Outlook, Temperature, Humidity, and Wind. The dependent variable is whether to play football or not.

As the first step, we have to find the parent node for our decision tree. For that follow the steps:

Find the entropy of the class variable.

$$E(S) = -[(9/14)\log(9/14) + (5/14)\log(5/14)] = 0.94$$

note: Here typically we will take log to base 2.

Here total there are 14 yes/no.

Out of which 9 yes and 5 no.

Based on it we calculated probability above.

From the above data for outlook we can arrive at the following table easily

		play		
		yes	no	total
	sunny	3	2	5
Outlook	overcast	4	0	4
	rainy	2	3	5
				14

Now we have to calculate average weighted entropy. ie, we have found the total of weights of each feature multiplied by probabilities.

$$E(S, \text{outlook}) = (5/14)*E(3,2) + (4/14)*E(4,0) + (5/14)*E(2,3) = (5/14)*(-(3/5)\log(3/5)-(2/5)\log(2/5)) + (4/14)*(0) + (5/14)*((2/5)\log(2/5)-(3/5)\log(3/5)) = 0.693$$

The next step is to find the information gain.

It is the difference between parent entropy and average weighted entropy we found above.

$$IG(S, \text{outlook}) = 0.94 - 0.693 = 0.247$$

Similarly find Information gain for Temperature, Humidity, and Windy.

$$\text{IG}(\text{S, Temperature}) = 0.940 - 0.911 = 0.029$$

$$\text{IG}(\text{S, Humidity}) = 0.940 - 0.788 = 0.152$$

$$\text{IG}(\text{S, Windy}) = 0.940 - 0.8932 = 0.048$$

Now select the feature having the largest entropy gain.

Here it is Outlook.

So it forms the first node(root node) of our decision tree.

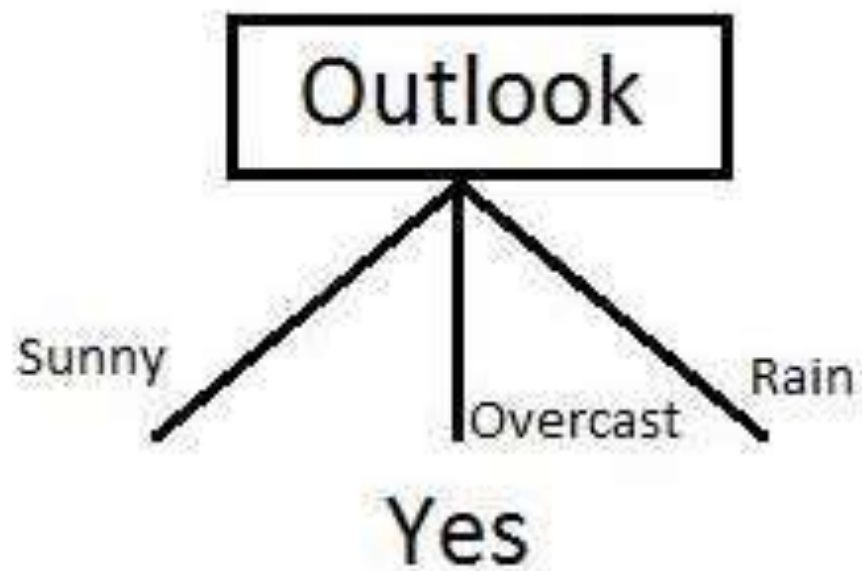
Now our data look as follows

Outlook ▼	Temperature	Humidity	Wind	Played football(yes/no)
Sunny	Hot	High	Weak	No
Sunny	Hot	High	Strong	No
Sunny	Mild	High	Weak	No
Sunny	Cool	Normal	Weak	Yes
Sunny	Mild	Normal	Strong	Yes

Outlook ▼	Temperature	Humidity	Wind	Played football(yes/no)
Overcast	Hot	High	Weak	Yes
Overcast	Cool	Normal	Strong	Yes
Overcast	Mild	High	Strong	Yes
Overcast	Hot	Normal	Weak	Yes

Outlook 	Temperature	Humidity	Wind	Played football(yes/no)
Rain	Mild	High	Weak	Yes
Rain	Cool	Normal	Weak	Yes
Rain	Cool	Normal	Strong	No
Rain	Mild	Normal	Weak	Yes
Rain	Mild	High	Strong	No

Since overcast contains only examples of class 'Yes' we can set it as yes.
That means If outlook is overcast football will be played.
Now our decision tree looks as follows.



The next step is to find the next node in our decision tree.

Now we will find one under sunny.

We have to determine which of the following Temperature, Humidity or Wind has higher information gain.

Outlook 	Temperature	Humidity	Wind	Played football(yes/no)
Sunny	Hot	High	Weak	No
Sunny	Hot	High	Strong	No
Sunny	Mild	High	Weak	No
Sunny	Cool	Normal	Weak	Yes
Sunny	Mild	Normal	Strong	Yes

Calculate parent entropy $E(\text{sunny})$

$$E(\text{sunny}) = -(3/5)\log(3/5) - (2/5)\log(2/5) = 0.971.$$

Now Calculate the information gain of Temperature. $IG(\text{sunny}, \text{Temperature})$

		play		
		yes	no	total
Temperature	hot	0	2	2
	cool	1	1	2
	mild	1	0	1
				5

$$E(\text{sunny}, \text{Temperature}) = (2/5)*E(0,2) + (2/5)*E(1,1) + (1/5)*E(1,0) = 2/5 = 0.4$$

Now calculate information gain.

$$IG(\text{sunny}, \text{Temperature}) = 0.971 - 0.4 = 0.571$$

Similarly we get

$$IG(\text{sunny}, \text{Humidity}) = 0.971$$

$$IG(\text{sunny}, \text{Windy}) = 0.020$$

Here $IG(\text{sunny}, \text{Humidity})$ is the largest value.
So Humidity is the node that comes under sunny.

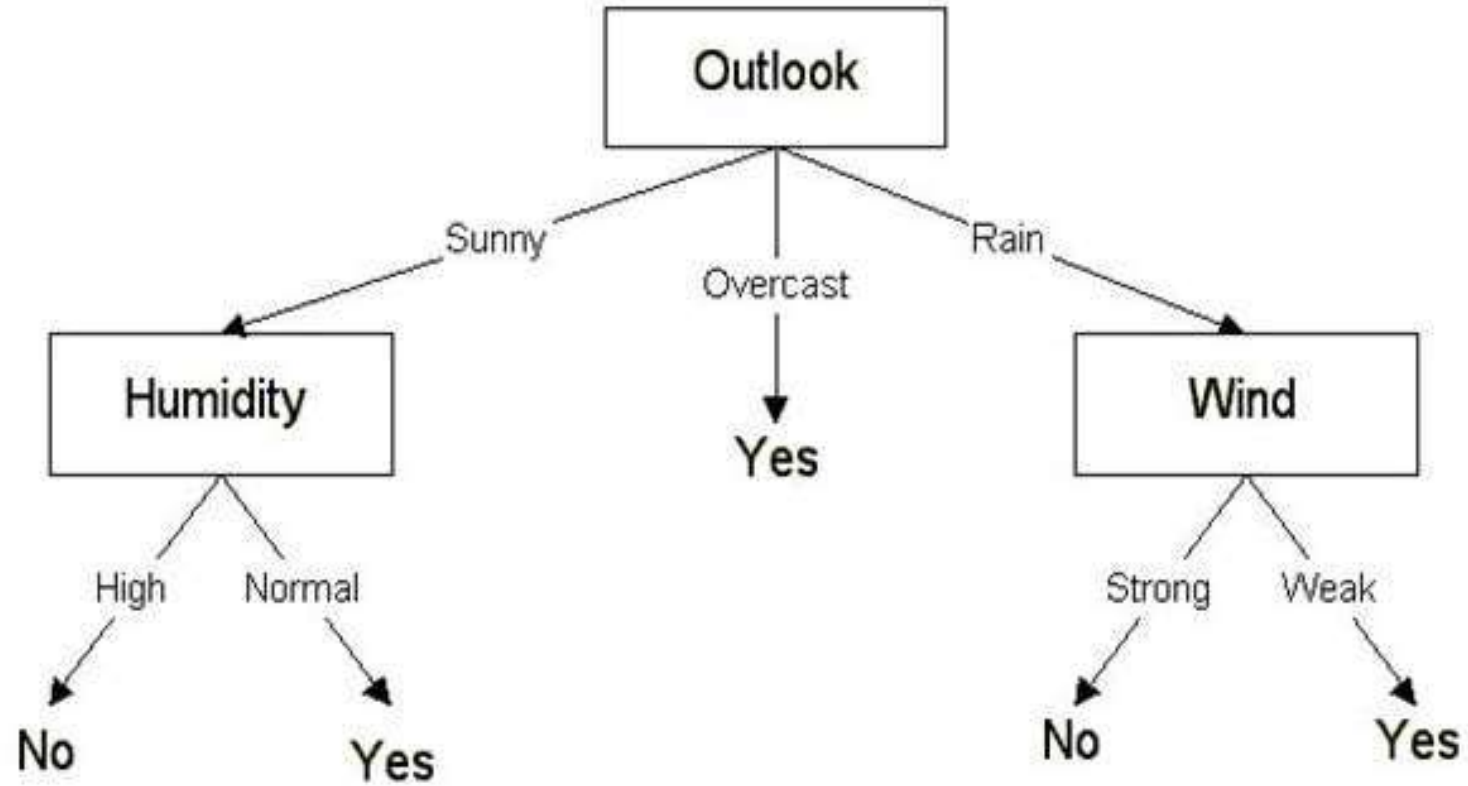
	play	
Humidity	yes	no
high	0	3
normal	2	0

For humidity from the above table, we can say that play will occur if humidity is normal and will not occur if it is high.

Similarly, find the nodes under rainy.

Note: A branch with entropy more than 0 needs further splitting.

Finally, our decision tree will look as below:



DECISION TREE CART ALGORITHM

Using Gini Index

Example 1:

Instance	a_1	a_2	a_3	Target Class
1	T	T	1.0	+
2	T	T	6.0	+
3	T	F	5.0	-
4	F	F	4.0	+
5	F	T	7.0	-
6	F	T	3.0	-
7	F	F	8.0	-
8	T	F	7.0	+
9	F	T	5.0	-

Compute the Gini Index of the attributes a_1 .

$$Gini = 1 - \sum_{i=1}^n (p_i)^2$$

$$Gini(T) = 1 - \left(\frac{3}{4}\right)^2 - \left(\frac{1}{4}\right)^2 = 0.375$$

$$Gini(F) = 1 - \left(\frac{1}{5}\right)^2 - \left(\frac{4}{5}\right)^2 = 0.32$$

Instance	a_1	a_2	a_3	Target Class
1	T	T	1.0	+
2	T	T	6.0	+
3	T	F	5.0	-
4	F	F	4.0	+
5	F	T	7.0	-
6	F	T	3.0	-
7	F	F	8.0	-
8	T	F	7.0	+
9	F	T	5.0	-

Compute the Gini Index of the attributes a_1 .

$$GiniIndex(a_1) = \sum_{v \in \{T, F\}} \frac{|S_v|}{|S|} Gini(S_v)$$

$$GiniIndex(a_1) = \left(\frac{4}{9}\right) * Gini(T) + \left(\frac{5}{9}\right) * Gini(F)$$

$$GiniIndex(a_1) = \left(\frac{4}{9}\right) * 0.375 + \left(\frac{5}{9}\right) * 0.32$$

$$GiniIndex(a_1) = 0.3444$$

Compute the Gini Index of the attributes a2.

Instance	a_1	a_2	a_3	Target Class
1	T	T	1.0	+
2	T	T	6.0	+
3	T	F	5.0	-
4	F	F	4.0	+
5	F	T	7.0	-
6	F	T	3.0	-
7	F	F	8.0	-
8	T	F	7.0	+
9	F	T	5.0	-

$$Gini = 1 - \sum_{i=1}^n (p_i)^2$$

$$Gini(T) = 1 - \left(\frac{2}{5}\right)^2 - \left(\frac{3}{5}\right)^2 = 0.48$$

$$Gini(F) = 1 - \left(\frac{2}{4}\right)^2 - \left(\frac{2}{4}\right)^2 = 0.5$$

Instance	a_1	a_2	a_3	Target Class
1	T	T	1.0	+
2	T	T	6.0	+
3	T	F	5.0	-
4	F	F	4.0	+
5	F	T	7.0	-
6	F	T	3.0	-
7	F	F	8.0	-
8	T	F	7.0	+
9	F	T	5.0	-

Compute the Gini Index of the attributes a_2 .

$$GiniIndex(a_2) = \sum_{v \in \{T, F\}} \frac{|S_v|}{|S|} Gini(S_v)$$

$$GiniIndex(a_2) = \left(\frac{5}{9}\right) * Gini(T) + \left(\frac{4}{9}\right) * Gini(F)$$

$$GiniIndex(a_2) = \left(\frac{5}{9}\right) * 0.48 + \left(\frac{4}{9}\right) * 0.5$$

$$GiniIndex(a_2) = 0.4889$$

Instance	a_1	a_2	a_3	Target Class
1	T	T	1.0	+
2	T	T	6.0	+
3	T	F	5.0	-
4	F	F	4.0	+
5	F	T	7.0	-
6	F	T	3.0	-
7	F	F	8.0	-
8	T	F	7.0	+
9	F	T	5.0	-

Which is the best splitting attribute between a_1 and a_2 .

$$GiniIndex(a_1) = 0.3444$$

$$GiniIndex(a_2) = 0.4889$$

Smaller GiniIndex Produces Better Split

Hence, attribute a_1 is the best split attribute

Example 2:

Outlook	Temperature	Humidity	Wind	Played football(yes/no)
Sunny	Hot	High	Weak	No
Sunny	Hot	High	Strong	No
Overcast	Hot	High	Weak	Yes
Rain	Mild	High	Weak	Yes
Rain	Cool	Normal	Weak	Yes
Rain	Cool	Normal	Strong	No
Overcast	Cool	Normal	Strong	Yes
Sunny	Mild	High	Weak	No
Sunny	Cool	Normal	Weak	Yes
Rain	Mild	Normal	Weak	Yes
Sunny	Mild	Normal	Strong	Yes
Overcast	Mild	High	Strong	Yes
Overcast	Hot	Normal	Weak	Yes
Rain	Mild	High	Strong	No

Gini index

Gini index is a metric for classification tasks in CART. It stores sum of squared probabilities of each class. We can formulate it as illustrated below.

$$\text{Gini} = 1 - \sum (P_i)^2 \text{ for } i=1 \text{ to number of classes}$$

Outlook

sunny, overcast or rain.

Outlook	Yes	No	Number of instances
Sunny	2	3	5
Overcast	4	0	4
Rain	3	2	5

$$\text{Gini}(\text{Outlook}=\text{Sunny}) = 1 - (2/5)^2 - (3/5)^2 = 1 - 0.16 - 0.36 = 0.48$$

$$\text{Gini}(\text{Outlook}=\text{Overcast}) = 1 - (4/4)^2 - (0/4)^2 = 0$$

$$\text{Gini}(\text{Outlook}=\text{Rain}) = 1 - (3/5)^2 - (2/5)^2 = 1 - 0.36 - 0.16 = 0.48$$

calculate weighted sum of gini indexes for outlook feature.

$$\text{Gini}(\text{Outlook}) = (5/14) \times 0.48 + (4/14) \times 0 + (5/14) \times 0.48 = 0.171 + 0 + 0.171 = 0.342$$

Temperature

values: Cool, Hot Mild.

Temperature	Yes	No	Number of instances
Hot	2	2	4
Cool	3	1	4
Mild	4	2	6

$$\text{Gini}(\text{Temp}=\text{Hot}) = 1 - (2/4)^2 - (2/4)^2 = 0.5$$

$$\text{Gini}(\text{Temp}=\text{Cool}) = 1 - (3/4)^2 - (1/4)^2 = 1 - 0.5625 - 0.0625 = 0.375$$

$$\text{Gini}(\text{Temp}=\text{Mild}) = 1 - (4/6)^2 - (2/6)^2 = 1 - 0.444 - 0.111 = 0.445$$

calculate weighted sum of gini index for temperature feature

$$\text{Gini}(\text{Temp}) = (4/14) \times 0.5 + (4/14) \times 0.375 + (6/14) \times 0.445 = 0.142 + 0.107 + 0.190 = 0.439$$

Humidity

It can be high or normal.

Humidity	Yes	No	Number of instances
High	3	4	7
Normal	6	1	7

$$\text{Gini}(\text{Humidity}=\text{High}) = 1 - (3/7)^2 - (4/7)^2 = 1 - 0.183 - 0.326 = 0.489$$

$$\text{Gini}(\text{Humidity}=\text{Normal}) = 1 - (6/7)^2 - (1/7)^2 = 1 - 0.734 - 0.02 = 0.244$$

Weighted sum for humidity feature will be calculated next

$$\text{Gini}(\text{Humidity}) = (7/14) \times 0.489 + (7/14) \times 0.244 = 0.367$$

Wind

weak and strong.

Wind	Yes	No	Number of instances
Weak	6	2	8
Strong	3	3	6

$$\text{Gini}(\text{Wind}=\text{Weak}) = 1 - (6/8)^2 - (2/8)^2 = 1 - 0.5625 - 0.0625 = 0.375$$

$$\text{Gini}(\text{Wind}=\text{Strong}) = 1 - (3/6)^2 - (3/6)^2 = 1 - 0.25 - 0.25 = 0.5$$

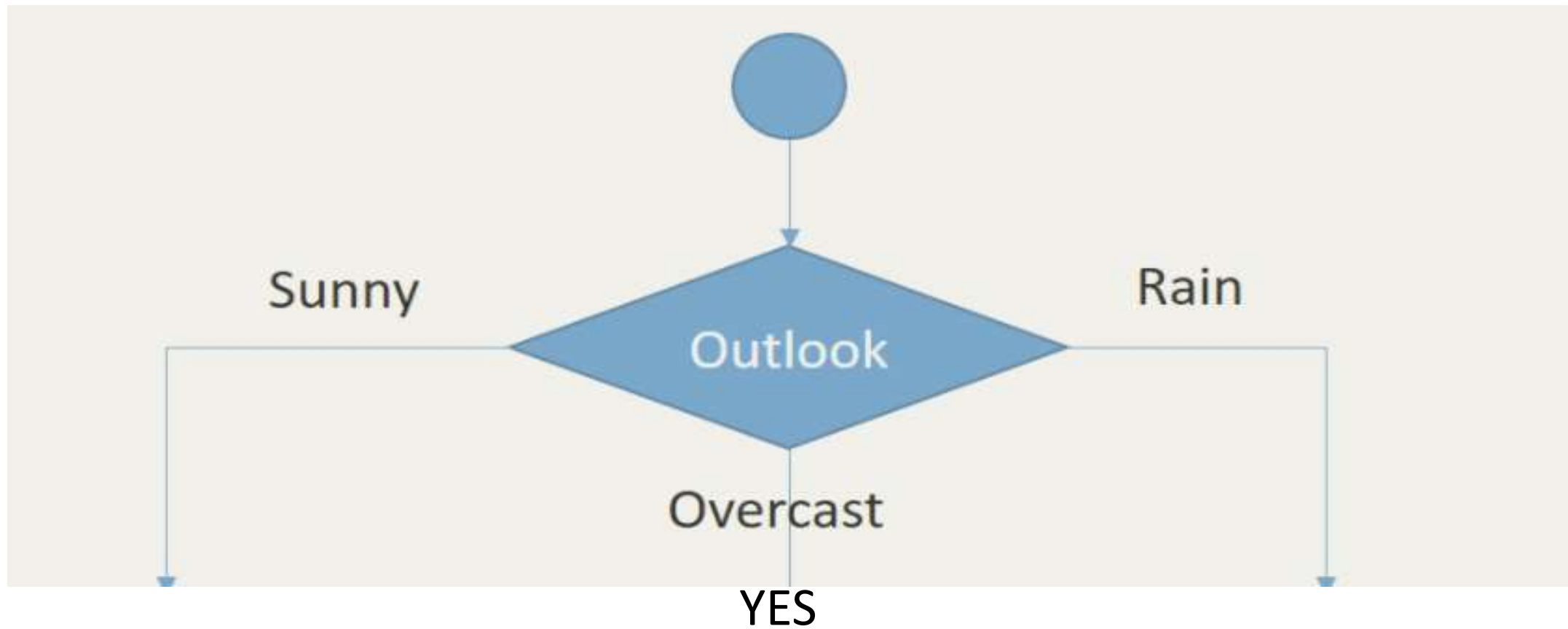
$$\text{Gini}(\text{Wind}) = (8/14) \times 0.375 + (6/14) \times 0.5 = 0.428$$

So which should be the root node?

The attribute having the smallest Gini Index value will be selected.

Feature	Gini index
Outlook	0.342
Temperature	0.439
Humidity	0.367
Wind	0.428

So it will be Outlook



sub dataset in the overcast leaf has only yes

This means that overcast leaf is over.

Now For Sunny: (Deciding Left branch node)

Day	Outlook	Temp.	Humidity	Wind	Decision
1	Sunny	Hot	High	Weak	No
2	Sunny	Hot	High	Strong	No
8	Sunny	Mild	High	Weak	No
9	Sunny	Cool	Normal	Weak	Yes
11	Sunny	Mild	Normal	Strong	Yes

Gini of temperature for sunny outlook

Temperature	Yes	No	Number of instances
Hot	0	2	2
Cool	1	0	1
Mild	1	1	2

$$\text{Gini}(\text{Outlook}=\text{Sunny and Temp.}=\text{Hot}) = 1 - (0/2)^2 - (2/2)^2 = 0$$

$$\text{Gini}(\text{Outlook}=\text{Sunny and Temp.}=\text{Cool}) = 1 - (1/1)^2 - (0/1)^2 = 0$$

$$\begin{aligned}\text{Gini}(\text{Outlook}=\text{Sunny and Temp.}=\text{Mild}) &= 1 - (1/2)^2 - (1/2)^2 = 1 - 0.25 - 0.25 \\ &= 0.5\end{aligned}$$

$$\text{Gini}(\text{Outlook}=\text{Sunny and Temp.}) = (2/5) \times 0 + (1/5) \times 0 + (2/5) \times 0.5 = 0.2$$

Gini of humidity for sunny outlook

Humidity	Yes	No	Number of instances
High	0	3	3
Normal	2	0	2

$$\text{Gini}(\text{Outlook}=\text{Sunny and Humidity}=\text{High}) = 1 - (0/3)^2 - (3/3)^2 = 0$$

$$\text{Gini}(\text{Outlook}=\text{Sunny and Humidity}=\text{Normal}) = 1 - (2/2)^2 - (0/2)^2 = 0$$

$$\text{Gini}(\text{Outlook}=\text{Sunny and Humidity}) = (3/5) \times 0 + (2/5) \times 0 = 0$$

Gini of wind for sunny outlook

Wind	Yes	No	Number of instances
Weak	1	2	3
Strong	1	1	2

$$\text{Gini}(\text{Outlook}=\text{Sunny and Wind}=\text{Weak}) = 1 - (1/3)^2 - (2/3)^2 = 0.266$$

$$\text{Gini}(\text{Outlook}=\text{Sunny and Wind}=\text{Strong}) = 1 - (1/2)^2 - (1/2)^2 = 0.2$$

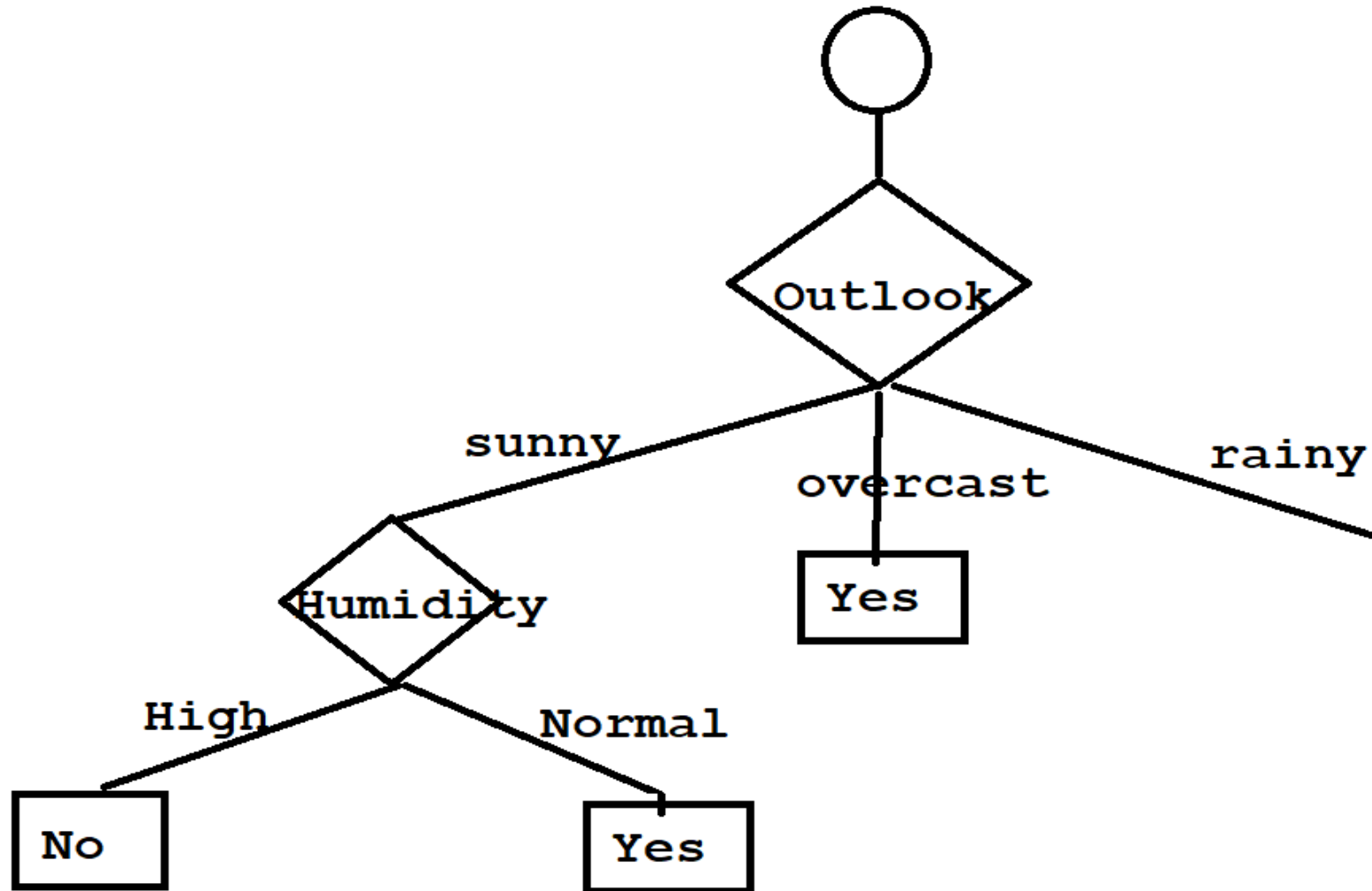
$$\text{Gini}(\text{Outlook}=\text{Sunny and Wind}) = (3/5) \times 0.266 + (2/5) \times 0.2 = 0.466$$

So which should be the node if Outlook is Sunny?

The attribute having the smallest Gini Index value will be selected.

Feature	Gini index
Temperature	0.2
Humidity	0
Wind	0.466

So Humidity will be selected as the node if outlook is sunny.



For Humidity there is no need to calculate again.

Since all humidity records with High as value, the decision is “No”

And all humidity records with Normal as value, the decision is “Yes”

Now For Rainy: (Deciding Right branch node)

Rain outlook

Day	Outlook	Temp.	Humidity	Wind	Decision
4	Rain	Mild	High	Weak	Yes
5	Rain	Cool	Normal	Weak	Yes
6	Rain	Cool	Normal	Strong	No
10	Rain	Mild	Normal	Weak	Yes
14	Rain	Mild	High	Strong	No

Calculate gini index scores for temperature, humidity and wind features when outlook is rain.

Gini of temprature for rain outlook

Temperature	Yes	No	Number of instances
Cool	1	1	2
Mild	2	1	3

$$\text{Gini}(\text{Outlook}=\text{Rain and Temp.}=\text{Cool}) = 1 - (1/2)^2 - (1/2)^2 = 0.5$$

$$\text{Gini}(\text{Outlook}=\text{Rain and Temp.}=\text{Mild}) = 1 - (2/3)^2 - (1/3)^2 = 0.444$$

$$\text{Gini}(\text{Outlook}=\text{Rain and Temp.}) = (2/5) \times 0.5 + (3/5) \times 0.444 = 0.466$$

Gini of humidity for rain outlook

Humidity	Yes	No	Number of instances
High	1	1	2
Normal	2	1	3

$$\text{Gini}(\text{Outlook}=\text{Rain and Humidity}=\text{High}) = 1 - (1/2)^2 - (1/2)^2 = 0.5$$

$$\text{Gini}(\text{Outlook}=\text{Rain and Humidity}=\text{Normal}) = 1 - (2/3)^2 - (1/3)^2 = 0.444$$

$$\text{Gini}(\text{Outlook}=\text{Rain and Humidity}) = (2/5) \times 0.5 + (3/5) \times 0.444 = 0.466$$

Gini of wind for rain outlook

Wind	Yes	No	Number of instances
Weak	3	0	3
Strong	0	2	2

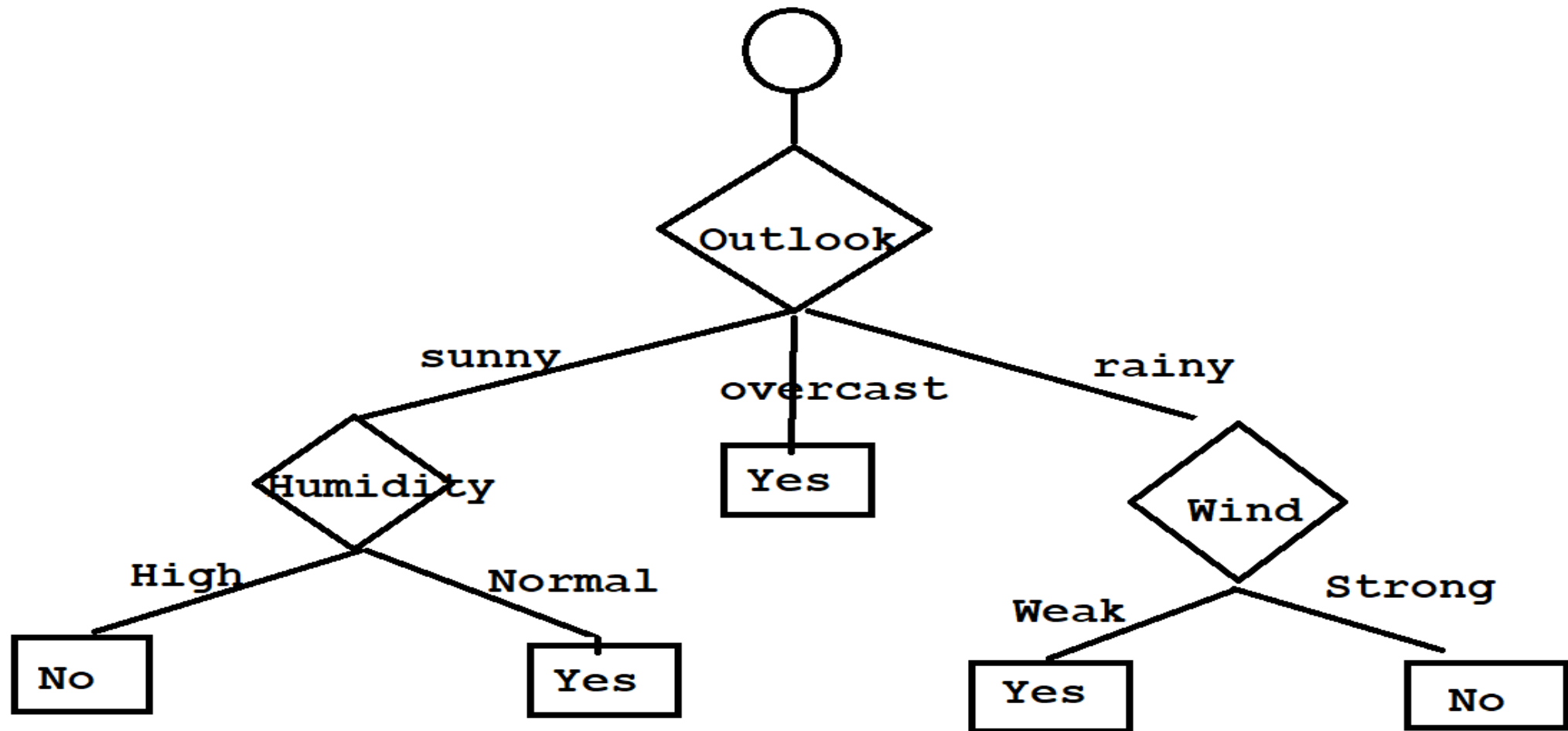
$$\text{Gini}(\text{Outlook}=\text{Rain and Wind}=\text{Weak}) = 1 - (3/3)^2 - (0/3)^2 = 0$$

$$\text{Gini}(\text{Outlook}=\text{Rain and Wind}=\text{Strong}) = 1 - (0/2)^2 - (2/2)^2 = 0$$

$$\text{Gini}(\text{Outlook}=\text{Rain and Wind}) = (3/5) \times 0 + (2/5) \times 0 = 0$$

The Node under rain will be selected as Wind because among temperature, humidity and wind, wind is having the lowest gini index.

Feature	Gini index
Temperature	0.466
Humidity	0.466
Wind	0



All records with wind Weak, the decision is “Yes”
All records with wind strong, the decision is “No”
So no need to calculate further.

So for a new record

Outlook	Temperature	Humidity	Wind	Play Tennis
Sunny	Mild	High	Weak	?

So according to the decision tree created the result will be

“No”

There will be no need to check for Temperature and Wind at all.

Decision Tree: ID3 Algorithm

Outlook	Temperature	Humidity	Wind	Played football(yes/no)
Sunny	Hot	High	Weak	No
Sunny	Hot	High	Strong	No
Overcast	Hot	High	Weak	Yes
Rain	Mild	High	Weak	Yes
Rain	Cool	Normal	Weak	Yes
Rain	Cool	Normal	Strong	No
Overcast	Cool	Normal	Strong	Yes
Sunny	Mild	High	Weak	No
Sunny	Cool	Normal	Weak	Yes
Rain	Mild	Normal	Weak	Yes
Sunny	Mild	Normal	Strong	Yes
Overcast	Mild	High	Strong	Yes
Overcast	Hot	Normal	Weak	Yes
Rain	Mild	High	Strong	No

Outlook	Temperature	Humidity	Wind	Played football(yes/no)
Sunny	Hot	High	Weak	No
Sunny	Hot	High	Strong	No
Overcast	Hot	High	Weak	Yes
Rain	Mild	High	Weak	Yes
Rain	Cool	Normal	Weak	Yes
Rain	Cool	Normal	Strong	No
Overcast	Cool	Normal	Strong	Yes
Sunny	Mild	High	Weak	No
Sunny	Cool	Normal	Weak	Yes
Rain	Mild	Normal	Weak	Yes
Sunny	Mild	Normal	Strong	Yes
Overcast	Mild	High	Strong	Yes
Overcast	Hot	Normal	Weak	Yes
Rain	Mild	High	Strong	No

Attribute : Outlook:

Values(Outlook) = Sunny, Overcast, Rain

$$S = [9+, 5 -] \quad Entropy(S) = -\frac{9}{14} \log_2 \frac{9}{14} - \frac{5}{14} \log_2 \frac{5}{14} = 0.94$$

$$S_{sunny} = [2+, 3 -] \quad Entropy(S_{sunny}) = -\frac{2}{5} \log_2 \frac{2}{5} - \frac{3}{5} \log_2 \frac{3}{5} = 0.971$$

$$S_{overcast} = [4+, 0 -] \quad Entropy(S_{overcast}) = -\frac{4}{4} \log_2 \frac{4}{4} - \frac{0}{4} \log_2 \frac{0}{4} = 0$$

$$S_{rainy} = [3+, 2 -] \quad Entropy(S_{rainy}) = -\frac{3}{5} \log_2 \frac{3}{5} - \frac{2}{5} \log_2 \frac{2}{5} = 0.971$$

$$Gain(S, outlook) = Entropy(S) - \sum_{v \in (sunny, overcast, rainy)} \frac{|S_v|}{|S|} Entropy(S_v)$$

$$Gain(S, outlook) = Entropy(S) - \frac{5}{14} Entropy(S_{sunny}) - \frac{4}{14} Entropy(S_{overcast}) - \frac{5}{14} Entropy(S_{rain})$$

$$Gain(S, outlook) = 0.94 - \frac{5}{14} 0.971 - \frac{4}{14} 0 - \frac{5}{14} 0.971 = 0.2464$$

Outlook	Temperature	Humidity	Wind	Played football(yes/no)
Sunny	Hot	High	Weak	No
Sunny	Hot	High	Strong	No
Overcast	Hot	High	Weak	Yes
Rain	Mild	High	Weak	Yes
Rain	Cool	Normal	Weak	Yes
Rain	Cool	Normal	Strong	No
Overcast	Cool	Normal	Strong	Yes
Sunny	Mild	High	Weak	No
Sunny	Cool	Normal	Weak	Yes
Rain	Mild	Normal	Weak	Yes
Sunny	Mild	Normal	Strong	Yes
Overcast	Mild	High	Strong	Yes
Overcast	Hot	Normal	Weak	Yes
Rain	Mild	High	Strong	No

Attribute: Temp

Values(Temp)=Hot, Mild, Cool

$$S = [9+, 5 -] \quad Entropy(S) = -\frac{9}{14} \log_2 \frac{9}{14} - \frac{5}{14} \log_2 \frac{5}{14} = 0.94$$

$$S_{Hot} = [2+, 2 -] \quad Entropy(S_{Hot}) = -\frac{2}{4} \log_2 \frac{2}{4} - \frac{2}{4} \log_2 \frac{2}{4} = 1.0$$

$$S_{Mild} = [4+, 2 -] \quad Entropy(S_{Mild}) = -\frac{4}{6} \log_2 \frac{4}{6} - \frac{2}{6} \log_2 \frac{2}{6} = 0.9183$$

$$S_{Cool} = [3+, 1 -] \quad Entropy(S_{Cool}) = -\frac{3}{4} \log_2 \frac{3}{4} - \frac{1}{4} \log_2 \frac{1}{4} = 0.8113$$

$$Gain(S, temp) = Entropy(S) - \sum_{v \in (Hot, Mild, Cool)} \frac{|S_v|}{|S|} Entropy(S_v)$$

$$Gain(S, temp) = Entropy(S) - \frac{4}{14} Entropy(S_{Hot}) - \frac{6}{14} Entropy(S_{Mild}) - \frac{4}{14} Entropy(S_{Cool})$$

$$Gain(S, temp) = 0.94 - \frac{4}{14} 1.0 - \frac{6}{14} 0.9183 - \frac{4}{14} 0.8113 = 0.0289$$

Outlook	Temperature	Humidity	Wind	Played football(yes/no)
Sunny	Hot	High	Weak	No
Sunny	Hot	High	Strong	No
Overcast	Hot	High	Weak	Yes
Rain	Mild	High	Weak	Yes
Rain	Cool	Normal	Weak	Yes
Rain	Cool	Normal	Strong	No
Overcast	Cool	Normal	Strong	Yes
Sunny	Mild	High	Weak	No
Sunny	Cool	Normal	Weak	Yes
Rain	Mild	Normal	Weak	Yes
Sunny	Mild	Normal	Strong	Yes
Overcast	Mild	High	Strong	Yes
Overcast	Hot	Normal	Weak	Yes
Rain	Mild	High	Strong	No

Attribute: Humidity

Values(Humidity)=High, Normal

$$S = [9+, 5-] \quad Entropy(S) = -\frac{9}{14} \log_2 \frac{9}{14} - \frac{5}{14} \log_2 \frac{5}{14} = 0.94$$

$$S_{High} = [3+, 4-] \quad Entropy(S_{High}) = -\frac{3}{7} \log_2 \frac{3}{7} - \frac{4}{7} \log_2 \frac{4}{7} = 0.9852$$

$$S_{Normal} = [6+, 1-] \quad Entropy(S_{Normal}) = -\frac{6}{7} \log_2 \frac{6}{7} - \frac{1}{7} \log_2 \frac{1}{7} = 0.5916$$

$$Gain(S, Humidity) = Entropy(S) - \sum_{v \in (High, Normal)} \frac{|S_v|}{|S|} Entropy(S_v)$$

$$Gain(S, Humidity) = Entropy(S) - \frac{7}{14} Entropy(S_{High}) - \frac{7}{14} Entropy(S_{Normal})$$

$$Gain(S, Humidity) = 0.94 - \frac{7}{14} 0.9852 - \frac{7}{14} 0.5916 - \frac{4}{14} 0.8113 = 0.1576$$

Outlook	Temperature	Humidity	Wind	Played football(yes/no)
Sunny	Hot	High	Weak	No
Sunny	Hot	High	Strong	No
Overcast	Hot	High	Weak	Yes
Rain	Mild	High	Weak	Yes
Rain	Cool	Normal	Weak	Yes
Rain	Cool	Normal	Strong	No
Overcast	Cool	Normal	Strong	Yes
Sunny	Mild	High	Weak	No
Sunny	Cool	Normal	Weak	Yes
Rain	Mild	Normal	Weak	Yes
Sunny	Mild	Normal	Strong	Yes
Overcast	Mild	High	Strong	Yes
Overcast	Hot	Normal	Weak	Yes
Rain	Mild	High	Strong	No

Attribute: Wind

Values(Wind)=Strong, Weak

$$S = [9+, 5-] \quad Entropy(S) = -\frac{9}{14} \log_2 \frac{9}{14} - \frac{5}{14} \log_2 \frac{5}{14} = 0.94$$

$$S_{Strong} = [3+, 3-] \quad Entropy(S_{Strong}) = 1.0$$

$$S_{Weak} = [6+, 2-] \quad Entropy(S_{Weak}) = -\frac{6}{8} \log_2 \frac{6}{8} - \frac{2}{8} \log_2 \frac{2}{8} = 0.8113$$

$$Gain(S, Wind) = Entropy(S) - \sum_{v \in (Strong, Weak)} \frac{|S_v|}{|S|} Entropy(S_v)$$

$$Gain(S, Wind) = Entropy(S) - \frac{6}{14} Entropy(S_{Strong}) - \frac{8}{14} Entropy(S_{Weak})$$

$$Gain(S, Wind) = 0.94 - \frac{6}{14} 1.0 - \frac{8}{14} 0.8113 - \frac{4}{14} 0.8113 = 0.0478$$

$$\textit{Gain}(S, \textit{Outlook}) = 0.2364$$

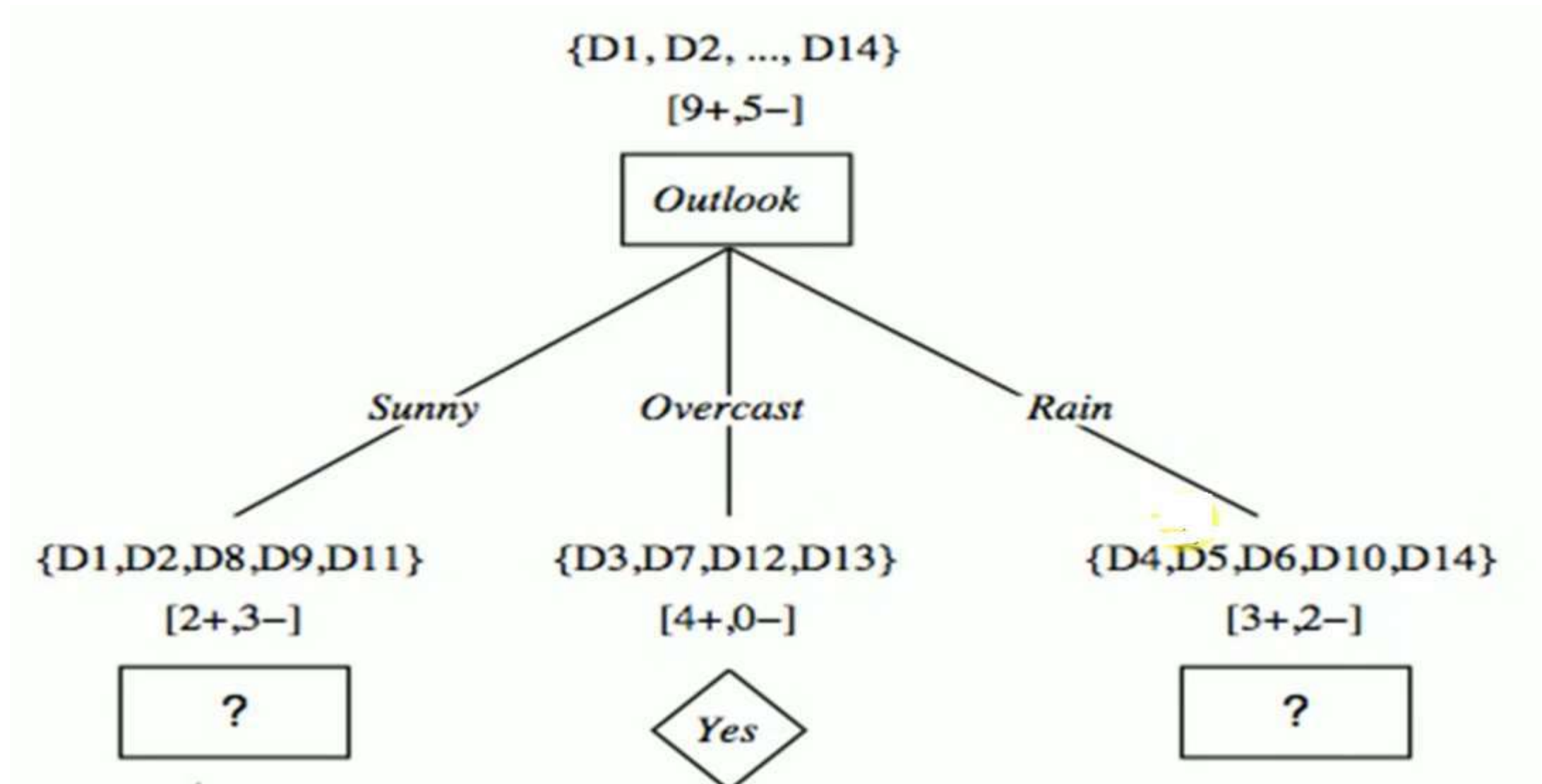
$$\textit{Gain}(S, \textit{Temp}) = 0.0289$$

$$\textit{Gain}(S, \textit{Humidity}) = 0.1576$$

$$\textit{Gain}(S, \textit{Wind}) = 0.0479$$

Maximum Information Gain is of Outlook, that is =0.2464

Now the tree will be as follows:



Now Considering Sunny:

Day	Outlook	Temp	Humidity	Wind	Play Tennis
D1	Sunny	Hot	High	Weak	No
D2	Sunny	Hot	High	Strong	No
D3	Overcast	Hot	High	Weak	Yes
D4	Rain	Mild	High	Weak	Yes
D5	Rain	Cool	Normal	Weak	Yes
D6	Rain	Cool	Normal	Strong	No
D7	Overcast	Cool	Normal	Strong	Yes
D8	Sunny	Mild	High	Weak	No
D9	Sunny	Cool	Normal	Weak	Yes
D10	Rain	Mild	Normal	Weak	Yes
D11	Sunny	Mild	Normal	Strong	Yes
D12	Overcast	Mild	High	Strong	Yes
D13	Overcast	Hot	Normal	Weak	Yes
D14	Rain	Mild	High	Strong	No

Day	Temp	Humidity	Wind	Play Tennis
D1	Hot	High	Weak	No
D2	Hot	High	Strong	No
D8	Mild	High	Weak	No
D9	Cool	Normal	Weak	Yes
D11	Mild	Normal	Strong	Yes

Attribute: Temp

Values(Temp)=Hot, Mild, Cool

$$S_{\text{sunny}} = [2+, 3-] \quad \text{Entropy}(S) = -\frac{2}{5}\log_2\frac{2}{5} - \frac{3}{5}\log_2\frac{3}{5} = 0.97$$

$$S_{\text{Hot}} = [0+, 2-] \quad \text{Entropy}(S_{\text{Hot}}) = 0.0$$

$$S_{\text{Mild}} = [1+, 1-] \quad \text{Entropy}(S_{\text{Mild}}) = 1.0$$

$$S_{\text{Cool}} = [1+, 0-] \quad \text{Entropy}(S_{\text{Cool}}) = 0.0$$

$$\text{Gain}(S_{\text{sunny}}, \text{temp}) = \text{Entropy}(S) - \sum_{v \in (\text{Hot}, \text{Mild}, \text{Cool})} \frac{|S_v|}{|S|} \text{Entropy}(S_v)$$

$$\text{Gain}(S_{\text{sunny}}, \text{temp}) = \text{Entropy}(S) - \frac{2}{5}\text{Entropy}(S_{\text{Hot}}) - \frac{2}{5}\text{Entropy}(S_{\text{Mild}}) - \frac{1}{5}\text{Entropy}(S_{\text{Cool}})$$

$$\text{Gain}(S_{\text{sunny}}, \text{temp}) = 0.97 - \frac{2}{5}0.0 - \frac{2}{5}1 - \frac{1}{5}0.0 = 0.570$$

Day	Temp	Humidity	Wind	Play Tennis
D1	Hot	High	Weak	No
D2	Hot	High	Strong	No
D8	Mild	High	Weak	No
D9	Cool	Normal	Weak	Yes
D11	Mild	Normal	Strong	Yes

Attribute: Humidity

Values(Humidity)=High, Normal

$$S_{sunny} = [2+, 3-] \quad Entropy(S) = -\frac{2}{5}\log_2\frac{2}{5} - \frac{3}{5}\log_2\frac{3}{5} = 0.97$$

$$S_{High} = [0+, 3-] \quad Entropy(S_{High}) = 0.0$$

$$S_{Normal} = [2+, 0-] \quad Entropy(S_{Normal}) = 0.0$$

$$Gain(S_{sunny}, Humidity) = Entropy(S) - \sum_{v \in (High, Normal)} \frac{|S_v|}{|S|} Entropy(S_v)$$

$$Gain(S_{sunny}, Humidity) = Entropy(S) - \frac{3}{5} Entropy(S_{High}) - \frac{2}{5} Entropy(S_{Normal})$$

$$Gain(S_{sunny}, Humidity) = 0.97 - \frac{3}{5} 0.0 - \frac{2}{5} 0.0 = 0.97$$

Day	Temp	Humidity	Wind	Play Tennis
D1	Hot	High	Weak	No
D2	Hot	High	Strong	No
D8	Mild	High	Weak	No
D9	Cool	Normal	Weak	Yes
D11	Mild	Normal	Strong	Yes

Attribute: Wind

Values(Wind)=Strong, Weak

$$S_{\text{sunny}} = [2+, 3-] \quad \text{Entropy}(S) = -\frac{2}{5}\log_2\frac{2}{5} - \frac{3}{5}\log_2\frac{3}{5} = 0.97$$

$$S_{\text{Strong}} = [1+, 1-] \quad \text{Entropy}(S_{\text{Strong}}) = 1.0$$

$$S_{\text{Weak}} = [1+, 2-] \quad \text{Entropy}(S_{\text{Weak}}) = -\frac{1}{3}\log_2\frac{1}{3} - \frac{2}{3}\log_2\frac{2}{3} = 0.9183$$

$$\text{Gain}(S_{\text{sunny}}, \text{Wind}) = \text{Entropy}(S) - \sum_{v \in (\text{Strong}, \text{Weak})} \frac{|S_v|}{|S|} \text{Entropy}(S_v)$$

$$\text{Gain}(S_{\text{sunny}}, \text{Wind}) = \text{Entropy}(S) - \frac{2}{5}\text{Entropy}(S_{\text{Strong}}) - \frac{3}{5}\text{Entropy}(S_{\text{Weak}})$$

$$\text{Gain}(S, \text{Wind}) = 0.94 - \frac{2}{5}1.0 - \frac{3}{5}0.9183 = 0.0192$$

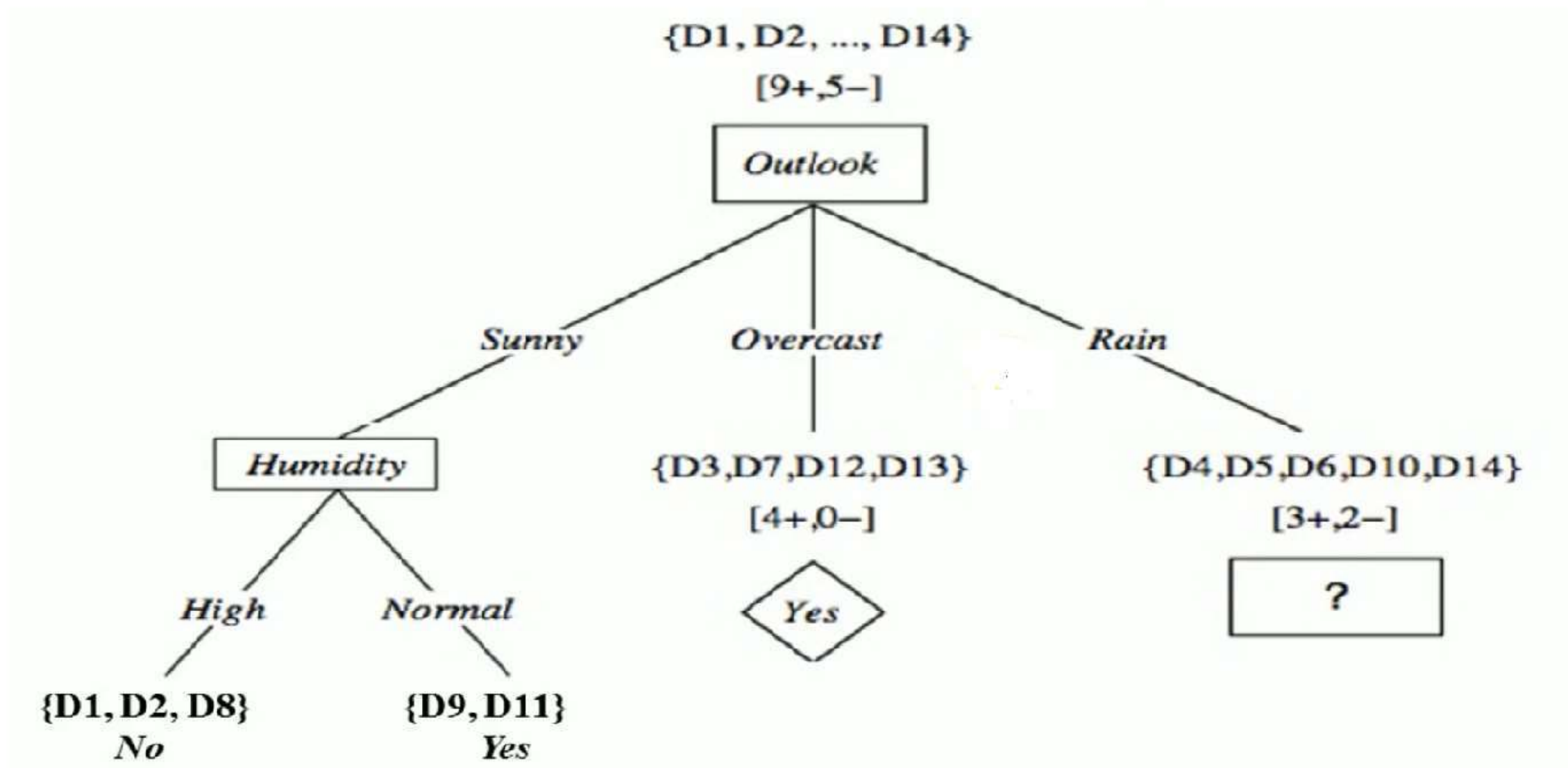
$$Gain(S_{sunny}, Temp) = 0.570$$

$$Gain(S_{sunny}, Humidity) = 0.97$$

$$Gain(S_{sunny}, Wind) = 0.0192$$

Maximum Information Gain is of Humidity, that is 0.97

So the tree can be drawn as:



Now For Rain :

Day	Outlook	Temp	Humidity	Wind	Play Tennis
D1	Sunny	Hot	High	Weak	No
D2	Sunny	Hot	High	Strong	No
D3	Overcast	Hot	High	Weak	Yes
D4	Rain	Mild	High	Weak	Yes
D5	Rain	Cool	Normal	Weak	Yes
D6	Rain	Cool	Normal	Strong	No
D7	Overcast	Cool	Normal	Strong	Yes
D8	Sunny	Mild	High	Weak	No
D9	Sunny	Cool	Normal	Weak	Yes
D10	Rain	Mild	Normal	Weak	Yes
D11	Sunny	Mild	Normal	Strong	Yes
D12	Overcast	Mild	High	Strong	Yes
D13	Overcast	Hot	Normal	Weak	Yes
D14	Rain	Mild	High	Strong	No

Day	Temp	Humidity	Wind	Play Tennis
D4	Mild	High	Weak	Yes
D5	Cool	Normal	Weak	Yes
D6	Cool	Normal	Strong	No
D10	Mild	Normal	Weak	Yes
D14	Mild	High	Strong	No

Attribute: Temp

Values(Temp)=Hot, Mild, Cool

$$S_{Rainy} = [3+, 2-] \quad Entropy(S_{Rainy}) = -\frac{3}{5}\log_2\frac{3}{5} - \frac{2}{5}\log_2\frac{2}{5} = 0.97$$

$$S_{Hot} = [0+, 0-] \quad Entropy(S_{Hot}) = 0.0$$

$$S_{Mild} = [2+, 1-] \quad Entropy(S_{Mild}) = -\frac{2}{3}\log_2\frac{2}{3} - \frac{1}{3}\log_2\frac{1}{3} = 0.9183$$

$$S_{Cool} = [1+, 1-] \quad Entropy(S_{Cool}) = 1.0$$

$$Gain(S_{rainy}, temp) = Entropy(S) - \sum_{v \in (Hot, Mild, Cool)} \frac{|S_v|}{|S|} Entropy(S_v)$$

$$Gain(S_{rainy}, temp) = Entropy(S) - \frac{0}{5} Entropy(S_{Hot}) - \frac{3}{5} Entropy(S_{Mild}) - \frac{2}{5} Entropy(S_{Cool})$$

$$Gain(S_{rainy}, temp) = 0.97 - \frac{0}{5} 0.0 - \frac{3}{5} 0.98 - \frac{2}{5} 1.0 = 0.0192$$

Day	Temp	Humidity	Wind	Play Tennis
D4	Mild	High	Weak	Yes
D5	Cool	Normal	Weak	Yes
D6	Cool	Normal	Strong	No
D10	Mild	Normal	Weak	Yes
D14	Mild	High	Strong	No

Attribute: Humidity

Values(Humidity)=High, Normal

$$S_{rainy} = [3+, 2-] \quad Entropy(S_{rainy}) = -\frac{3}{5} \log_2 \frac{3}{5} - \frac{2}{5} \log_2 \frac{2}{5} = 0.97$$

$$S_{High} = [1+, 1-] \quad Entropy(S_{High}) = 1.0$$

$$S_{Normal} = [2+, 1-] \quad Entropy(S_{Normal}) = -\frac{2}{3} \log_2 \frac{2}{3} - \frac{1}{3} \log_2 \frac{1}{3} = 0.9183$$

$$Gain(Rainy, Humidity) = Entropy(S) - \sum_{v \in (High, Normal)} \frac{|S_v|}{|S|} Entropy(S_v)$$

$$Gain(Rainy, Humidity) = Entropy(S) - \frac{2}{5} Entropy(S_{High}) - \frac{3}{5} Entropy(S_{Normal})$$

$$Gain(Rainy, Humidity) = 0.97 - \frac{2}{5} 1.0 - \frac{3}{5} 0.918 = 0.0192$$

Day	Temp	Humidity	Wind	Play Tennis
D4	Mild	High	Weak	Yes
D5	Cool	Normal	Weak	Yes
D6	Cool	Normal	Strong	No
D10	Mild	Normal	Weak	Yes
D14	Mild	High	Strong	No

Attribute: Wind

Values(Wind)=Strong, Weak

$$S_{rainy} = [3+, 2-] \quad Entropy(S_{rainy}) = -\frac{3}{5} \log_2 \frac{3}{5} - \frac{2}{5} \log_2 \frac{2}{5} = 0.97$$

$$S_{Strong} = [0+, 2-] \quad Entropy(S_{Strong}) = 0.0$$

$$S_{Weak} = [3+, 0-] \quad Entropy(S_{Weak}) = 0.0$$

$$Gain(S_{rainy}, Wind) = Entropy(S) - \sum_{v \in (Strong, Weak)} \frac{|S_v|}{|S|} Entropy(S_v)$$

$$Gain(S_{rainy}, Wind) = Entropy(S) - \frac{2}{5} Entropy(S_{Strong}) - \frac{3}{5} Entropy(S_{Weak})$$

$$Gain(S_{rainy}, Wind) = 0.97 - \frac{2}{5} 0.0 - \frac{3}{5} 0.0 = 0.97$$

$$Gain(S_{rainy}, Temp) = 0.0192$$

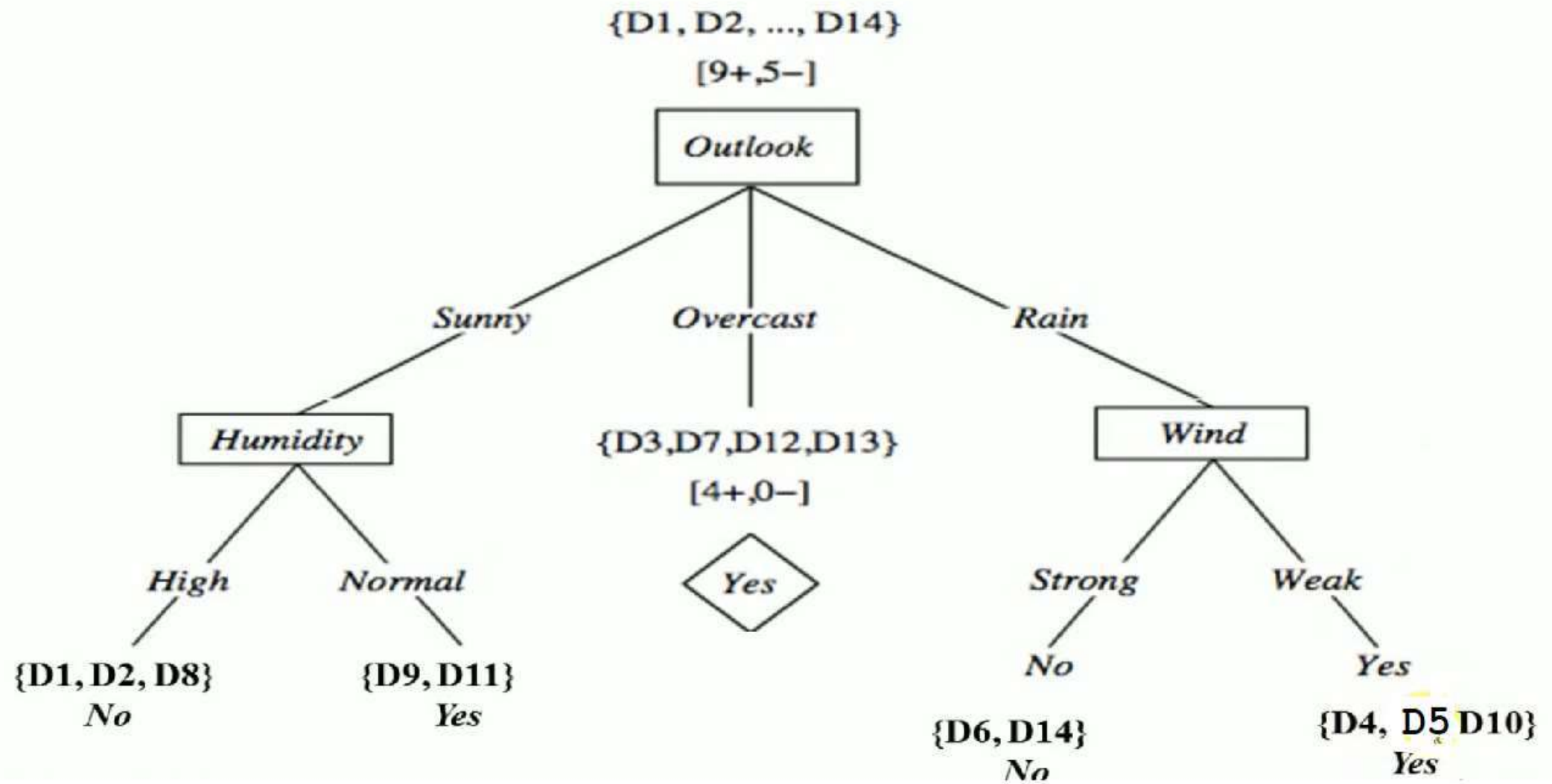
$$Gain(S_{rainy}, Humidity) = 0.0192$$

$$Gain(S_{rainy}, Wind) = 0.97$$

Therefore 0.97 is maximum

Wind will be considered at that level.

So the final tree will be:



SUPPORT VECTOR MACHINE

What is Support Vector Machine?

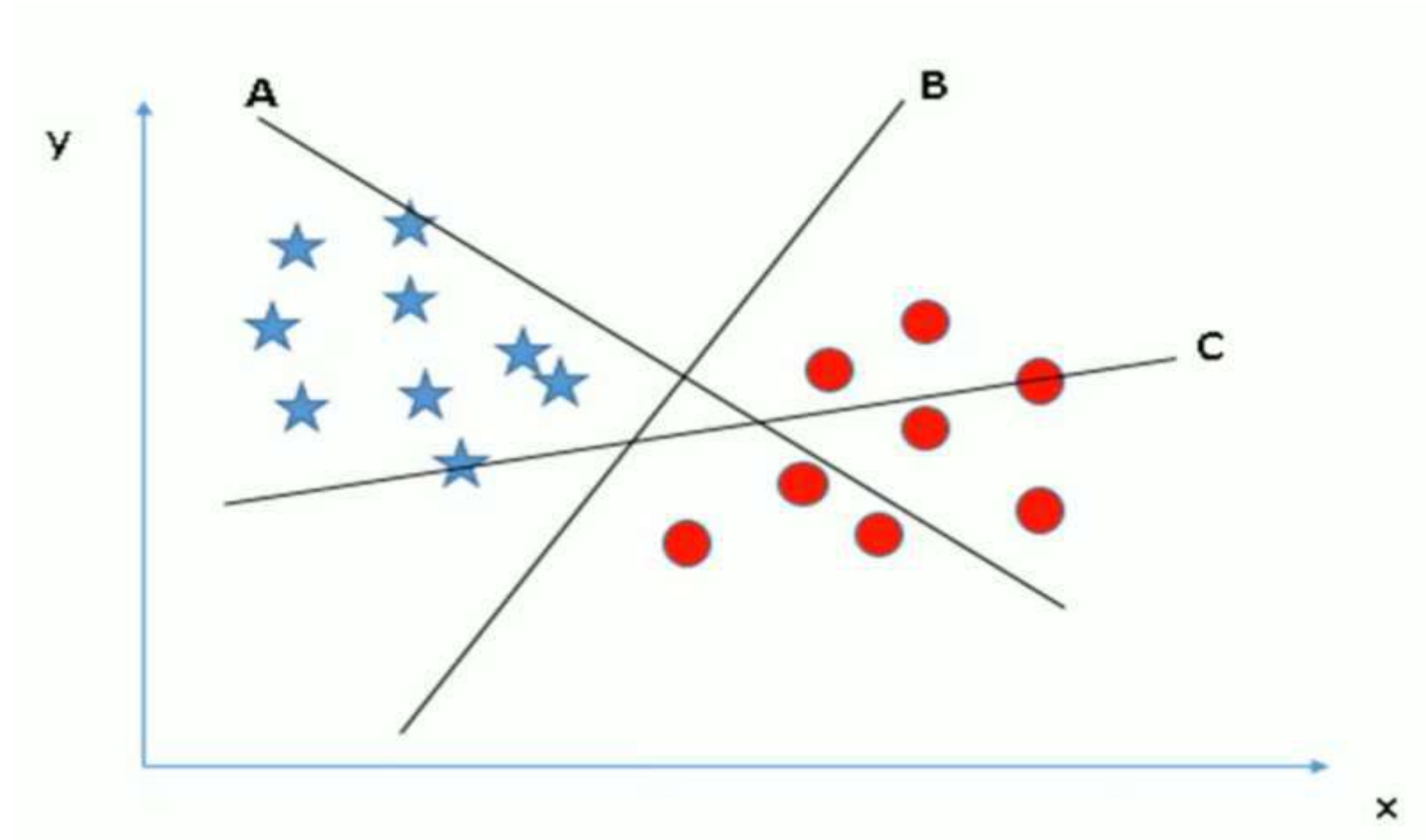
Support Vector Machine (SVM) is a supervised machine learning algorithm which can be used for both classification or regression analysis.

However it is mostly used in classification problems.

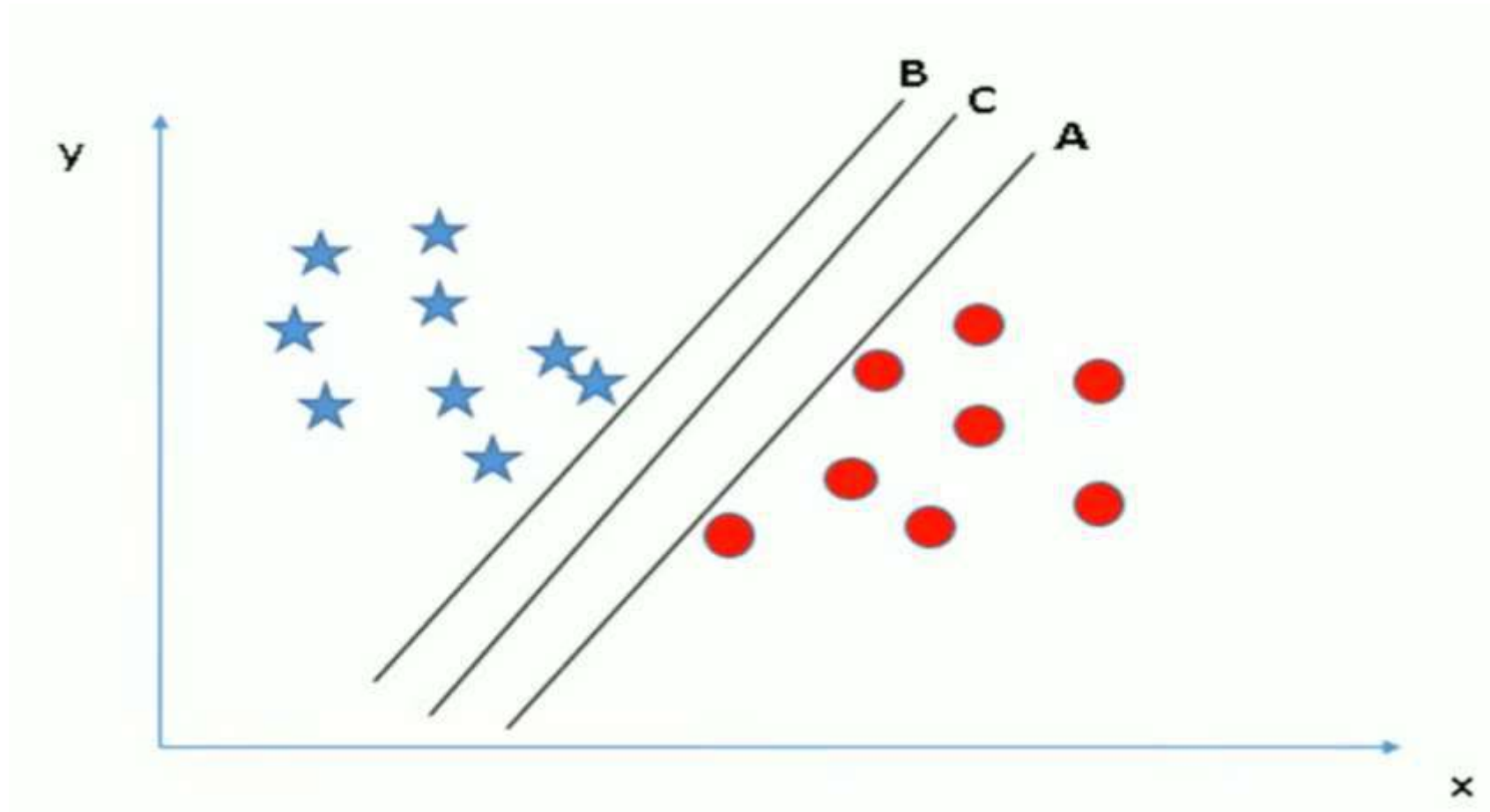
In this algorithm we plot each data item as a point in n -dimensional space (where n is the number of features you have) with the value of each feature being the value of a particular coordinate.

Then we perform classification by finding the hyper-plane that differentiate the two classes very well.

How does it work?



How does it work?



How does it work?

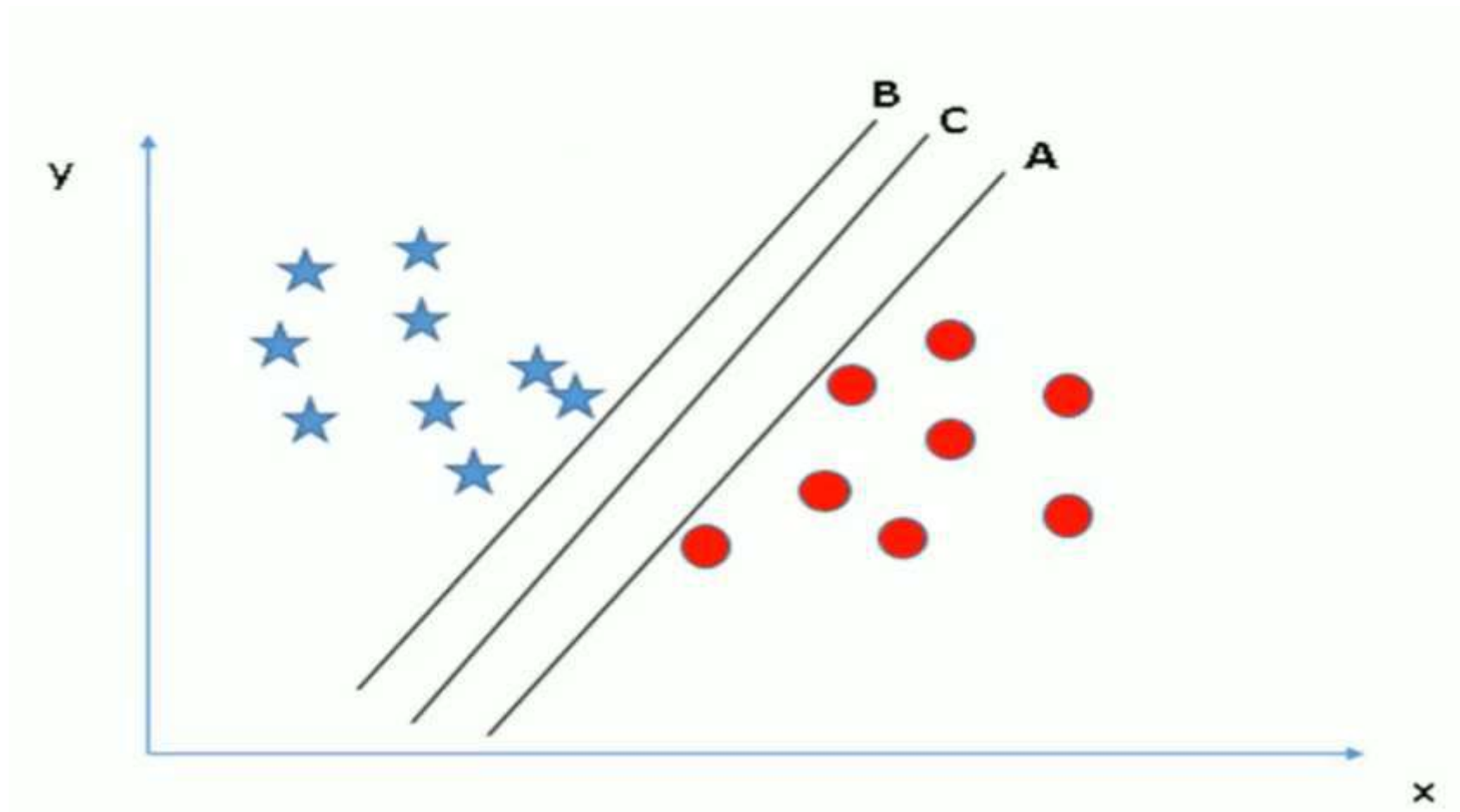
Thumb rule to identify the right hyper-plane

Select the hyper-plane that segregates the two classes better.

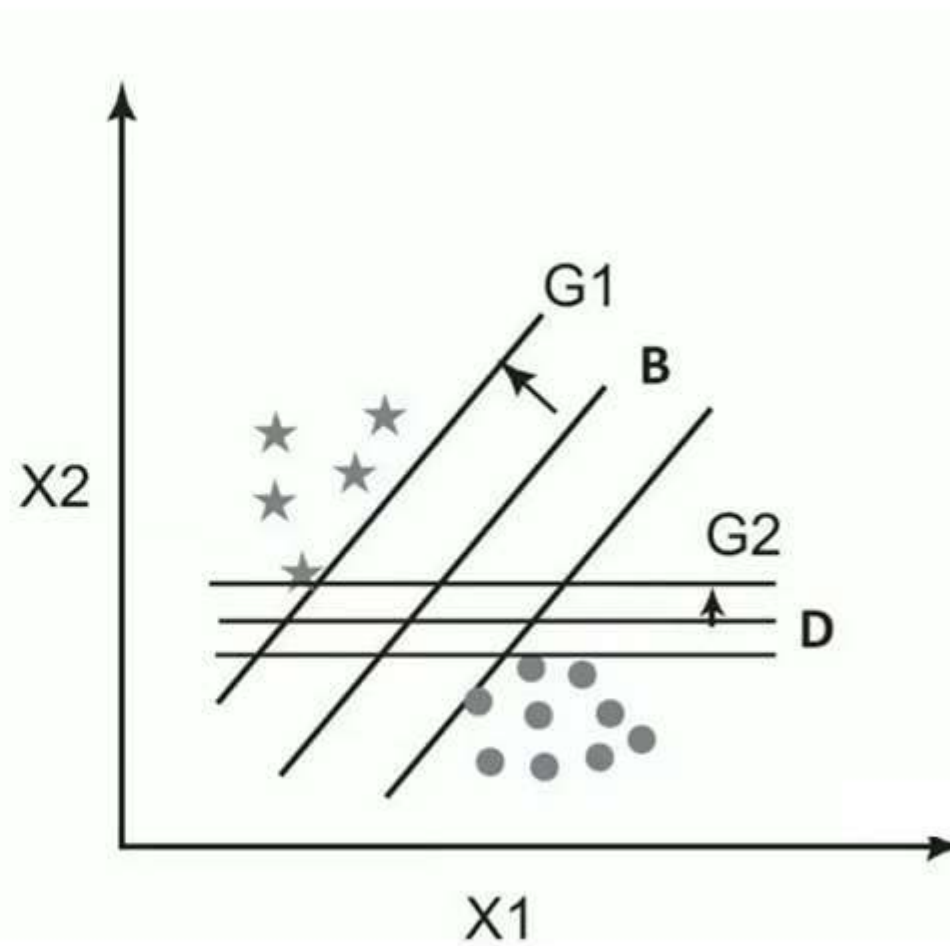
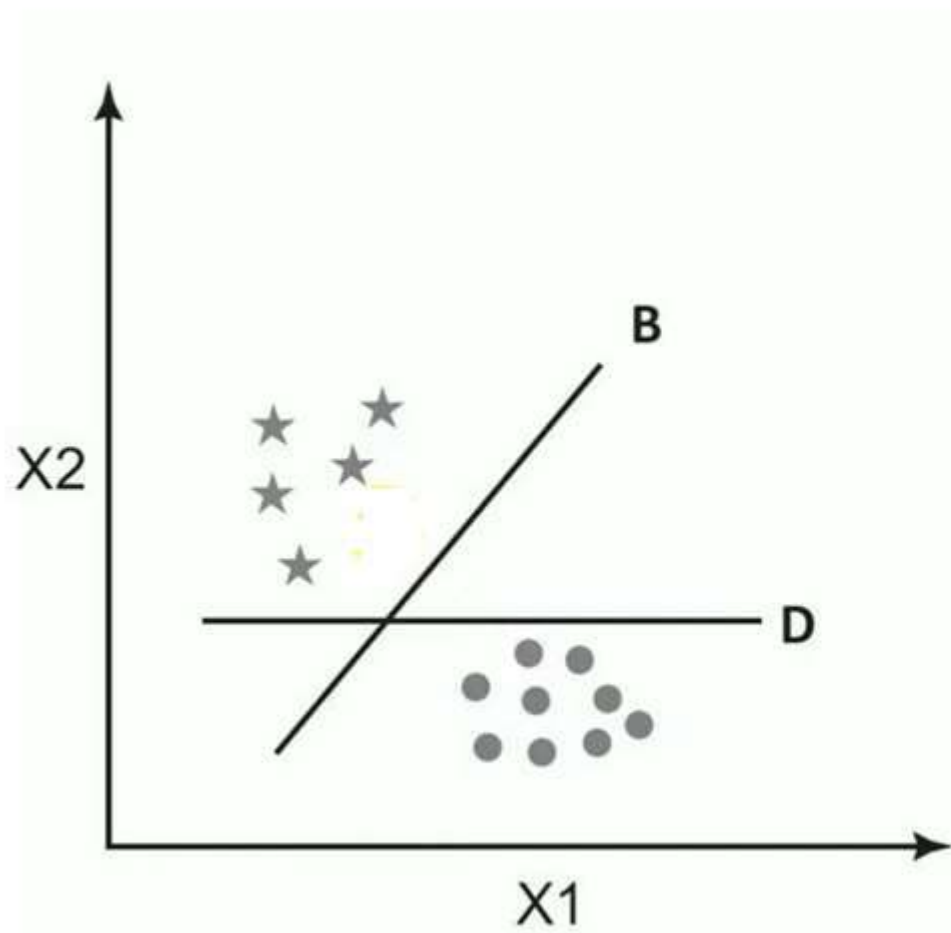
Maximizing the distances between nearest data point (either class) and the hyper-plane.

This distance is called the Margin

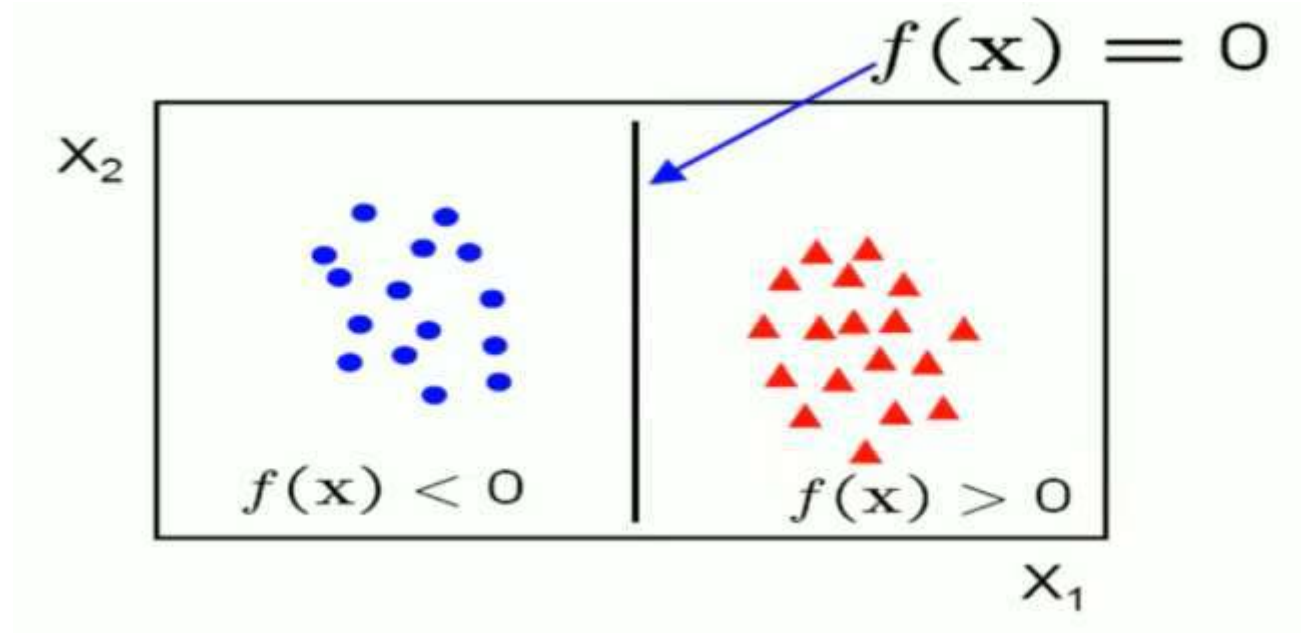
How does it work?



How does it work?



How does it work?

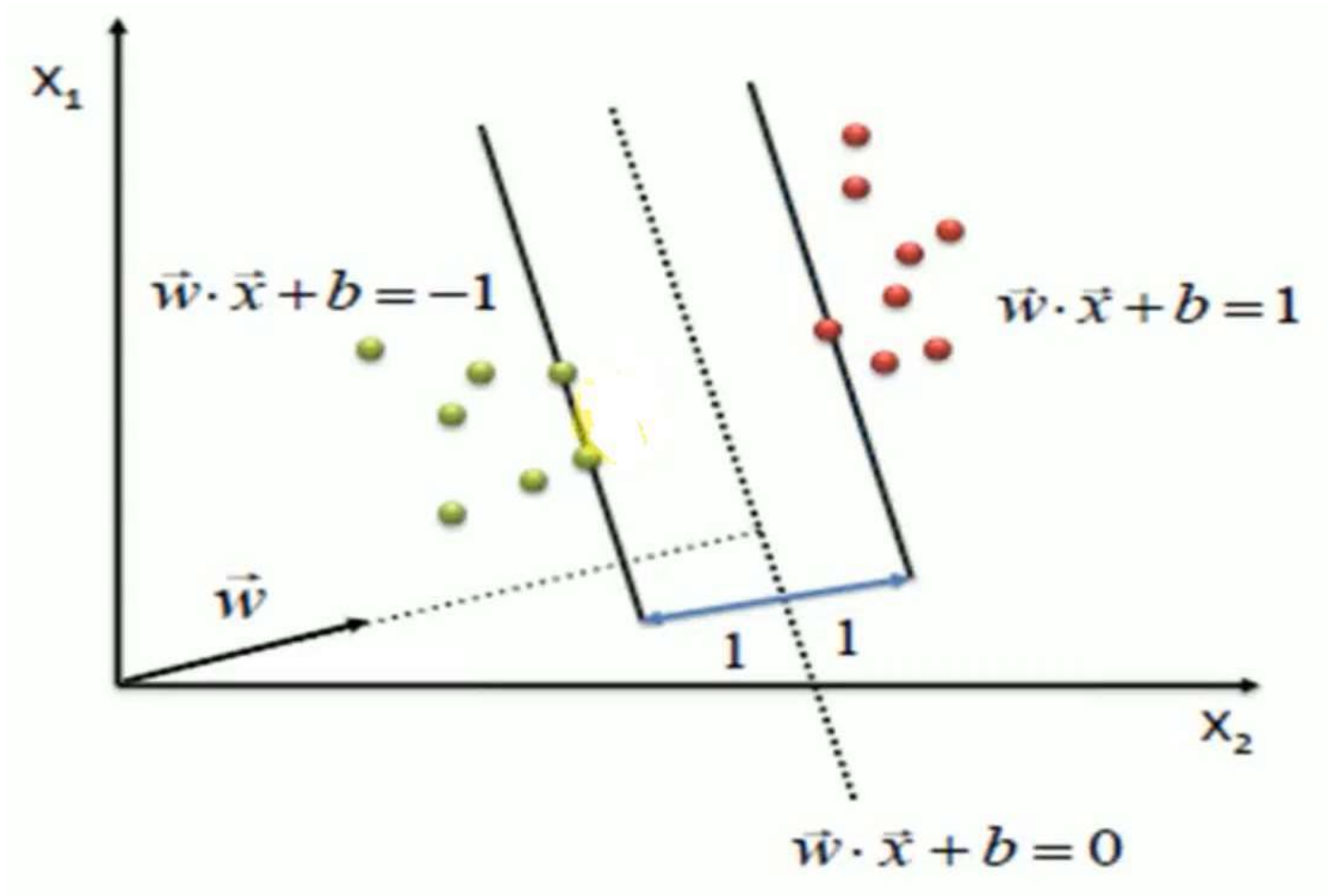


$$f(\mathbf{X}) = \mathbf{W} \cdot \mathbf{X} + b$$

$\mathbf{W}$ is the slope or normal to the line, $\mathbf{X}$ is the input vector and b is the bias or the intercept.

$\mathbf{W}$ is known as the weight vector.

How does it work?



$$\max \frac{2}{\|\vec{w}\|}$$

s.t.

$$(\vec{w} \cdot \vec{x} + b) \geq 1, \forall \vec{x} \text{ of class 1}$$

$$(\vec{w} \cdot \vec{x} + b) \leq -1, \forall \vec{x} \text{ of class 2}$$

Advantages of SVM

The main strength of SVM is that they work well even when the number of SVM features is much larger than the number of instances.

It can work on datasets with huge feature space, such is the case in spam filtering , where a large number of words are the potential signifiers of a message being spam.

Even when the optimal decision boundary is a nonlinear curve, the SVM transforms the variables to create new dimensions such that the representation of the classifier is a linear function of those transformed dimensions of data.

SVMs are conceptually easy to understand. They create an easy to understand linear classifier.

SVMs are now available with almost all data analytics toolsets.

Disadvantages of SVM

The SVM technique has two major constraints.

- it works well with real numbers, i.e, all the data points in all the dimensions must be defined by numeric values only.
- It works only with binary classification problems. One can make a series of cascaded SVMs to get around the constraint.

Training the SVMs is inefficient and time consuming process, when the data is large.

It does not work well when there is much noise in the data, and thus has to compute soft margins.

The SVMs will also not provide a probability estimate of classification, i.e, the confidence level for classifying an instance.

Applications of SVMs

Classification

Regression Analysis

Pattern Recognition

Outliers detection

Relevance based applications

Linear Support Vector Machine

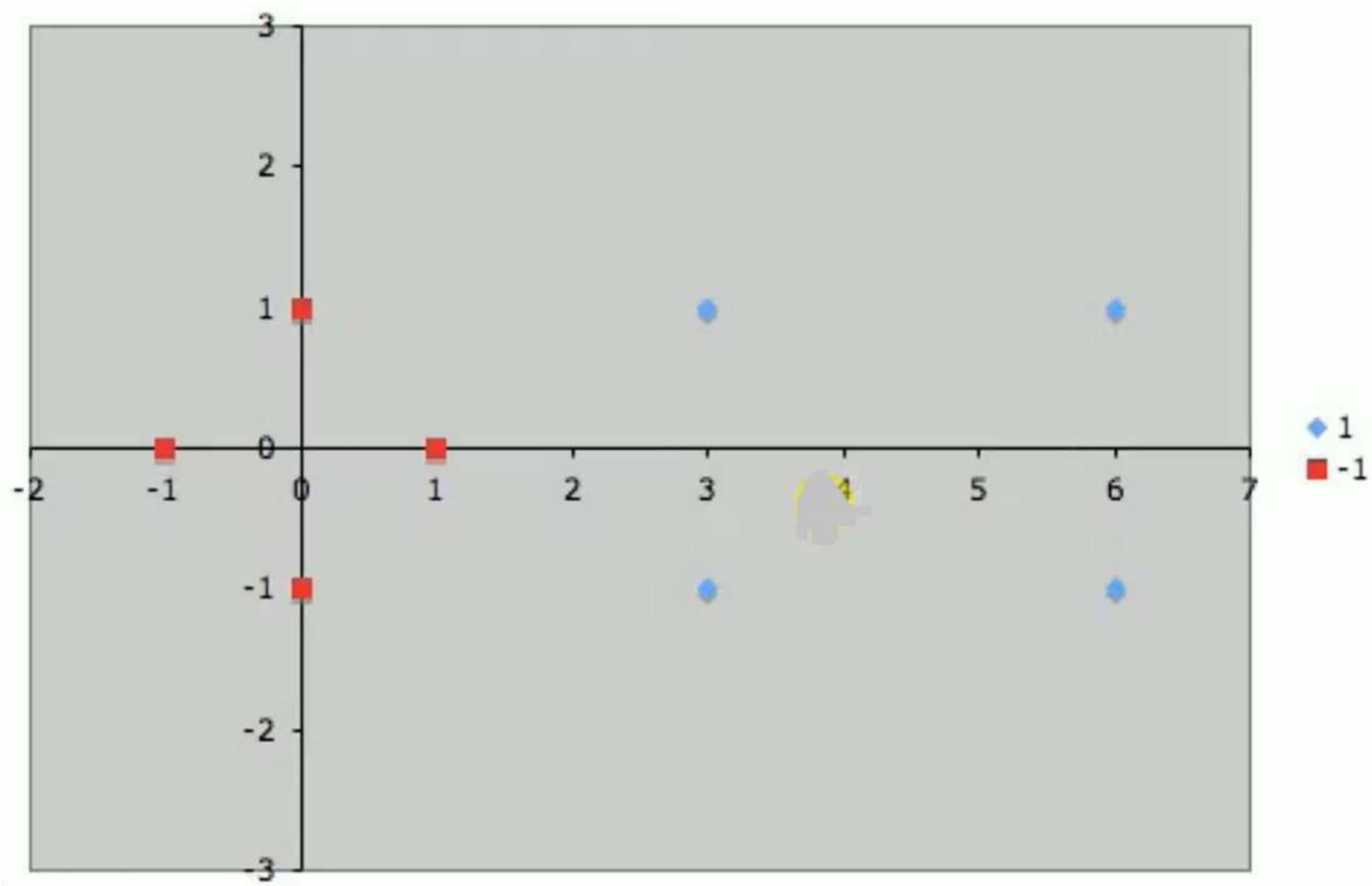
Support Vector Machine – Linear Example

Suppose we are given the following positively labeled data points,

$$\left\{ \begin{pmatrix} 3 \\ 1 \end{pmatrix}, \begin{pmatrix} 3 \\ -1 \end{pmatrix}, \begin{pmatrix} 6 \\ 1 \end{pmatrix}, \begin{pmatrix} 6 \\ -1 \end{pmatrix} \right\}$$

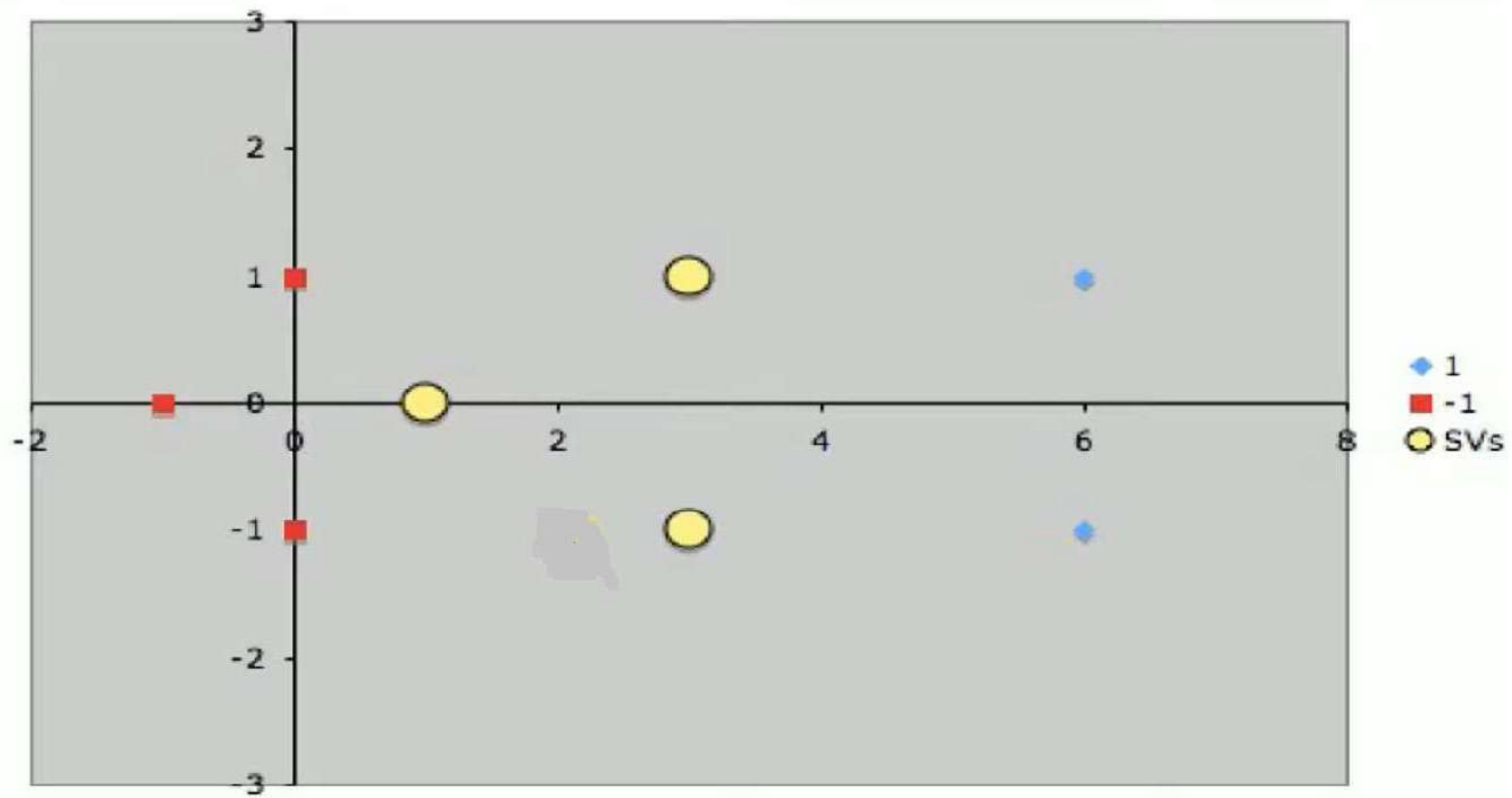
and the following negatively labeled data points,

$$\left\{ \begin{pmatrix} 1 \\ 0 \end{pmatrix}, \begin{pmatrix} 0 \\ 1 \end{pmatrix}, \begin{pmatrix} 0 \\ -1 \end{pmatrix}, \begin{pmatrix} -1 \\ 0 \end{pmatrix} \right\}$$



- By inspection, it should be obvious that there are **three** support vectors,

$$\left\{ s_1 = \begin{pmatrix} 1 \\ 0 \end{pmatrix}, s_2 = \begin{pmatrix} 3 \\ 1 \end{pmatrix}, s_3 = \begin{pmatrix} 3 \\ -1 \end{pmatrix} \right\}$$



- Each vector is augmented with a 1 as a bias input

- So, $s_1 = \begin{pmatrix} 1 \\ 0 \end{pmatrix}$, then $\tilde{s}_1 = \begin{pmatrix} 1 \\ 0 \\ 1 \end{pmatrix}$

- Similarly,

- $s_2 = \begin{pmatrix} 3 \\ 1 \end{pmatrix}$, then $\tilde{s}_2 = \begin{pmatrix} 3 \\ 1 \\ 1 \end{pmatrix}$ and $s_3 = \begin{pmatrix} 3 \\ -1 \end{pmatrix}$, then $\tilde{s}_3 = \begin{pmatrix} 3 \\ -1 \\ 1 \end{pmatrix}$

Support Vector Machine – Linear Example

$$\alpha_1 \tilde{s}_1 \cdot \tilde{s}_1 + \alpha_2 \tilde{s}_2 \cdot \tilde{s}_1 + \alpha_3 \tilde{s}_3 \cdot \tilde{s}_1 = -1$$

$$\alpha_1 \tilde{s}_1 \cdot \tilde{s}_2 + \alpha_2 \tilde{s}_2 \cdot \tilde{s}_2 + \alpha_3 \tilde{s}_3 \cdot \tilde{s}_2 = +1$$

$$\alpha_1 \tilde{s}_1 \cdot \tilde{s}_3 + \alpha_2 \tilde{s}_2 \cdot \tilde{s}_3 + \alpha_3 \tilde{s}_3 \cdot \tilde{s}_3 = +1$$

$$\alpha_1(1+0+1)+\alpha_2(3+0+1)+\alpha_3(3+0+1)=-1$$

$$\alpha_1(3+0+1)+\alpha_2(9+1+1)+\alpha_3(9-1+1)=1$$

$$\alpha_1(3+0+1)+\alpha_2(9-1+1)+\alpha_3(9+1+1)=1$$

$$2\alpha_1+4\alpha_2+4\alpha_3=-1$$

$$4\alpha_1+11\alpha_2+9\alpha_3=1$$

$$4\alpha_1+9\alpha_2+11\alpha_3=1$$

$$\alpha_1 \begin{pmatrix} 1 \\ 0 \\ 1 \end{pmatrix} \begin{pmatrix} 1 \\ 0 \\ 1 \end{pmatrix} + \alpha_2 \begin{pmatrix} 3 \\ 1 \\ 1 \end{pmatrix} \begin{pmatrix} 1 \\ 0 \\ 1 \end{pmatrix} + \alpha_3 \begin{pmatrix} 3 \\ -1 \\ 1 \end{pmatrix} \begin{pmatrix} 1 \\ 0 \\ 1 \end{pmatrix} = -1$$

$$\alpha_1 \begin{pmatrix} 1 \\ 0 \\ 1 \end{pmatrix} \begin{pmatrix} 3 \\ 1 \\ 1 \end{pmatrix} + \alpha_2 \begin{pmatrix} 3 \\ 1 \\ 1 \end{pmatrix} \begin{pmatrix} 3 \\ 1 \\ 1 \end{pmatrix} + \alpha_3 \begin{pmatrix} 3 \\ -1 \\ 1 \end{pmatrix} \begin{pmatrix} 3 \\ 1 \\ 1 \end{pmatrix} = 1$$

$$\alpha_1 \begin{pmatrix} 1 \\ 0 \\ 1 \end{pmatrix} \begin{pmatrix} 3 \\ -1 \\ 1 \end{pmatrix} + \alpha_2 \begin{pmatrix} 3 \\ 1 \\ 1 \end{pmatrix} \begin{pmatrix} 3 \\ -1 \\ 1 \end{pmatrix} + \alpha_3 \begin{pmatrix} 3 \\ -1 \\ 1 \end{pmatrix} \begin{pmatrix} 3 \\ -1 \\ 1 \end{pmatrix} = 1$$

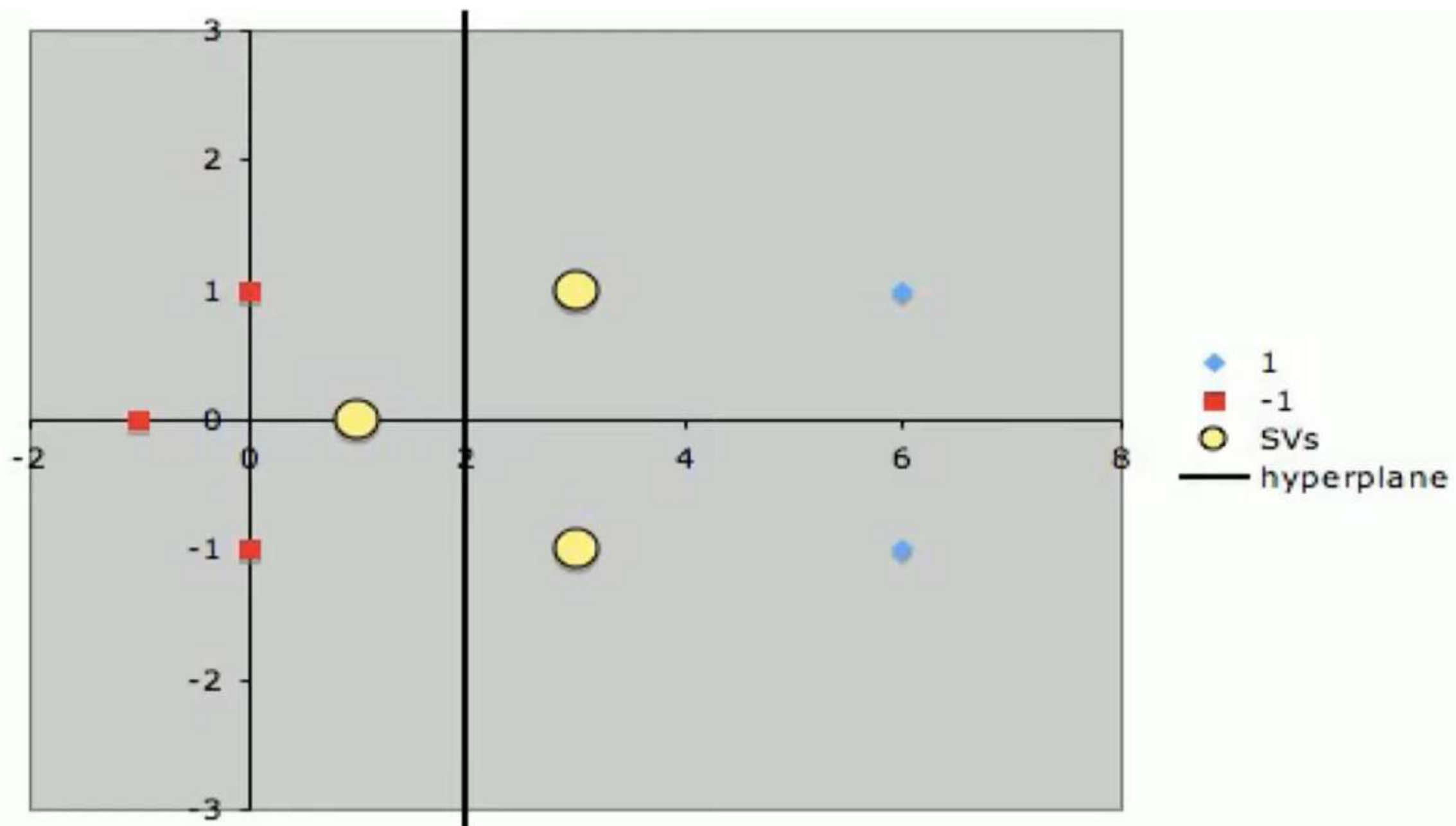
$$\alpha_1 = -3.5$$

$$\alpha_2 = 0.75$$

$$\alpha_3 = 0.75$$

$$\begin{aligned}\tilde{w} &= \sum_i \alpha_i \tilde{s}_i \\ &= -3.5 \begin{pmatrix} 1 \\ 0 \\ 1 \end{pmatrix} + 0.75 \begin{pmatrix} 3 \\ 1 \\ 1 \end{pmatrix} + 0.75 \begin{pmatrix} 3 \\ -1 \\ 1 \end{pmatrix} \\ &= \begin{pmatrix} 1 \\ 0 \\ -2 \end{pmatrix}\end{aligned}$$

- Finally, remembering that our vectors are augmented with a bias.
- We can equate the last entry in $\tilde{w}$ as the hyperplane offset b and write the separating
- Hyperplane equation $\mathbf{y} = \mathbf{w}\mathbf{x} + \mathbf{b}$
- with $\mathbf{w} = \begin{pmatrix} 1 \\ 0 \end{pmatrix}$ and $\mathbf{b} = -2$.



- Let's take the 5 support vector version
- $\tilde{w} = [0 \ 1 \ -1]$. This is a horizontal line passing through $x_2=1$.
- Let's classify the point $(x_1, x_2)=(4, 2)$.
- $w \cdot x = \begin{pmatrix} 0 \\ 1 \end{pmatrix} \cdot \begin{pmatrix} 4 \\ 2 \end{pmatrix} = 2 > 1$
- Hence this point belongs to the red class
- Let's classify the point $(x_1, x_2)=(2, -2)$.
- $w \cdot x = \begin{pmatrix} 0 \\ 1 \end{pmatrix} \cdot \begin{pmatrix} 2 \\ -2 \end{pmatrix} = -2 < 1$
- Hence this point belongs to the blue class
- We can do the same for any new point.

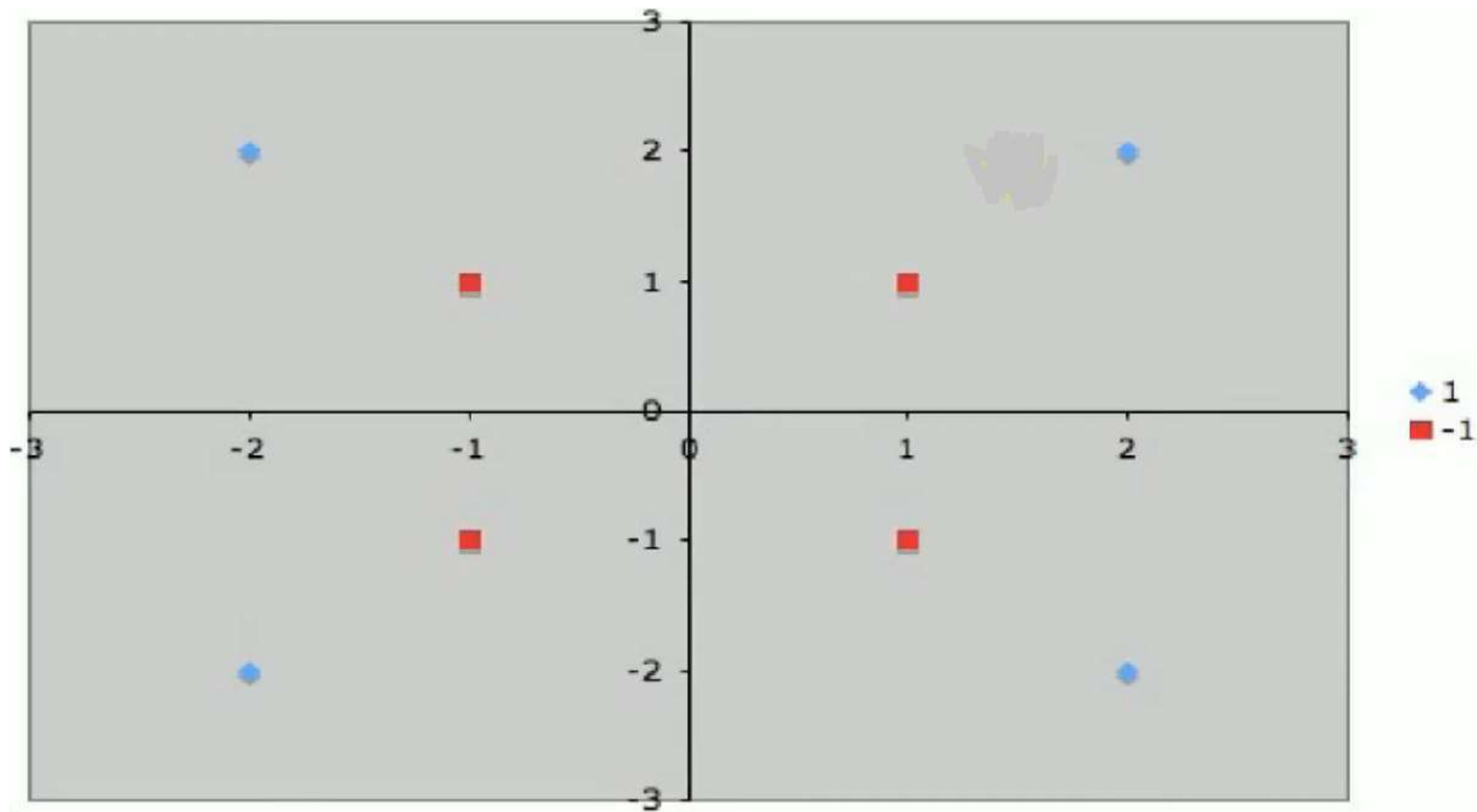
NON-LINEAR SUPPORT VECTOR MACHINE

Suppose we are given the following positively labeled data points,

$$\left\{ \begin{pmatrix} 2 \\ 2 \end{pmatrix}, \begin{pmatrix} 2 \\ -2 \end{pmatrix}, \begin{pmatrix} -2 \\ -2 \end{pmatrix}, \begin{pmatrix} -2 \\ 2 \end{pmatrix} \right\}$$

and the following negatively labeled data points,

$$\left\{ \begin{pmatrix} 1 \\ 1 \end{pmatrix}, \begin{pmatrix} 1 \\ -1 \end{pmatrix}, \begin{pmatrix} -1 \\ -1 \end{pmatrix}, \begin{pmatrix} -1 \\ 1 \end{pmatrix} \right\}$$



- Our goal, again, is to discover a separating hyperplane that accurately discriminates the two classes.
- Of course, it is obvious that no such hyperplane exists in the input space
- Therefore, we must use a nonlinear SVM (that is, we need to convert data from one feature space to another.

$$\Phi_1 \begin{pmatrix} x_1 \\ x_2 \end{pmatrix} = \begin{cases} \begin{pmatrix} 4 - x_2 + |x_1 - x_2| \\ 4 - x_1 + |x_1 - x_2| \end{pmatrix} & \text{if } \sqrt{x_1^2 + x_2^2} > 2 \\ \begin{pmatrix} x_1 \\ x_2 \end{pmatrix} & \text{otherwise} \end{cases}$$

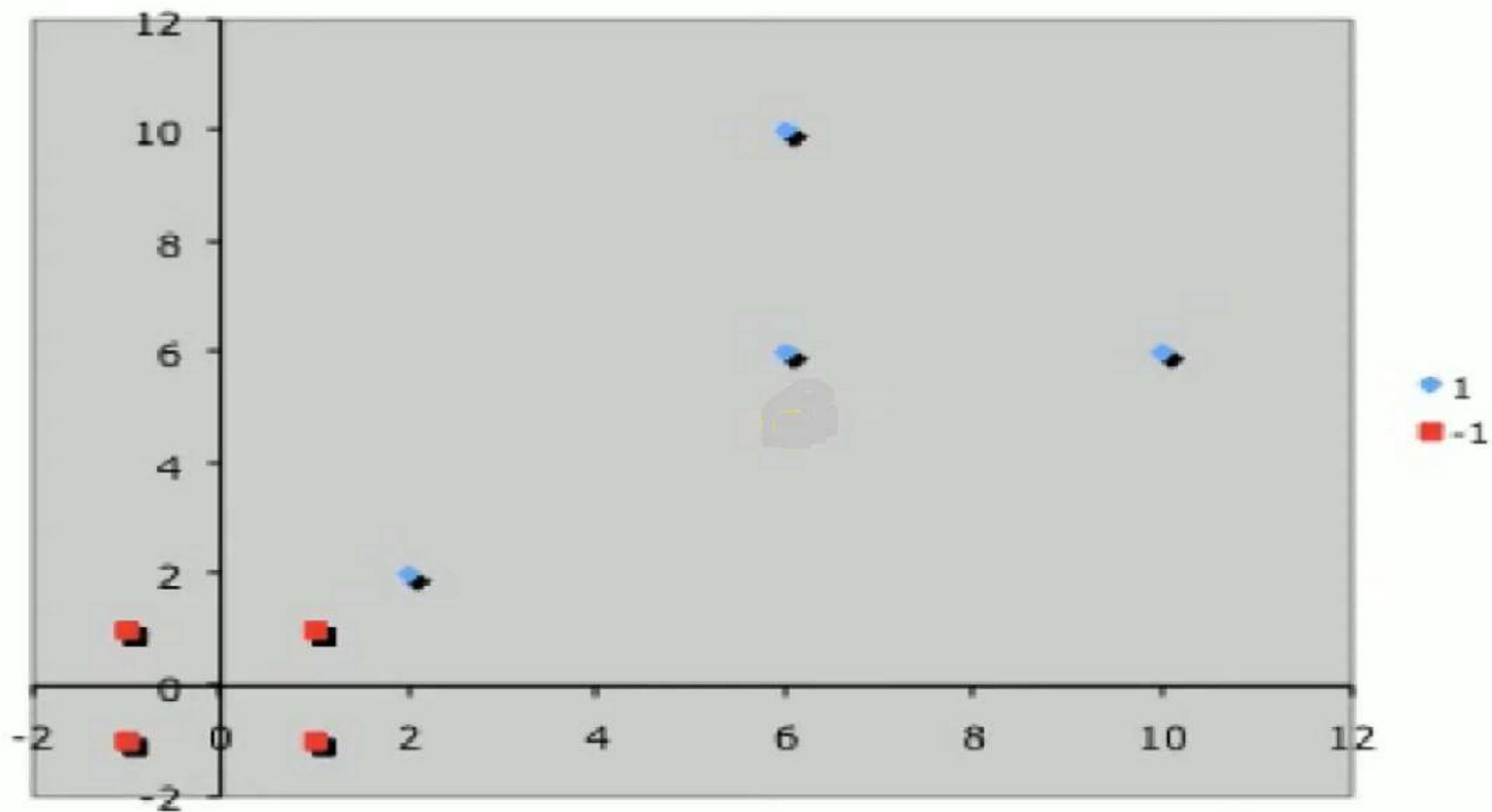
$$\Phi_1 \begin{pmatrix} x_1 \\ x_2 \end{pmatrix} = \begin{cases} \begin{pmatrix} 4 - x_2 + |x_1 - x_2| \\ 4 - x_1 + |x_1 - x_2| \end{pmatrix} & \text{if } \sqrt{x_1^2 + x_2^2} > 2 \\ \begin{pmatrix} x_1 \\ x_2 \end{pmatrix} & \text{otherwise} \end{cases}$$

Positive Examples

$$\left\{ \begin{pmatrix} 2 \\ 2 \end{pmatrix}, \begin{pmatrix} 2 \\ -2 \end{pmatrix}, \begin{pmatrix} -2 \\ -2 \end{pmatrix}, \begin{pmatrix} -2 \\ 2 \end{pmatrix} \right\} \rightarrow \left\{ \binom{2}{2}, \binom{10}{6}, \binom{6}{6}, \binom{6}{10} \right\}$$

Negative Examples

$$\left\{ \begin{pmatrix} 1 \\ 1 \end{pmatrix}, \begin{pmatrix} 1 \\ -1 \end{pmatrix}, \begin{pmatrix} -1 \\ -1 \end{pmatrix}, \begin{pmatrix} -1 \\ 1 \end{pmatrix} \right\} \rightarrow \left\{ \begin{pmatrix} 1 \\ 1 \end{pmatrix}, \begin{pmatrix} 1 \\ -1 \end{pmatrix}, \begin{pmatrix} -1 \\ -1 \end{pmatrix}, \begin{pmatrix} -1 \\ 1 \end{pmatrix} \right\}$$



- Now we can easily identify the support vectors,

$$\left\{ s_1 = \begin{pmatrix} 1 \\ 1 \end{pmatrix}, s_2 = \begin{pmatrix} 2 \\ 2 \end{pmatrix} \right\}$$

- Each vector is augmented with a 1 as a bias input

$$\tilde{s}_1 = \begin{pmatrix} 1 \\ 1 \\ 1 \end{pmatrix} \quad \tilde{s}_2 = \begin{pmatrix} 2 \\ 2 \\ 1 \end{pmatrix}$$

$$\alpha_1 \tilde{s}_1 \cdot \tilde{s}_1 + \alpha_2 \tilde{s}_2 \cdot \tilde{s}_1 = -1 \quad \alpha_1(1+1+1) + \alpha_2(2+2+1) = -1$$

$$\alpha_1 \tilde{s}_1 \cdot \tilde{s}_2 + \alpha_2 \tilde{s}_2 \cdot \tilde{s}_2 = +1 \quad \alpha_1(2+2+1) + \alpha_2(4+4+1) = 1$$

$$\alpha_1 \begin{pmatrix} 1 \\ 1 \\ 1 \end{pmatrix} \begin{pmatrix} 1 \\ 1 \\ 1 \end{pmatrix} + \alpha_2 \begin{pmatrix} 2 \\ 2 \\ 1 \end{pmatrix} \begin{pmatrix} 1 \\ 1 \\ 1 \end{pmatrix} = -1$$

$$3\alpha_1 + 5\alpha_2 = -1$$

$$5\alpha_1 + 9\alpha_2 = 1$$

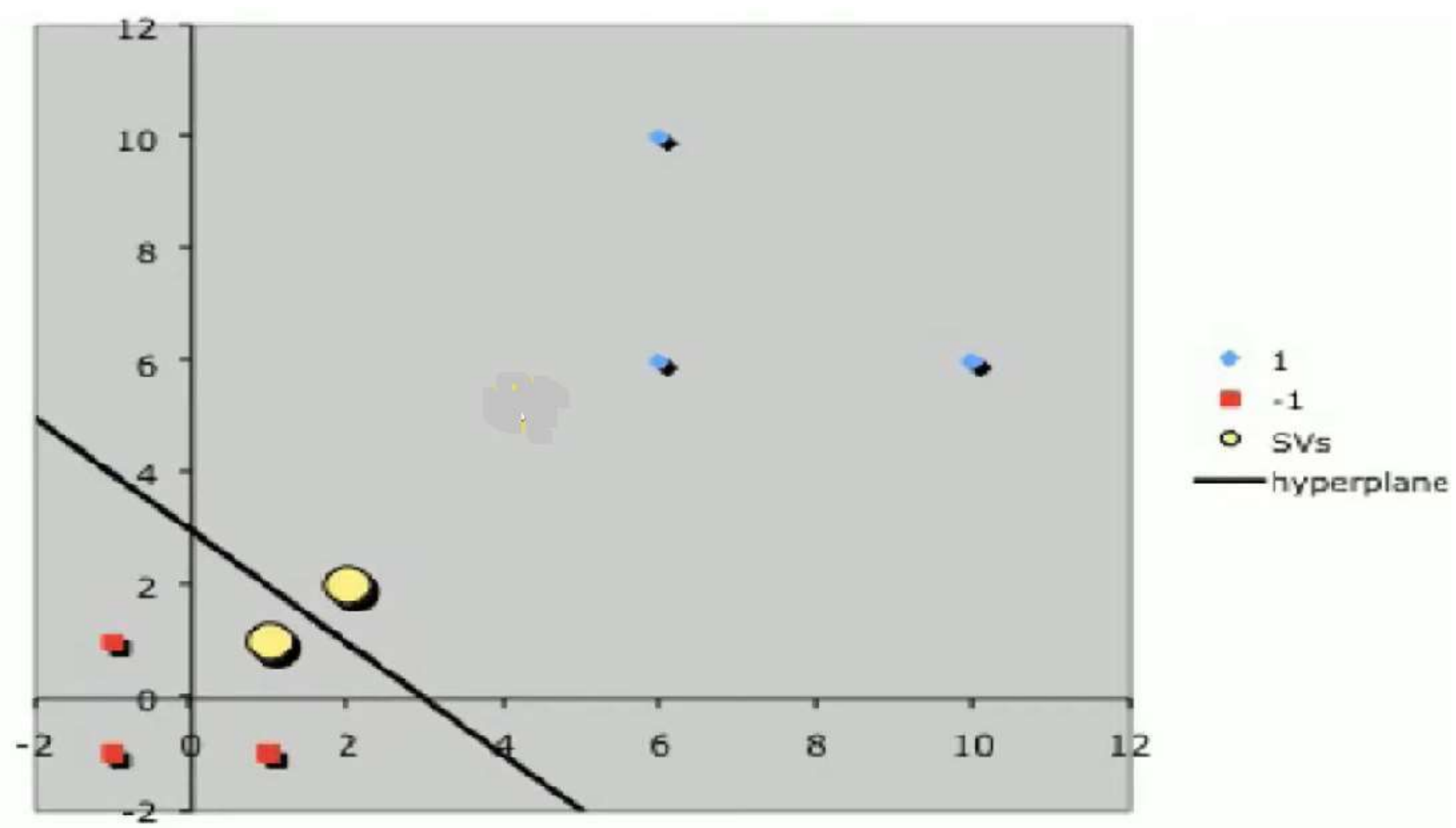
$$\alpha_1 \begin{pmatrix} 1 \\ 1 \\ 1 \end{pmatrix} \begin{pmatrix} 2 \\ 2 \\ 1 \end{pmatrix} + \alpha_2 \begin{pmatrix} 2 \\ 2 \\ 1 \end{pmatrix} \begin{pmatrix} 2 \\ 2 \\ 1 \end{pmatrix} = 1$$

$$\alpha_1 = -7$$

$$\alpha_2 = 4$$

$$\begin{aligned}\tilde{w} &= \sum_i \alpha_i \tilde{s}_i = -7 \begin{pmatrix} 1 \\ 1 \\ 1 \end{pmatrix} + 4 \begin{pmatrix} 2 \\ 2 \\ 1 \end{pmatrix} \\ &= \begin{pmatrix} 1 \\ 1 \\ -3 \end{pmatrix}\end{aligned}$$

- Finally, remembering that our vectors are augmented with a bias.
- We can equate the last entry in $\tilde{w}$ as the hyperplane offset b and write the separating
- Hyperplane equation $\mathbf{y} = \mathbf{w}\mathbf{x} + \mathbf{b}$
- with $\mathbf{w} = \begin{pmatrix} 1 \\ 1 \end{pmatrix}$ and $\mathbf{b} = -3$.



NAÏVE BAYES

NAÏVE BAYES

Naïve Bayes Analysis

- Naive-Bayes (NB) technique is a supervised learning technique that uses probability-theory-based analysis.
- It is a machine-learning technique that computes the probabilities of an instance belonging to each one of many target classes, given the prior probabilities of classification using individual factors.
- Naïve Bayes technique is used often in classifying text documents into one of multiple predefined categories.

- NB algorithm is easy to understand and works fast.
- It also performs well in multiclass prediction, such as when the target class has multiple options beyond binary yes/no classification.
- NB can perform well even in case of categorical input variables compared to numerical variable(s)

- In the abstract, Naive-Bayes is a conditional probability model for classification purposes.
- The goal is to find a way to predict the class variable (Y) using a vector of independent variables i.e., finding the function $f: X \rightarrow Y$.
- In probability terms, the goal is to find $P(Y|X)$, i.e., the probability of Y belonging to a certain class X .
- Y is generally assumed to be a categorical variable with two or more discrete values.

- Given an instance to be classified, represented by a vector $x = (x_1, x_2, \dots, x_n)$ representing 'n' features (independent variables), the Naive-Bayes model assigns, to an instance, probabilities of belonging to any of the K classes.
- The class K with the highest posterior probability is the label assigned to the instance.

- The posterior probability (of belonging to a Class K) is calculated as a function of prior probabilities and current likelihood value, as shown in the equation,

$$p(c_k|x) = \frac{p(c_k) * p(x|c_k)}{p(x)}$$

- $p(c_k|x)$ is the posterior probability of class K, given predictor X.
- $p(c_k)$ is the prior probability of class K.
- $p(x)$ is the prior probability of predictor.
- $p(x|c_k)$ is the current likelihood of predictor given class.

Naïve Bayes Model – Simple Classification Example

- Suppose a **XYZ** company needs to predict the service required by the incoming customer.
- If there are only two services offered — R and M.
- Then the value to be predicted is whether the next customer will be for R or M.
- The number of classes k are 2 that is, $k=2$.

- The first step is to compute prior probability.
- Suppose the data gathered for the last one year showed that during that period there were 2500 customers for R and 1500 customers for M.
- Thus, prior probability for the next customer to be for R is $2500/4000$ or $5/8$.
- Prior probability for the next customer to be for M is $1500/4000$ or $3/8$.
- Based on this information alone, the next customer would likely be for R.

- Another way to predict the service requirement by the next customer is to look at the most recent data.
- One can look at the last few (choose a number) customers, to predict the next customer.
- Suppose the last five customers were for the services ... R, M, R, M, M order.
- Thus, the data shows the recent probability of R is $2/5$ and that of M is $3/5$.
- Based on just this information, the next customer will likely to be for M.

- Thomas Bayes suggested that the prior probability should be informed by the more recent data.
- Naive-Bayes posterior probability for a class is computed by multiplying the prior probability and the recent probability.
- NB posterior probability $P(R)$ is $(5/8 \times 2/5) = 10/40$.
- Similarly, the NB probability $P(M)$ is $(3/8 \times 3/5) = 9/40$.
- Since $P(R)$ is greater than $P(M)$, it follows that there is a greater probability of the next customer to be for R.
- Thus the expected class label assigned to the next customer would be R.

- Suppose, however the next customer coming in was for M service.
- The last five customer sequence now becomes M, R, M, M, M.
- Thus, the recent data shows the probability for R to be $1/5$ and that of M to be $4/5$.
- Now the NB probability for R is $(5/8 \times 1/5) = 5/40$.
- Similarly, the NB probability for M is $(3/8 \times 4/5) = 12/40$.
- Since $P(M)$ is greater than $P(R)$, it follows that there is a greater probability of the next customer to be for M.
- Thus the expected class label assigned to the next customer is M.

Naïve Bayes Model – Advantages and Disadvantages

- The NB logic is simple and so is the NB posterior probability computation.
- Conditional probabilities can be computed for discrete data and for probabilistic distributions.
- Naive-Bayes assumes that all the features are independent for most instances that work fine. However, it can be a limitation. If there are no joint occurrences at all of a class label with a certain attribute, then the frequency-based conditional probability will be zero.
- A limitation of NB is that the posterior probability computations are good for comparison and classification of the instances. However, the probability values themselves are not good estimates of the event happening.

NAÏVE BAYES ALGORITHM

<i>Day</i>	<i>Outlook</i>	<i>Temperature</i>	<i>Humidity</i>	<i>Wind</i>	<i>PlayTennis</i>
D1	Sunny	Hot	High	Weak	No
D2	Sunny	Hot	High	Strong	No
D3	Overcast	Hot	High	Weak	Yes
D4	Rain	Mild	High	Weak	Yes
D5	Rain	Cool	Normal	Weak	Yes
D6	Rain	Cool	Normal	Strong	No
D7	Overcast	Cool	Normal	Strong	Yes
D8	Sunny	Mild	High	Weak	No
D9	Sunny	Cool	Normal	Weak	Yes
D10	Rain	Mild	Normal	Weak	Yes
D11	Sunny	Mild	Normal	Strong	Yes
D12	Overcast	Mild	High	Strong	Yes
D13	Overcast	Hot	Normal	Weak	Yes
D14	Rain	Mild	High	Strong	No

(Outlook = sunny, Temperature = cool, Humidity = high, Wind = strong)

Day	Outlook	Temperature	Humidity	Wind	PlayTennis
D1	Sunny	Hot	High	Weak	No
D2	Sunny	Hot	High	Strong	No
D3	Overcast	Hot	High	Weak	Yes
D4	Rain	Mild	High	Weak	Yes
D5	Rain	Cool	Normal	Weak	Yes
D6	Rain	Cool	Normal	Strong	No
D7	Overcast	Cool	Normal	Strong	Yes
D8	Sunny	Mild	High	Weak	No
D9	Sunny	Cool	Normal	Weak	Yes
D10	Rain	Mild	Normal	Weak	Yes
D11	Sunny	Mild	Normal	Strong	Yes
D12	Overcast	Mild	High	Strong	Yes
D13	Overcast	Hot	Normal	Weak	Yes
D14	Rain	Mild	High	Strong	No

Prior Probability

$$P(\text{PlayTennis} = \text{yes}) = 9/14 = .64$$

$$P(\text{PlayTennis} = \text{no}) = 5/14 = .36$$

Current or Conditional Probability of individual Attributes

Outlook	Y	N		Humidity	Y	N
sunny	2/9	3/5		high	3/9	4/5
overcast	4/9	0		normal	6/9	1/5
rain	3/9	2/5				
Temperature				Windy		
hot	2/9	2/5		Strong	3/9	3/5
mild	4/9	2/5		Weak	6/9	2/5
cool	3/9	1/5				

NAIVE BAYES CLASSIFIER

For New Instance:

(Outlook = sunny, Temperature = cool, Humidity = high, Wind = strong)

$$v_{NB} = \operatorname{argmax}_{v_j \in \{yes, no\}} P(v_j) \prod_i P(a_i | v_j)$$

$$= \operatorname{argmax}_{v_j \in \{yes, no\}} P(v_j) \quad P(Outlook = sunny | v_j) P(Temperature = cool | v_j) \\ P(Humidity = high | v_j) P(Wind = strong | v_j)$$

$$v_{NB}(yes) = P(yes) P(sunny|yes) P(cool|yes) P(high|yes) P(strong|yes) = .0053$$

$$v_{NB}(no) = P(no) P(sunny|no) P(cool|no) P(high|no) P(strong|no) = .0206$$

$$v_{NB}(yes) = \frac{v_{NB}(yes)}{v_{NB}(yes) + v_{NB}(no)} = \mathbf{0.205}$$

$$v_{NB}(no) = \frac{v_{NB}(no)}{v_{NB}(yes) + v_{NB}(no)} = \mathbf{0.795}$$

- Estimate conditional probabilities of each attributes {color, legs, height, smelly} for the species classes: {M, H} using the data given in the table.
- Using these probabilities estimate the probability values for the new instance – (Color=Green, legs=2, Height=Tall, and Smelly=No).

No	Color	Legs	Height	Smelly	Species
1	White	3	Short	Yes	M
2	Green	2	Tall	No	M
3	Green	3	Short	Yes	M
4	White	3	Short	Yes	M
5	Green	2	Short	No	H
6	White	2	Tall	No	H
7	White	2	Tall	No	H
8	White	2	Short	Yes	H

No	Color	Legs	Height	Smelly	Species
1	White	3	Short	Yes	M
2	Green	2	Tall	No	M
3	Green	3	Short	Yes	M
4	White	3	Short	Yes	M
5	Green	2	Short	No	H
6	White	2	Tall	No	H
7	White	2	Tall	No	H
8	White	2	Short	Yes	H

New Instance

(Color=Green, legs=2, Height=Tall, and Smelly=No)

$$P(M) = \frac{4}{8} = 0.5 \quad P(H) = \frac{4}{8} = 0.5$$

Color	M	H
White	2/4	3/4
Green	2/4	1/4

Legs	M	H
2	1/4	4/4
3	3/4	0/4

Height	M	H
Tall	3/4	2/4
Short	1/4	2/4

Smelly	M	H
Yes	3/4	1/4
No	1/4	3/4

$$P(M) = \frac{4}{8} = 0.5 \quad P(H) = \frac{4}{8} = 0.5$$

Color	M	H
White	2/4	3/4
Green	2/4	1/4

Legs	M	H
2	1/4	4/4
3	3/4	0/4

Height	M	H
Tall	3/4	2/4
Short	1/4	2/4

Smelly	M	H
Yes	3/4	1/4
No	1/4	3/4

$$p(M|New Instance) = p(M) * p(Color = Green|M) * p(Legs = 2|M) * p(Height = tall|M) * p(Smelly = no |M)$$

$$p(M|New Instance) = 0.5 * \frac{2}{4} * \frac{1}{4} * \frac{3}{4} * \frac{1}{4} = 0.0117$$

$$p(H|New Instance) = p(H) * p(Color = Green|H) * p(Legs = 2|H) * p(Height = tall|H) * p(Smelly = no |H)$$

$$p(H|New Instance) = 0.5 * \frac{1}{4} * \frac{4}{4} * \frac{2}{4} * \frac{3}{4} = 0.047$$

$$p(H|New Instance) > p(M|New Instance)$$

Hence the new instance belongs to Speices H

UNIT 4 - Clustering

K-Means

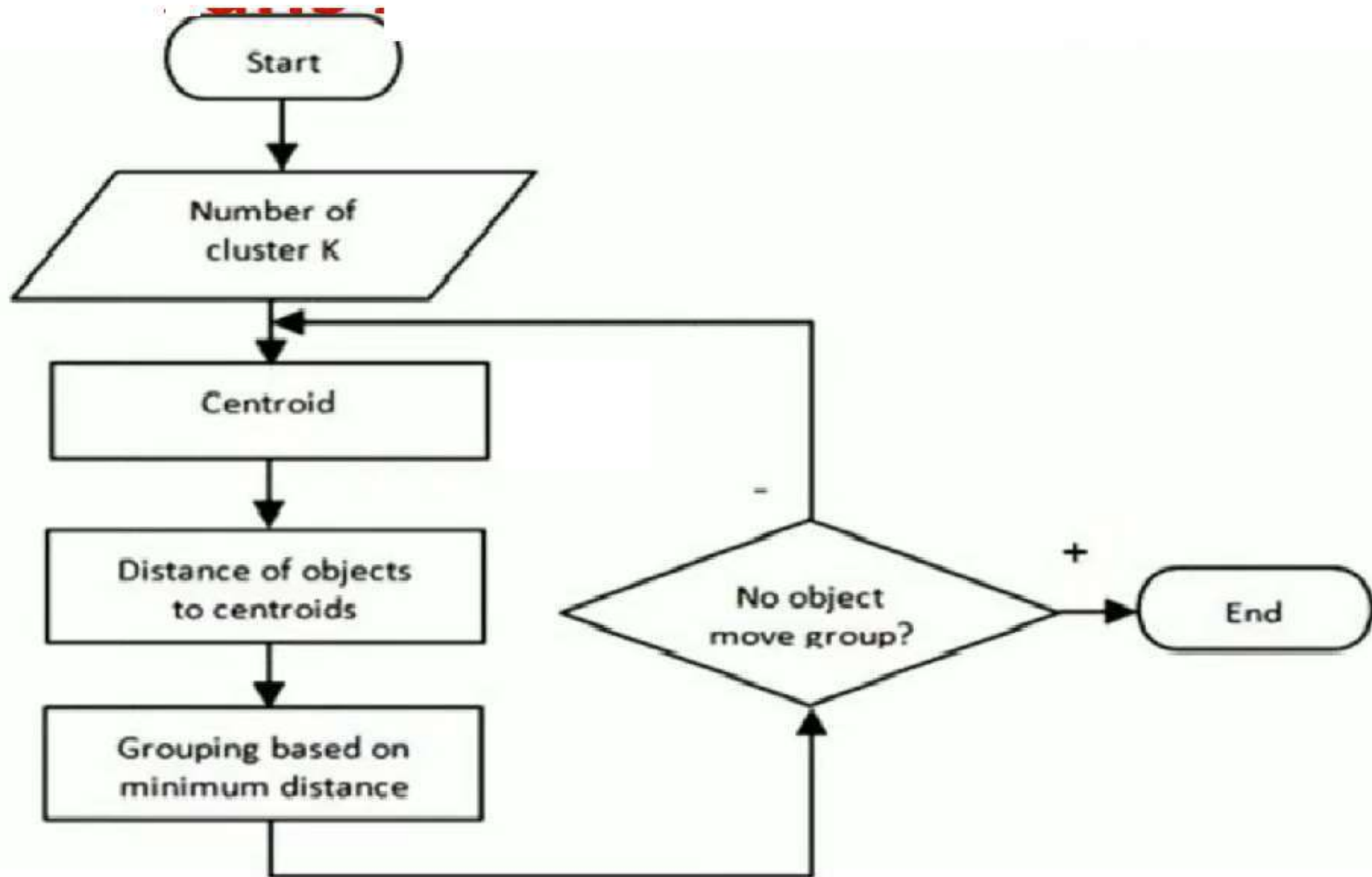
- kMeans algorithm is an unsupervised learning algorithm
- Given a data set of items, with certain features, and values for these features, the algorithm will categorize the items into k groups or clusters of similarity.
- To calculate the similarity, we can use the Euclidean distance, Manhattan distance, Hamming distance, Cosine distance as measurement.

K-Means Clustering Algorithm

Here is the pseudocode for implementing a K-means algorithm.

Input: Algorithm K-Means (K number of clusters, D list of data points)

1. Choose K number of random data points as initial centroids (cluster centers).
2. Repeat till cluster centers stabilize:
 - a. Allocate each point in D to the nearest of Kth centroids.
 - b. Compute centroid for the cluster using all points in the cluster.



Advantages and Disadvantages of K-Means Algorithm

Advantages of K-Means Algorithm

1. K-means algorithm is simple, easy to understand, and easy to implement.
2. It is also efficient, in which the time taken to cluster K-means rises linearly with the number of data points.
3. No other clustering algorithm performs better than K-means.

Disadvantages of K-Means Algorithm

1. The user needs to specify an initial value of K.
2. The process of finding the clusters may not converge.
3. It is not suitable for discovering clusters that are not hyper ellipsoids or hyper spheres,.

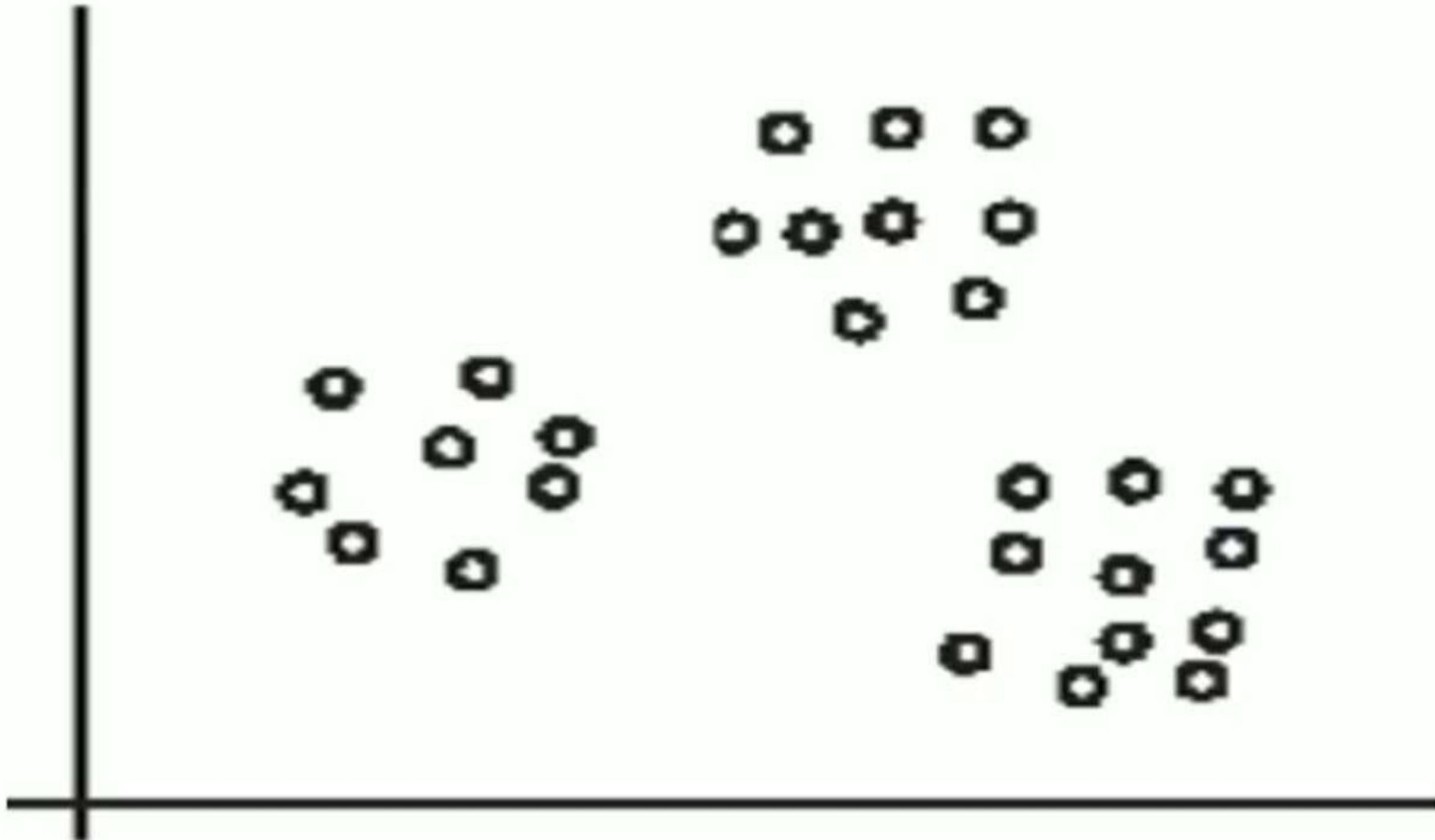
Cluster Analysis & K-Means Clustering

Cluster Analysis

- Cluster analysis is used for automatic identification of natural groupings of things. It is also known as the **segmentation technique**.
- In cluster analysis , data instances that are **similar to (or near)** each other are categorized into one cluster.
- Similarly, data instances that are **very different (or far away)** from each other are moved into different clusters.
- Clustering is an unsupervised learning technique as there is no output or dependent variable for which a right or wrong answer can be computed.
- The correct number of clusters or the definition of those clusters is not known ahead of time.
- Clustering techniques can only suggest to the user how many clusters would make sense from the characteristics of the data.

This is how output of cluster analysis looks like

Cluster Analysis



Applications of Cluster Analysis

- **Market segmentation:** Categorizing customers according to their similarities, for example by their common wants and needs and propensity to pay, can help with targeted marketing.
- **Product portfolio:** People of similar sizes can be grouped together to make small, medium, and large sizes for clothing items.
- **Text mining:** Clustering can help organize a given collection of text documents according to their content similarities into clusters of related topics.

Clustering Techniques

- Cluster analysis is a machine-learning technique.
- The quality of a clustering result depends on the algorithm, the distance function, and the application.
- First, consider the distance function.
- Most cluster analysis methods use a distance measure to calculate the closeness between pairs of items.
- There are two major measures of distances: **Euclidian distance** is the most intuitive measure.
- The other popular measure is the **Manhattan (rectilinear) distance**, where one can go only in orthogonal directions.
- In either case, the key objective of the clustering algorithm is the same:
 - **Intercluster distance $\Rightarrow$ maximized**
 - **Intracluster distance $\Rightarrow$ minimized**

Here is the generic Pseudo code for clustering

- 1. Pick an arbitrary number of groups/segments to be created.*
- 2. Start with some initial randomly chosen center values for groups.*
- 3. Classify instances to closest groups.*
- 4. Compute new values for the group centers.*
- 5. Repeat Steps 3 and 4 till groups converge.*
- 6. If clusters are not satisfactory, go to Step 1 and pick a different number of groups/segments.*

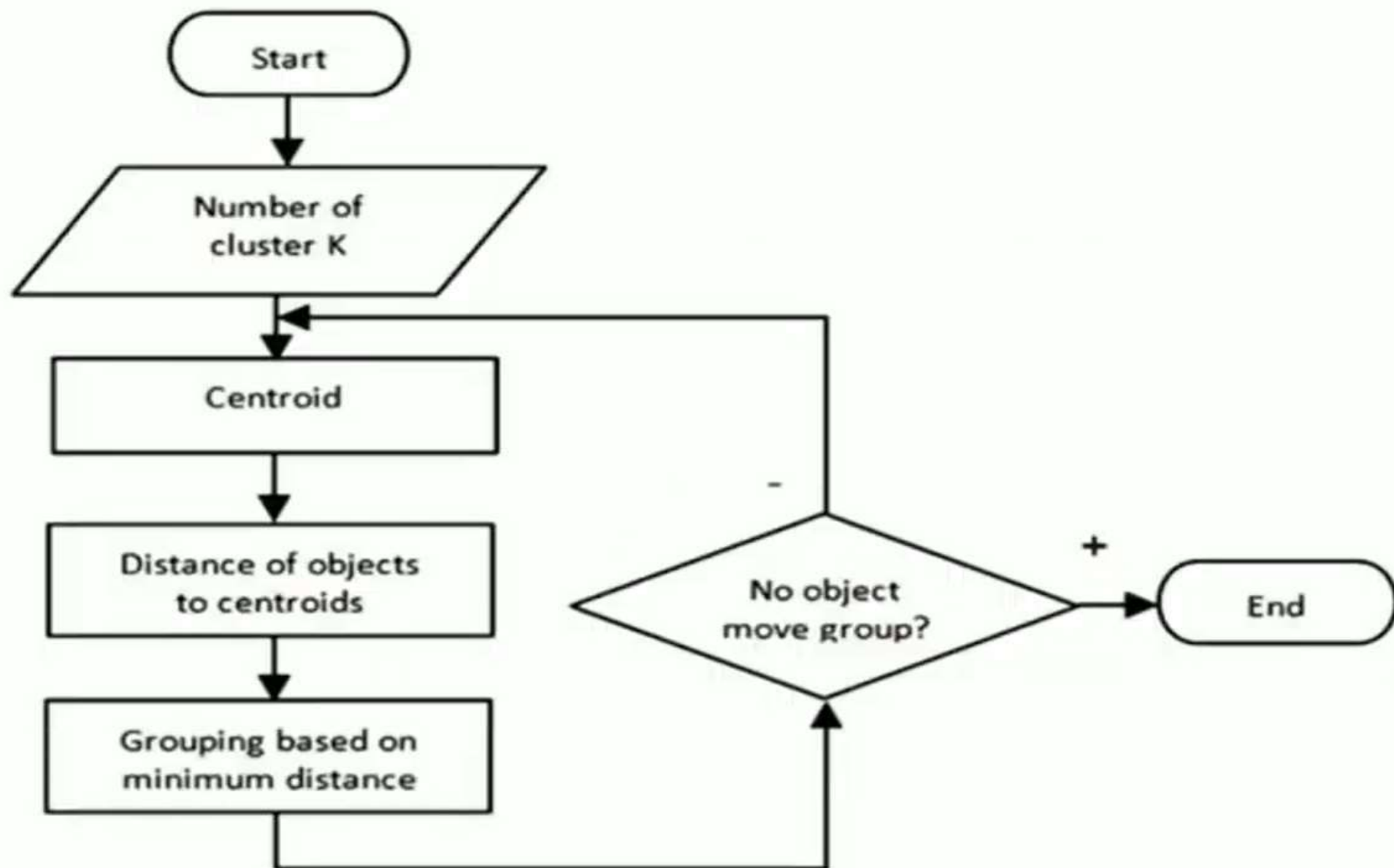
K-Means Algorithm for Clustering

Here is the pseudocode for implementing a K-means algorithm.

Input: Algorithm K-Means (K number of clusters, D list of data points)

1. Choose K number of random data points as initial centroids (cluster centers).
2. Repeat till cluster centers stabilize:
 - a. Allocate each point in D to the nearest of Kth centroids.
 - b. Compute centroid for the cluster using all points in the cluster.

K-Means Algorithm for Clustering



Advantages and Disadvantages of K-Means Algorithm

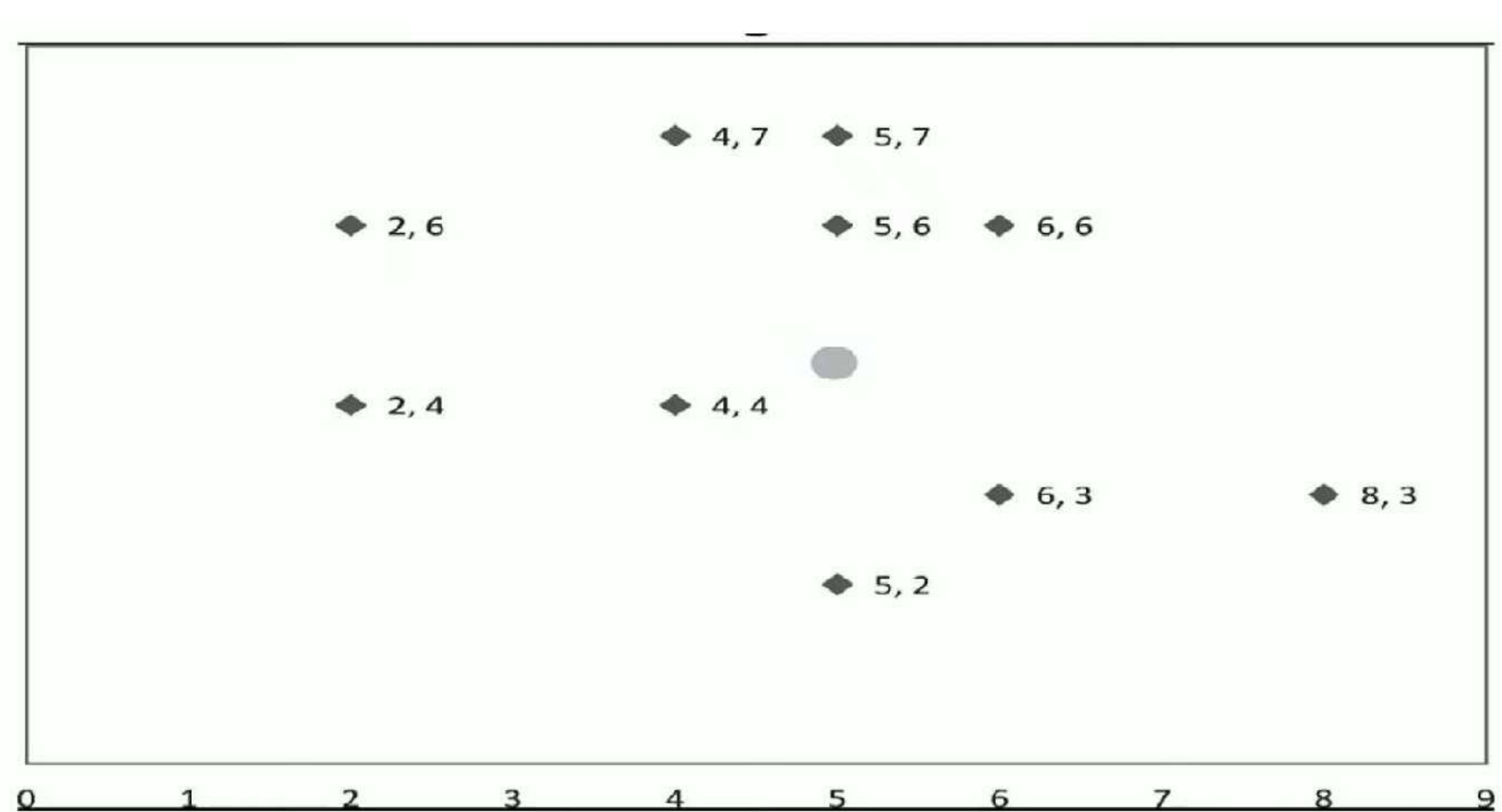
There are many advantages of **K-Means Algorithm**

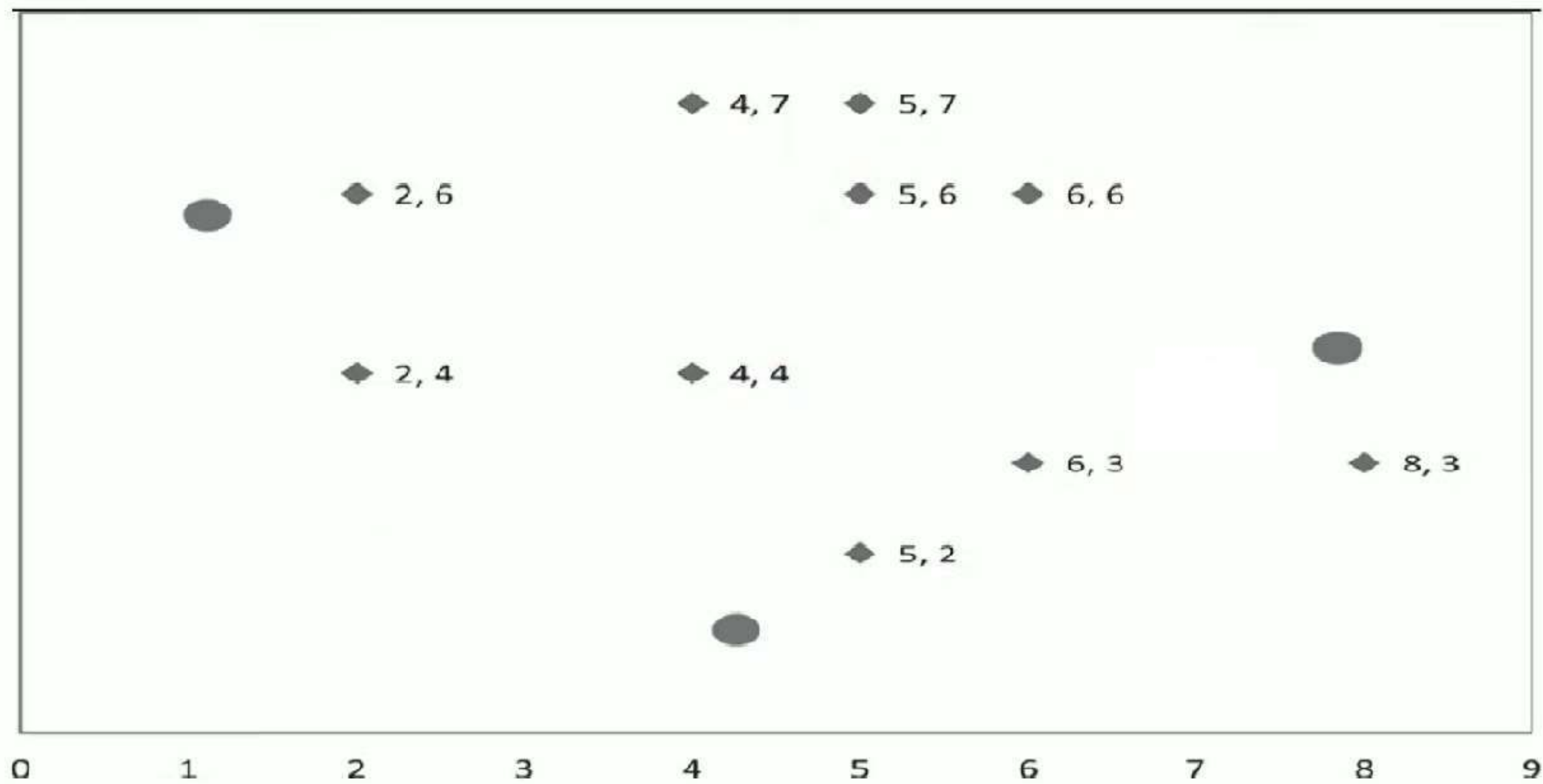
1. K-means algorithm is simple, easy to understand, and easy to implement.
2. It is also efficient, in which the time taken to cluster K-means rises linearly with the number of data points.
3. No other clustering algorithm performs better than K-means, in general.

There are many disadvantages of **K-Means Algorithm**

1. The user needs to specify an initial value of K.
2. The process of finding the clusters may not converge.
3. It is not suitable for discovering clusters that are not hyper ellipsoids or hyper spheres).

K-MEANS ALGORITHM





Iteration - 1

C1 - Seed Point1 – (1, 5)

C2 - Seed Point2 – (4, 1)

C3 - Seed Point3 – (8, 4)

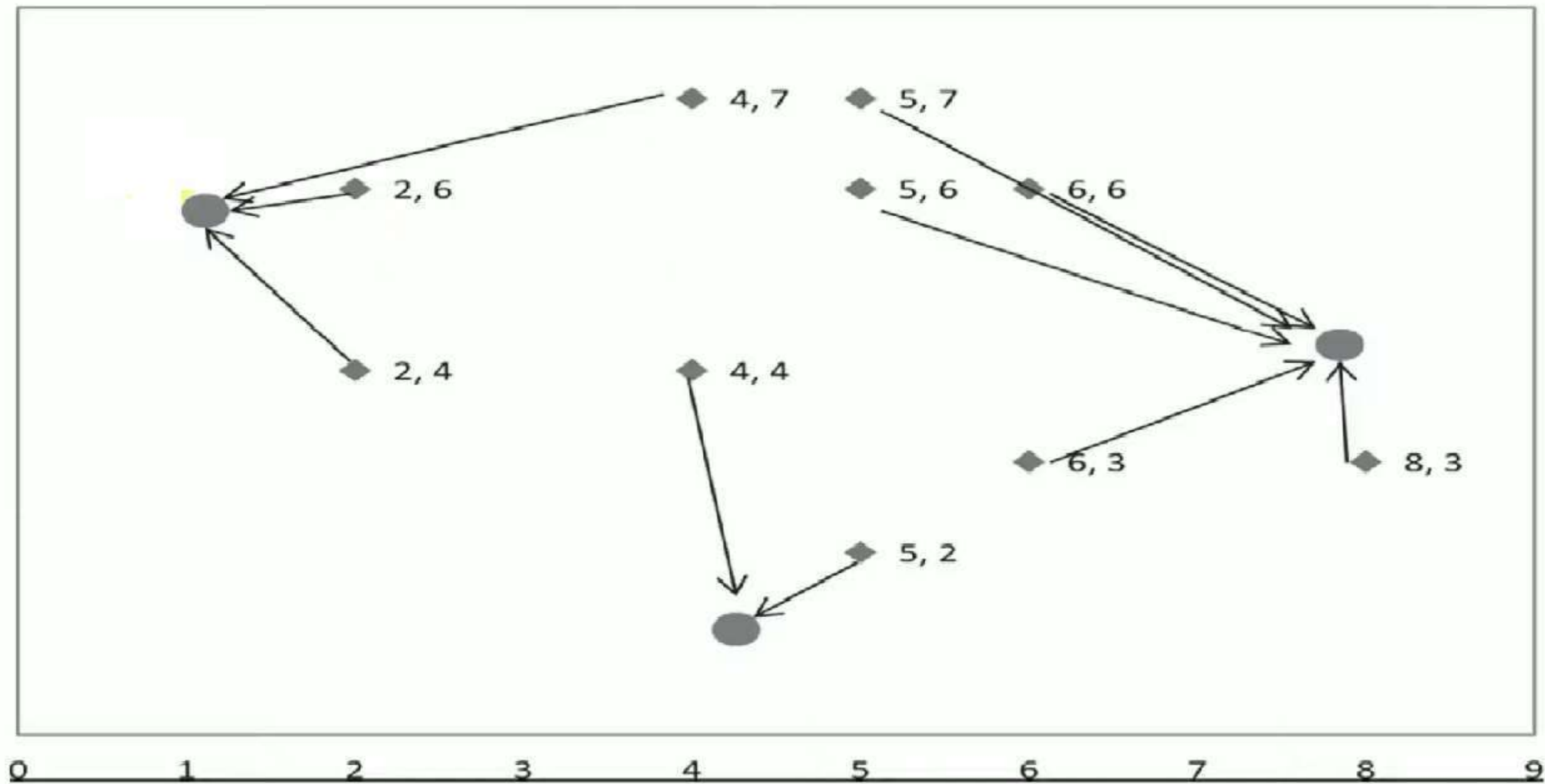
$$D = \sqrt{((x_2 - x_1)^2 + (y_2 - y_1)^2)}$$

C1 – Centroid – (2.66, 5.66)

C2 – Centroid – (4.5, 3)

C3 – Centroid – (6, 5)

X	Y	Distance to			Cluster Number
		(1, 5)	(4, 1)	(8, 4)	
2	4	1.41	3.61	6.00	C1
2	6	1.41	5.39	6.32	C1
5	6	4.12	5.10	3.61	C3
4	7	3.61	6.00	5.00	C1
8	3	7.28	4.47	1.00	C3
6	6	5.10	5.39	2.83	C3
5	2	5.00	1.41	3.61	C2
5	7	4.47	6.08	4.24	C3
6	3	5.39	2.83	2.24	C3
4	4	3.16	3.00	4.00	C2



Iteration - 2

C1 – Centroid – (2.66, 5.66)

C2 – Centroid – (4.5, 3)

C3 – Centroid – (6, 5)

C1 – Centroid – (2.66, 5.66)

C2 – Centroid – (5, 3)

C3 – Centroid – (6, 5.5)

X	Y	Distance to			Cluster Number
		(2.66, 5.66)	(4.5, 3)	(6, 5)	
2	4	1.79	2.69	4.12	C1
2	6	0.74	3.91	4.12	C1
5	6	2.36	3.04	1.41	C3
4	7	1.90	4.03	2.83	C1
8	3	5.97	3.5	2.83	C3
6	6	3.36	3.35	1	C3
5	2	4.34	1.12	3.16	C2
5	7	2.70	4.03	2.24	C3
6	3	4.27	1.5	2	C2
4	4	2.13	1.12	2.24	C2

Iteration - 3

C1 – Centroid – (2.66, 5.66)

C2 – Centroid – (5, 3)

C3 – Centroid – (6, 5.5)

C1 – Centroid – (2.66, 5.66)

C2 – Centroid – (5.75, 3)

C3 – Centroid – (5.33, 6.33)

X	Y	Distance to			Cluster Number
		(2.66, 5.66)	(5, 3)	(6, 5.5)	
2	4	1.79	3.16	4.27	C1
2	6	0.74	4.24	4.03	C1
5	6	2.36	3.00	1.12	C3
4	7	1.90	4.12	2.50	C1
8	3	5.97	3.00	3.20	C2
6	6	3.36	3.16	0.50	C3
5	2	4.34	1.00	3.64	C2
5	7	2.70	4.00	1.80	C3
6	3	4.27	1.00	2.50	C2
4	4	2.13	1.41	2.50	C2

Iteration - 4

C1 – Centroid – (2.66, 5.66)

C2 – Centroid – (5.75, 3)

C3 – Centroid – (5.33, 6.33)

C1 – Centroid – (2, 5)

C2 – Centroid – (5.75, 3)

C3 – Centroid – (5, 6.5)

		Distance to			Cluster Number
X	Y	(2.66, 5.66)	(5.75, 3)	(5.33, 6.33)	
2	4	1.79	3.88	4.06	C1
2	6	0.74	4.80	3.35	C1
5	6	2.36	3.09	0.47	C3
4	7	1.90	4.37	1.49	C3
8	3	5.97	2.25	4.27	C2
6	6	3.36	3.01	0.75	C3
5	2	4.34	1.25	4.34	C2
5	7	2.70	4.07	0.75	C3
6	3	4.27	0.25	3.40	C2
4	4	2.13	2.02	2.68	C2

Iteration - 5

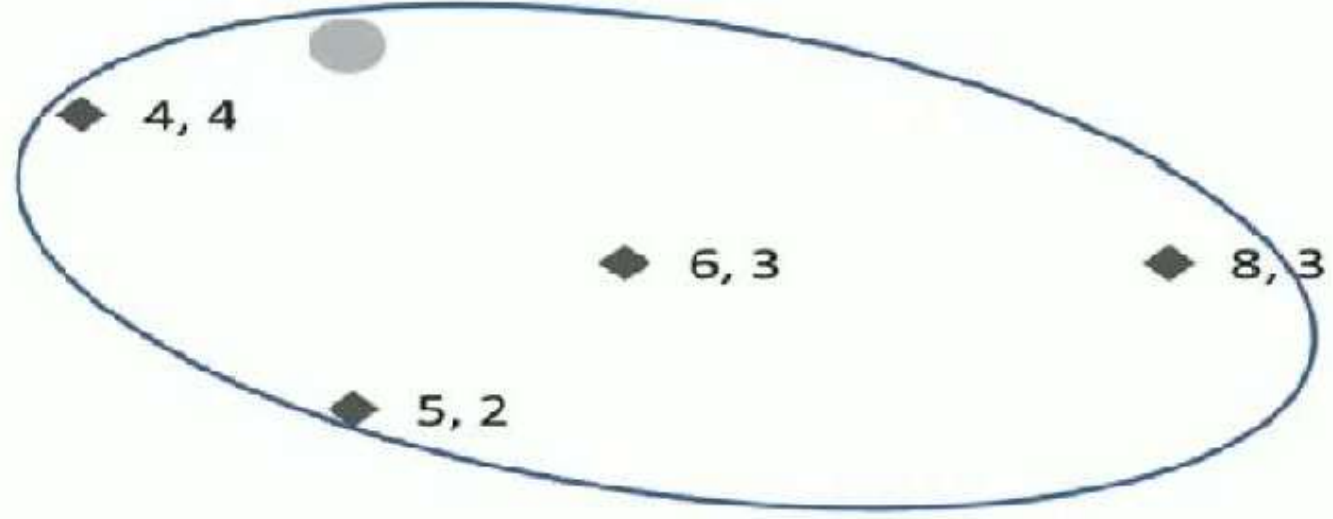
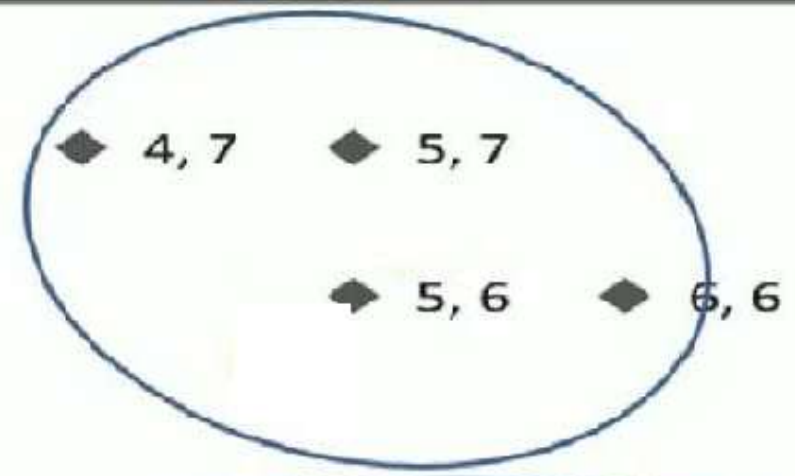
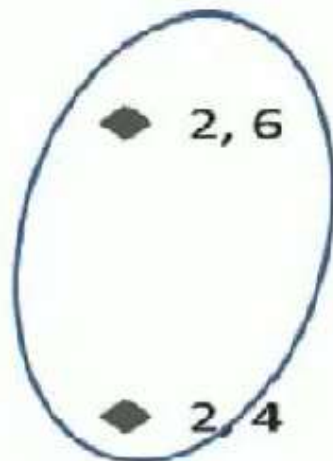
C1 – Centroid – (2, 5)

C2 – Centroid – (5.75, 3)

C3 – Centroid – (5, 6.5)

No movement of data Points
Hence these are the final
positions

X	Y	Distance to			Cluster Number
		(2, 5)	(5.75, 3)	(5, 6.5)	
2	4	1.00	3.88	3.91	C1
2	6	1.00	4.80	3.04	C1
5	6	3.16	3.09	0.50	C3
4	7	2.83	4.37	1.12	C3
8	3	6.32	2.25	4.61	C2
6	6	4.12	3.01	1.12	C3
5	2	4.24	1.25	4.50	C2
5	7	3.61	4.07	0.50	C3
6	3	4.47	0.25	3.64	C2
4	4	2.24	2.02	2.69	C2

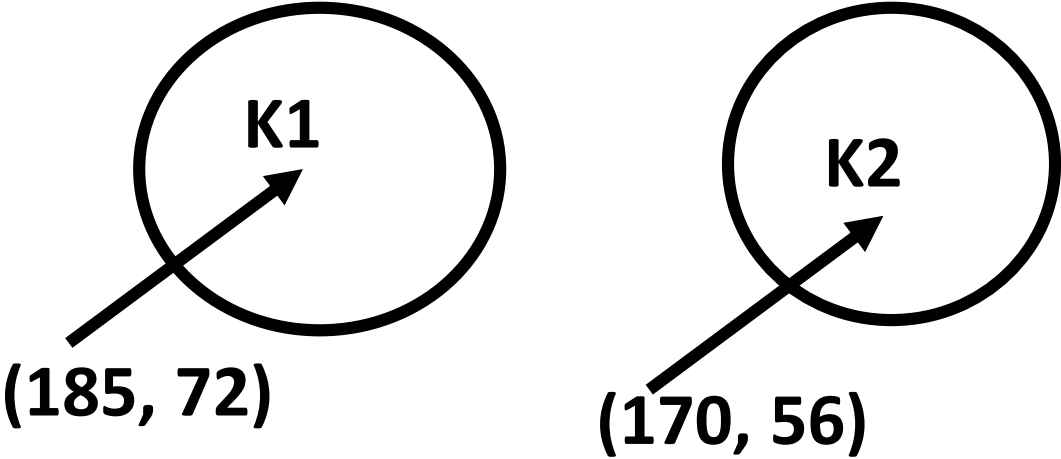


0 1 2 3 4 5 6 7 8 9

K MEANS CLUSTERING

K MEANS CLUSTERING

	Height	Weight
1	185	72
2	170	56
3	168	60
4	179	68
5	182	72
6	188	77
7	180	71
8	180	70
9	183	84
10	180	88
11	180	67
12	177	76



Euclidean Distance

$$\sqrt{(X_o - X_c)^2 + (Y_o - Y_c)^2}$$

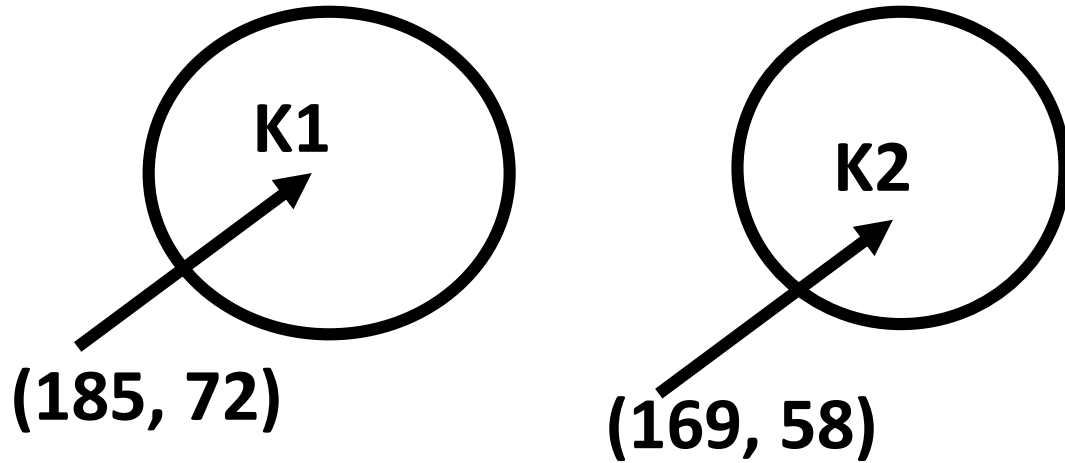
Euclidean Distance for 3:

$$K1 = \sqrt{(168 - 185)^2 + (60 - 72)^2} = 20.80$$

$$K2 = \sqrt{(68 - 170)^2 + (60 - 56)^2} = 4.48$$

New Centroid Calculation:

For K2= $((170+168)/2, (60+56)/2) = (169, 58)$



Euclidean Distance for 4:

$$K1 = \sqrt{(179 - 185)^2 + (68 - 72)^2} = 6.32$$

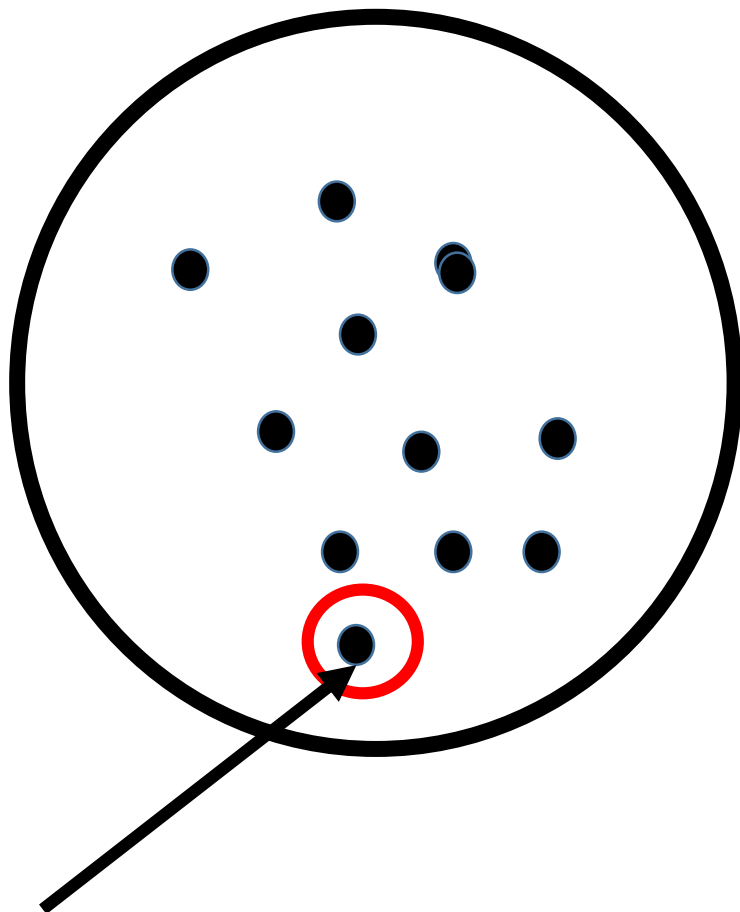
$$K2 = \sqrt{(179 - 169)^2 + (68 - 58)^2} = 14.14$$

K1 -> {1,4,5,6,7,8,9,10,11,12}

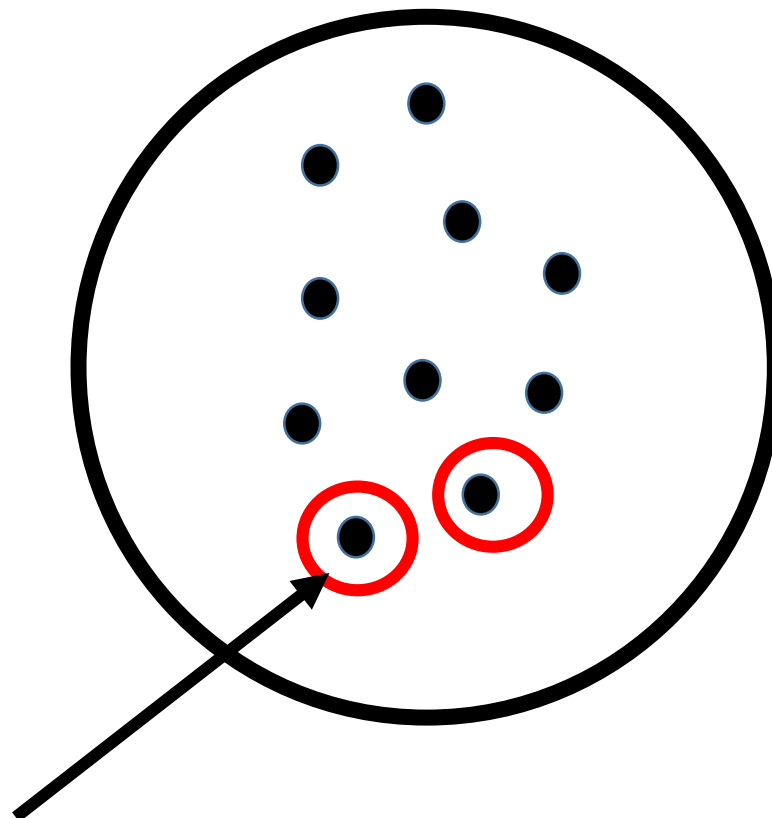
K2 -> {2,3}

Hierarchical Clustering

K2



K1



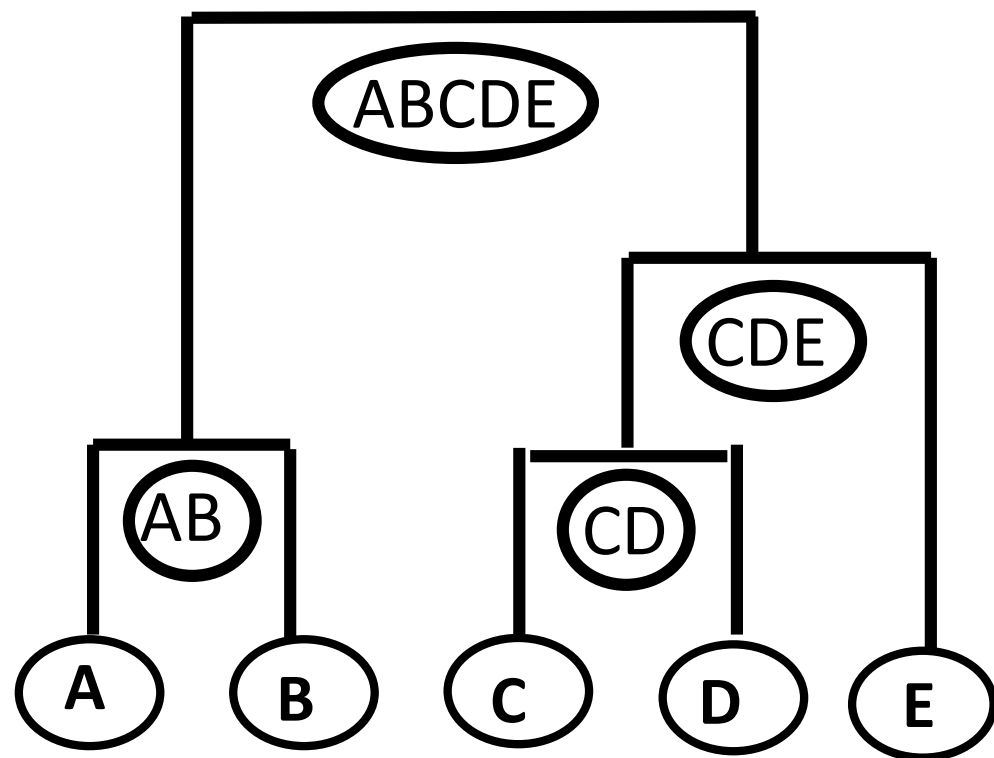
Hierarchical Clustering

- Agglomerative
- Divisive

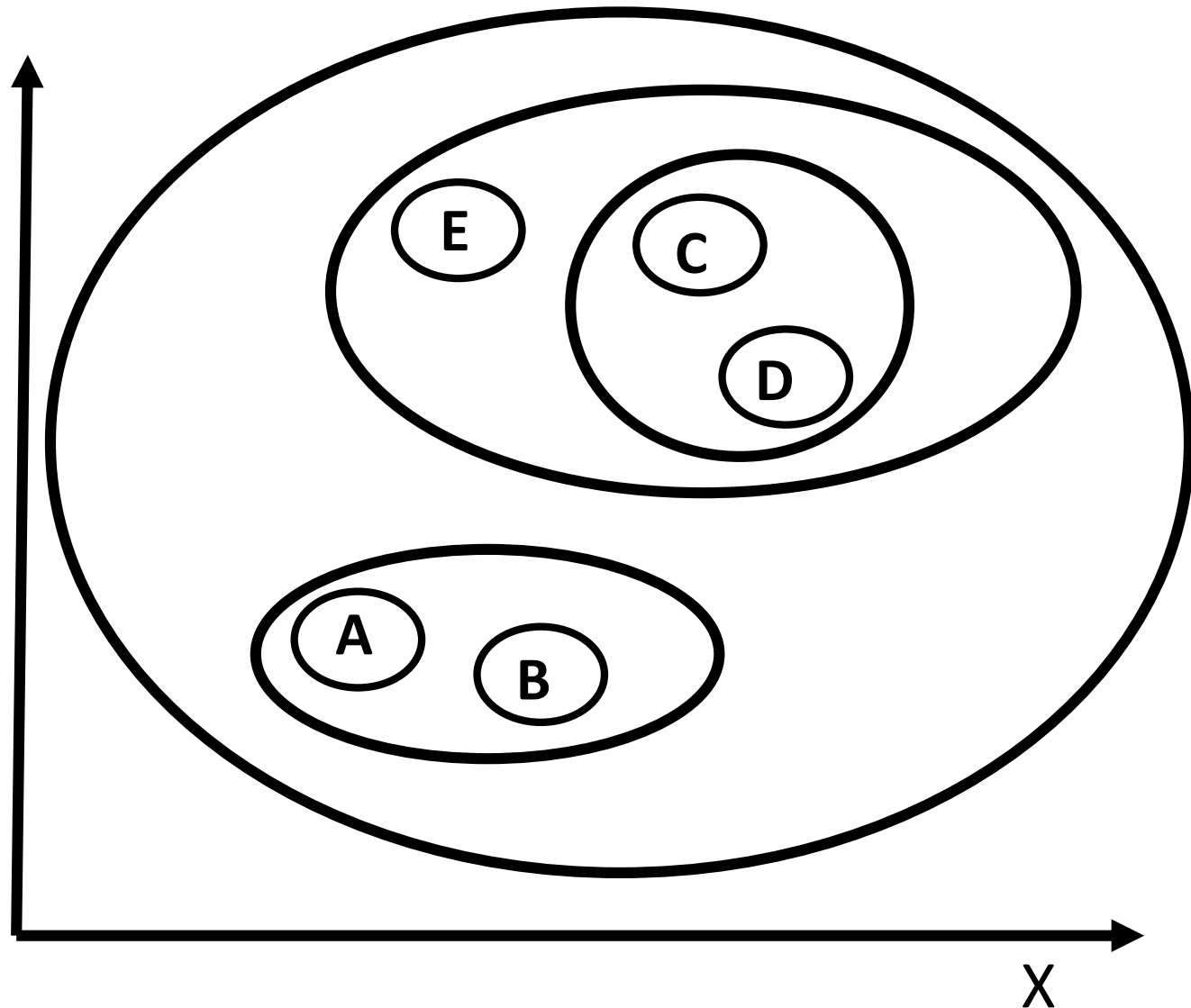
Agglomerative

Bottom Up

Dendrogram

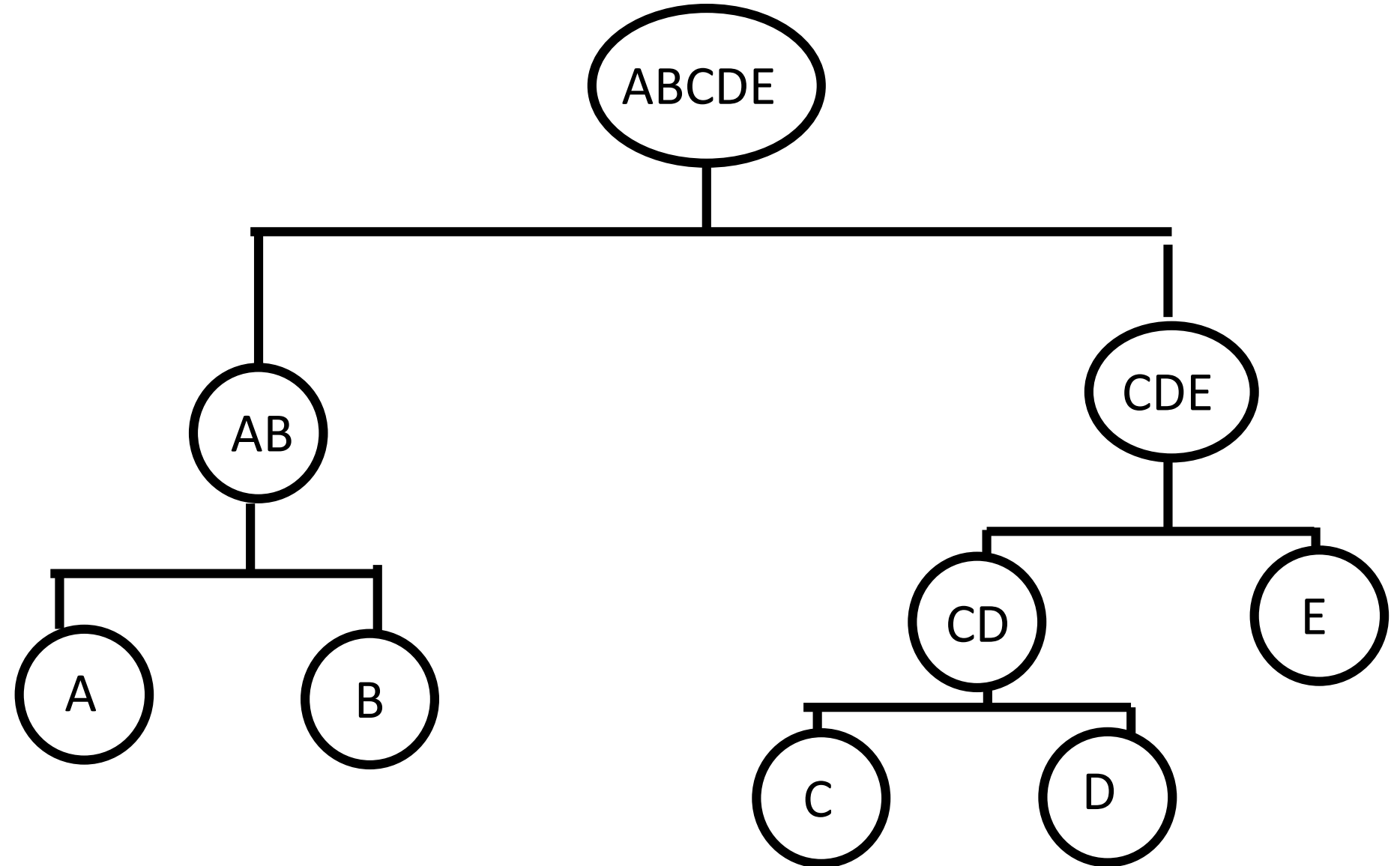


Y



Divisive

Top Down

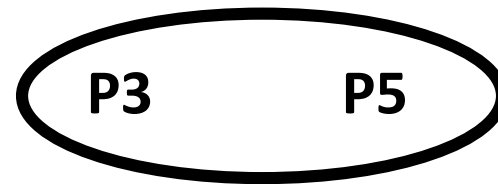


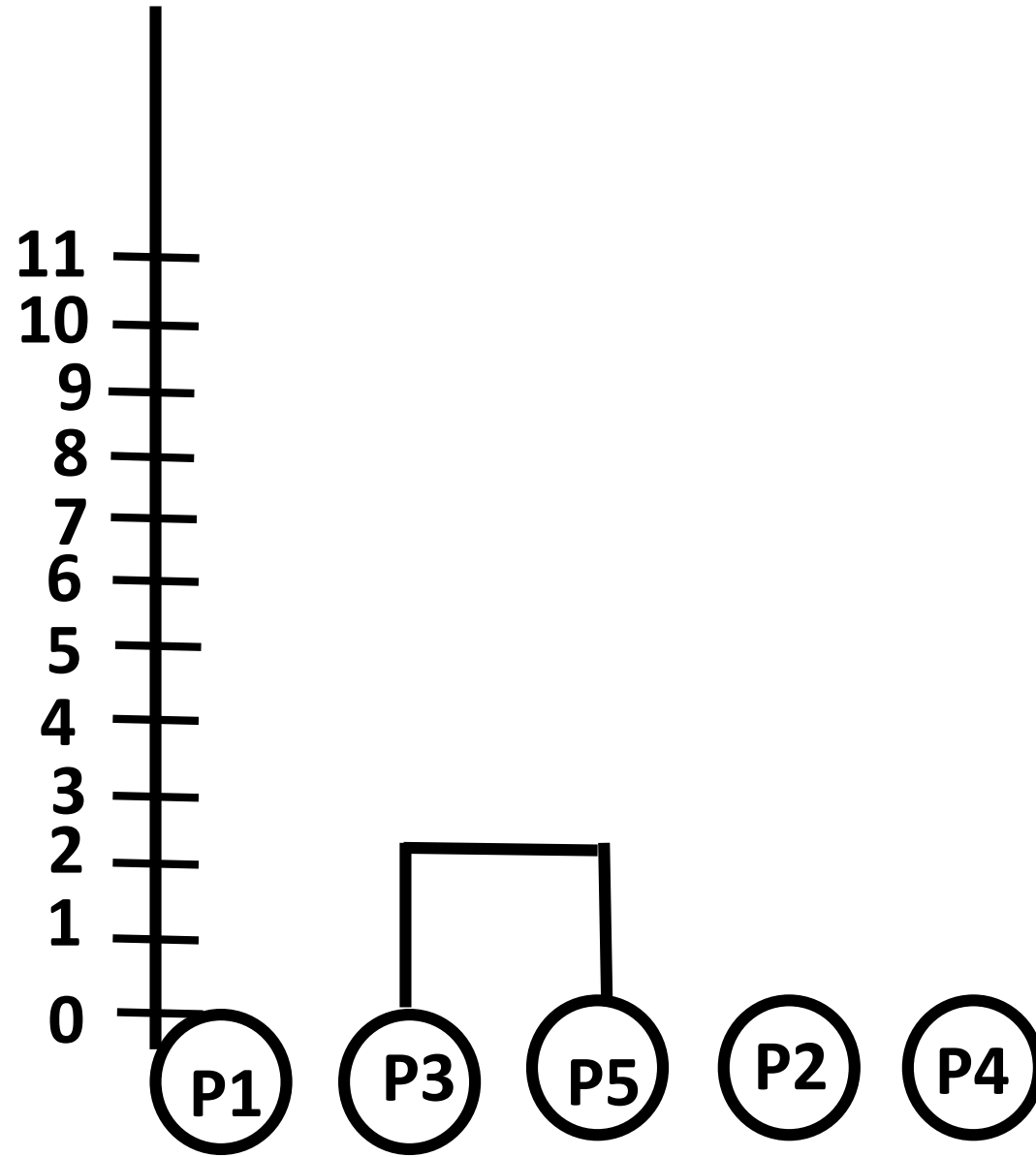
AGGLOMERATIVE CLUSTERING

COMPLETE LINKAGE

	P1	P2	P3	P4	P5
P1	0				
P2	9	0			
P3	3	7	0		
P4	6	5	9	0	
P5	11	10	2	8	0

(P3, P5)



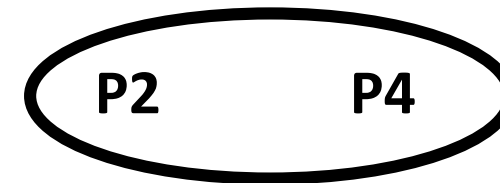


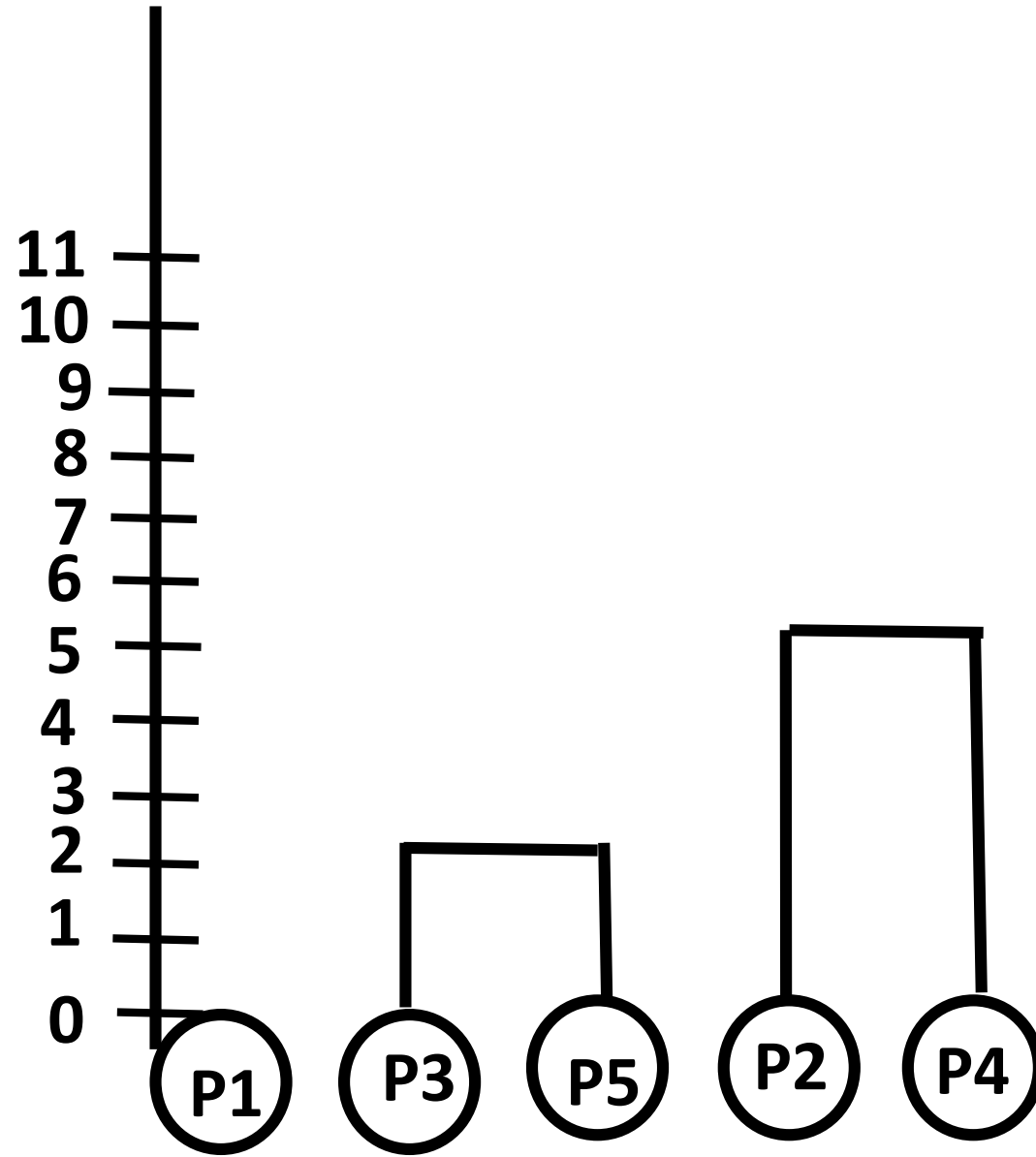
$$d(P2, [P3, P5]) \\ = \text{MAX}(d(P2, P3), d(P2, P5)) = \text{MAX}((7, 10)) = 10$$

$$d(P1, [P3, P5]) \\ = \text{MAX}(d(P1, P3), d(P1, P5)) = \text{MAX}((3, 11)) = 11$$

$$d(P4, [P3, P5]) \\ = \text{MAX}(d(P4, P3), d(P4, P5)) = \text{MAX}((9, 8)) = 9$$

	P1	P2	[P3,P5]	P4
P1	0			
P2	9	0		
[P3.P5]	11	10	0	
P4	6	5	9	0

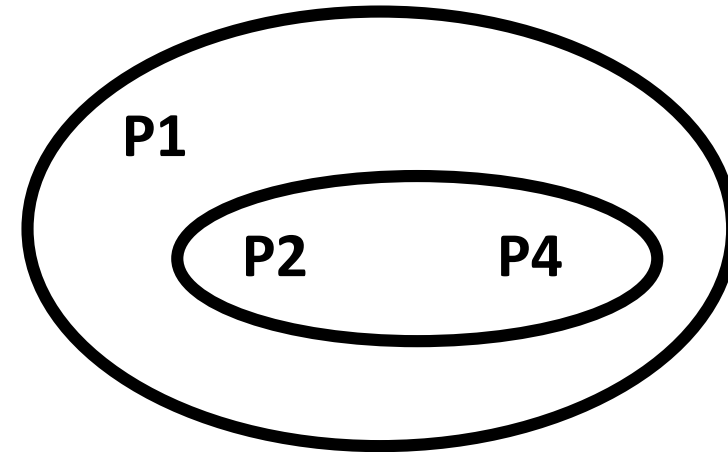


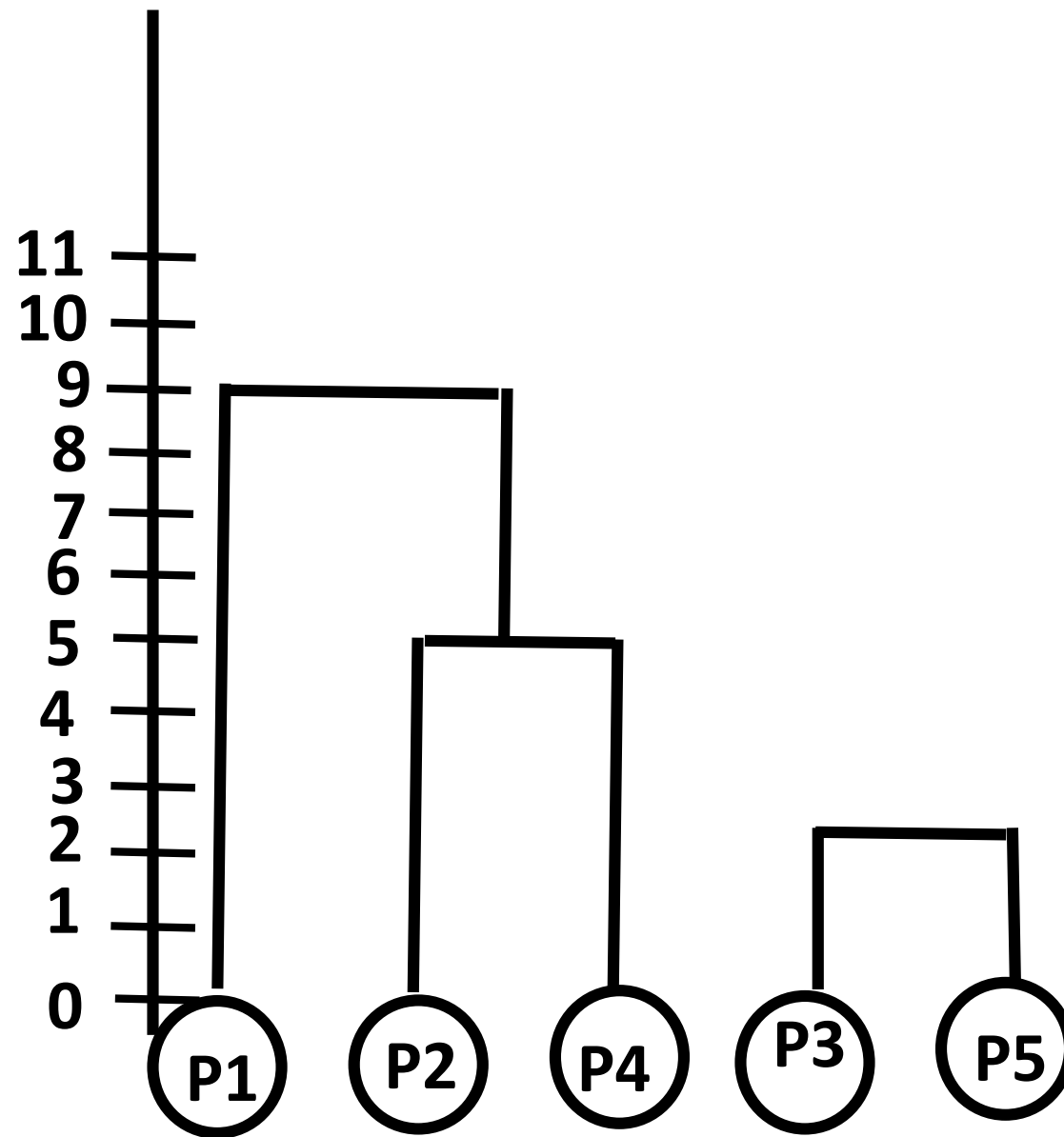


$$d(P1, [P2, P4]) \\ = \text{MAX}(d(P1, P2), d(P1, P4)) = \text{MAX}((9, 6)) = 9$$

$$d([P2, P4], [P3, P5]) \\ = \text{MAX}(d(P2, P3), d(P2, P5), d(P4, P3), d(P4, P5)) = \text{MAX}((7, 10, 9, 8)) = 10$$

	P1	[P2,P4]	[P3,P5]
P1	0		
[P2,P4]	9	0	
[P3,P5]	11	10	0



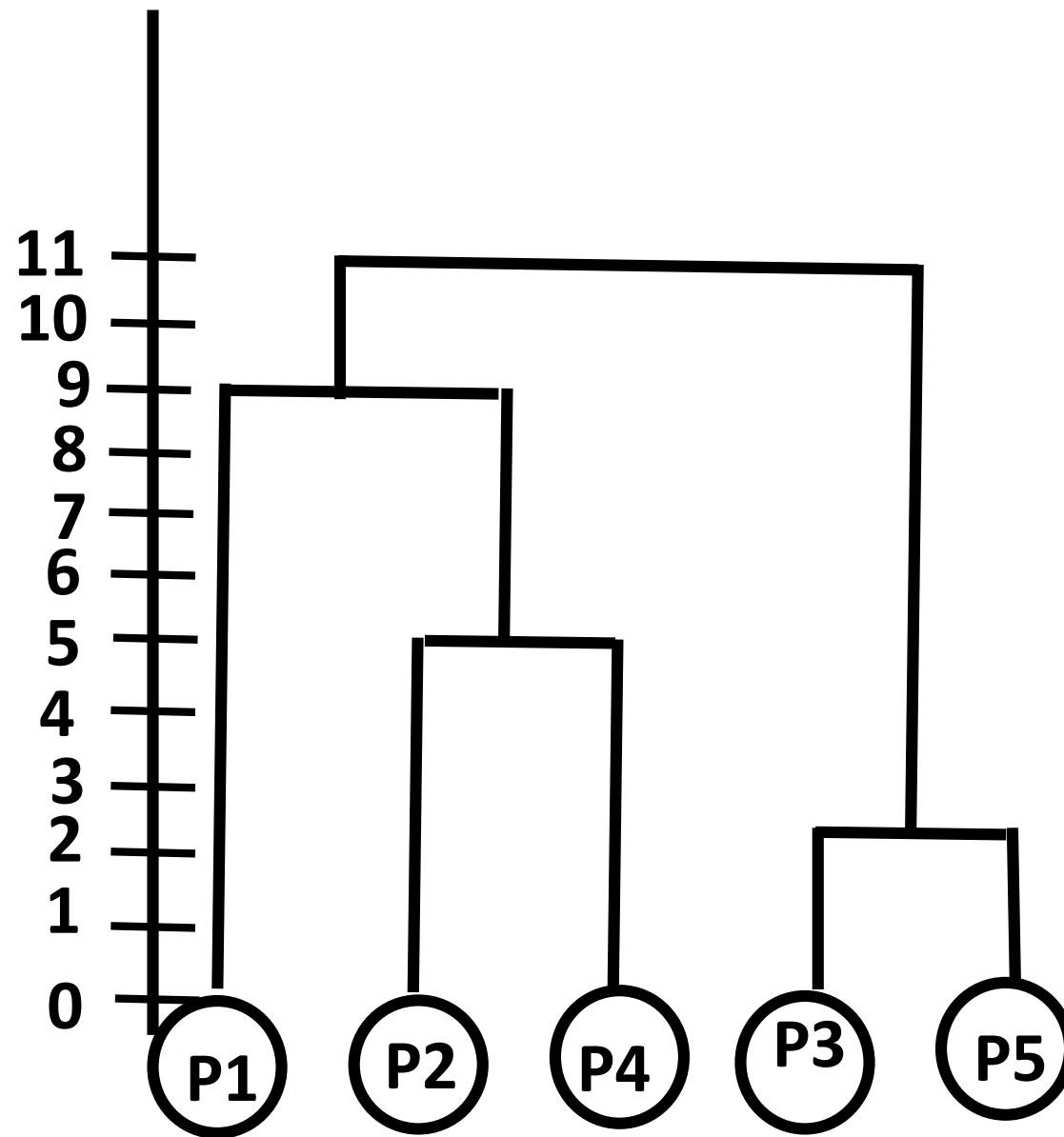


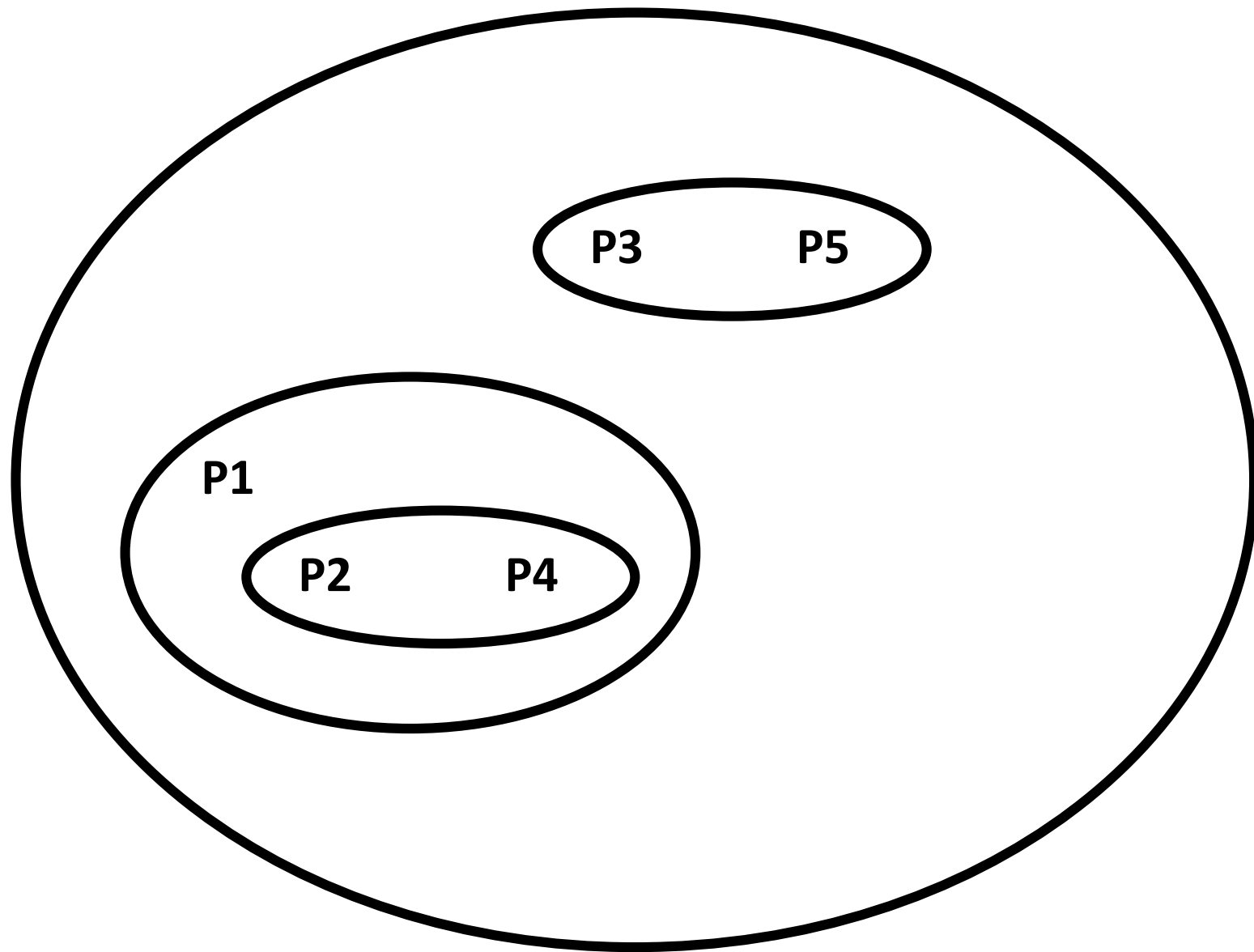
$d([P3, P5], [P1, P2, P4])$

$= \text{MAX}(d(P3, P1), d(P3, P2), d(P3, P4), d(P5, P1), d(P5, P2), d(P5, P4))$

$= \text{MAX}((3, 7, 9, 11, 10, 8)) = 11$

	[P1,P2,P4]	[P3,P5]
[P1,P2,P4]	0	
[P3,P5]	11	0





AGGLOMERATIVE CLUSTERING

SINGLE LINKAGE TECHNIQUE

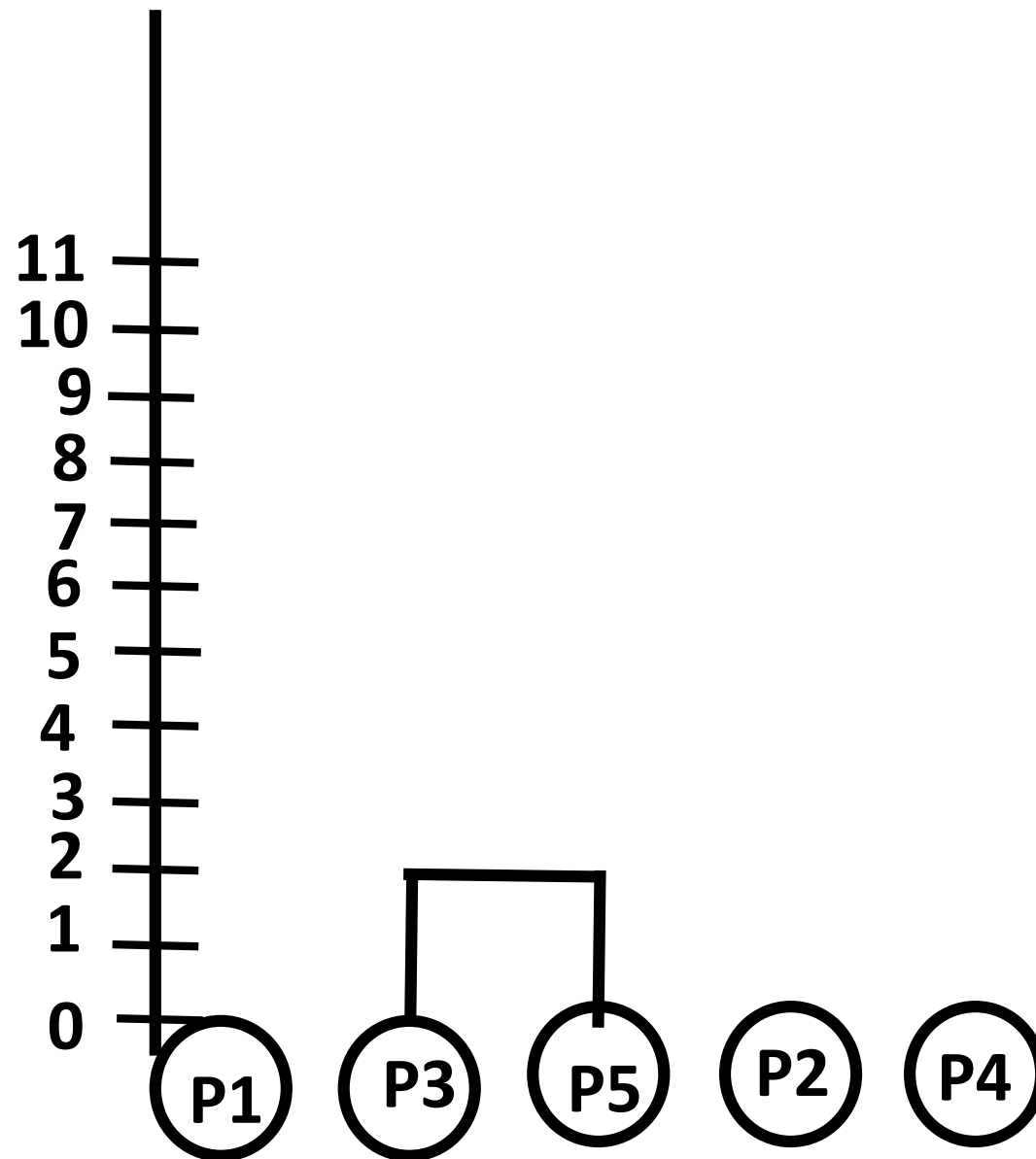
Bottom Up Approach

We are using single linkage

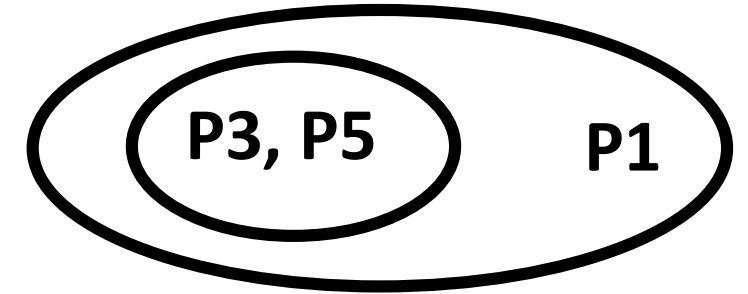
Therefore we find the minimum distance between the points.

	P1	P2	P3	P4	P5
P1	0				
P2	9	0			
P3	3	7	0		
P4	6	5	9	0	
P5	11	10	2	8	0

P3 P5 => One Cluster



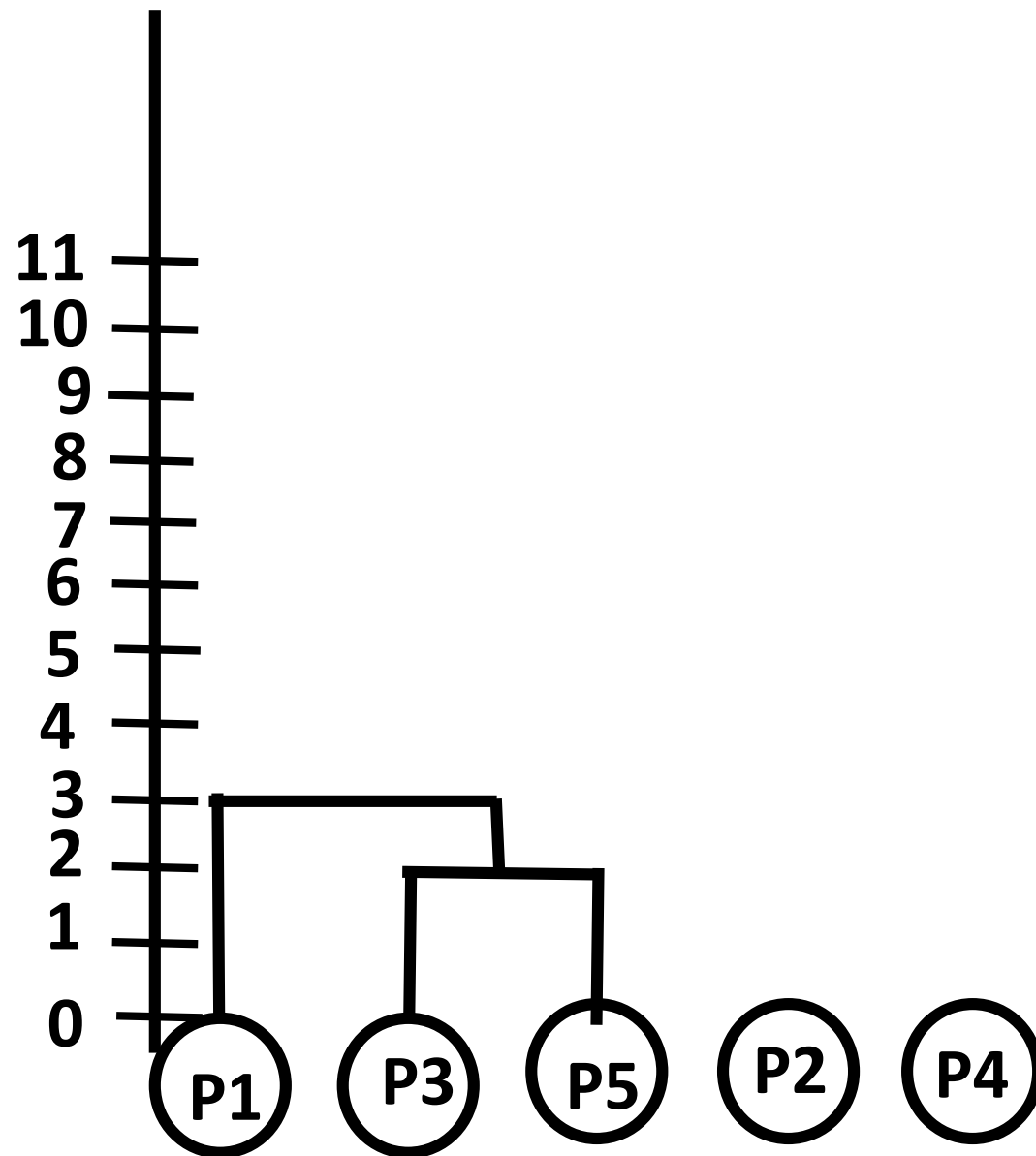
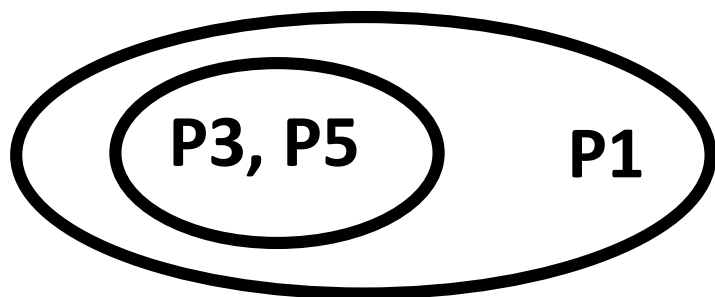
	P1	P2	[P3, P5]	P4
P1	0			
P2	9	0		
[P3,P5]	3	7	0	
P4	6	5	8	0



$\Rightarrow d(p1, [p3,p5])$
 $\Rightarrow \text{Min}(d(p1,p3), d(p1,p5))$
 $\Rightarrow \text{Min}(3, 11)$
 $\Rightarrow 3$

$\Rightarrow d(p2, [p3,p5])$
 $\Rightarrow \text{Min}(d(p2,p3), d(p2,p5))$
 $\Rightarrow \text{Min}(7, 10)$
 $\Rightarrow 7$

$\Rightarrow d(p4, [p3,p5])$
 $\Rightarrow \text{Min}(d(p4,p3), d(p4,p5))$
 $\Rightarrow \text{Min}(9, 8)$
 $\Rightarrow 8$



$\Rightarrow d(p_4, [P_1, p_3, p_5])$

$\Rightarrow \text{Min}(d(p_4, p_1), d(p_4, p_3), d(p_4, p_5))$

$\Rightarrow \text{Min}(6, 9, 8)$

$\Rightarrow 6$

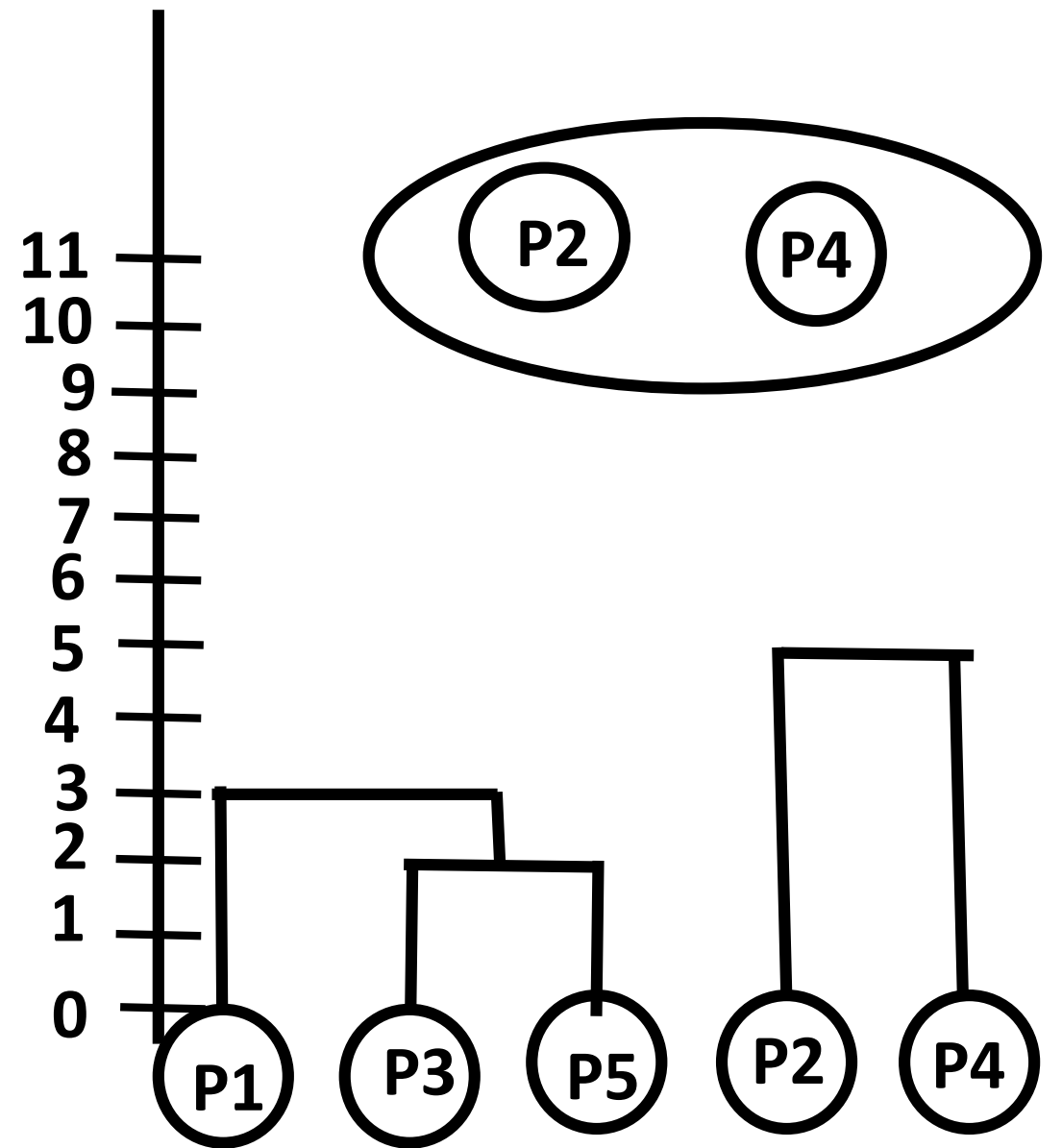
$\Rightarrow d(p_2, [P_1, p_3, p_5])$

$\Rightarrow \text{Min}(d(p_2, p_1), d(p_2, p_3), d(p_2, p_5))$

$\Rightarrow \text{Min}(9, 7, 10)$

$\Rightarrow 7$

	[P1,P3,P5]	P2	P4
[P1,P3,P5]	0		
P2	7	0	
P4	6	5	0



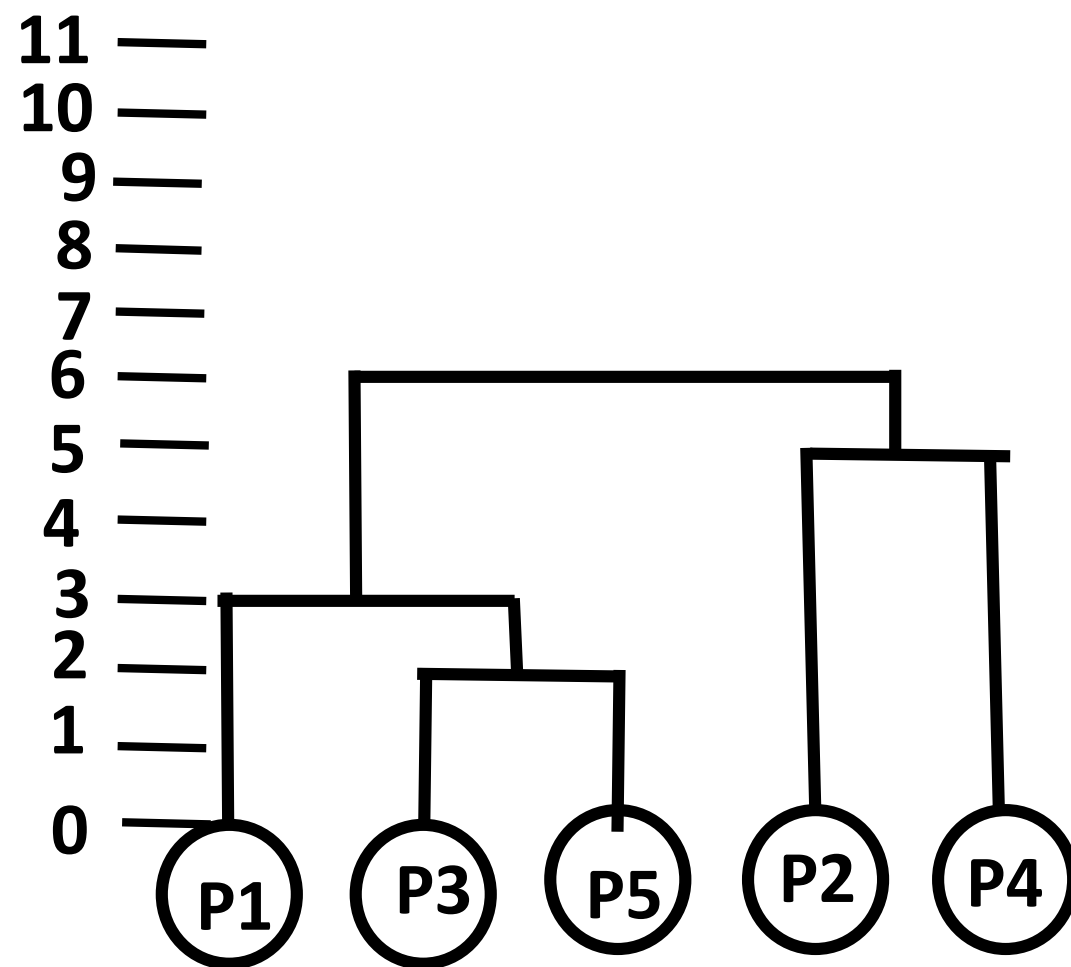
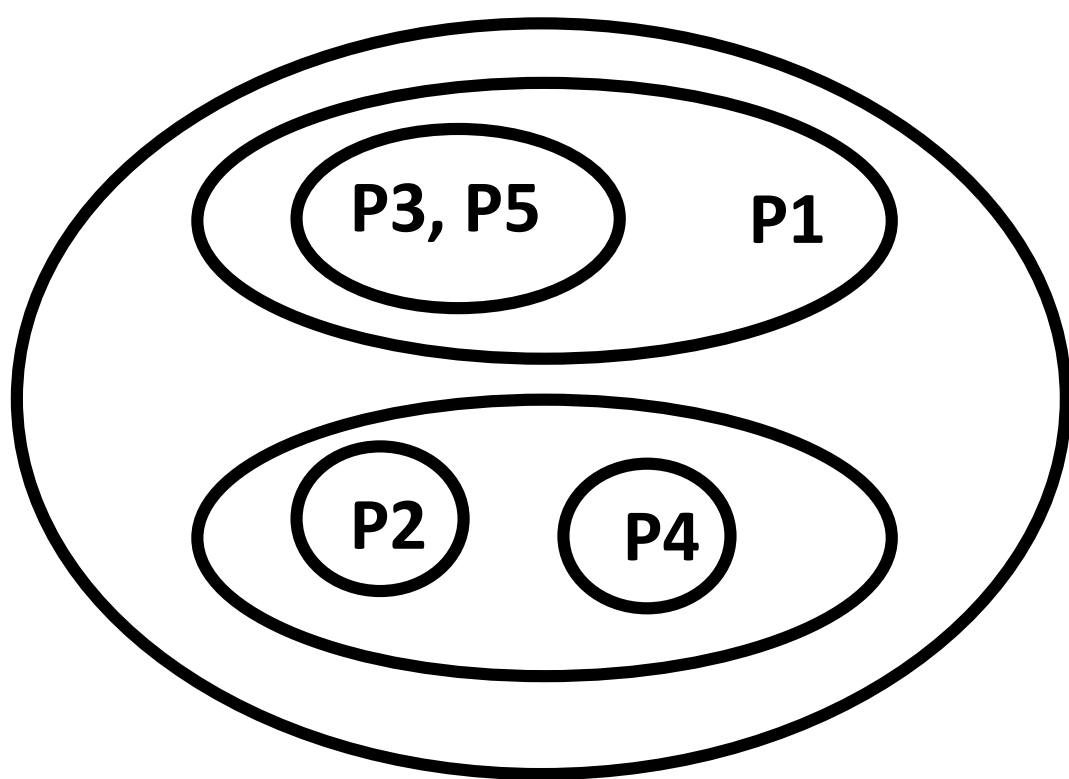
=> $d([p1, p3, p5], [p2, p4])$

=> $\text{Min}(d(p2, p1), d(p2, p3), d(p2, p5), d(p4, p1), d(p4, p3), d(p4, p5),)$

=> $\text{Min}(9, 7, 10, 6, 9, 8)$

=> 6

	[P1,P3,P5]	[P2, P4]
[P1,P3,P5]	0	
[P2,P4]	6	0



AL-ML-DL

AI

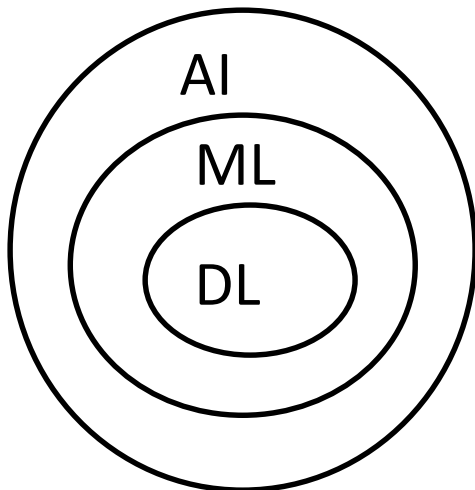
ML

DL

Machine to
Mimic human
Behaviour

Machine to
'learn'

Algo inspired
by structure
and function
of human brain.



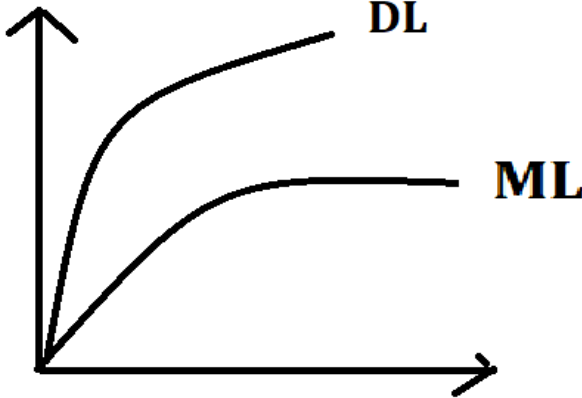
Training ↓
Testing ↑

Training ↑
Testing ↓

ML: Less Data

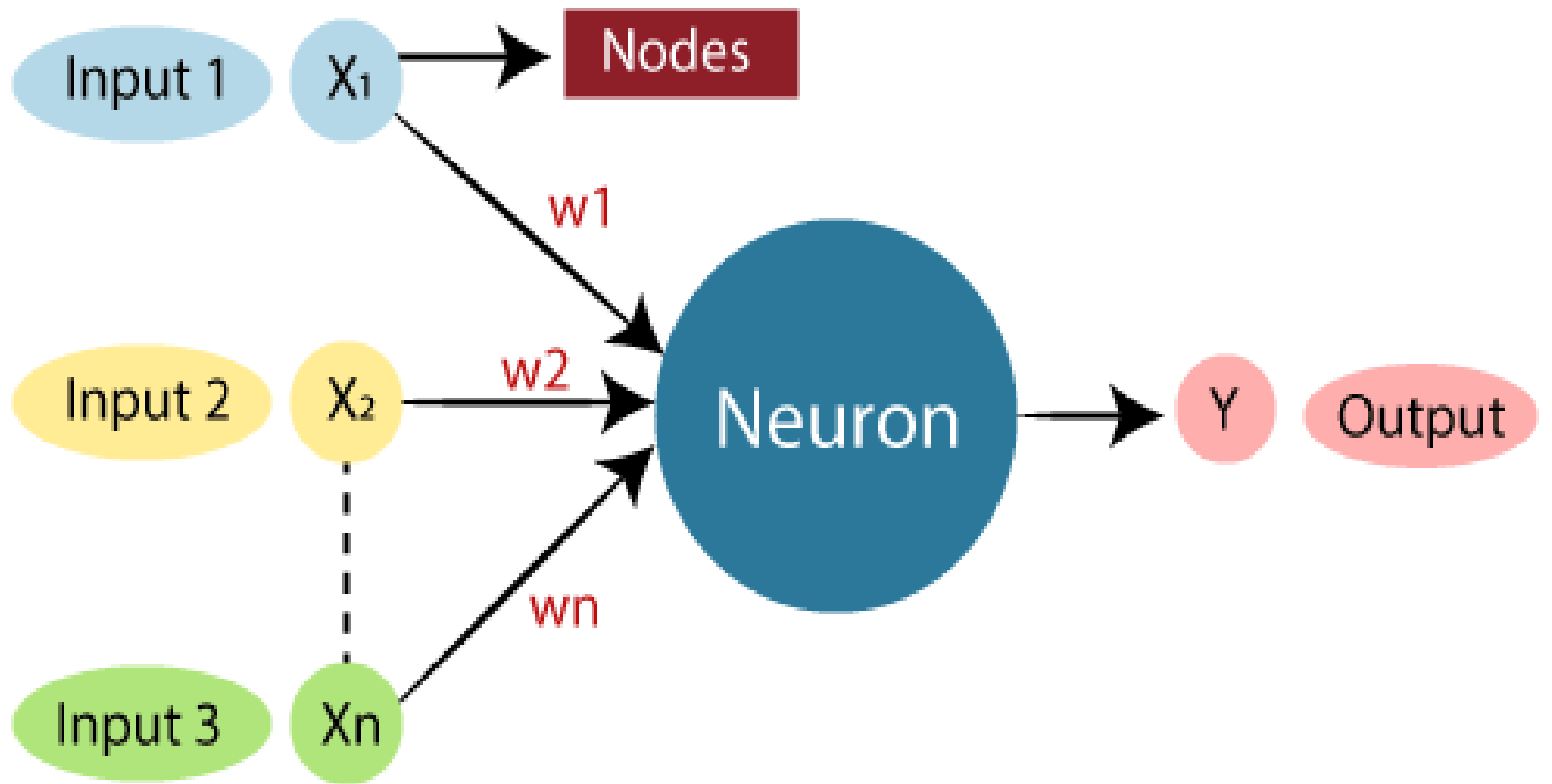
DL: Huge amount of Complex data

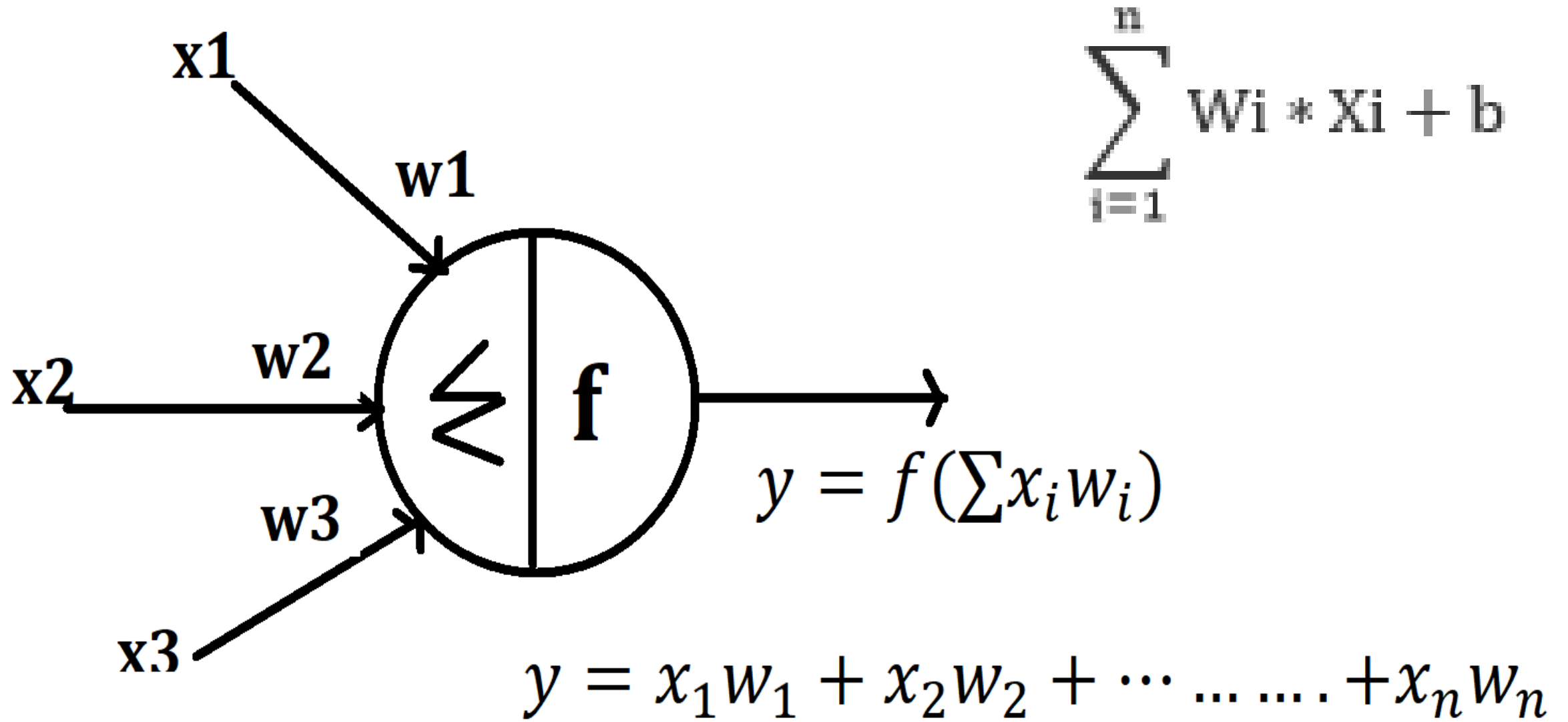
Deep Learning

Why?	What?	Where?
Huge Amount of Data	Handle Huge Amount of Structured and Unstructured Data	Medical Field
Complex Problems		Robotics
Feature Extraction	Complex Operations Problems Solved	Self Driving Cars
		Translation

ARTIFICIAL NEURAL NETWORK

ANN





An **Artificial Neural Network** attempts to mimic the network of neurons that makes up a human brain so that computers will have an option to understand things and make decisions in a human-like manner.

The artificial neural network is designed by programming computers to behave simply like interconnected brain cells.

ANNs are composed of multiple **nodes**, which imitate biological **neurons** of human brain.

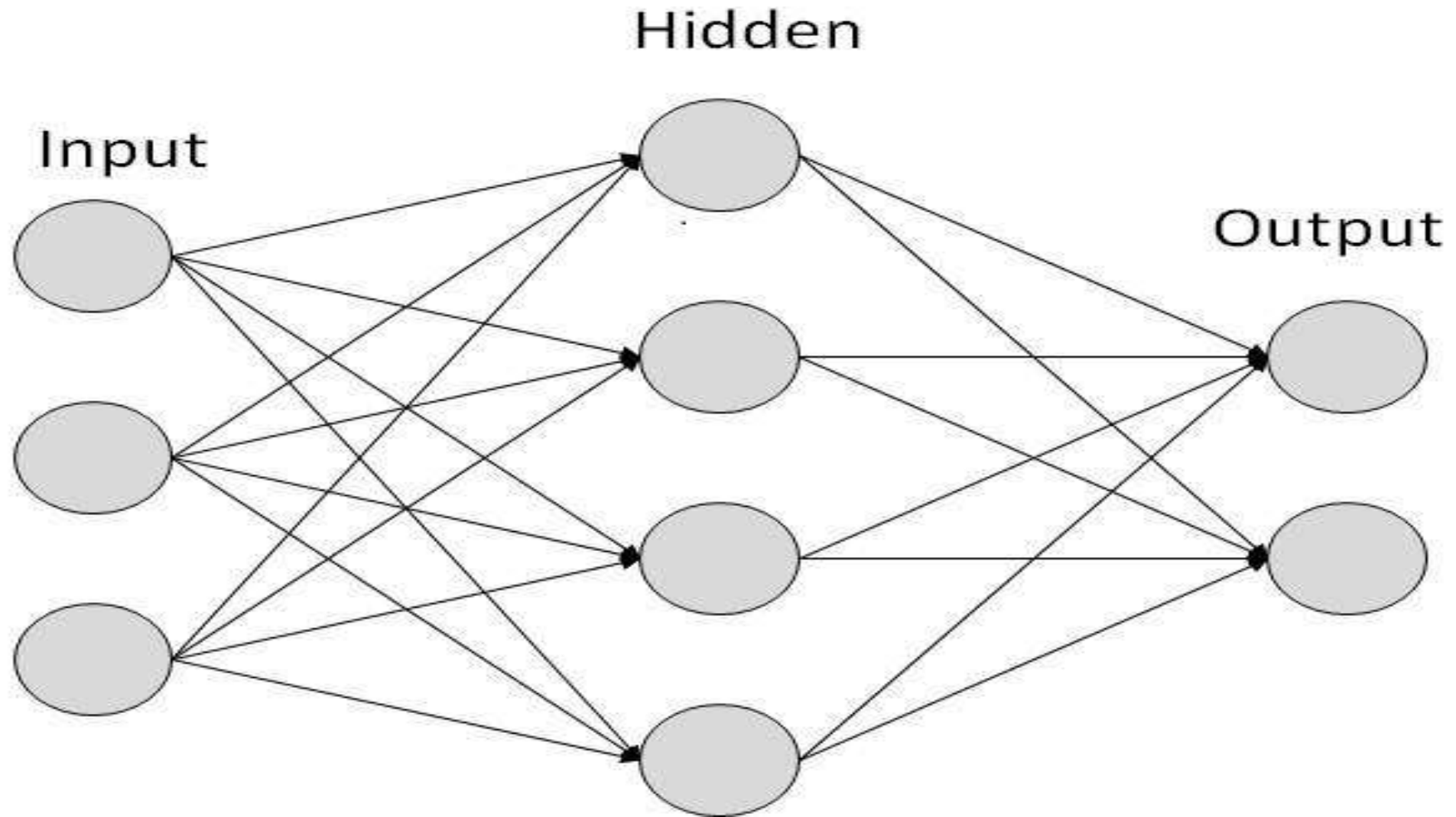
The neurons are connected by links and they interact with each other.

The nodes can take input data and perform simple operations on the data.

The result of these operations is passed to other neurons. The output at each node is called its **activation** or **node value**.

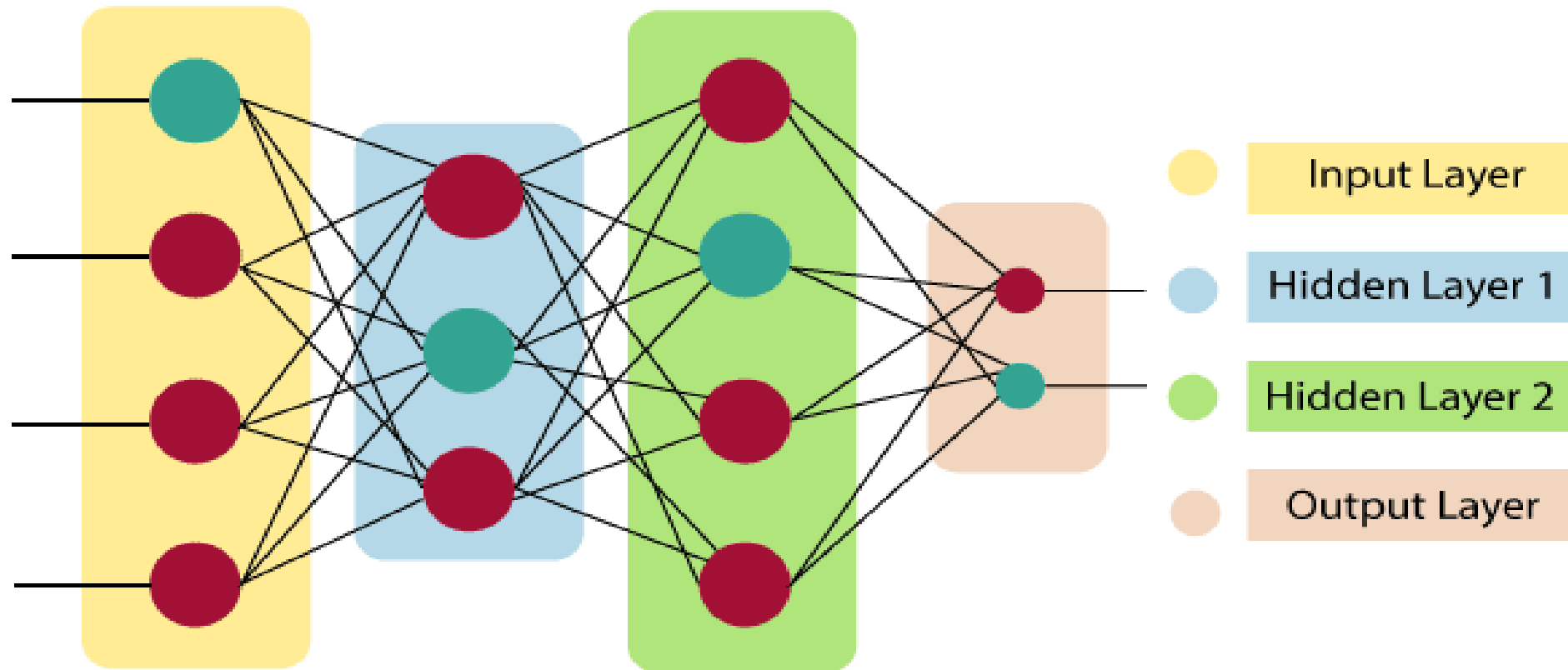
Each link is associated with weight. ANNs are capable of learning, which takes place by altering weight values.

Simple ANN -



The architecture of an artificial neural network:

Artificial Neural Network primarily consists of three layers:



Input Layer:

As the name suggests, it accepts inputs in several different formats provided by the programmer.

This layer accepts input features.

It provides information from the outside world to the network.

No computation is performed at this layer, nodes here just pass on the information(features) to the hidden layer.

Hidden Layer:

The hidden layer is in-between input and output layers. It performs all the calculations to find hidden features and patterns.

Nodes of this layer are not exposed to the outer world, they are the part of the abstraction provided by any neural network.

Hidden layer performs all sort of computation on the features entered through the input layer and transfer the result to the output layer.

Output Layer:

The input goes through a series of transformations using the hidden layer, which finally results in output that is conveyed using this layer.

This layer gives the information learned by the network to the outer world.

The artificial neural network takes input and computes the weighted sum of the inputs and includes a bias.

This computation is represented in the form of a transfer function.

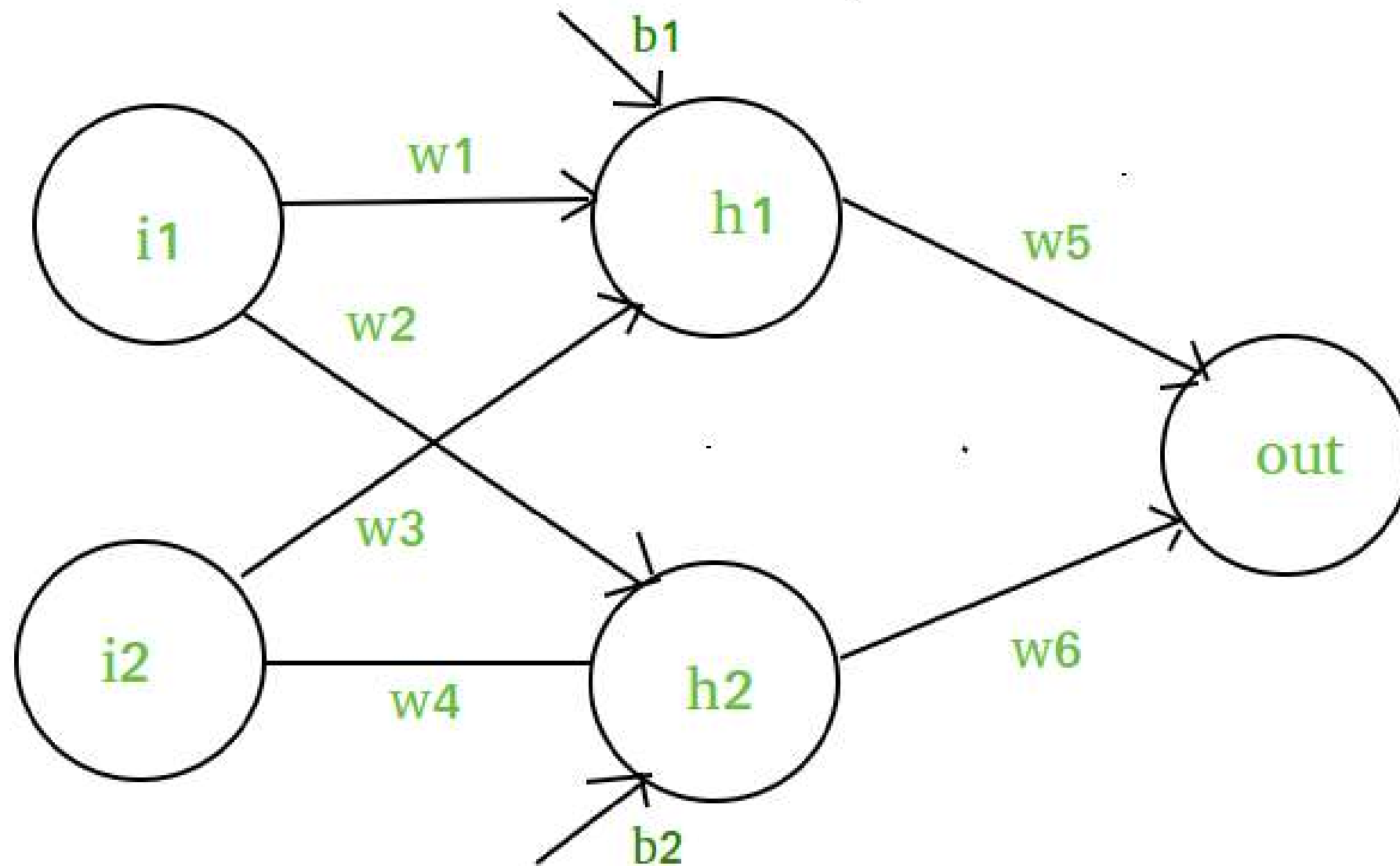
It determines weighted total which is passed as an input to an activation function to produce the output.

Activation functions choose whether a node should fire or not. Only those who are fired make it to the output layer. There are distinctive activation functions available that can be applied upon the sort of task we are performing.

Activation Function:

The activation function refers to the set of transfer functions used to achieve the desired output.

Definition of activation function:- Activation function decides, whether a neuron should be activated or not by calculating weighted sum and further adding bias with it. The purpose of the activation function is to **introduce non-linearity** into the output of a neuron.



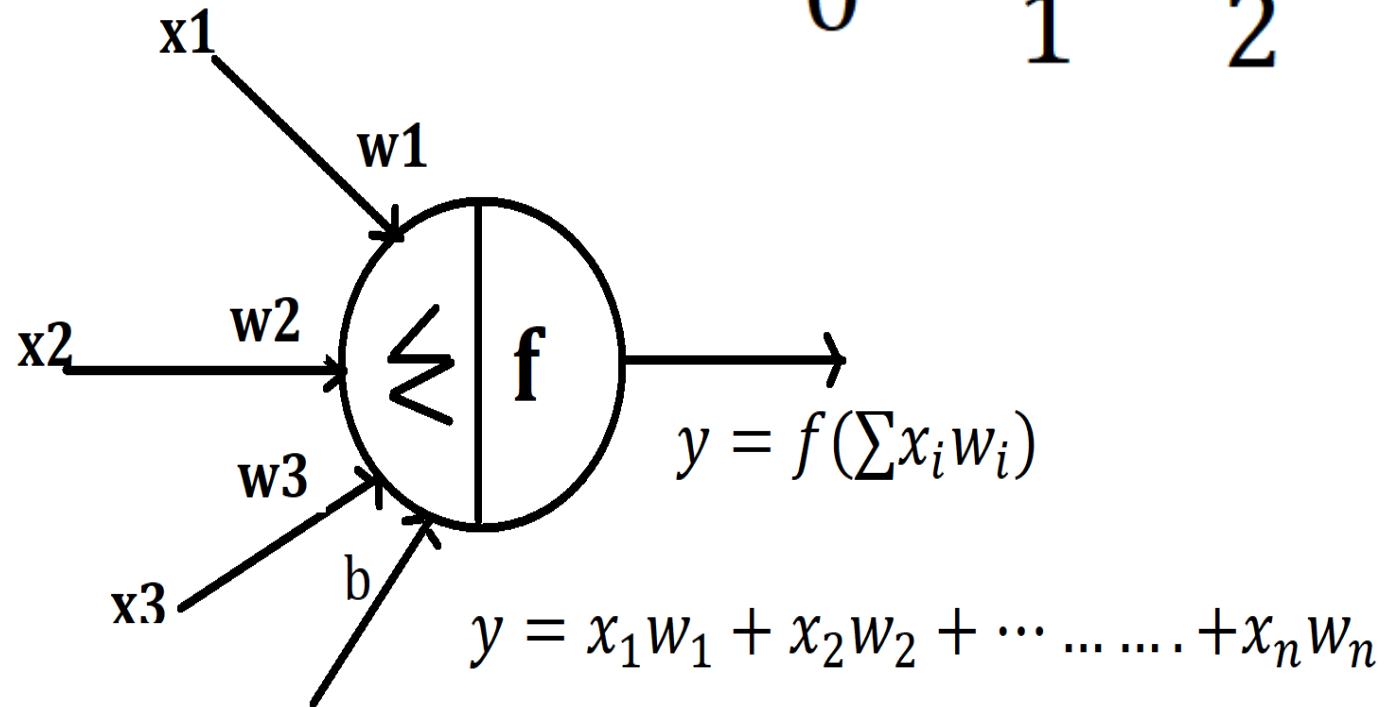
Activation Function:

1. Linear Activation Function:

$$f(v) = a + v$$

$$a + \sum x_i w_i$$

Where $a \rightarrow$ bias



1). Linear Function :-

- **Equation** : Linear function has the equation similar to as of a straight line
i.e. $y = ax$

- No matter how many layers we have, if all are linear in nature, the final activation function of last layer is nothing but just a linear function of the input of first layer.

- **Range** : $-\infty$ to $+\infty$

- **Uses** : **Linear activation function** is used at just one place i.e. output layer.

- **Issues** : If we will differentiate linear function to bring non-linearity, result will no more depend on *input* “ x ” and function will become constant, it won't introduce any ground-breaking behavior to our algorithm.

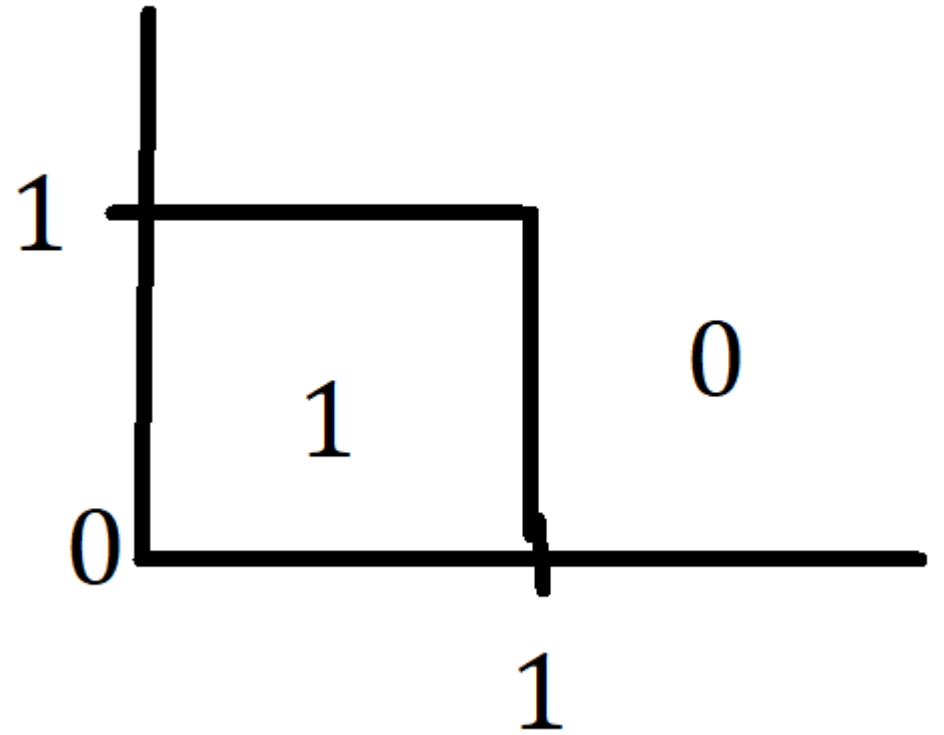
For example : Calculation of price of a house is a regression problem. House price may have any big/small value, so we can apply linear activation at output layer. Even in this case neural net must have any non-linear function at hidden layers.

2, Heviside Step Function:

$f(v) = 1$ if $v \geq \text{threshold value}$

0 otherwise

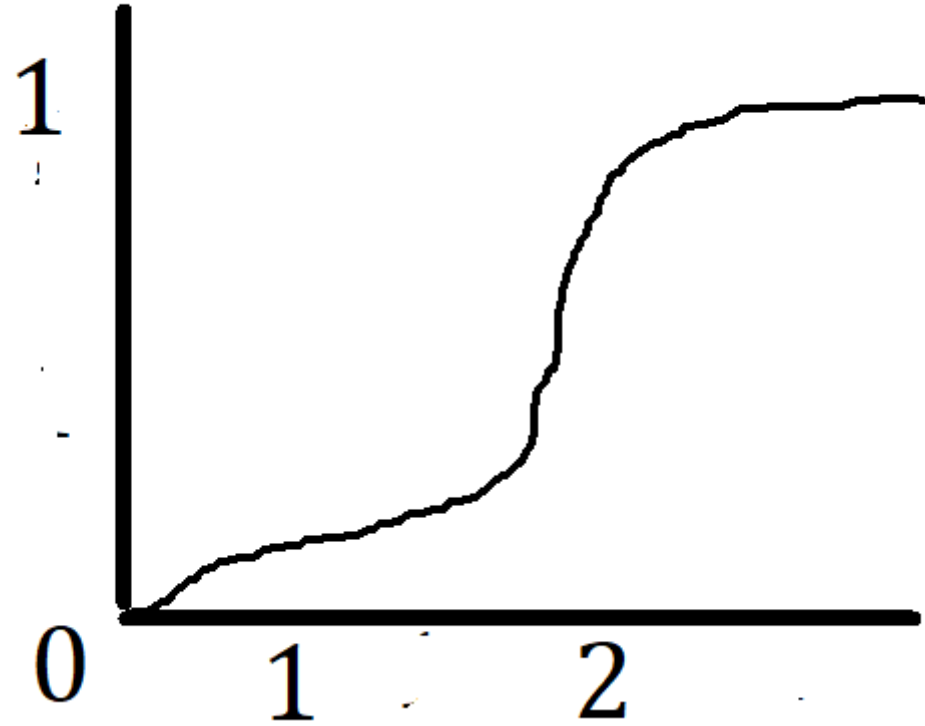
$$V = \sum x_i w_i$$



3. Sigmoid Function

$$f(v) = \frac{1}{1+e^{-v}}$$

$$V = \sum x_i w_i$$



2). Sigmoid Function :-

- It is a function which is plotted as '**S**' shaped graph.

- **Equation :**

$$A = 1/(1 + e^{-x})$$

- **Nature :** Non-linear. Notice that X values lies between -2 to 2, Y values are very steep. This means, small changes in x would also bring about large changes in the value of Y.

- **Value Range :** 0 to 1

- **Uses :** Usually used in output layer of a binary classification, where result is either 0 or 1, as value for sigmoid function lies between 0 and 1 only so, result can be predicted easily to be **1** if value is greater than **0.5** and **0** otherwise.

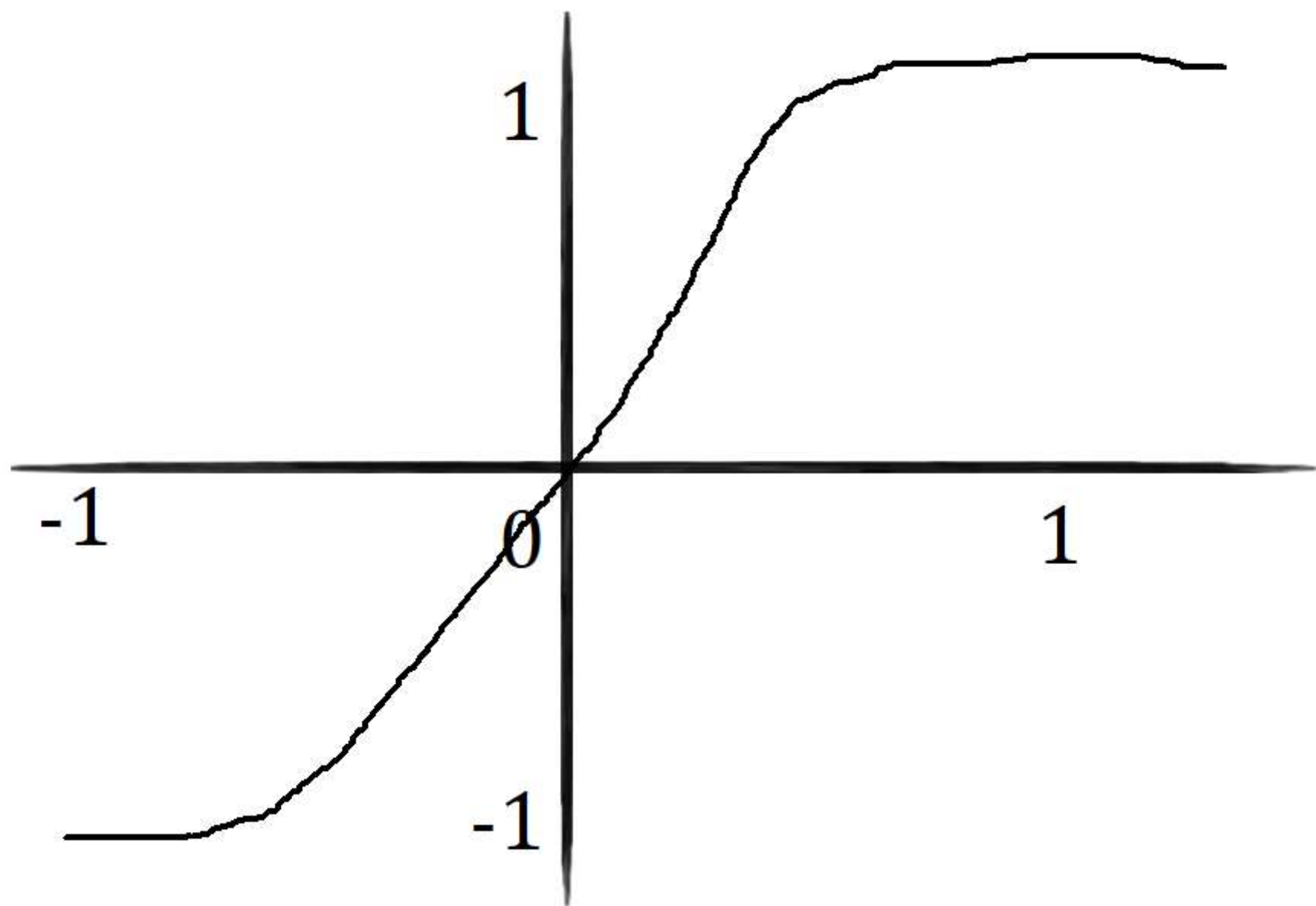
similar and can be derived from each other.

- **Equation** :- $f(x) = \tanh(x) = \frac{2}{1 + e^{-2x}} - 1$ OR $\tanh(x) = 2 * \text{sigmoid}(2x) - 1$

- **Value Range** :- -1 to +1

- **Nature** :- non-linear

- **Uses** :- Usually used in hidden layers of a neural network as its values lie between **-1 to 1** hence the mean for the hidden layer comes out to be 0 or very close to it, hence helps in *centering the data* by bringing mean close to 0. This makes learning for the next layer much easier.



•**4). RELU** :- Stands for *Rectified linear unit*. It is the most widely used activation function. Chiefly implemented in *hidden layers* of Neural network.

•**Equation** :- $A(x) = \max(0, x)$. It gives an output x if x is positive and 0 otherwise.

•**Value Range** :- $[0, \infty)$

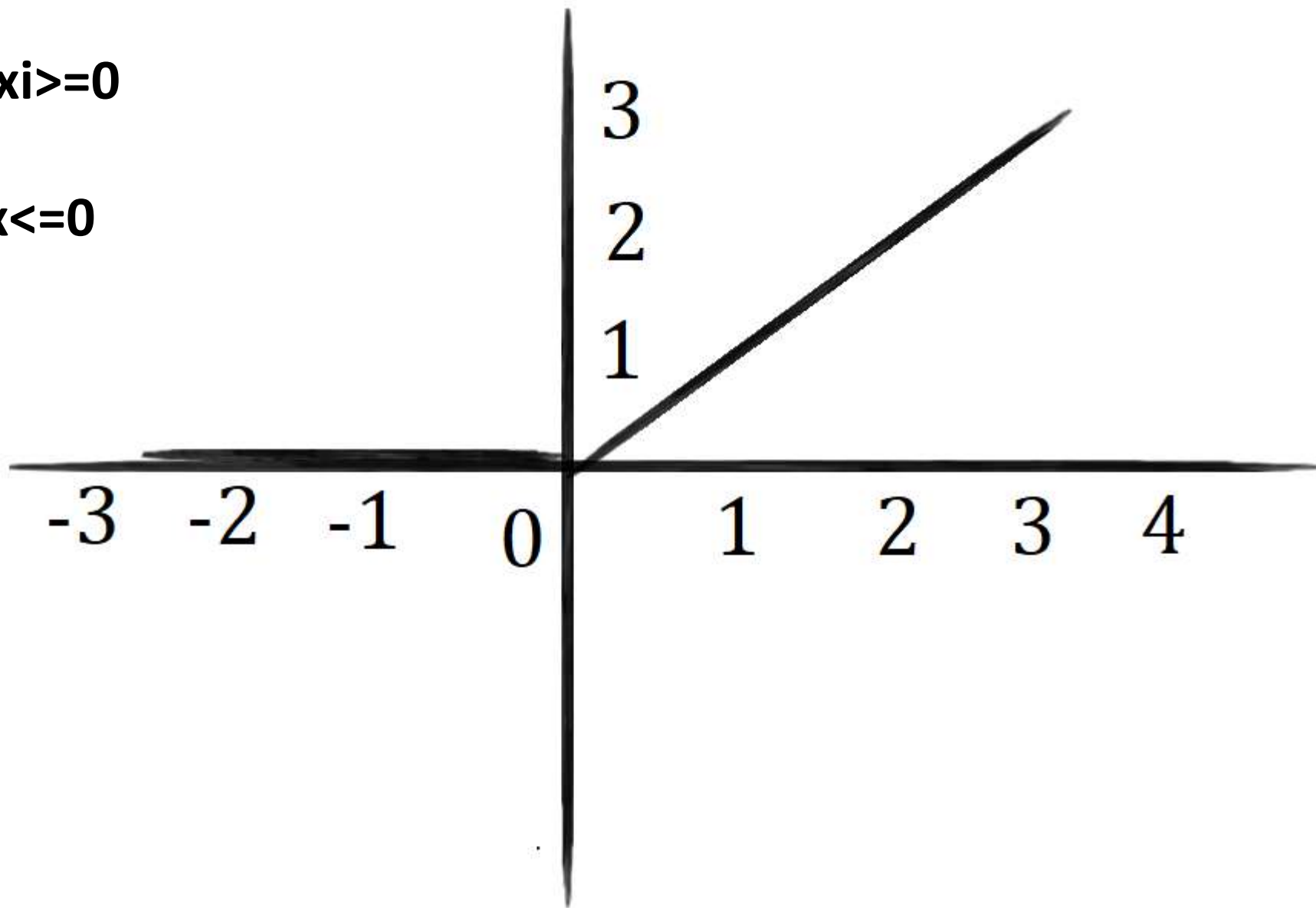
•**Nature** :- non-linear, which means we can easily backpropagate the errors and have multiple layers of neurons being activated by the ReLU function.

•**Uses** :- ReLu is less computationally expensive than tanh and sigmoid because it involves simpler mathematical operations. At a time only a few neurons are activated making the network sparse making it efficient and easy for computation.

In simple words, RELU learns *much faster* than sigmoid and Tanh function.

$$F(x) = \begin{cases} x & \text{if } x \geq 0 \end{cases}$$

$$0 \text{ if } x < 0$$



5). Softmax Function :- The softmax function is also a type of sigmoid function but is handy when we are trying to handle classification problems.

- Nature :-** non-linear

- Uses :-** Usually used when trying to handle multiple classes. The softmax function would squeeze the outputs for each class between 0 and 1 and would also divide by the sum of the outputs.

- Output:-** The softmax function is ideally used in the output layer of the classifier where we are actually trying to attain the probabilities to define the class of each input.

CHOOSING THE RIGHT ACTIVATION FUNCTION

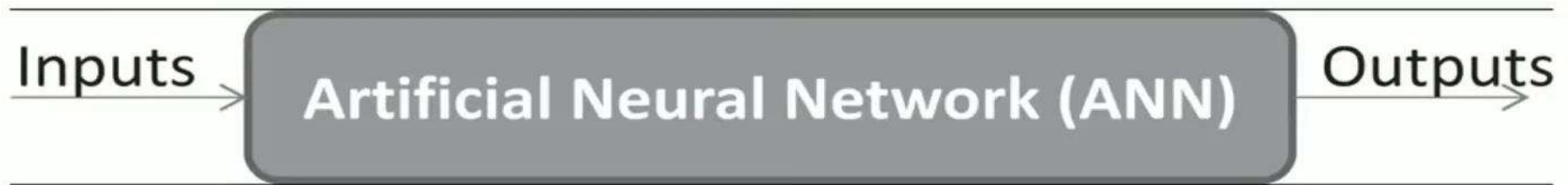
- The basic rule of thumb is if you really don't know what activation function to use, then simply use *RELU* as it is a general activation function and is used in most cases these days.

- If your output is for binary classification then, *sigmoid function* is very natural choice for output layer.

ARTIFICIAL NEURAL NETWORK

Artificial Neural Networks

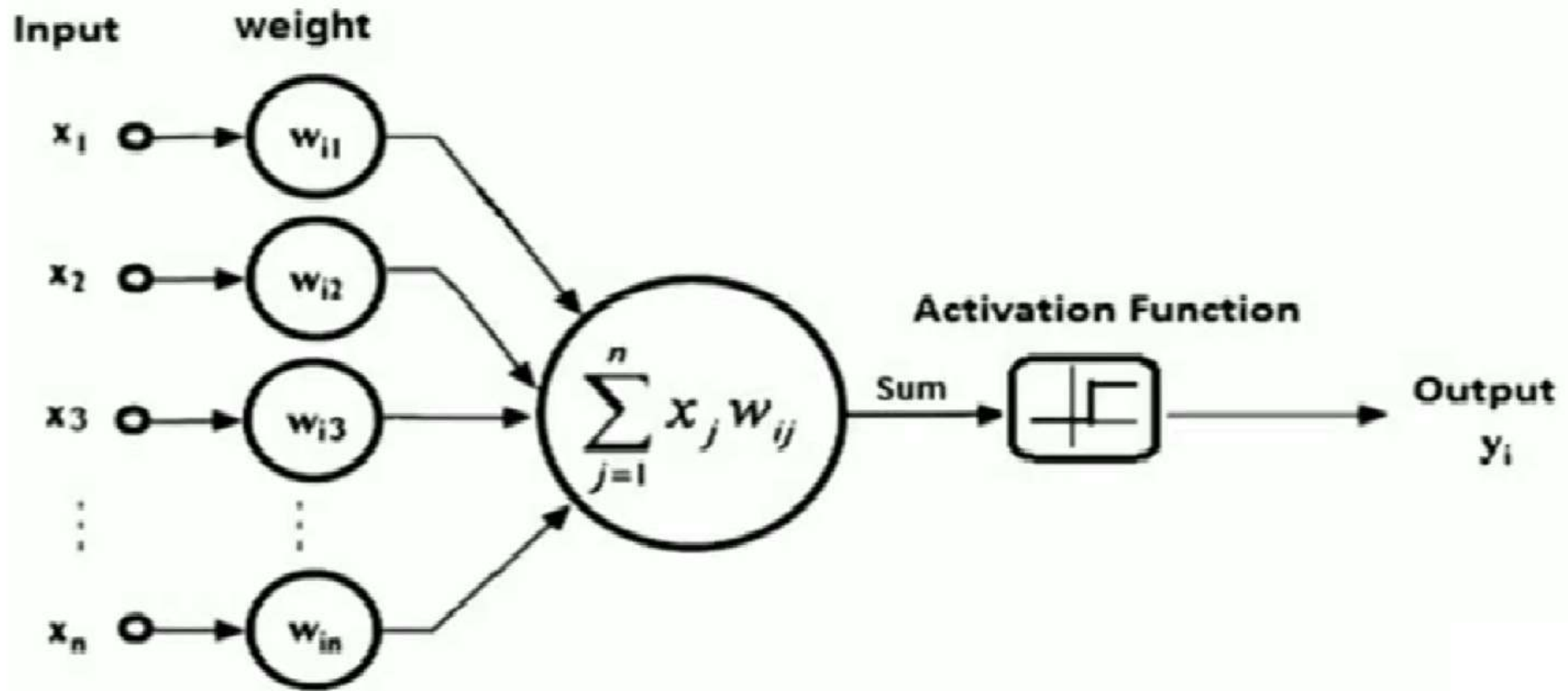
- Artificial neural networks (ANNs) are inspired by the information processing model of the human mind/brain.
- The human brain consists of billions of neurons that link with one another in an intricate pattern.
- Every neuron receives information from many other neurons, processes it, gets excited or not, and passes its state information to other neurons.



Business Applications of ANN

1. They are used in stock price prediction where the rules of the game are extremely complicated, and a lot of data needs to be processed very quickly.
2. They are used for character recognition, as in recognizing handwritten text, or damaged or mangled text.
3. They are used in recognizing finger prints. These are complicated patterns and are unique for each person. Layers of neurons can progressively clarify the pattern.
4. They are also used in traditional classification problems, such as approving a loan application.

Artificial neural network (ANN)



Developing an ANN

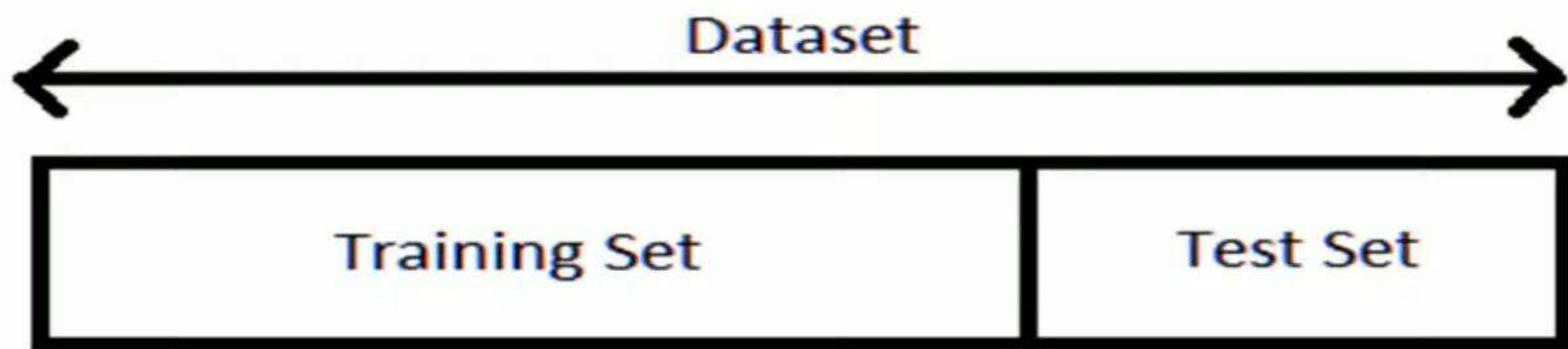
The steps required to build an ANN are as follows:

1. Gather data: Divide into training data and test data. The training data needs to be further divided into training data, validation data and testing data.
2. Select the network architecture, such as feedforward network.
3. Select the algorithm, such as Multilayer Perception.
4. Set network parameters.
5. Train the ANN with training data.
6. Validate the model with validation data.
7. Freeze the weights and other parameters.
8. Test the trained network with test data.
9. Deploy the ANN when it achieves good predictive accuracy.

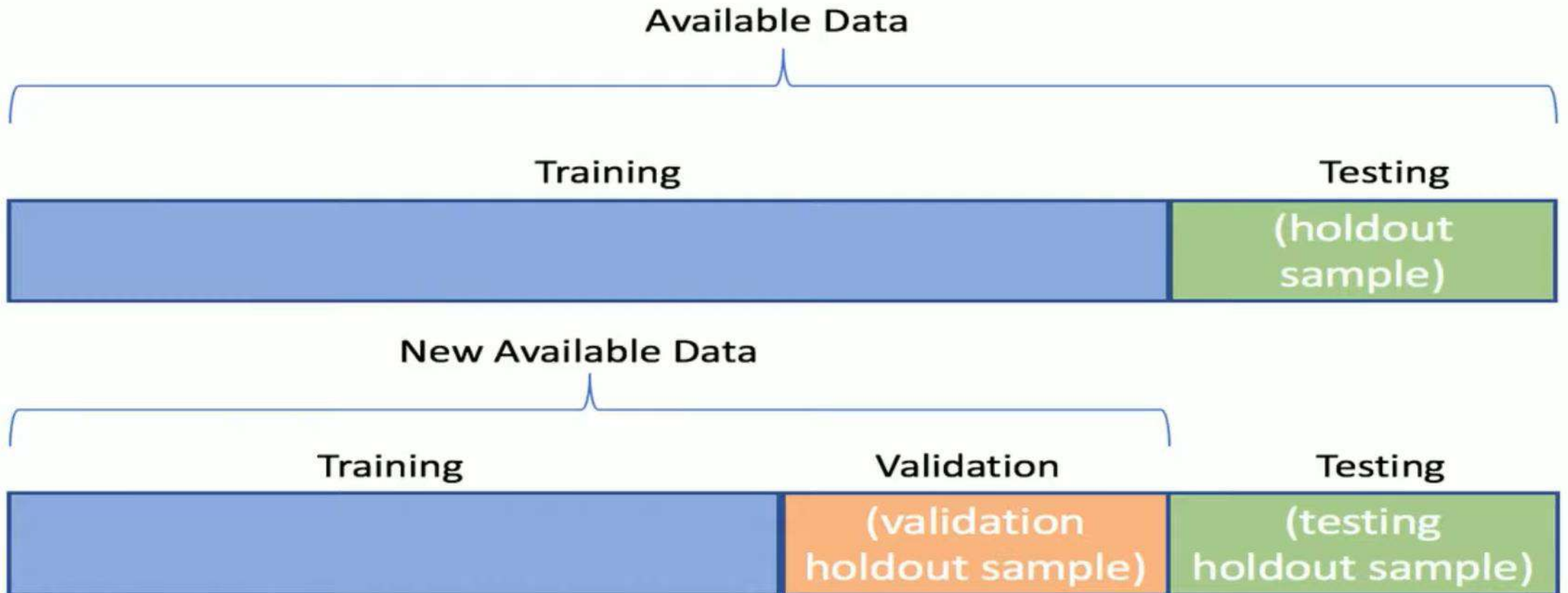
Training an ANN: Training data is split into three parts

Training set	This data set is used to adjust the weights on the neural network ($\sim 60\%$).
Validation set	This data set is used to minimize overfitting and verifying accuracy ($\sim 20\%$).
Testing set	This data set is used only for testing the final solution in order to confirm the actual predictive power of the network ($\sim 20\%$).
k-fold cross-validation	approach means that the data is divided into k equal pieces, and the learning process is repeated k -times with each pieces becoming the training set. This process leads to less bias and more accuracy, but is more time consuming.

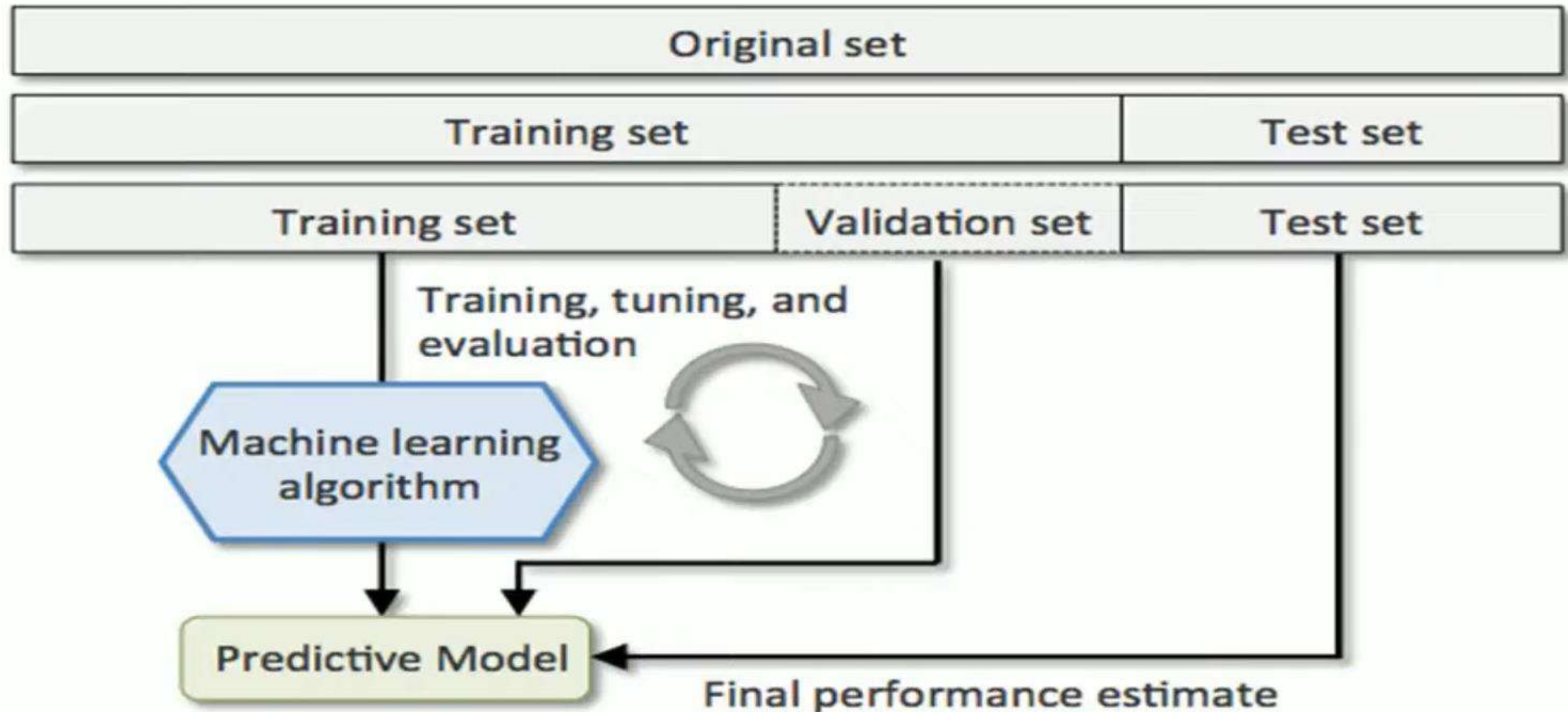
Train Test Distribution



Train Test Validation

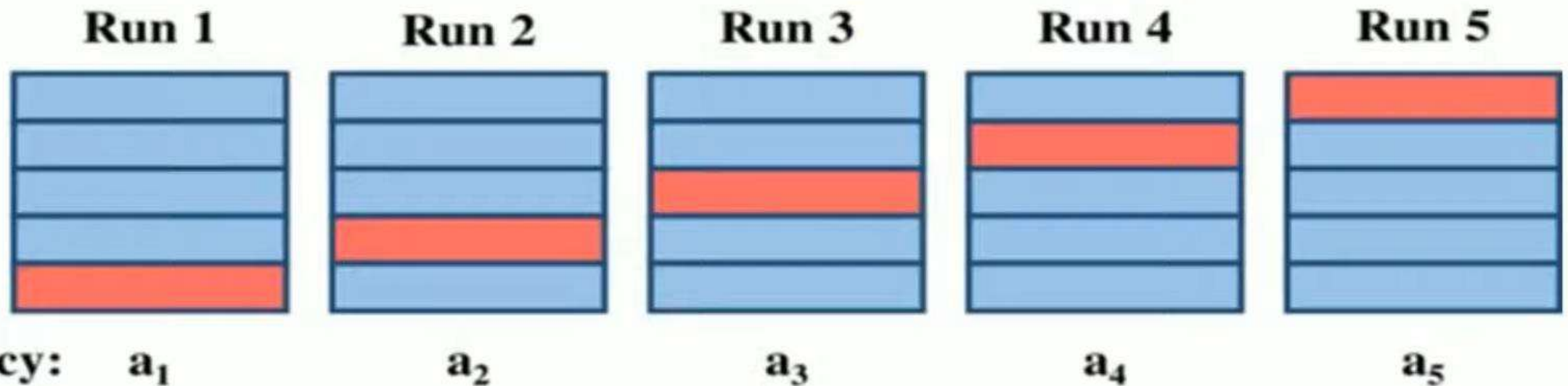


Train Test Validation



K-Fold Cross Validation

 Training dataset  Test dataset



$$\text{Average accuracy} = (a_1 + a_2 + a_3 + a_4 + a_5)/5$$

Advantages of Using ANNs

1. ANNs impose very little restrictions on their use.
2. There is no need to program ANN neural networks, as they learn from examples. They get better with use, without much programming effort.
3. ANN can handle a variety of problem types, including classification, clustering, associations, and so on.
4. ANNs are tolerant of data quality issues, and they do not restrict the data to follow strict normality and/or independence assumptions.
5. ANN can handle both numerical and categorical variables.
6. ANNs can be much faster than other techniques.
7. Most importantly, ANN usually provide better results (prediction and/or clustering) compared to statistical counterparts, once they have been trained enough.

Disadvantages of Using ANNs

1. They are deemed to be black-box solutions, lacking explainability.
2. Optimal design of ANN is still an art: It requires expertise and extensive experimentation.
3. It can be difficult to handle a large number of variables (especially the rich nominal attributes) with an ANN.
4. It takes large data sets to train an ANN.

Appropriate Problems – for ANN

Most appropriate for problems where

- Instances have many attribute-value pairs
- Target function output may be discrete-valued, real-valued, or a vector of several real- or discrete-valued attributes
- Training examples may contain errors
- Long training times are acceptable
- Fast evaluation of the learned target function may be required
- The ability for humans to understand the learned target function is not important

ARTIFICIAL NEURAL NETWORK TRAINING

Back Propagation

$$Y = X_1W_1 + X_2W_2 + X_3W_3 + B$$

$$Z = \frac{1}{1 + e^Y}$$

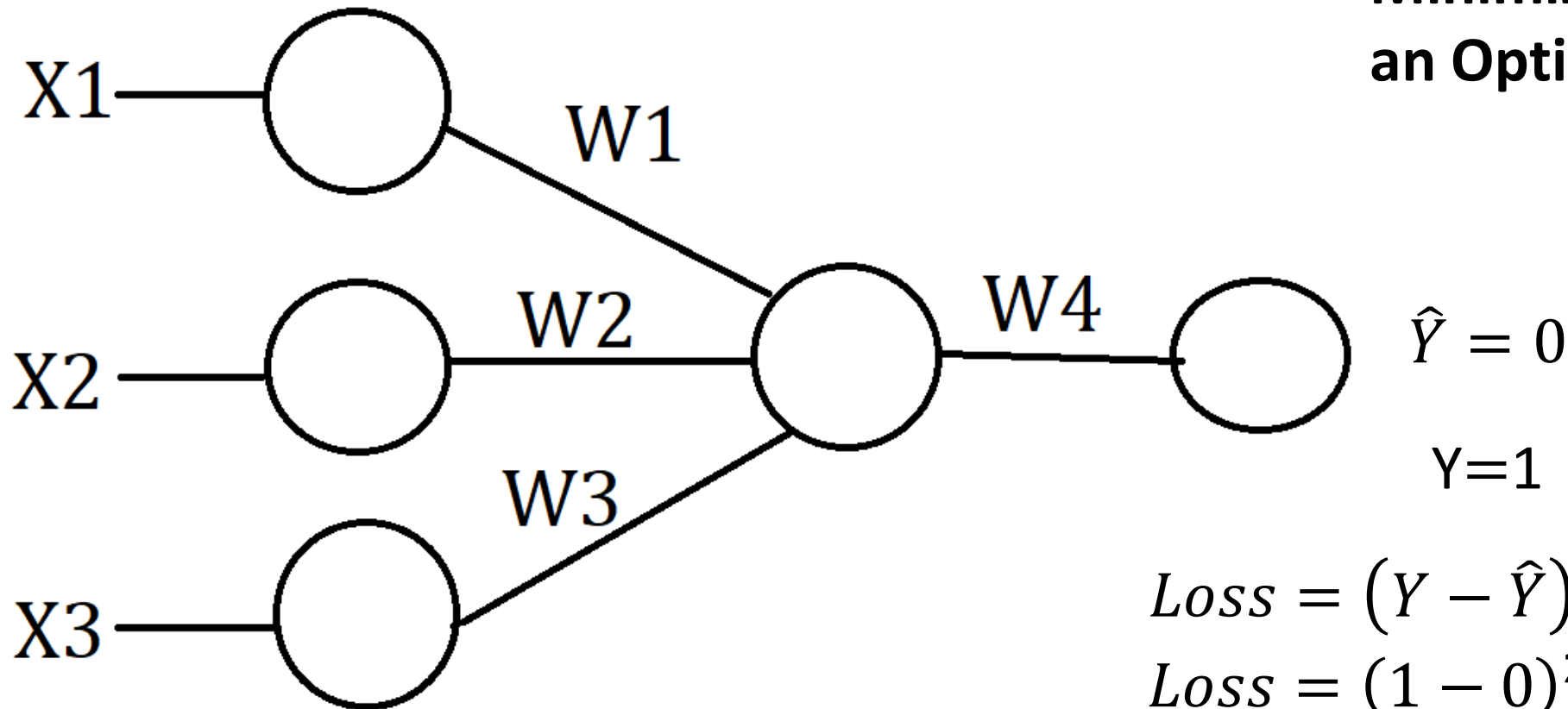
PLAY
2H

STUDY
4H

SLEEP
8H

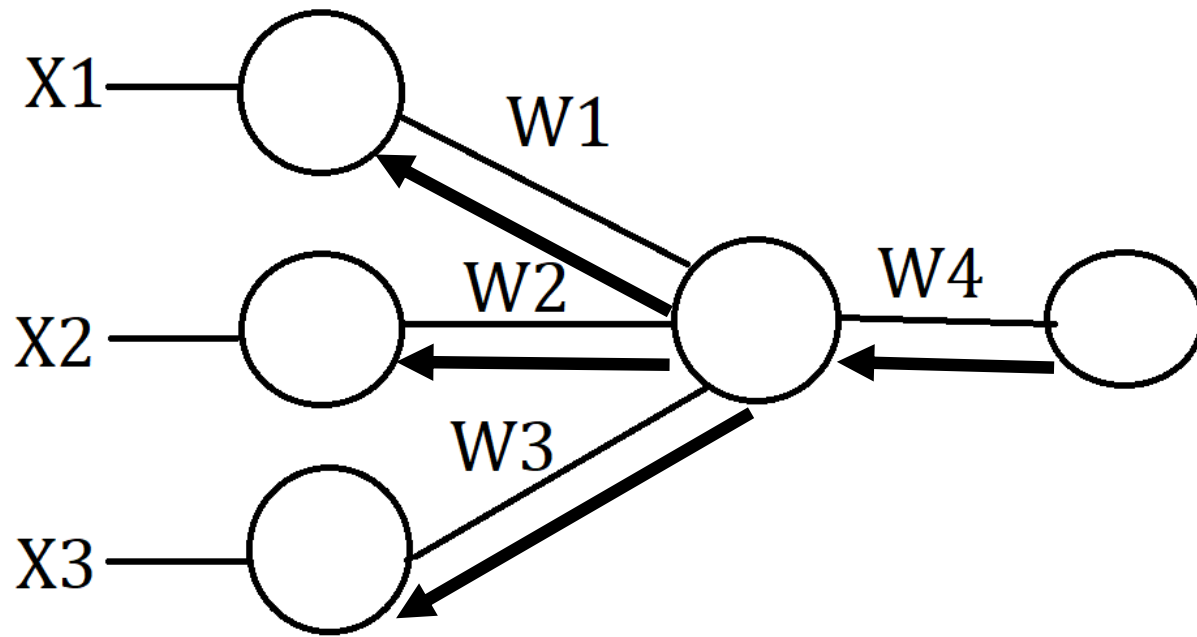
O/P
1

**Minimize Loss using
an Optimizer**



$$Loss = (Y - \hat{Y})^2$$

$$Loss = (1 - 0)^2 = 1 \rightarrow \text{reduce}$$



$$W4_{new} = W4_{old} - \eta \frac{\partial L}{\partial W4_{old}}$$

Where η is the learning rate.

η can be a small value like 0.001

$$W1_{new} = W1_{old} - \eta \frac{\partial L}{\partial W1_{old}}$$

$$W2_{new} = W2_{old} - \eta \frac{\partial L}{\partial W2_{old}}$$

$$W3_{new} = W3_{old} - \eta \frac{\partial L}{\partial W3_{old}}$$

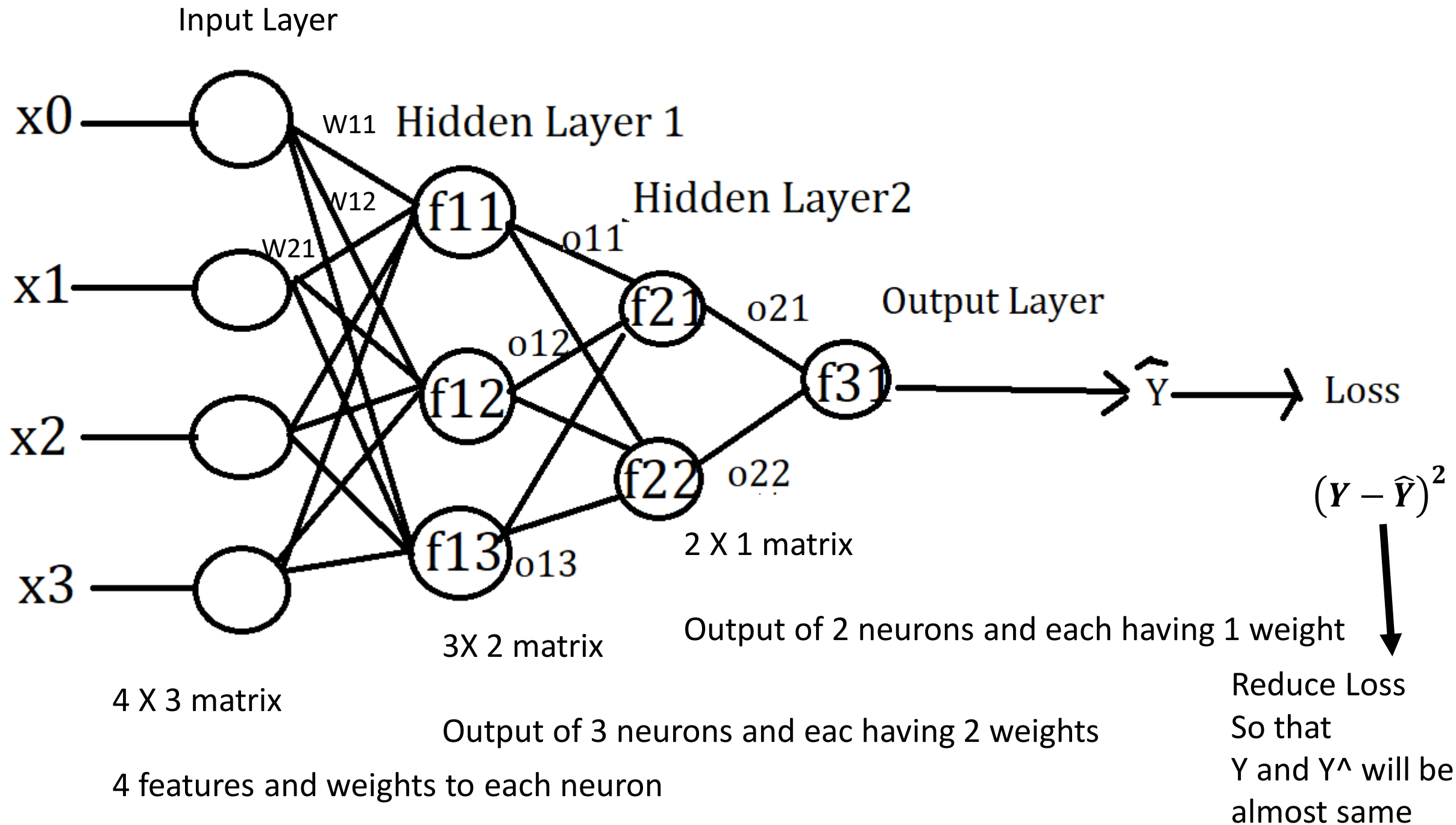
Since we have considered only one record, we calculate the loss function.

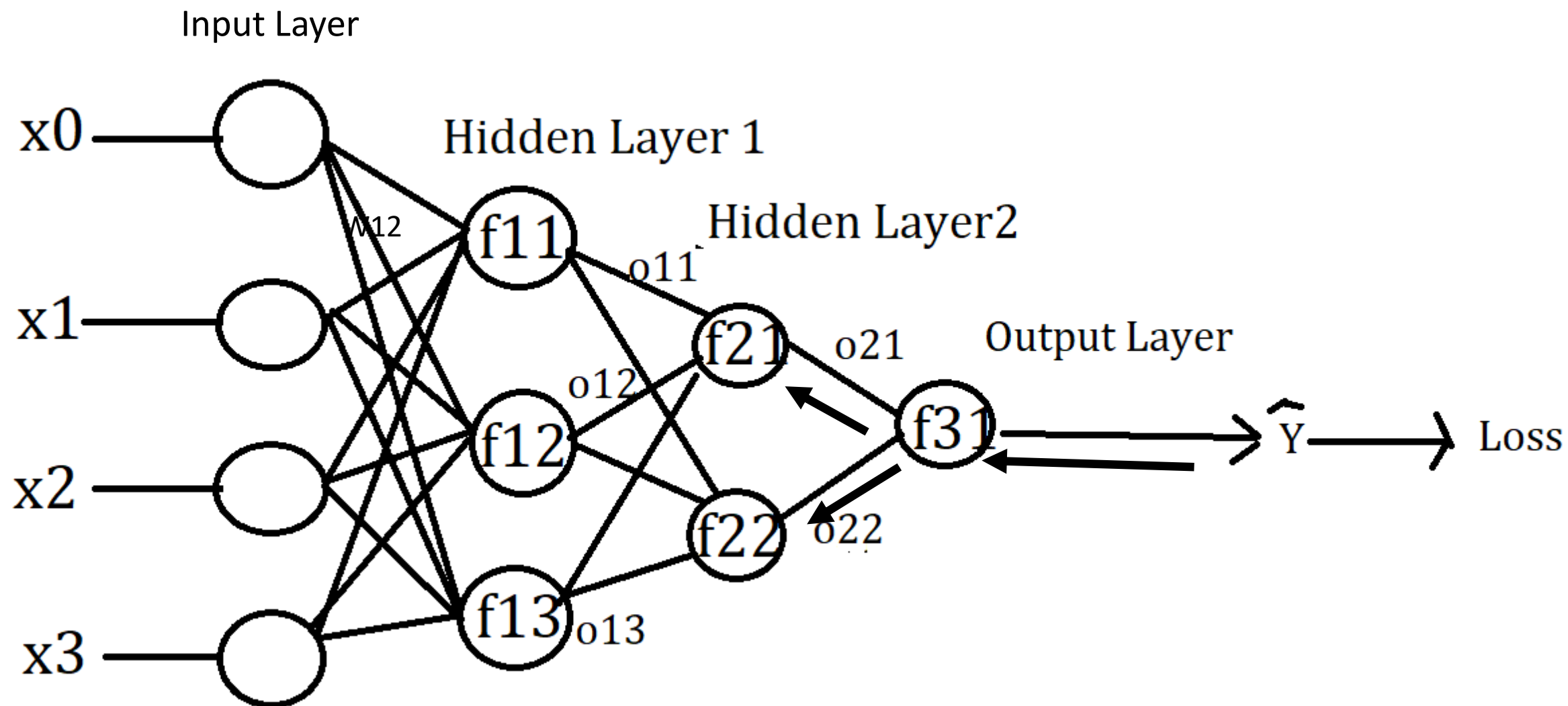
But if we have multiple records, then we calculate the cost function

$$Cost = \sum_{i=1}^n (Y - \hat{Y})^2$$

Artificial Neural Network

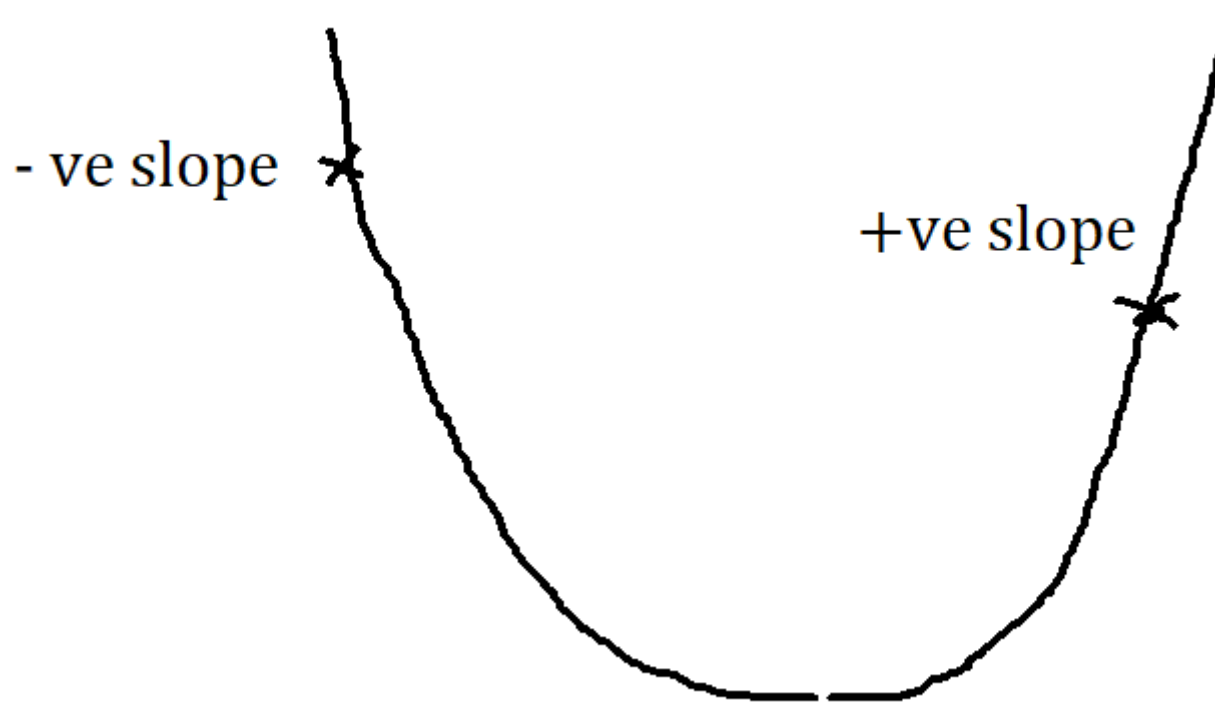
Gradient Decsnt





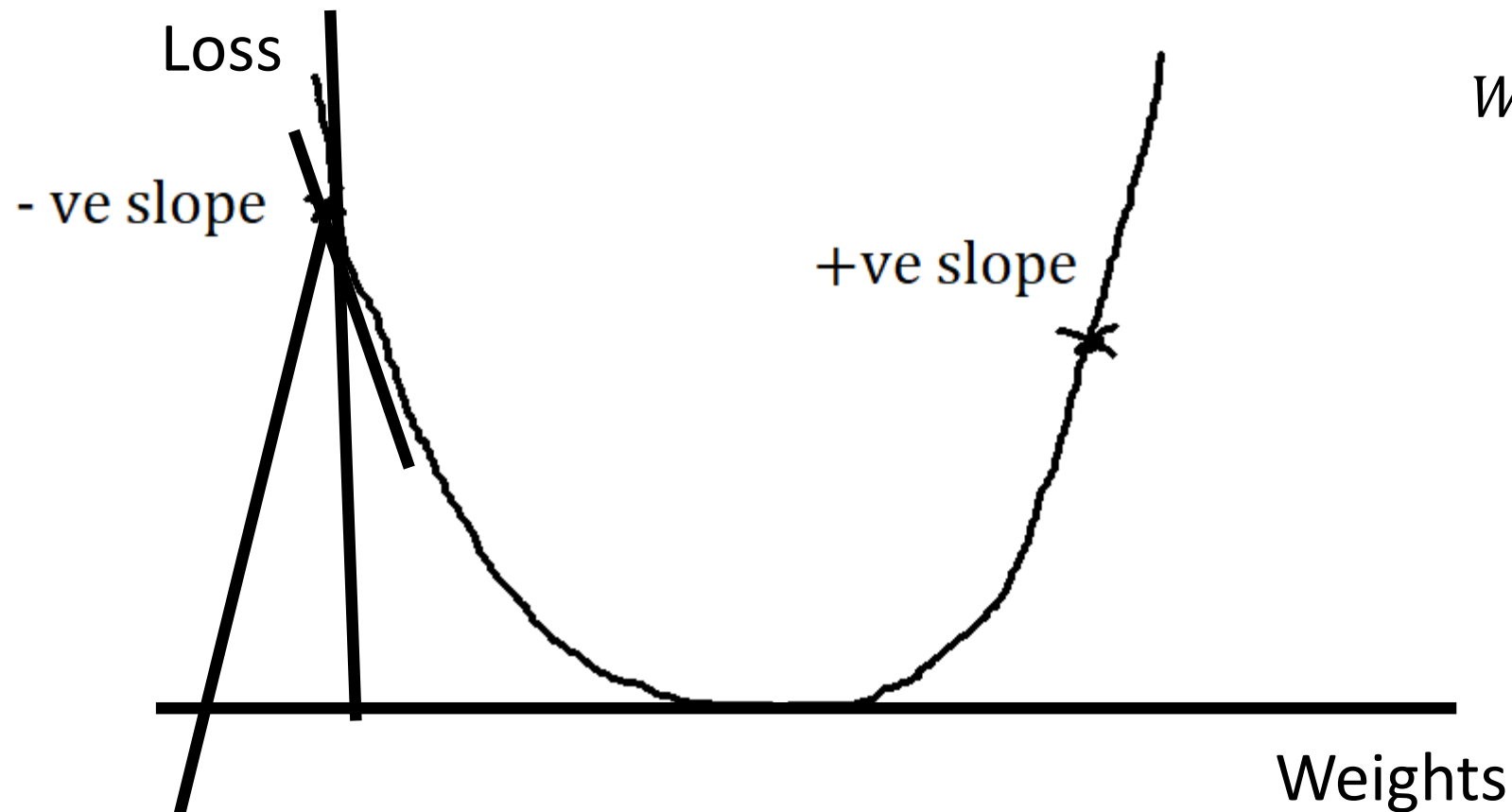
GRADIENT DESCENT: Is an Optimizer

Weight Optimization



$$W_{new} = W_{old} - \eta \frac{\delta L}{\delta W_{old}}$$

Its job is to optimize the weights in such a way that $\hat{Y}$ become almost similar to or equal to Y .



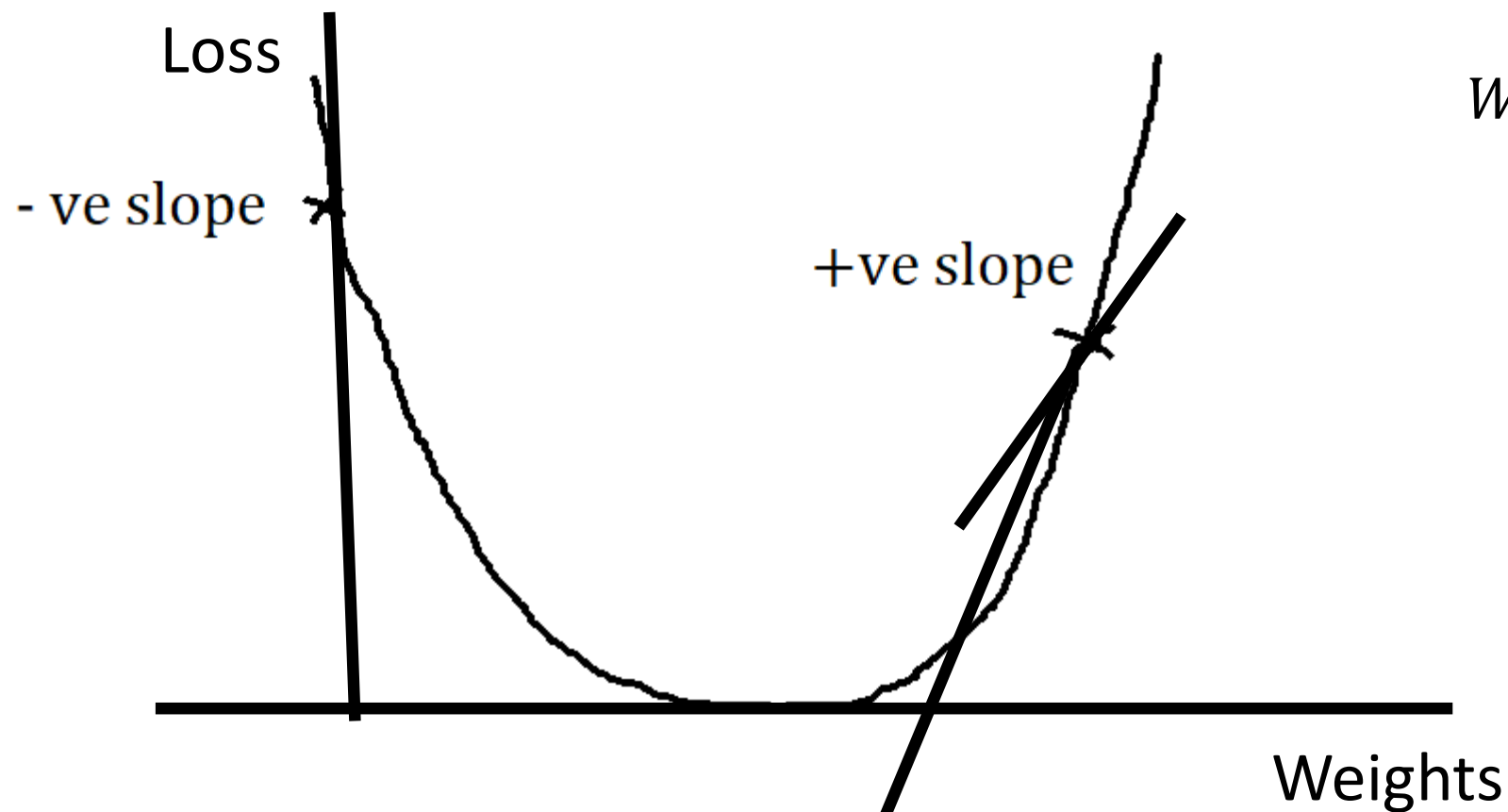
$$W_{new} = W_{old} - \eta \frac{\delta L}{\delta W_{old}}$$

Derivative is basically the slope.

Imagine this is the initial weights.

The derivative will help is find out if the slope is +ve or -ve.

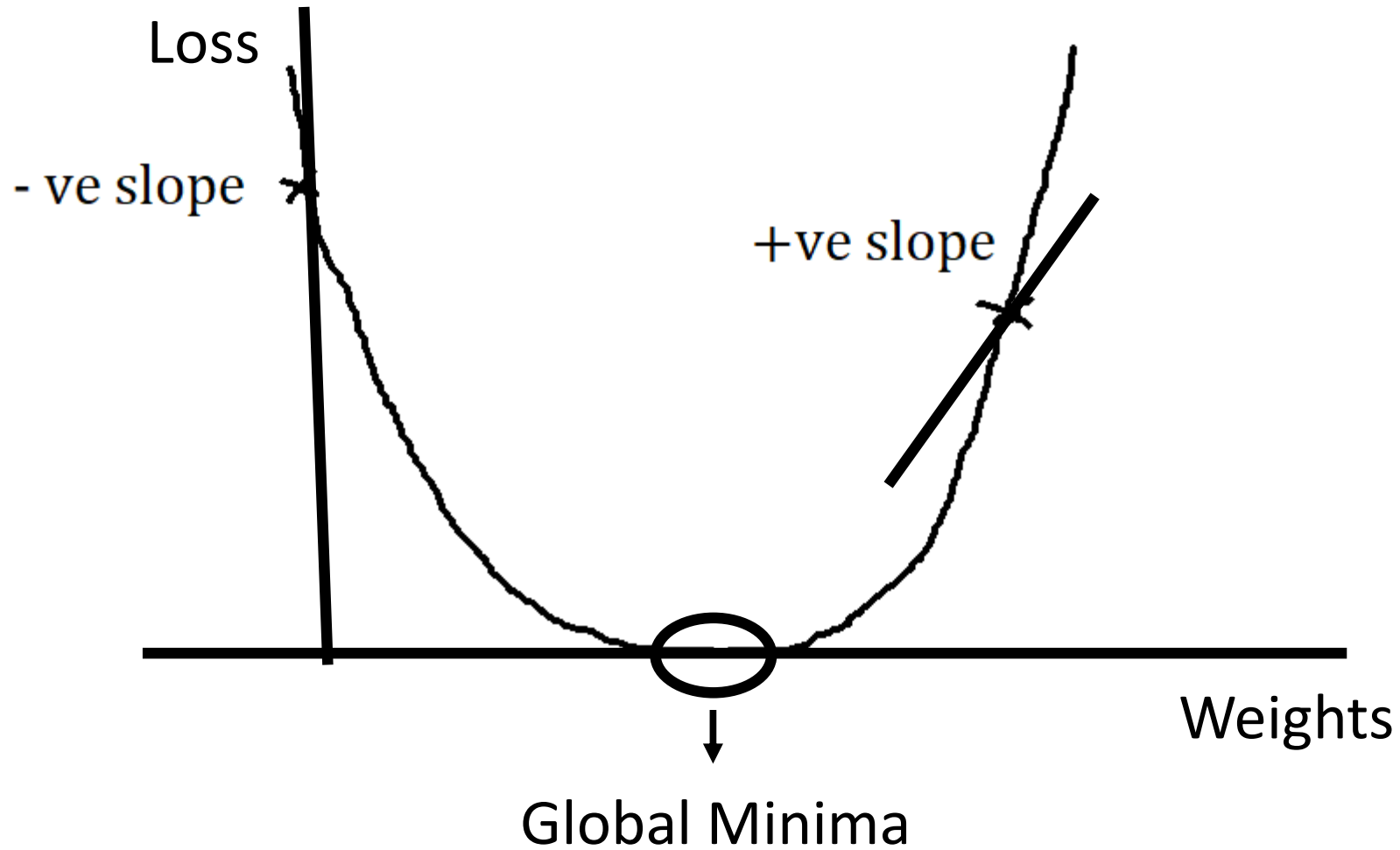
When we draw a line at the point, if the right side of the line is pointing down, then the slope of the line is $-ve$.



$$W_{new} = W_{old} - \eta \frac{\delta L}{\delta W_{old}}$$

Imagine this is the initial weights.

When we draw a line at the point, if the right side of the line is pointing up, then the slope of the line is +ve.



Our main aim is to reach the global minima.

Till we reach the global minima, we will be continuously back propagating.

The loss is decreasing means, we are approaching global minima.

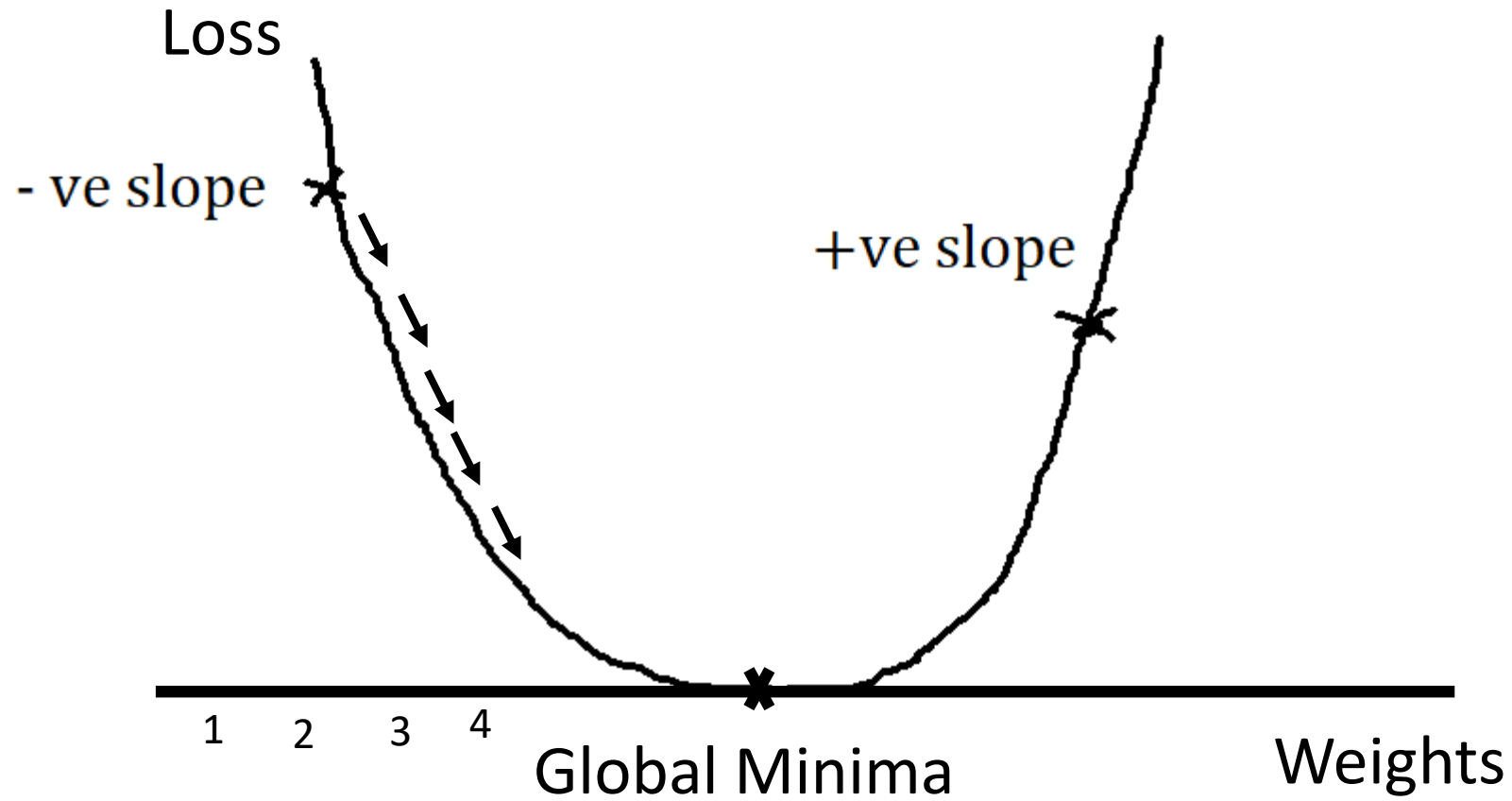
Or the difference between Y and $Y^{\wedge}$ is decreasing.

If the loss is not decreasing means, the point is rotating or bouncing between the gradient descent curve.

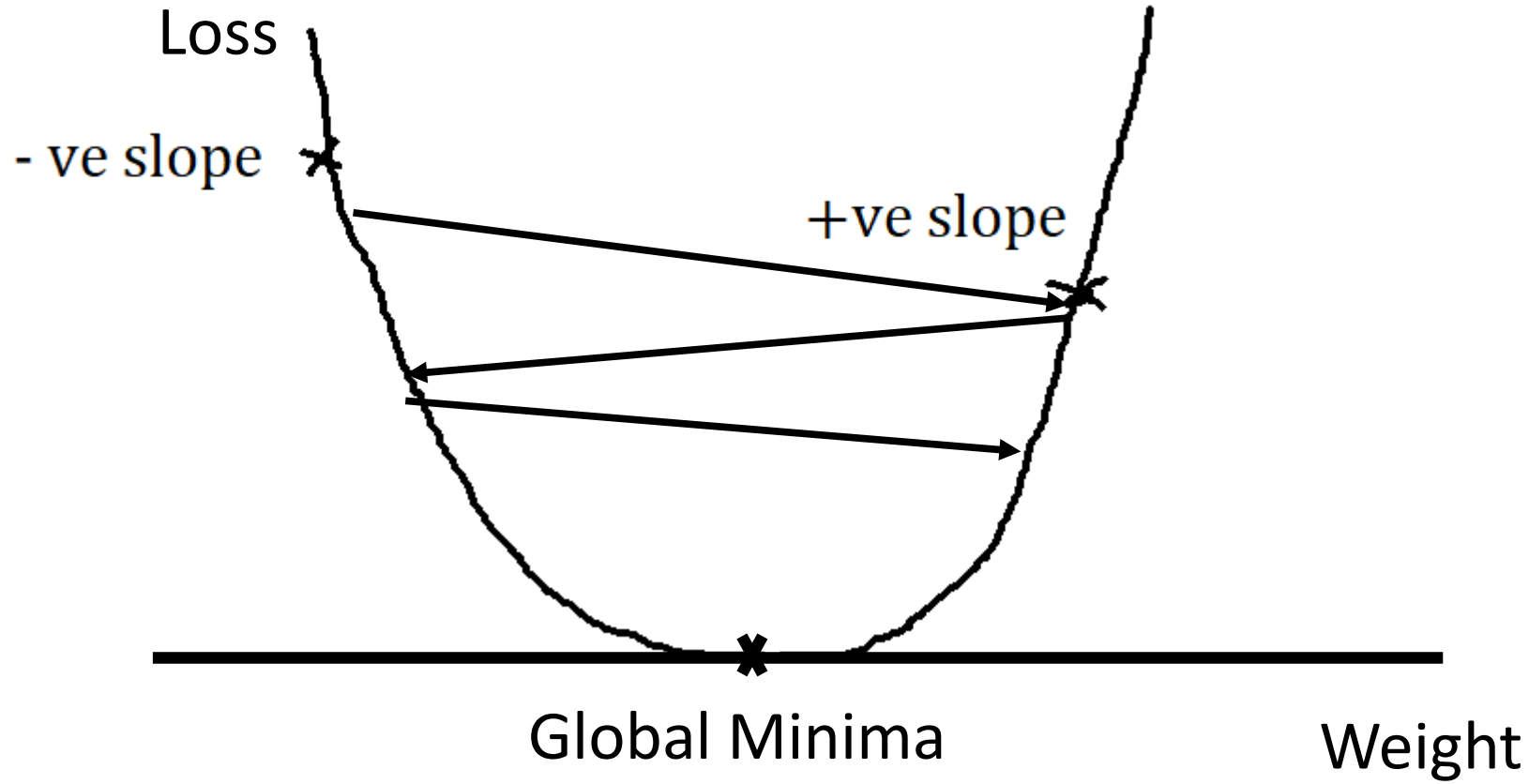
Now if the slope is $-ve$, then the derivative is negative.

So $-\eta \left(-\frac{\delta L}{\delta W_{old}} \right)$ *will be a + ve value.*

That is our point goes on moving towards the global minima.



If the learning rate is very small, then the point will go on slowly moving towards the global minima.



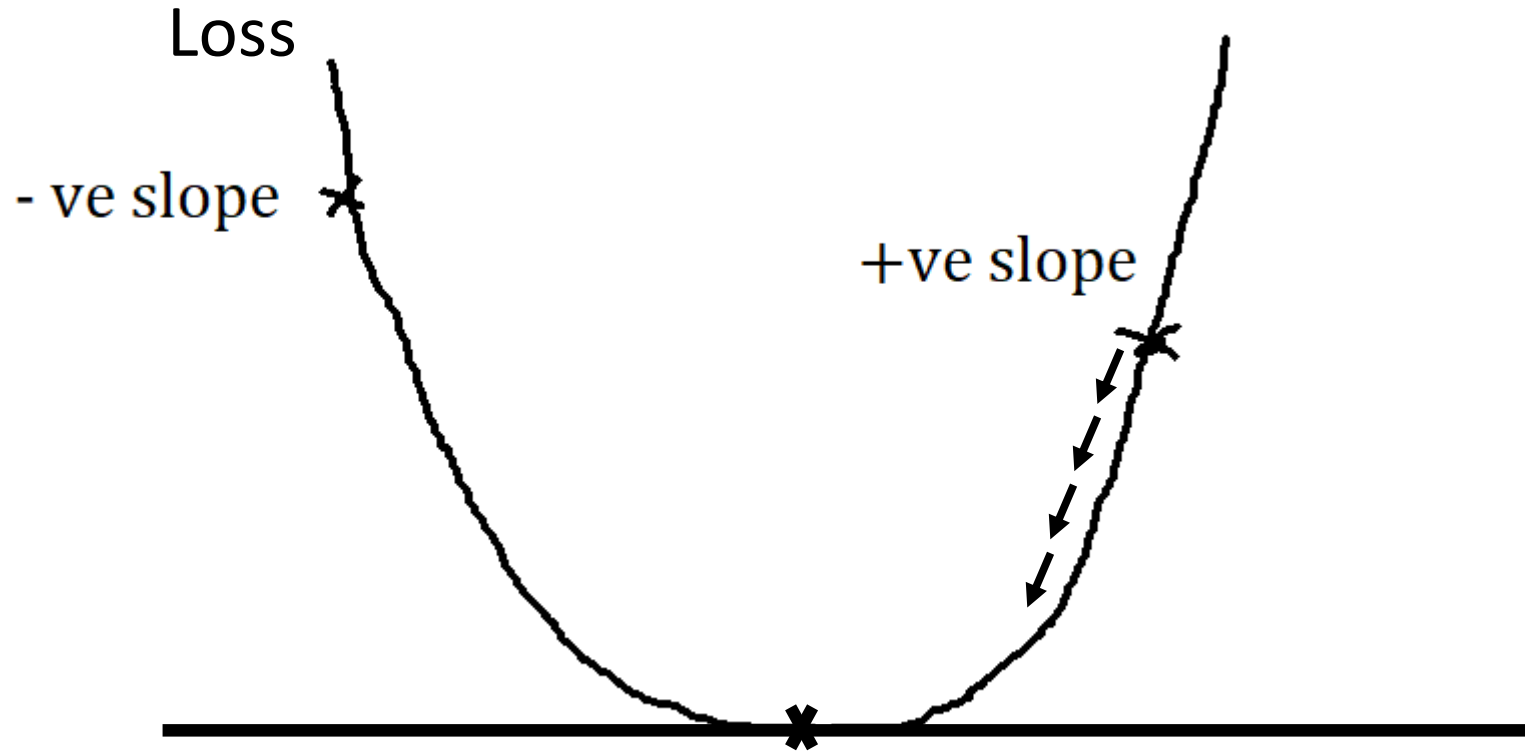
If the learning rate is high, then the point will be jumping around, not reaching the global minima.

Now if the slope is +ve, then the derivative is positive.

So $-\eta \left(\frac{\delta L}{\delta W_{old}} \right)$ will be a -ve value.

That is our point goes on moving towards the global minima.

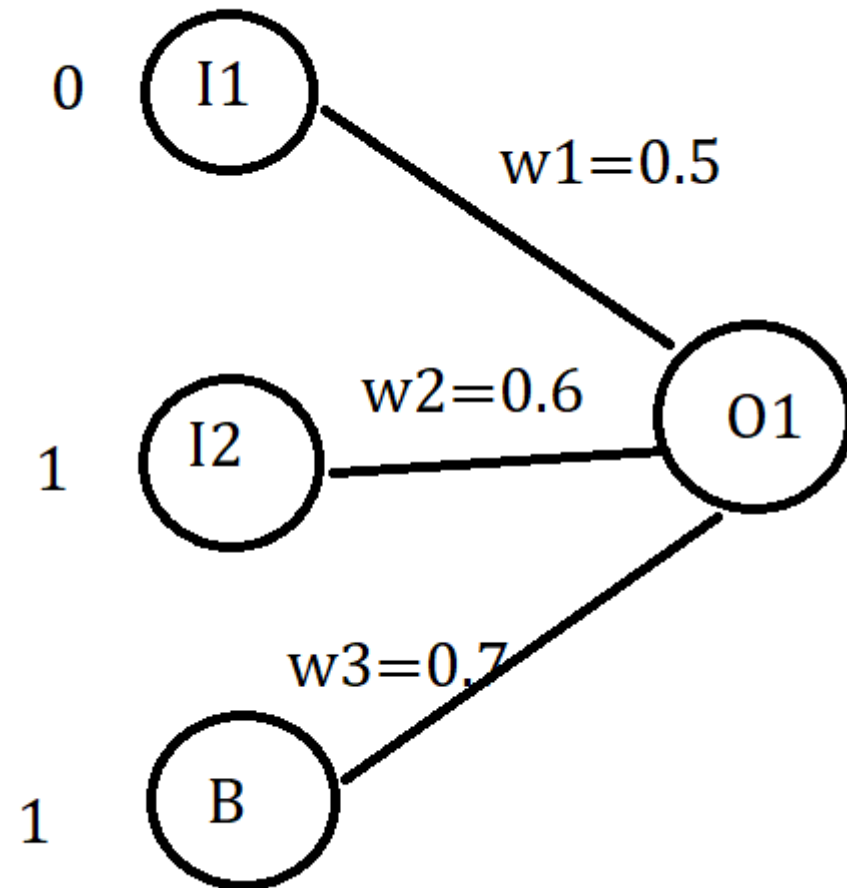
Select a learning rate which is small like 0.001.



One forward and backward movement is considered 1 epoch.

If it is not converging to the global minima, that means your learning rate or derivative is wrong.

ARTIFICIAL NEURAL NETWORK



Sigmoid

$$f(x) = \frac{1}{1 + e^{-z}}$$

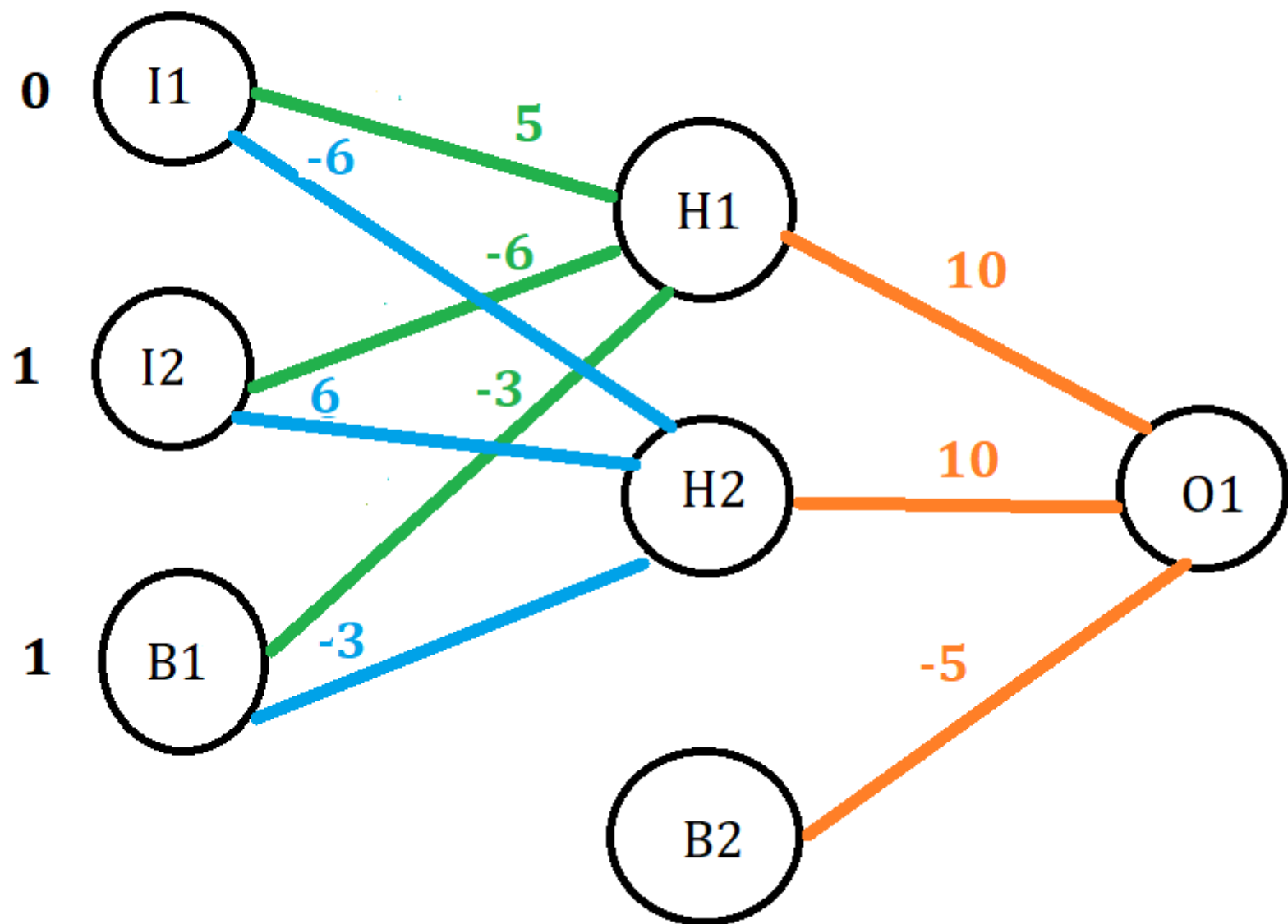
$$O1 = \text{sig}(I1w1 + I2w2 + Bw3)$$

$$O1 = \text{sig}(0 * 0.5 + 1 * 0.6 + 1 * 0.7)$$

$$O1 = \text{sig}(1.3)$$

$$O1 = \frac{1}{1 + e^{-1.3}}$$

$$O1 = 0.79$$



$$H1 = \text{sig}((0*5)+(1*-6)+(-3))$$

$$H1 = \text{sig}(-6-3)$$

$$H1 = 0.88$$

$$H2 = \text{sig}((0*-6)+(1*6)+(-3))$$

$$H2 = \text{sig}(0+6-3)$$

$$H2 = \text{sig}(3)$$

$$H2 = 0.00012$$

$$O1 = \text{sig}(0.88*10)+(0.00012*10)+(-5)$$

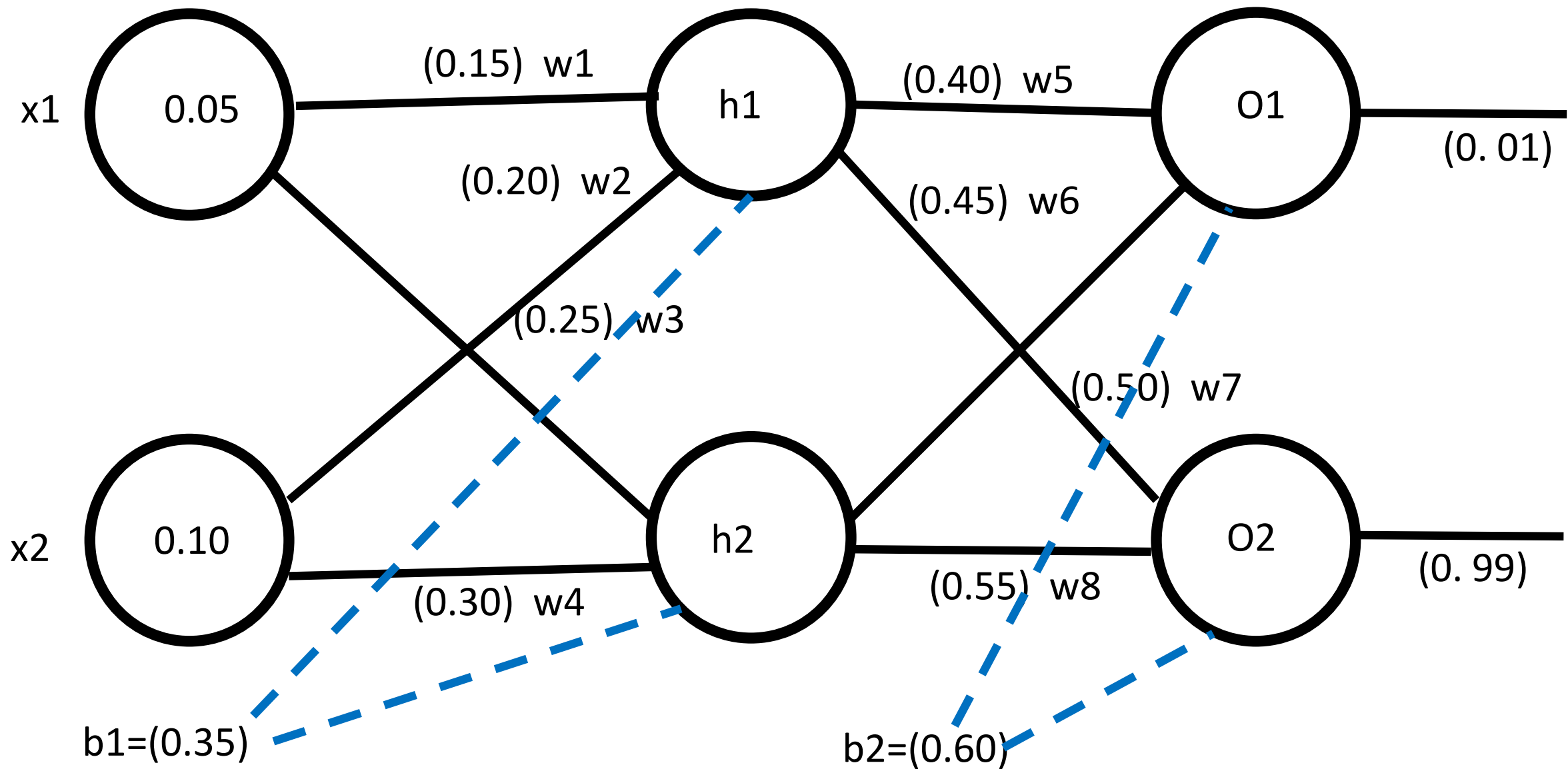
$$O1 = \text{sig}(3.8012)$$

$$O1 = \frac{1}{1+e^{-3.805}} = 0.98$$

$$O1 = 1$$

Artificial Neural Network

Backpropagation



$$\begin{aligned} H1(in) &= w1x1 + w2x2 + b1 \\ &= (0.15*0.05) + (0.20*0.10) + 0.35 \\ &= 0.377 \end{aligned}$$

$$h1(out) = \frac{1}{1 + e^{-h1(in)}} = \frac{1}{1 + 2.71^{-0.377}} = 0.5932$$

$$\begin{aligned} H2(in) &= w3x1 + w4x2 + b1 \\ &= (0.25*0.05) + (0.30*0.10) + 0.35 \\ &= 0.4 \end{aligned}$$

$$h2(out) = \frac{1}{1 + e^{-h2(in)}} = \frac{1}{1 + 2.71^{-0.4}} = 0.5968$$

$$\begin{aligned}
 O1(in) &= w5h1(out) + w6h2(out) + b2 \\
 &= (0.40 * 0.593) + (0.45 * 0.596) + 0.60 \\
 &= 1.105
 \end{aligned}$$

$$O1(out) = \frac{1}{1 + e^{-O1(in)}} = \frac{1}{1 + 2.71^{-1.105}} = 0.7513$$

$$\begin{aligned}
 O2(in) &= w7h1(out) + w8h2(out) + b2 \\
 &= (0.50 * 0.593) + (0.55 * 0.596) + 0.60 \\
 &= 1.2
 \end{aligned}$$

$$O2(out) = \frac{1}{1 + e^{-O2(in)}} = \frac{1}{1 + 2.71^{-1.2}} = 0.7729$$

$$ETotal = \sum \frac{1}{2} (target - output)^2$$

$$EO1 = \frac{(0.01 - 1.105)^2}{2} = 0.274$$

$$EO2 = \frac{(0.99 - 0.7729)^2}{2} = 0.0235$$

$$ETotal = EO1 + EO2$$

$$ETotal = 0.274 + 0.0235$$

$$ETotal = 0.2983$$

E_{Total}=0.2983

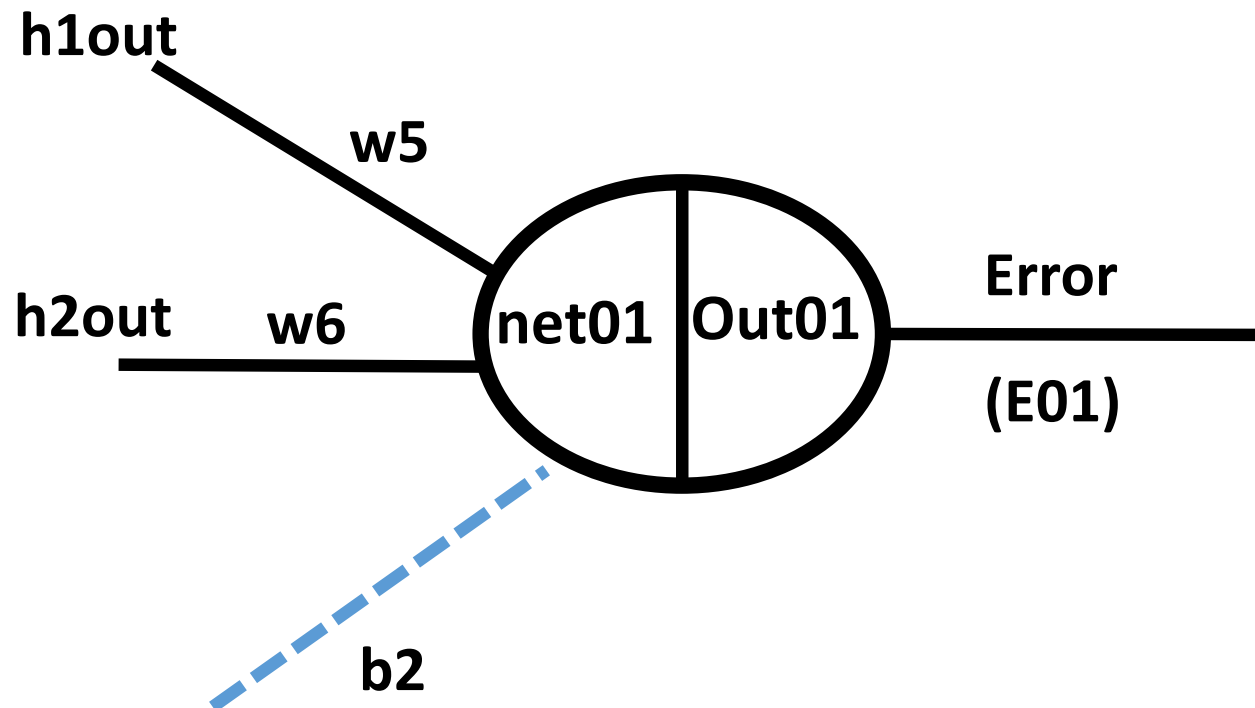
$$\frac{\partial E_{Total}}{\partial w_5} = \frac{\partial E_{Total}}{\partial Out_{01}} * \frac{\partial Out_{01}}{\partial net_{01}} * \frac{\partial net_{01}}{\partial w_5}$$

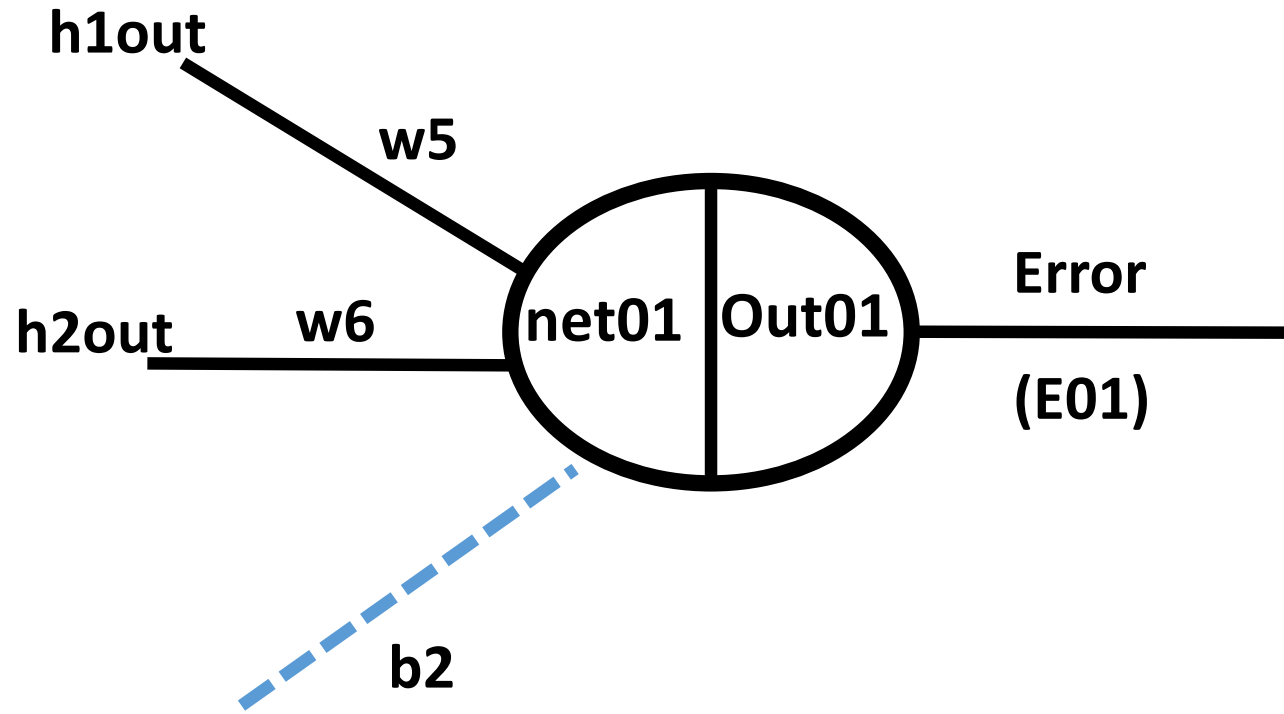
$$\begin{aligned}\frac{\partial E_{Total}}{\partial Out_{01}} &= Out_{01} - Target_{01} \\ &= 0.751365 - 0.01 \\ &= 0.741365\end{aligned}$$

$$\begin{aligned}\frac{\partial Out_{01}}{\partial net_{01}} &= Out_{01}(1 - Out_{01}) \\ &= 0.751365(1 - 0.751365) \\ &= 0.186815602\end{aligned}$$

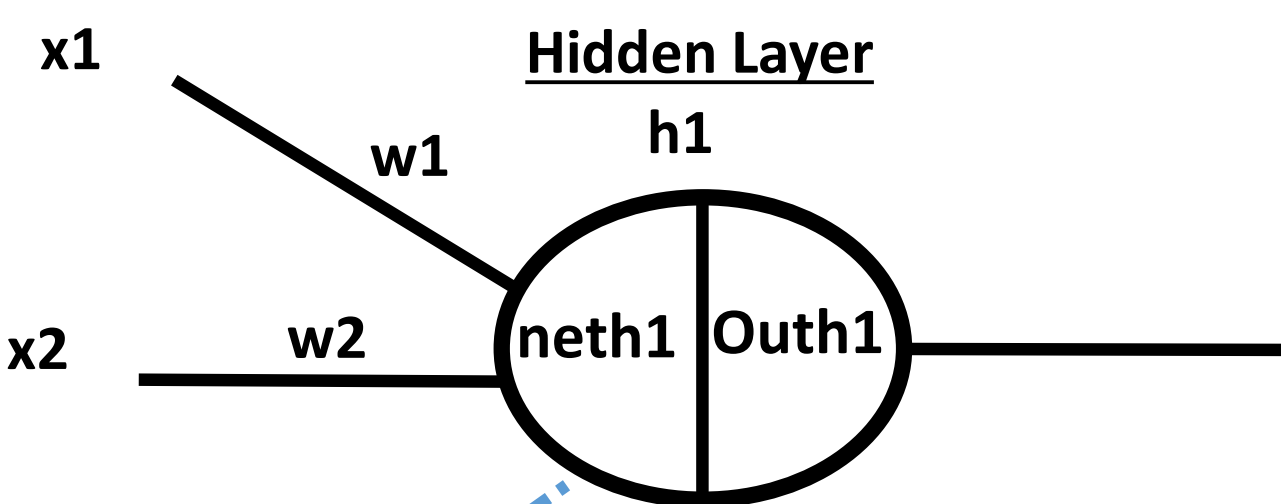
$$\frac{\partial net_{01}}{\partial w_5} = Out_{01} = 0.593269992$$

$$\frac{\partial E_{Total}}{\partial w_5} = 0.08216704$$





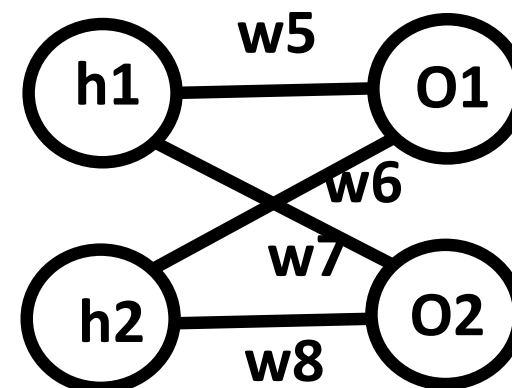
$$w5_{new} = w5 - \eta \frac{\delta E_{Total}}{\delta w5} = 0.4 - 0.6 * 0.08216704 = 0.350699776$$



$$\frac{\partial E_{Total}}{\partial w1} = \frac{\partial E_{Total}}{\partial Outh1} * \frac{\partial Outh1}{\partial neth1} * \frac{\partial neth1}{\partial w1}$$

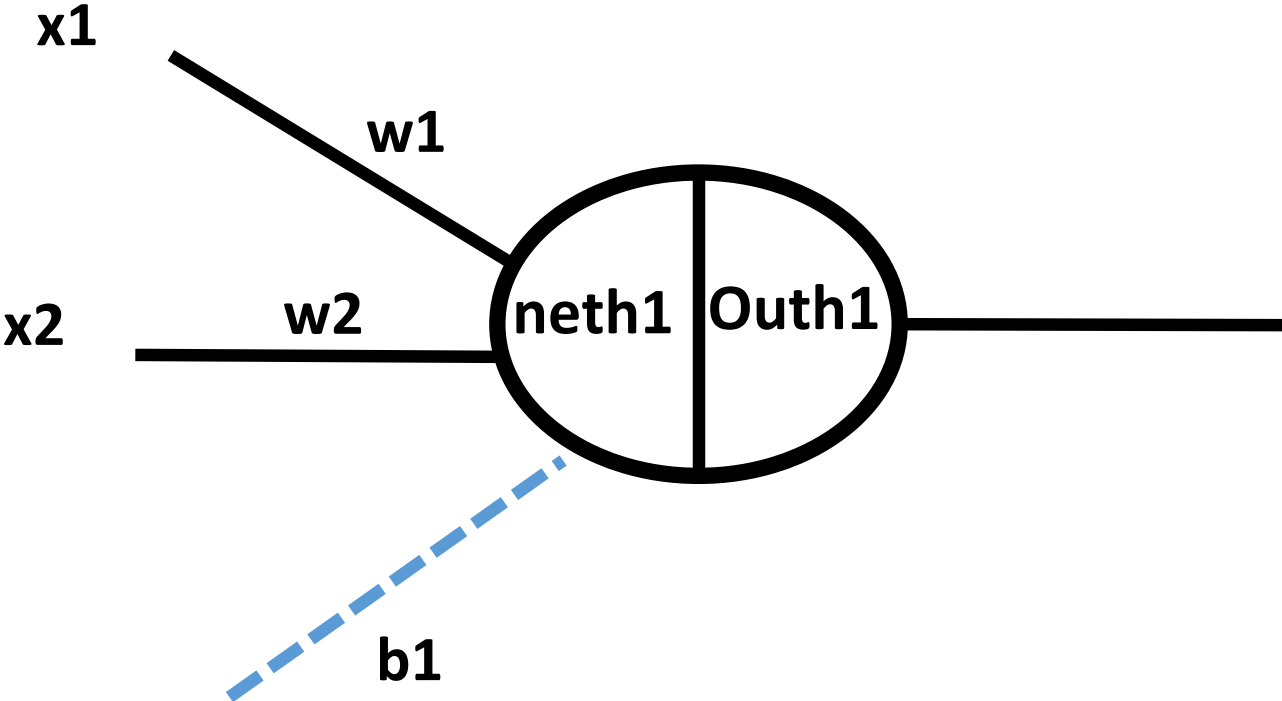
$$\begin{aligned} \frac{\partial E02}{\partial Out02} &= (Out02 - target02) \\ &= 0.772928465 - 0.99 \\ &= -0.217071535 \end{aligned}$$

$$\begin{aligned} \frac{\partial E_{Total}}{\partial Outh1} &= \frac{\partial E01}{\partial Outh1} + \frac{\partial E02}{\partial Outh1} \\ \frac{\partial E01}{\partial net01} * \frac{\partial net01}{\partial Outh1} &+ \frac{\partial E02}{\partial net02} * \frac{\partial net02}{\partial outh1} \\ \downarrow \quad \searrow & \quad \downarrow \quad \searrow \\ \frac{\partial E01}{\partial out01} * \frac{\partial out01}{\partial net01} & \quad \frac{\partial E02}{\partial out02} * \frac{\partial out02}{\partial net02} \\ \mathbf{0.13849856} & \quad \mathbf{0.4} \quad \quad \mathbf{-0.0380982} \quad \mathbf{0.50} \end{aligned}$$



$$\begin{aligned} Out02(1 - Out02) \\ = 0.175510052 \end{aligned}$$

$$=0.055399425+(-0.019049119)$$
$$=0.036350306$$



$$\frac{\partial E_{Total}}{\partial w1} = \frac{\partial E_{Total}}{\partial Outh1} * \frac{\partial Outh1}{\partial neth1} * \frac{\partial neth1}{\partial w1}$$

$$\frac{\partial E_{Total}}{\partial Outh1} = 0.055399425 + (-0.019049119)$$

$$= 0.036350306$$

$$\frac{\partial Outh1}{\partial neth1} = outh1(1 - Outh1)$$

$$= 0.241300709$$

$$neth1 = w1x1 + w2x2 + b1 * 1$$

$$\frac{\partial neth1}{\partial w1} = x1 = 0.05$$

$$\frac{\partial E_{Total}}{\partial w1} = 0.000438568$$

$$w1_{new} = w1 - \eta * \frac{\partial E_{Total}}{\partial w1} = 0.15 - 0.6 * 0.000438568$$

$$= 0.1497368592$$

$$\Rightarrow 0.055399425 + (-0.019049119)$$

$$= 0.036350306$$

CONFUSION MATRIX AND ACCURACY

Performance Metrics

A confusion matrix is the easiest way to measure the performance of a classification problem where the output can be of two or more type of classes.

A confusion matrix is a summary of prediction results on a classification problem.

The number of correct and incorrect predictions are summarized with count values and broken down by each class.

Below is the process for calculating a confusion Matrix.

- 1.You need a test dataset or a validation dataset with expected outcome values.
- 2.Make a prediction for each row in your test dataset.
- 3.From the expected outcomes and predictions count:
 - 1.The number of correct predictions for each class.
 - 2.The number of incorrect predictions for each class, organized by the class that was predicted.

A confusion matrix is a table with two dimensions.

“Actual” and “Predicted”

Both the dimensions have “True Positives (TP)”, “True Negatives (TN)”, “False Positives (FP)”, “False Negatives (FN)”

Actual

1

0

1

True Positives (TP)

False Positives (FP)

Predicted

0

False Negative (FN)

True Negatives (TN)

- **True Positives (TP)** – It is the case when both actual class & predicted class of data point is 1.
- **True Negatives (TN)** – It is the case when both actual class & predicted class of data point is 0.
- **False Positives (FP)** – It is the case when actual class of data point is 0 & predicted class of data point is 1.
- **False Negatives (FN)** – It is the case when actual class of data point is 1 & predicted class of data point is 0.

	1	0
1	73	7
0	4	144

Accuracy

It may be defined as the number of correct predictions made by our ML model. We can easily calculate it by confusion matrix with the help of following formula –

$$\textit{Accuracy} = \frac{TP + TN}{TP + FP + FN + TN}$$

For above built binary classifier,

$$TP + TN = 73 + 144 = 217 \text{ and } TP + FP + FN + TN = 73 + 7 + 4 + 144 = 228.$$

$$\text{Hence Accuracy} = 217/228 = 0.951754385965$$

Precision

Precision, used in document retrievals, may be defined as the number of correct documents returned by our ML model. We can easily calculate it by confusion matrix with the help of following formula –

$$Precision = \frac{TP}{TP + FP}$$

For the above built binary classifier,

$TP = 73$ and $TP+FP = 73+7 = 80$.

Hence, Precision = $73/80 = 0.915$

Recall or Sensitivity

Recall may be defined as the number of positives returned by our ML model. We can easily calculate it by confusion matrix with the help of following formula –

$$\textit{Recall} = \frac{TP}{TP + FN}$$

For above built binary classifier,

$TP = 73$ and $TP+FN = 73+4 = 77$.

Hence, Precision = $73/77 = 0.94805$

Specificity

Specificity, in contrast to recall, may be defined as the number of negatives returned by our ML model. We can easily calculate it by confusion matrix with the help of following formula –

$$\textit{Specificity} = \frac{TN}{TN + FP}$$

For the above built binary classifier,

$TN = 144$ and $TN+FP = 144+7 = 151$.

Hence, Precision = $144/151 = 0.95364$

F1 Score

F1 which is a function of Precision and Recall.

$$F1 = 2 \times \frac{Precision * Recall}{Precision + Recall}$$

Precision quantifies the number of positive class predictions that actually belong to the positive class.

Recall quantifies the number of positive class predictions made out of all positive examples in the dataset.

F-Measure provides a single score that balances both the concerns of precision and recall in one number.

2-Class Confusion Matrix Case Study

we have a two-class classification problem of predicting whether a photograph contains a man or a woman.

We have a test dataset of 10 records with expected outcomes and a set of predictions from our classification algorithm.

Expected,
man,
man,
woman,
man,
woman,
woman,
woman,
man,
man,
woman,

Predicted
woman
man
woman
man
man
woman
woman
man
woman
woman

The algorithm made 7 of the 10 predictions correct with an accuracy of 70%.

$\text{accuracy} = \text{total correct predictions} / \text{total predictions made} * 100$

$\text{accuracy} = 7 / 10 * 100$

confusion matrix

calculate the number of correct predictions for each class.

men classified as men: 3

women classified as women: 4

Now, calculate the number of incorrect predictions for each class, organized by the predicted value.

men classified as women: 2

woman classified as men: 1

We can now arrange these values into the 2-class confusion matrix:

	men	women
men	3	1
women	2	4

$$\textit{Accuracy} = \frac{3 + 4}{3 + 1 + 2 + 4} = \frac{7}{10} = 0.7$$

$$Precision = \frac{TP}{TP + FP}$$

$$Precision = \frac{3}{3 + 1} = 0.75$$

$$Recall = \frac{TP}{TP + FN}$$

$$Recall = \frac{3}{3 + 2} = 0.6$$

$$F1 = 2 \times \frac{Precision * Recall}{Precision + Recall}$$

$$F1 = 2 * \frac{0.75 * 0.6}{0.75 + 0.6} = 0.333$$

```
from sklearn.metrics import confusion_matrix

expected = [1, 1, 0, 1, 0, 0, 1, 0, 0, 0]
predicted = [1, 0, 0, 1, 0, 0, 1, 1, 1, 0]
results = confusion_matrix(expected, predicted)
print(results)
```

Convolutional Neural Network

CNNs are a class of Deep Neural Networks that can recognize and classify particular features from images and are widely used for analyzing visual images.

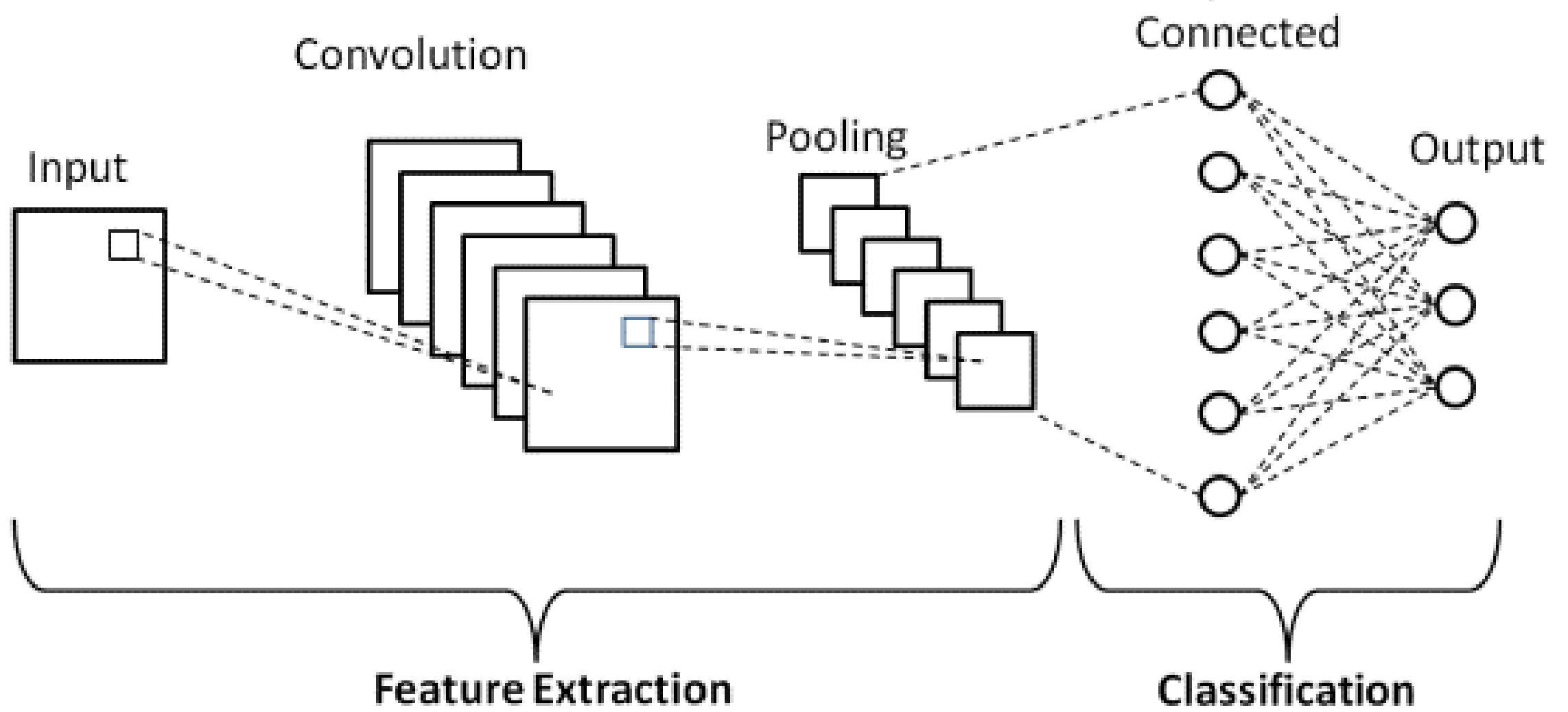
Their applications range from image and video recognition, image classification, medical image analysis, computer vision and natural language processing.

The term ‘Convolution’ in CNN denotes the mathematical function of convolution which is a special kind of linear operation wherein two functions are multiplied to produce a third function which expresses how the shape of one function is modified by the other.

Basic Architecture

There are two main parts to a CNN architecture

- A convolution tool that separates and identifies the various features of the image for analysis in a process called as Feature Extraction
- A fully connected layer that utilizes the output from the convolution process and predicts the class of the image based on the features extracted in previous stages.



Convolution Layers

There are three types of layers that make up the CNN which are the

- convolutional layers,
- pooling layers, and
- fully-connected (FC) layers.

When these layers are stacked, a CNN architecture will be formed.

In addition to these three layers, there are two more important parameters which are the dropout layer and the activation function.

1. Convolutional Layer

This layer is the first layer that is used to extract the various features from the input images.

In this layer, the mathematical operation of convolution is performed between the input image and a filter of a particular size $M \times M$.

By sliding the filter over the input image, the dot product is taken between the filter and the parts of the input image with respect to the size of the filter ($M \times M$).

The output is termed as the Feature map which gives us information about the image such as the corners and edges.

Later, this feature map is fed to other layers to learn several other features of the input image.

2. Pooling Layer

In most cases, a Convolutional Layer is followed by a Pooling Layer.

The primary aim of this layer is to decrease the size of the convolved feature map to reduce the computational costs.

This is performed by decreasing the connections between layers and independently operates on each feature map.

Depending upon method used, there are several types of Pooling operations.

In Max Pooling, the largest element is taken from feature map.

Average Pooling calculates the average of the elements in a predefined sized Image section. T

he total sum of the elements in the predefined section is computed in Sum Pooling.

The Pooling Layer usually serves as a bridge between the Convolutional Layer and the FC Layer

3. Fully Connected Layer

The Fully Connected (FC) layer consists of the weights and biases along with the neurons and is used to connect the neurons between two different layers.

These layers are usually placed before the output layer and form the last few layers of a CNN Architecture.

In this, the input image from the previous layers are flattened and fed to the FC layer.

The flattened vector then undergoes few more FC layers where the mathematical functions operations usually take place.

In this stage, the classification process begins to take place.

4. Dropout

Usually, when all the features are connected to the FC layer, it can cause overfitting in the training dataset.

Overfitting occurs when a particular model works so well on the training data causing a negative impact in the model's performance when used on a new data.

To overcome this problem, a dropout layer is utilised wherein a few neurons are dropped from the neural network during training process resulting in reduced size of the model.

On passing a dropout of 0.3, 30% of the nodes are dropped out randomly from the neural network.

5. Activation Functions

Finally, one of the most important parameters of the CNN model is the activation function.

They are used to learn and approximate any kind of continuous and complex relationship between variables of the network.

In simple words, it decides which information of the model should fire in the forward direction and which ones should not at the end of the network.

It adds non-linearity to the network.

There are several commonly used activation functions such as the ReLU, Softmax, tanH and the Sigmoid functions.

Each of these functions have a specific usage.

For a binary classification CNN model, sigmoid and softmax functions are preferred an for a multi-class classification, generally softmax us used.

Kernel or Filter:

One way to understand the convolution operation is to imagine placing the **convolution filter** or **kernel** on the top of the input image, positioned in a way so that the **kernel** and the image upper left corners coincide, and then multiplying the values of the input image matrix with the corresponding values in the **convolution filter**.

All of the multiplied values are then added together resulting in a single scalar, which is placed in the first position of a result matrix.

The **kernel** is then moved $\backslash(x\backslash)$ pixels to the right, where $\backslash(x\backslash)$ is denoted **stride length** and is a parameter of the ConvNet structure.

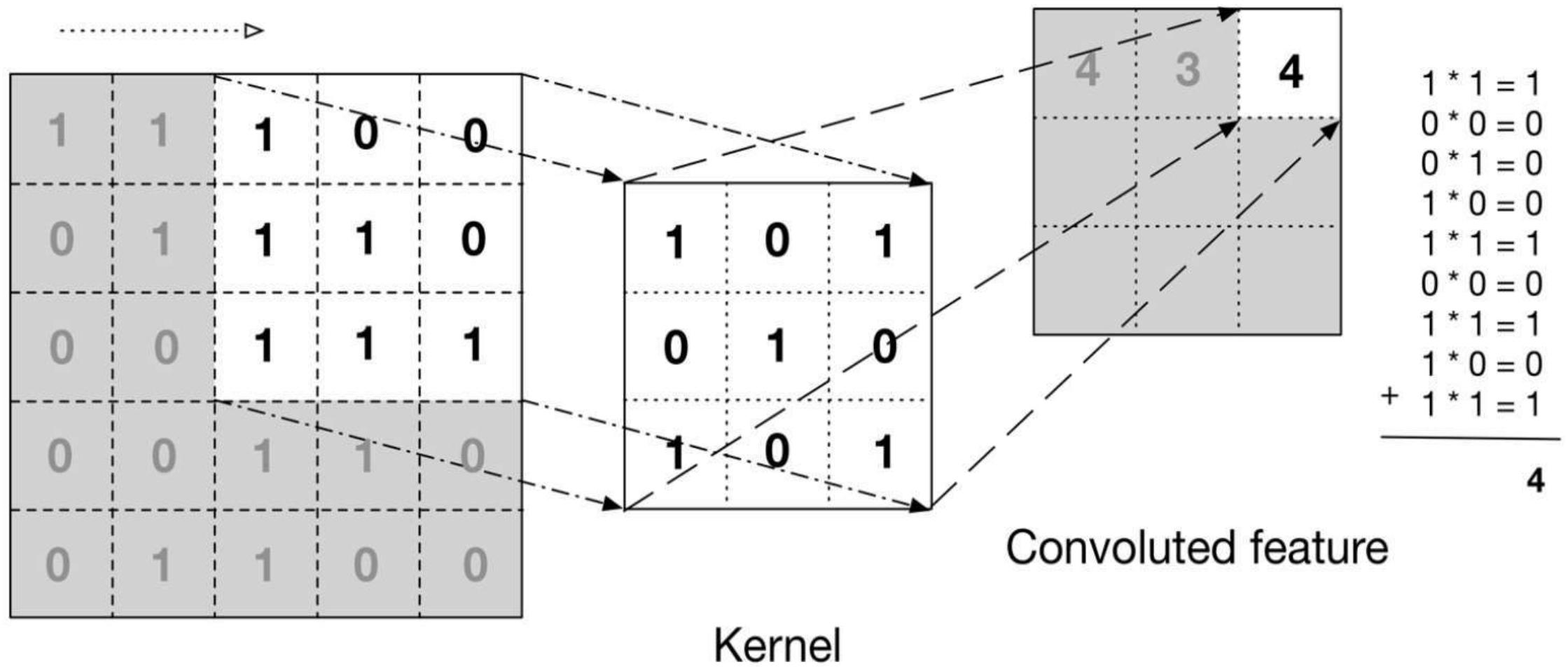
The process of multiplication is then repeated, so that the next value in the result matrix is computed and filled.

Stride:

This process is then repeated, by first covering an entire row, and then shifting down the columns by the same **stride length**, until all the entries in the input image have been covered.

The output of this process is a matrix with all its entries filled, called the **convoluted feature** or **input feature map**.

An input image can be convolved with multiple convolution kernels at once, creating one output for each kernel.



Input data

Kernel

Convoluted feature

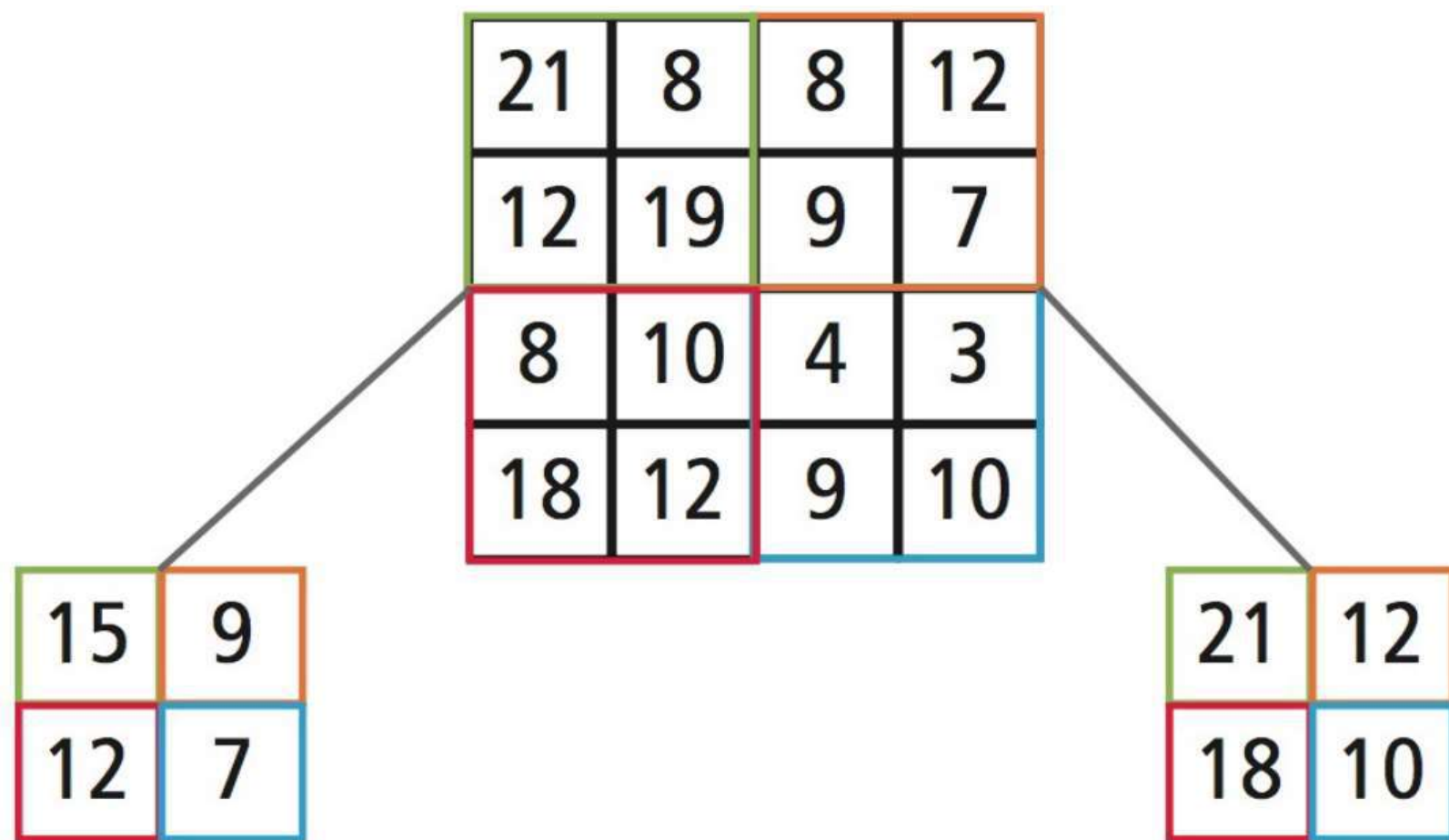
Example of a convolution operation.

(Image adapted from "*Deep Learning*" by Adam Gibson, Josh Patterson)

Pooling

Next comes the **pooling** or **downsampling** layer, which consists of applying some operation over regions/patches in the **input feature map** and extracting some representative value for each of the analysed regions/patches.

Two of the most common pooling operations are max- and average-pooling. **Max-pooling** selects the maximum of the values in the **input feature map** region of each step and **average-pooling** the average value of the values in the region. The output in each step is therefore a single scalar, resulting in significant size reduction in output size.



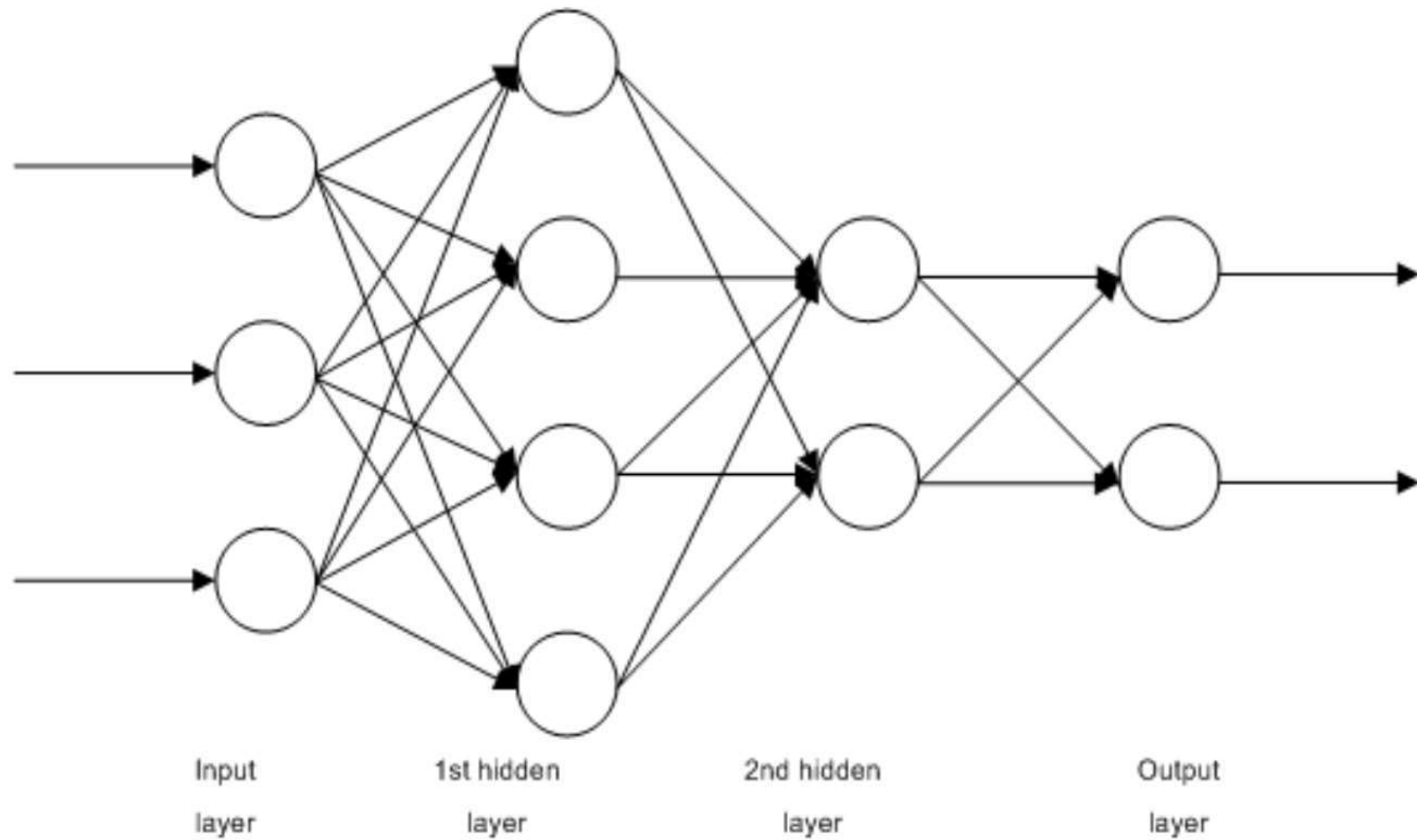
Average Pooling

Max Pooling

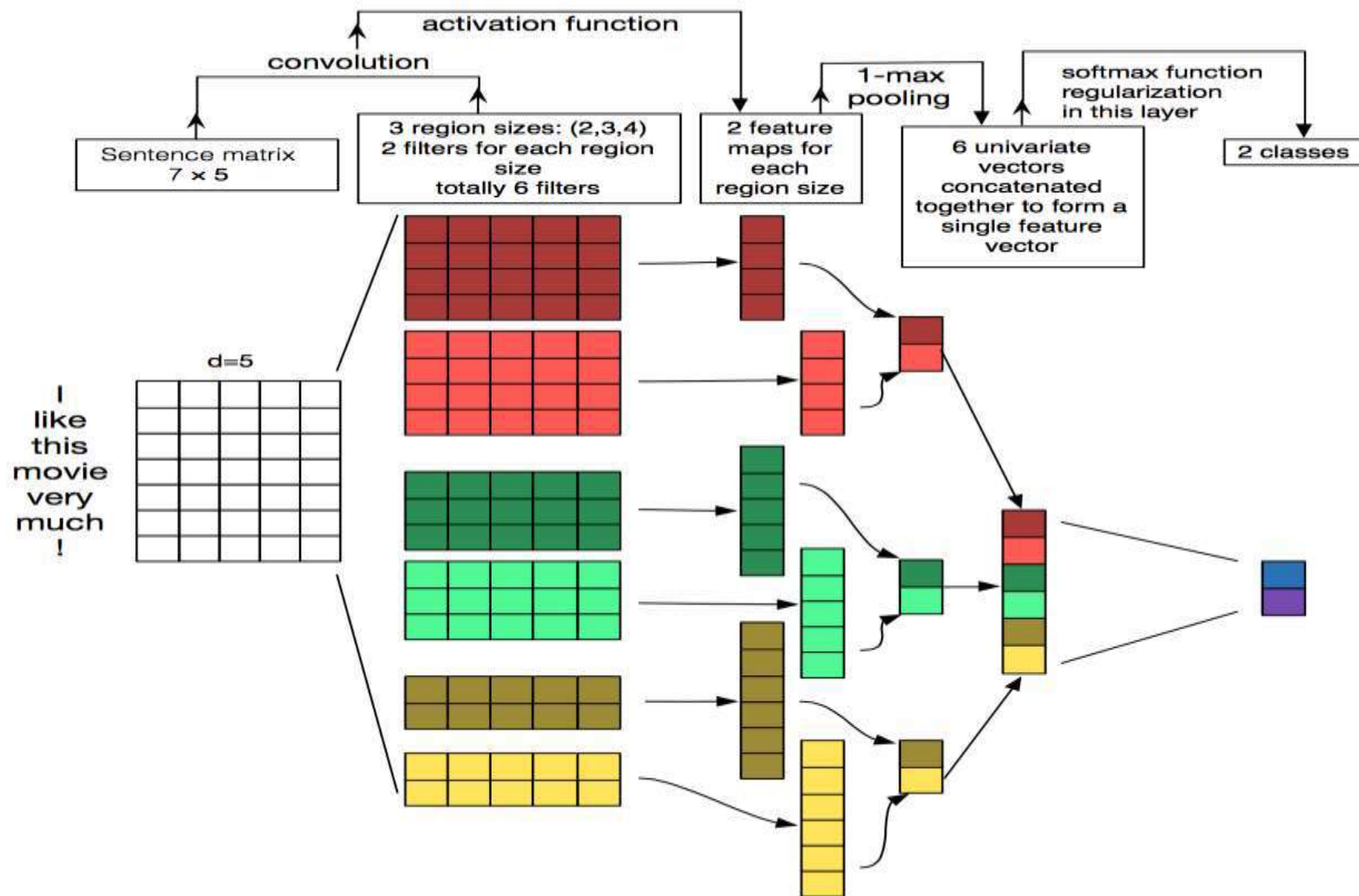
Example of a pooling operation with stride length of 2.

Fully Connected

The two processes described before i.e.: convolutions and pooling, can be thought of as feature extractors, then we pass these features, usually as a reshaped vector of one row, further to the network, for instance, a multi-layer perceptron to be trained for classification.



Example of multi-layer perceptron network used to train for classification.



CNN



Koala's **eye**? = Y



Koala's **nose**? = Y



Koala's **ears**? = Y



Koala's **hands**? = Y



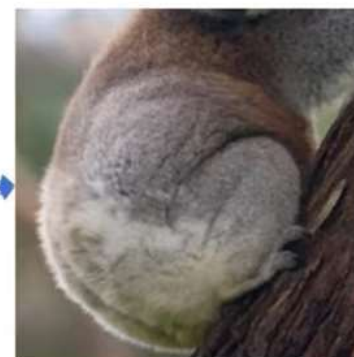
Koala's **legs**? = Y



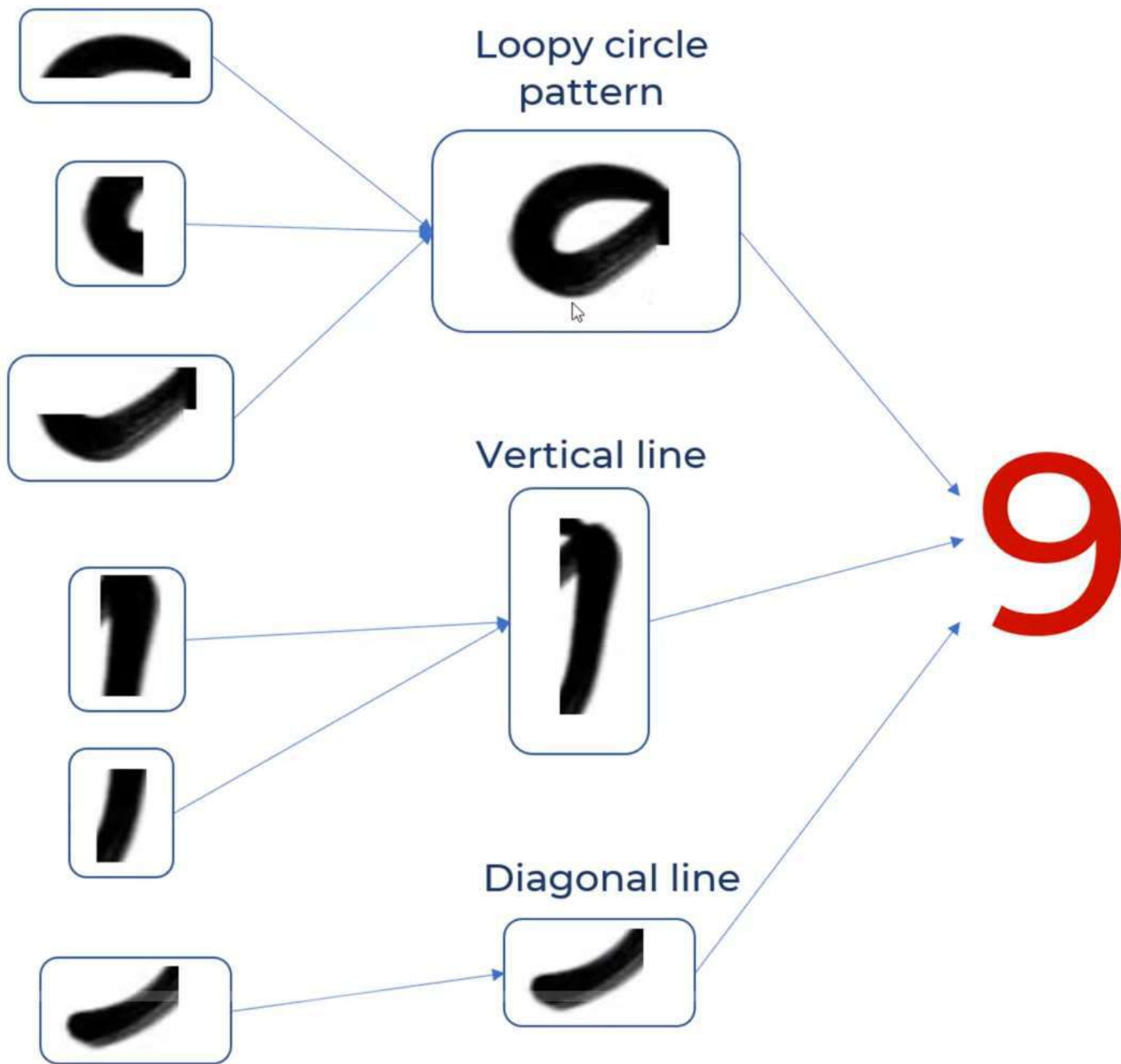
Koala's **head**? = Y



Koala's **body**? = Y



Is it **Koala**? = Y



$$-1+1+1-1-1-1-1+1+1 = -1 \rightarrow -1/9 = -0.11$$

-1	1	1	1	-1
-1	1	-1	1	-1
-1	1	1	1	-1
-1	-1	-1	1	-1
-1	-1	-1	1	-1
-1	-1	1	-1	-1
-1	1	-1	-1	-1

*

1	1	1
1	-1	1
1	1	1

-0.11		

-1	1	1	1	-1
-1	1	-1	1	-1
-1	1	1	1	-1
-1	-1	-1	1	-1
-1	-1	-1	1	-1
-1	-1	1	-1	-1
-1	1	-1	-1	-1

*

1	1	1
1	-1	1
1	1	1

-0.11	1	-0.11
-0.55	0.11	-0.33
-0.33	0.33	-0.33
-0.22	-0.11	-0.22
-0.33	-0.33	-0.33

Feature Map

9

Loopy pattern detector

$$\begin{matrix}
 1 & 1 & 1 \\
 1 & -1 & 1 \\
 1 & 1 & 1
 \end{matrix}
 =
 \begin{matrix}
 & 1 & \\
 & & \\
 & & \\
 & & \\
 & & \\
 & &
 \end{matrix}$$

6

Loopy pattern detector

$$\begin{matrix}
 1 & 1 & 1 \\
 1 & -1 & 1 \\
 1 & 1 & 1
 \end{matrix}
 =
 \begin{matrix}
 & & \\
 & & \\
 & & \\
 & & \\
 & 1 & \\
 & &
 \end{matrix}$$

8

Loopy pattern detector

$$\begin{matrix}
 1 & 1 & 1 \\
 1 & -1 & 1 \\
 1 & 1 & 1
 \end{matrix}
 =
 \begin{matrix}
 & 1 & \\
 & & \\
 & & \\
 & & \\
 & & \\
 & 1 &
 \end{matrix}$$

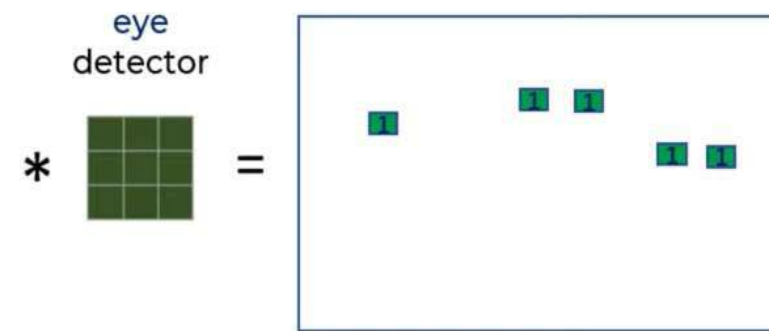
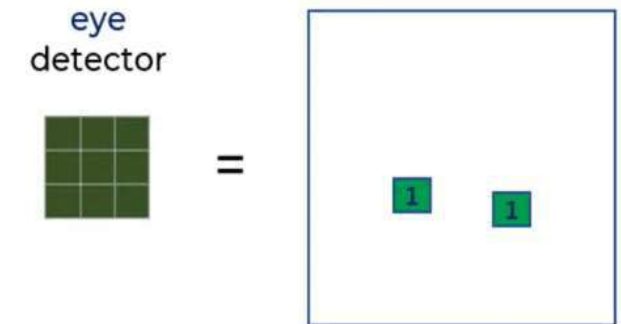
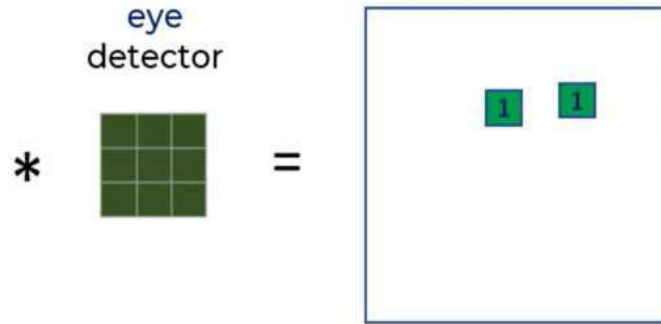
96

Loopy pattern detector

$$\begin{matrix}
 1 & 1 & 1 \\
 1 & -1 & 1 \\
 1 & 1 & 1
 \end{matrix}
 =
 \begin{matrix}
 & 1 & & \\
 & & & \\
 & & & \\
 & & & \\
 & & & \\
 & & & 1
 \end{matrix}$$

Filters are nothing
but the feature
detectors

Location invariant: It can detect eyes in any location of the image





hands
detector

*



=



Loopy pattern detector

$$\begin{bmatrix} 1 & 1 & 1 \\ 1 & -1 & 1 \\ 1 & 1 & 1 \end{bmatrix} = \begin{bmatrix} & 1 & & & \\ & & & & \\ & & & & \\ & & & & \\ & & & & \end{bmatrix}$$

Vertical line detector

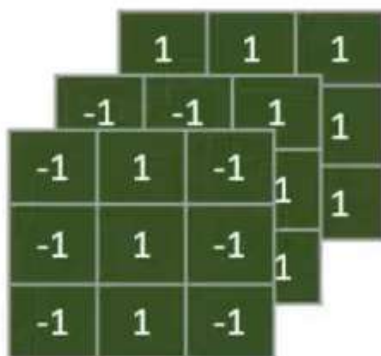
$$\begin{bmatrix} -1 & 1 & -1 \\ -1 & 1 & -1 \\ -1 & 1 & -1 \end{bmatrix} = \begin{bmatrix} & & & & \\ & & & & \\ & & 1 & & \\ & & & & \\ & & & & \end{bmatrix}$$

Diagonal line detector

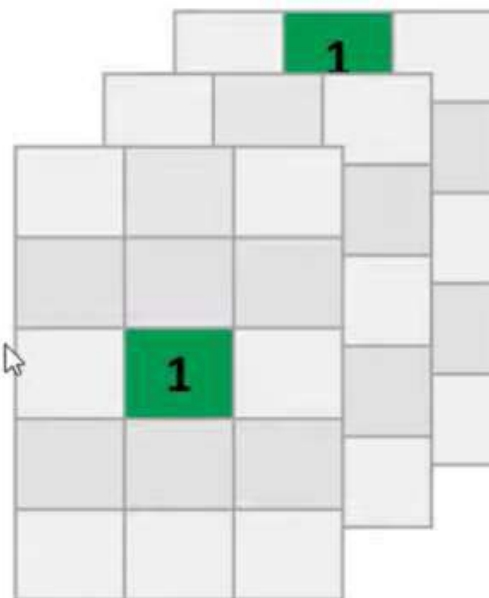
$$\begin{bmatrix} -1 & -1 & 1 \\ -1 & 1 & -1 \\ 1 & -1 & -1 \end{bmatrix} = \begin{bmatrix} & & & & \\ & & & & \\ & & & & \\ & & 1 & & \\ & & & & \end{bmatrix}$$

9

Filters


$$=$$

Feature Maps



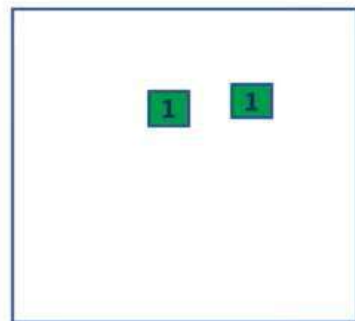


eye



*

=



nose



*

=

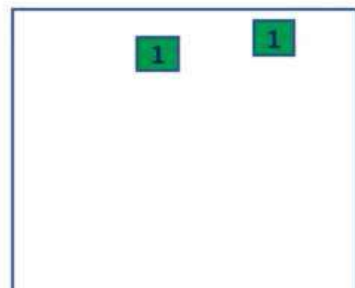


ears



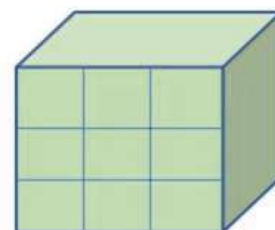
*

=

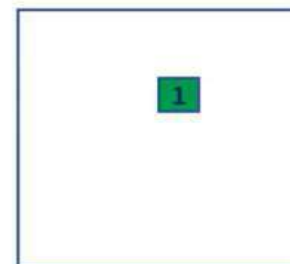


Filter for head

*



→





$$\begin{matrix} * & \text{eye} \\ & \begin{smallmatrix} \blacksquare & \blacksquare \\ \blacksquare & \blacksquare \end{smallmatrix} \\ & = \end{matrix} \begin{matrix} & \blacksquare & \blacksquare \\ & & \\ & & \end{matrix}$$



$$\begin{matrix} * & \text{nose} \\ & \begin{smallmatrix} \blacksquare & \blacksquare \\ \blacksquare & \blacksquare \end{smallmatrix} \\ & = \end{matrix} \begin{matrix} & \blacksquare \\ & & \\ & & \end{matrix}$$



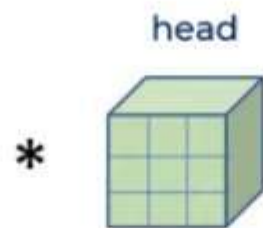
$$\begin{matrix} * & \text{ears} \\ & \begin{smallmatrix} \blacksquare & \blacksquare \\ \blacksquare & \blacksquare \end{smallmatrix} \\ & = \end{matrix} \begin{matrix} & \blacksquare & \blacksquare \\ & & \\ & & \end{matrix}$$



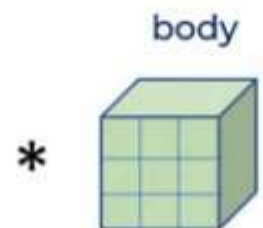
$$\begin{matrix} * & \text{hands} \\ & \begin{smallmatrix} \blacksquare & \blacksquare \\ \blacksquare & \blacksquare \end{smallmatrix} \\ & = \end{matrix} \begin{matrix} & \blacksquare \\ & & \\ & & \end{matrix}$$



$$\begin{matrix} * & \text{legs} \\ & \begin{smallmatrix} \blacksquare & \blacksquare \\ \blacksquare & \blacksquare \end{smallmatrix} \\ & = \end{matrix} \begin{matrix} & \blacksquare \\ & & \\ & & \end{matrix}$$

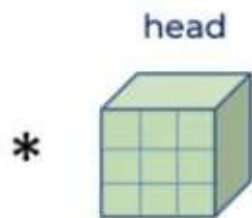
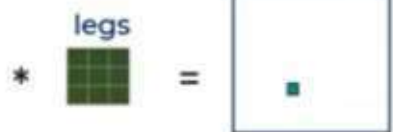
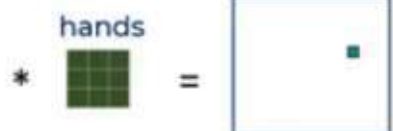
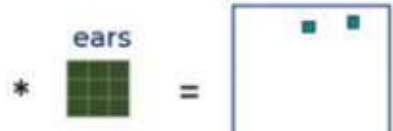
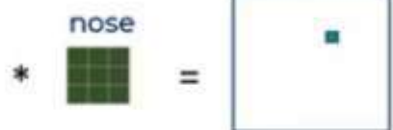
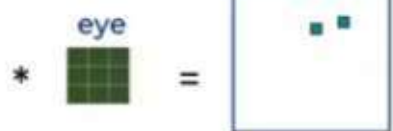


flatten



flatten

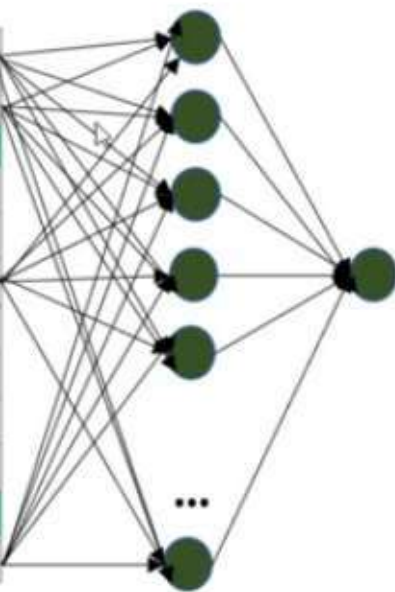




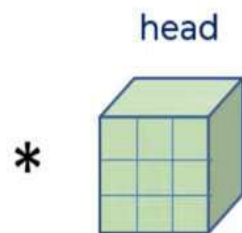
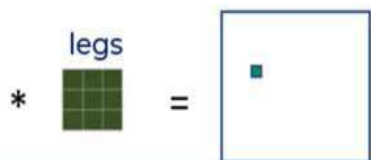
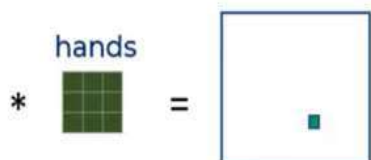
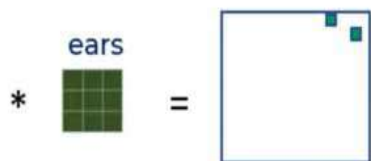
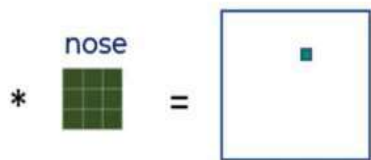
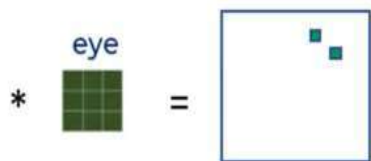
flatten



flatten



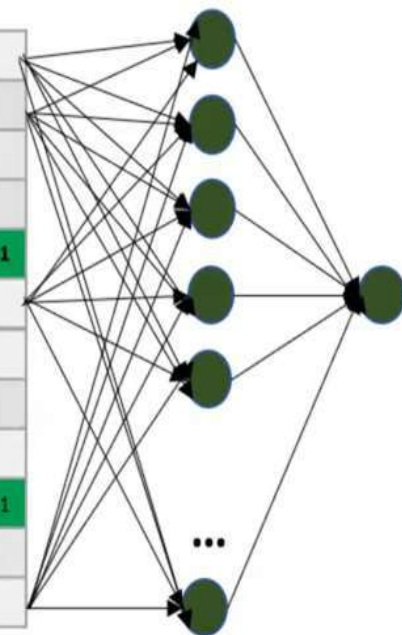
Is this Koala?



flatten



flatten



Is this
Koala?

-1	1	1	1	-1
-1	1	-1	1	-1
-1	1	1	1	-1
-1	-1	-1	1	-1
-1	-1	-1	1	-1
-1	-1	1	-1	-1
-1	1	-1	-1	-1

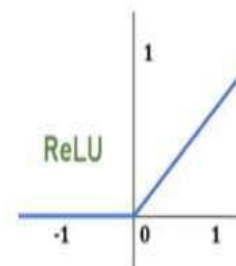
*

Loopy pattern
filter

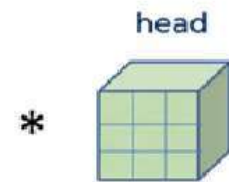
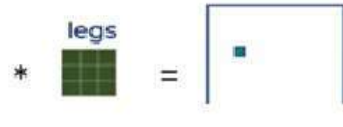
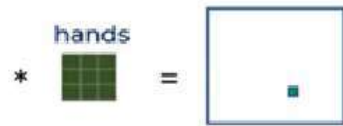
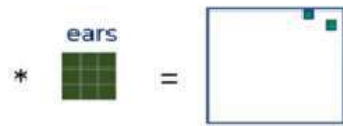
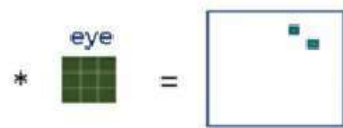
1	1	1
1	-1	1
1	1	1



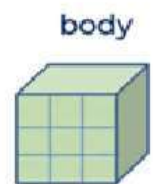
-0.11	1	-0.11
-0.55	0.11	-0.33
-0.33	0.33	-0.33
-0.22	-0.11	-0.22
-0.33	-0.33	-0.33



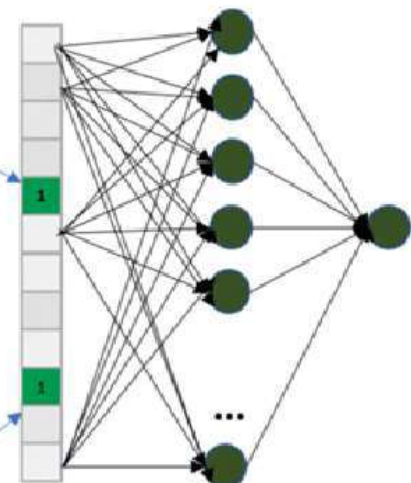
0	1	0
0	0.11	0
0	0.33	0
0	0	0
0	0	0



flatten



flatten



Is this Koala?



Feature Extraction

Classification

ReLU helps with making
the model nonlinear

-1	1	1	1	-1
-1	1	-1	1	-1
-1	1	1	1	-1
-1	-1	-1	1	-1
-1	-1	-1	1	-1
-1	-1	1	-1	-1
-1	1	-1	-1	-1

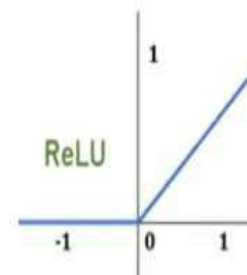
*

Loopy pattern
filter

1	1	1
1	-1	1
1	1	1

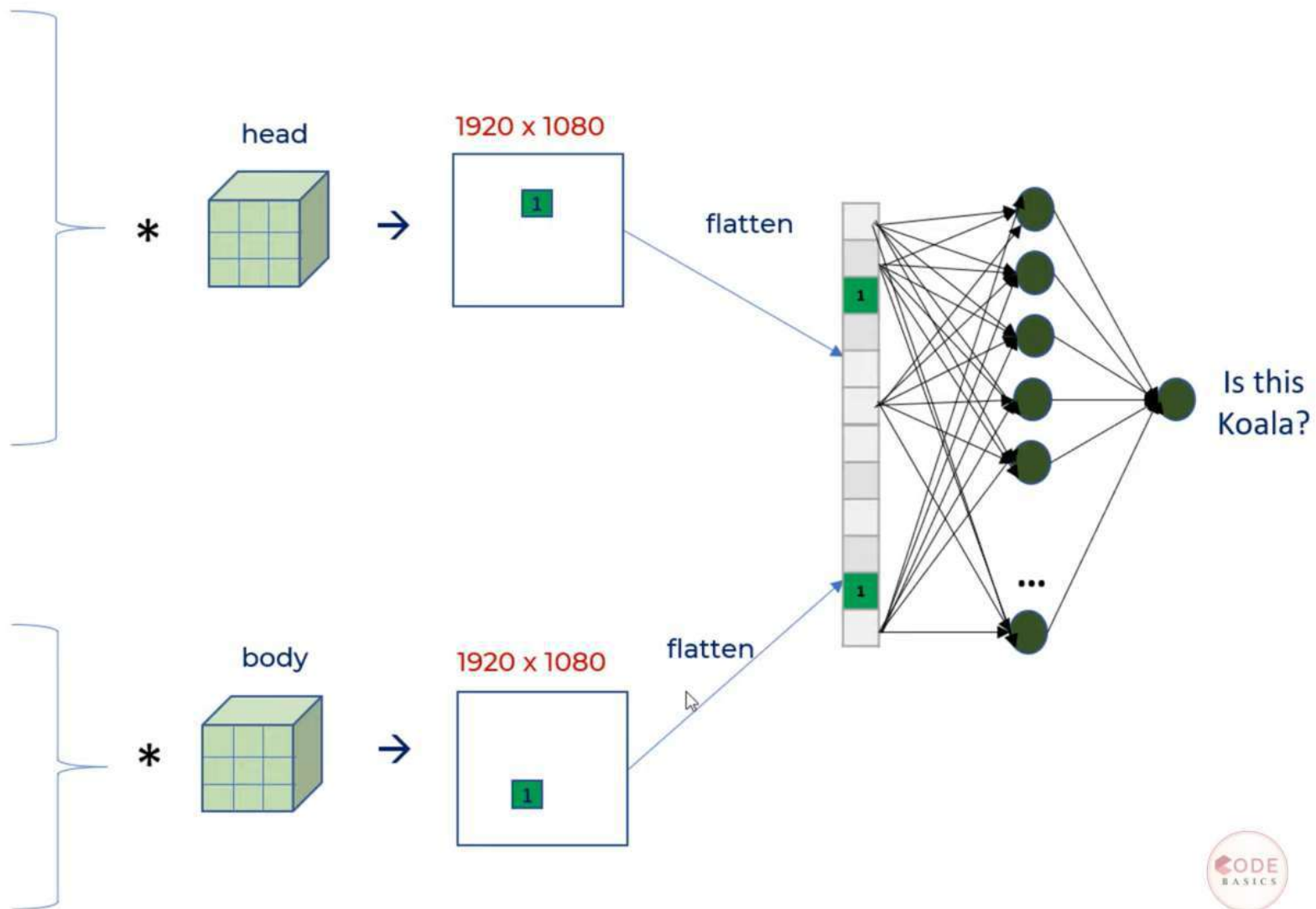
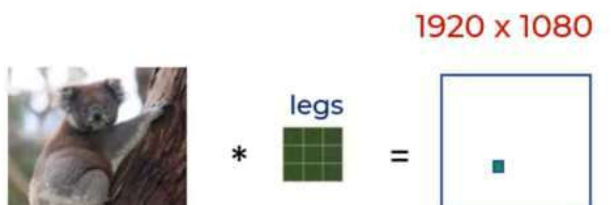
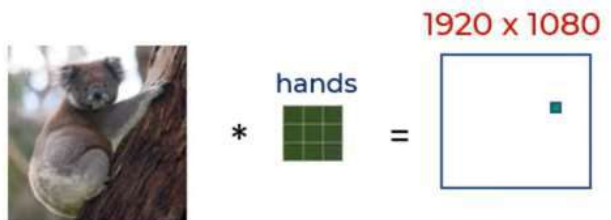
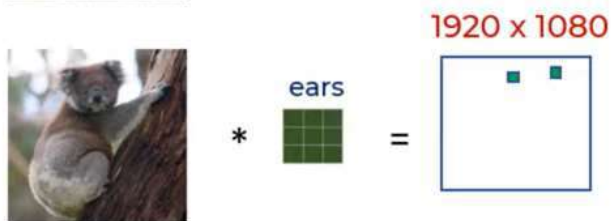
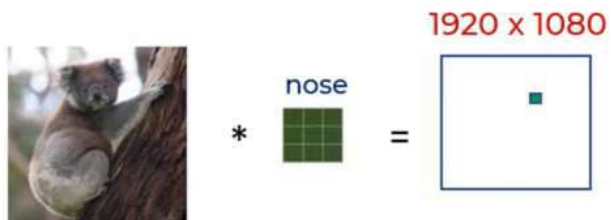
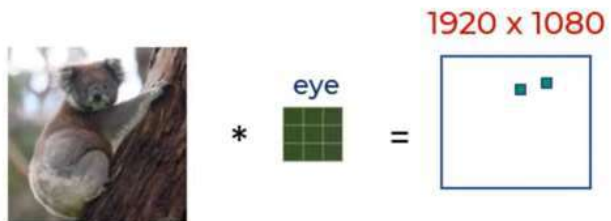


-0.11	1	-0.11
-0.55	0.11	-0.33
-0.33	0.33	-0.33
-0.22	-0.11	-0.22
-0.33	-0.33	-0.33



0	1	0
0	0.11	0
0	0.33	0
0	0	0
0	0	0

Pooling layer is used to
reduce the size



5	1	3	4
8	2	9	2
1	3	0	1
2	2	2	0

8	9
3	2

2 by 2 filter with stride = 2

-1	1	1	1	-1
-1	1	-1	1	-1
-1	1	1	1	-1
-1	-1	-1	1	-1
-1	-1	-1	1	-1
-1	-1	1	-1	-1
-1	1	-1	-1	-1

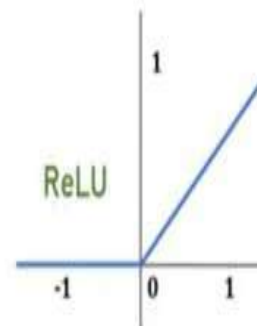
Loopy pattern
filter

*

1	1	1
1	-1	1
1	1	1



-0.11	1	-0.11
-0.55	0.11	-0.33
-0.33	0.33	-0.33
-0.22	-0.11	-0.22
-0.33	-0.33	-0.33



0	1	0
0	0.11	0
0	0.33	0
0	0	0
0	0	0

Max
pooling



0	1	0
0	0.11	0
0	0.33	0
0	0	0
0	0	0

1	1

2 by 2 filter with stride = 1

Shifted 9 at different position

1	1	1	-1	-1
1	-1	1	-1	-1
1	1	1	-1	-1
-1	-1	1	-1	-1
-1	-1	1	-1	-1
-1	1	-1	-1	-1
1	-1	-1	-1	-1

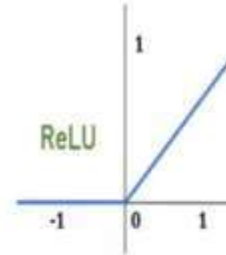
Loopy pattern
filter

*

1	1	1
1	-1	1
1	1	1



1	-0.11	-0.11
0.11	-0.33	0.33
0.33	-0.33	-0.33
-0.11	-0.55	-0.33
-0.55	-0.33	-0.55



1	0	0
0.11	0	0.33
0.33	0	0
0	0	0
0	0	0

Max
pooling



1	0.33
0.33	0.33
0.33	0
0	0

There is average pooling also...

5	1	3	4
8	2	9	2
1	3	0	1
2	2	2	0



4	4.5
2	0.75

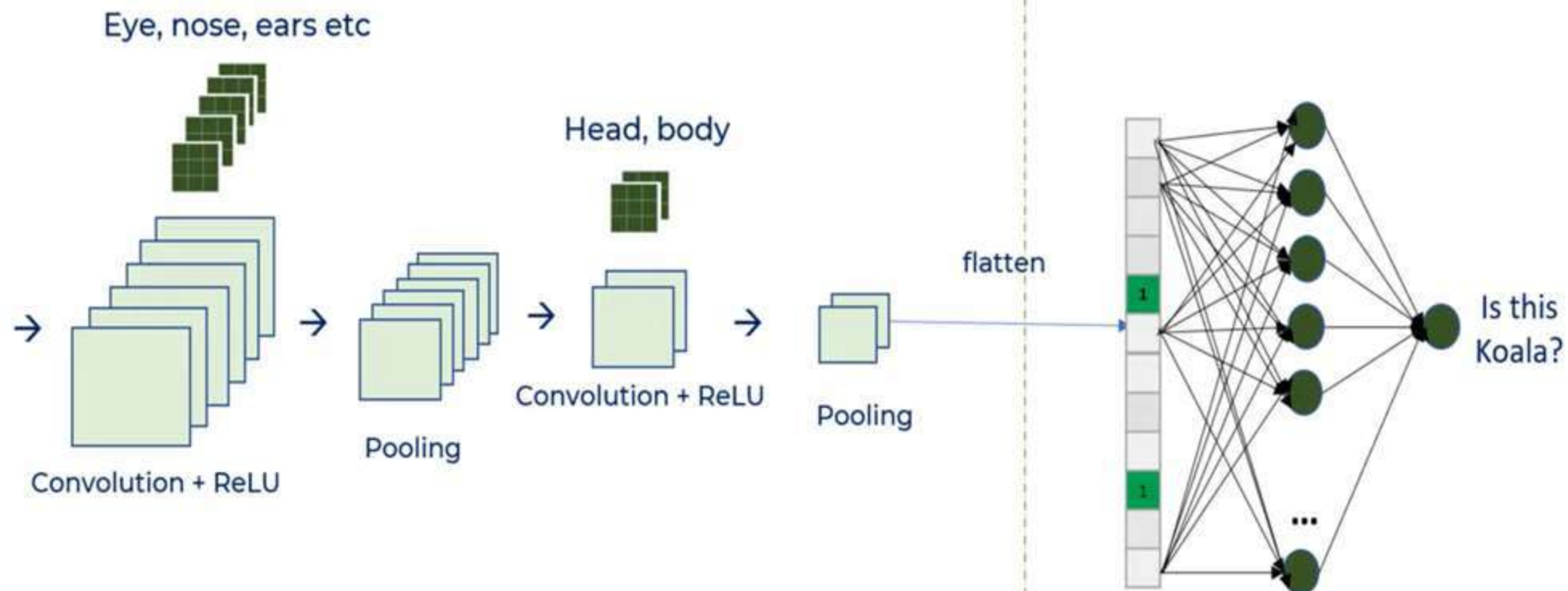
Benefits of pooling

Reduces
dimensions &
computation

Reduce overfitting
as there are less
parameters

Model is tolerant
towards variations,
distortions





Feature Extraction

Classification

Convolution

- Connections sparsity reduces overfitting
- Conv + Pooling gives location invariant feature detection
- Parameter sharing

ReLU

- Introduces nonlinearity
- Speeds up training, faster to compute

Pooling

- Reduces dimensions and computation
- Reduces overfitting
- Makes the model tolerant towards small distortion and variations

